

Multi-Cloud Engineering Program

Recorrido de Google Cloud

27 clases · 116 horas

La plataforma esencial de Google Cloud y su arquitectura de datos y operación en producción, con las clases de otras partes que tratan el proveedor.

Extracto del manual integral, generado desde las mismas fuentes versionadas.

Alcance de este cuaderno

Componente	Cobertura
Partes propias	04, 19
Clases de esas partes	24
Clases de otras partes	3
Clases totales	27
Horas estimadas	116

Índice

Alcance de este cuaderno	2
Índice	3
Parte 04 — Google Cloud: plataforma esencial	4
049 — Organización, folders, proyectos, billing y cuotas	4
050 — IAM, service accounts y Workload Identity Federation	14
051 — VPC global, subredes regionales, firewall y Cloud NAT	24
052 — Compute Engine, managed instance groups y load balancing	34
053 — Cloud Storage, clases, lifecycle y replicación	44
054 — Cloud SQL, Spanner, Firestore y Memorystore	54
055 — Cloud Run, Cloud Functions y API Gateway	64
056 — Pub/Sub, Cloud Tasks y Workflows	74
057 — Cloud Logging, Monitoring, Trace y Audit Logs	84
058 — Cloud KMS, Secret Manager y Security Command Center	94
059 — Terraform y despliegues reproducibles en GCP	104
060 — Proyecto: aplicación de tres capas en Google Cloud	115
Parte 06 — Kubernetes y plataformas administradas	126
083 — EKS, AKS y GKE: similitudes y diferencias	126
Parte 09 — Datos, mensajería, serverless e integración	135
114 — Pub/sub, streams, particiones y orden	135
Parte 19 — Google Cloud: arquitectura de datos y operación en producción	144
229 — Resource Manager, folders, Shared VPC y guardrails	144
230 — Workload Identity Federation, IAM Conditions y PAM	155
231 — Red global, load balancing, PSC y Cloud DNS	166
232 — Terraform, Infrastructure Manager y policy validation	176
233 — Cloud Run, Functions, API Gateway y Workflows	187
234 — GKE Autopilot, Workload Identity y Config Sync	198
235 — Cloud SQL, Spanner, Firestore y Bigtable	209
236 — BigQuery, Dataflow, Dataproc y gobernanza de datos	220
237 — Pub/Sub, Eventarc y entrega exactamente-una-vez	230
238 — Cloud Operations, Trace y OpenTelemetry	241
239 — SCC, VPC Service Controls, KMS y FinOps	251
240 — Proyecto: CloudShop productivo en Google Cloud	261
Parte 22 — Especializaciones, certificaciones y práctica profesional	272
273 — Mapeo AWS, Azure, Google Cloud, Kubernetes y FinOps	272

Parte 04 — Google Cloud: plataforma esencial

049 — Organización, folders, proyectos, billing y cuotas

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: governance · Estado: EXECUTABLECORE

Propósito

Montar la estructura de una organización en Google Cloud sabiendo que aquí el proyecto no equivale a la suscripción de Azure ni a la cuenta de AWS: es barato, desechable y la unidad de casi todo —API, cuotas, IAM, nombres—, y esa diferencia cambia el diseño en vez de solo cambiar los nombres. Es la clase donde se comprueba la primera parte de la hipótesis de la clase 048: qué contratos de las partes 02 y 03 reaparecen intactos y dónde aparece la primera asimetría propia.

Resultados de aprendizaje

Al finalizar podrás:

1. Diseñar una jerarquía de carpetas y proyectos sabiendo qué se hereda y qué se decide en cada nivel.
2. Usar el identificador correcto de un proyecto y explicar por qué el nombre no sirve para automatizar.
3. Dimensionar cuotas por proyecto y por región, y planificar los aumentos con antelación en vez de descubrirlos.
4. Separar el eje de facturación del eje jerárquico, y activar la exportación de costos antes de necesitarla.
5. Aplicar políticas de organización distinguiéndolas de IAM, con su prueba negativa.

Conceptos centrales

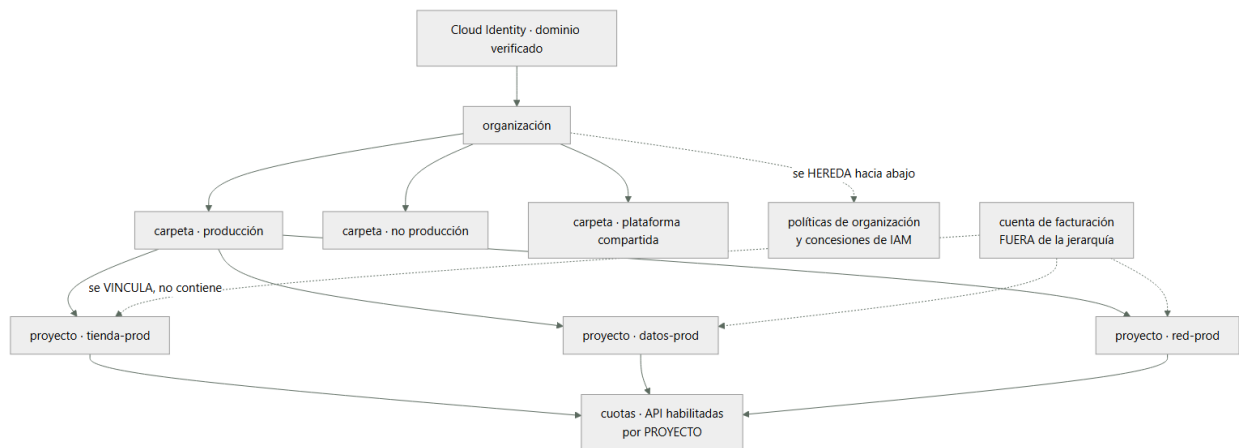
Concepto	Comprensión verificable
organización	Nodo raíz, vinculado a un dominio verificado en Cloud Identity o Workspace. Como el inquilino de Azure, la identidad precede a la jerarquía: sin dominio no hay organización.
proyecto	Unidad de API habilitadas, cuotas, IAM, red y nombres. Es barato y se crea en segundos, así que el idioma de Google Cloud es tener muchos, no pocos y grandes.
identificador de proyecto	Cadena global, única e inmutable que usan todas las API. Distinta del nombre, que se puede cambiar, y del número, que también es inmutable. Automatizar con el nombre es el error clásico.
cuenta de facturación	Objeto fuera de la jerarquía: se vincula a proyectos, no los contiene. Desvincularla no borra nada y detiene los recursos del proyecto.
cuota	Límite por proyecto y a menudo por región, de dos tipos: de tasa —por minuto— y de asignación —recursos existentes—. Ampliarlas es una solicitud con plazo, no un ajuste.
política de organización	Restricción sobre configuraciones permitidas, heredada por la jerarquía. No es IAM: acota lo que se puede configurar, con independencia de quién tenga permiso.
proyecto huérfano	Proyecto creado fuera de la organización. No hereda ninguna política, no aparece en la exportación de costos y nadie lo gobierna.

Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. El proyecto es barato, y eso cambia el diseño

La primera traducción que hace todo el mundo al llegar de AWS o de Azure es esta:

cuenta de AWS ≈ suscripción de Azure ≈ proyecto de Google Cloud

Es correcta en lo que concentra —aislamiento, cuotas, facturación, IAM— y falla en lo que cuesta. Abrir una cuenta de AWS implica un proceso organizativo; crear una suscripción de Azure, un acuerdo comercial. Crear un proyecto es una llamada de API que tarda segundos:

```
$ gcloud projects create cloudshop-tienda-prod-01 \
  --folder $CARPETA_PROD --labels entorno=prod,sistema=tienda
$ gcloud beta billing projects link cloudshop-tienda-prod-01 \
  --billing-account $CUENTA_FACTURACION
```

Esa diferencia de costo administrativo produce un idioma distinto. En AWS y en Azure la presión es a agrupar para no multiplicar cuentas o suscripciones; en Google Cloud la presión es la contraria, y por tres razones concretas:

1. las cuotas son POR PROYECTO
más proyectos = más margen, sin pedir nada a nadie
2. las API habilitadas son POR PROYECTO
un proyecto solo tiene encendido lo que usa: menos superficie
3. borrar un proyecto borra todo lo que contiene
la limpieza de un entorno efímero es una operación, no un inventario

La tercera es la que más rinde en la práctica. Un entorno de pruebas completo se destruye con una llamada, y el borrado es reversible durante 30 días antes de hacerse definitivo, lo que da un margen que ni el grupo de recursos de Azure ni la cuenta de AWS ofrecen de la misma forma.

Y hay un detalle que rompe automatizaciones el primer día: un proyecto tiene tres identificadores y solo uno sirve.

nombre	"CloudShop tienda producción"	legible, MUTABLE
identificador	cloudshop-tienda-prod-01	lo usan las API, INMUTABLE y ÚNICO EN GOOGLE CLOUD ENTERO
número	418293047512	inmutable, lo usan algunos servicios

Dos consecuencias. La primera: cualquier automatización debe usar el identificador; usar el nombre funciona hasta que alguien lo edita. La segunda sorprende más: el identificador es único globalmente, entre todos los clientes de Google Cloud. cloudshop-prod puede estar cogido por una empresa que no conoces, y además un identificador no se libera nunca, ni siquiera al borrar el proyecto. La convención de nombres tiene que asumirlo desde el principio:

[illegible]

El prefijo no es estética: es lo que hace improbable la colisión global. Y el límite de 30 caracteres del identificador obliga a abreviar, así que conviene fijarlo antes de crear el primero y no a mitad del segundo entorno.

2. Carpetas, herencia y el proyecto que nadie gobierna

La jerarquía tiene cuatro niveles y hereda hacia abajo, igual que en las dos plataformas anteriores:

organización	vinculada a un dominio verificado
carpeta	anidable hasta 10 niveles
carpeta	
proyecto	la unidad real
recurso	

Lo que se hereda es IAM y políticas de organización, y es acumulativo: un permiso concedido en la carpeta vale para todos los proyectos que contiene. Es el mismo comportamiento aditivo de Azure RBAC de la clase 038, y aquí Google Cloud ofrece dos mecanismos que restan en vez de uno: las políticas de denegación de IAM —que la clase 050 desarrolla— y las políticas de organización, que ocupan el mismo lugar que Azure Policy allí. Con tres plataformas ya se puede enunciar el patrón: el permiso suma, y lo que resta vive en otro sistema, con otro error y otras reglas de herencia. Lo que cambia entre proveedores es cuántos sistemas hay para restar, no que hagan falta.

El criterio para diseñar las carpetas es el mismo que la clase 037 estableció y conviene repetirlo porque la tentación es la misma: se ordena por lo que necesita una política distinta, no por el organigrama. Una estructura que sirve:

organización	
produccion/	políticas estrictas, acceso restringido
tienda/ datos/ red/	
no-produccion/	políticas relajadas, presupuestos bajos
desarrollo/ pruebas/	
plataforma/	red compartida, registro central, artefactos
aislados/	terceros, pruebas de concepto, sin datos reales

Las carpetas se reorganizan sin recrear proyectos —mover un proyecto de carpeta es una operación— así que equivocarse aquí es barato. Lo que no es barato es no tener organización en absoluto.

Y ese es el problema propio de Google Cloud que no existe en las otras dos: un proyecto puede crearse fuera de cualquier organización. Cualquiera con una cuenta de Google puede crear uno, vincularlo a una tarjeta y desplegar. Ese proyecto:

- no hereda ninguna política de organización
- no aparece en la exportación de facturación de la empresa
- no lo ve ningún panel de seguridad corporativo
- si la persona se va, puede quedar sin propietario accesible

Es el equivalente al «TI en la sombra» con infraestructura real dentro, y suele descubrirse por una factura o por un incidente. Las dos medidas que lo cierran:

```
# 1. solo un grupo concreto puede crear proyectos en la organización
$ gcloud organizations add-iam-policy-binding $ORG_ID \
  --member "group:plataforma@cloudshop.example" \
  --role roles/resourcemanager.projectCreator

# 2. solo identidades del dominio propio pueden recibir permisos
$ gcloud resource-manager org-policies allow \
  constraints/iam.allowedPolicyMemberDomains $ID_CLIENTE_DOMINIO \
  --organization $ORG_ID
```

La segunda es una de las políticas de mayor rendimiento de toda la plataforma: impide conceder acceso a una cuenta personal de Gmail o al dominio de otra empresa. Sin ella, un roles/editor concedido a una dirección externa es una puerta permanente que ningún inventario de usuarios de la empresa muestra.

Y la parte que no cubre ninguna política: los proyectos que ya existen fuera de la organización. Se localizan por la facturación —una cuenta de facturación corporativa vinculada a un proyecto sin organización— y se migran, que es una operación posible y con requisitos. Es el mismo patrón de la clase 046: la política guarda la puerta y no limpia lo que ya está dentro.

3. API y cuotas: dos límites que se descubren tarde

Ningún servicio existe en un proyecto hasta que se habilita su API. No hay equivalente a esto en AWS ni en Azure, y produce dos efectos, uno bueno y uno molesto.

```
$ gcloud services enable run.googleapis.com sqladmin.googleapis.com \
secretmanager.googleapis.com --project cls-tienda-prod-euw1-01
$ gcloud services list --enabled --project cls-tienda-prod-euw1-01 | wc -l
14
```

El efecto bueno es que la superficie de un proyecto es explícita y auditable: catorce API habilitadas es una afirmación verificable sobre lo que ese proyecto puede hacer. El molesto es que un despliegue falla con `SERVICEDISABLED` la primera vez, y que habilitar una API tiene una propagación de hasta unos minutos — así que habilitarla y usarla en la misma ejecución de una plantilla puede fallar de forma intermitente. La solución es habilitar las API en un paso previo, no en el mismo despliegue que las usa.

Las cuotas son la segunda sorpresa, y esta cuesta calendario. Hay dos tipos y se confunden:

de tasa	peticiones por minuto a una API
	se reinician solas; el síntoma es 429 <code>RESOURCE_EXHAUSTED</code>
de asignación	cantidad de recursos que pueden existir a la vez
	CPU por región, direcciones IP, instancias, reglas de firewall
	el síntoma es un despliegue que no puede crear más

Y el detalle que decide el diseño: casi todas son por proyecto, y muchas además por región. Eso convierte la granularidad de proyectos en una decisión de capacidad, no solo de orden:

- un proyecto grande con todo
 - una sola bolsa de CPU por región para todos los servicios
 - una prueba de carga de un equipo agota el margen de otro

- varios proyectos por servicio
 - bolsas independientes
 - el ruido de un equipo no llega al de al lado

Lo que hay que anticipar es el plazo. Un aumento de cuota es una solicitud que se revisa, y puede tardar de horas a días:

```
$ gcloud compute regions describe europe-west1 \
--format="table(quotas.metric,quotas.limit,quotas.usage)" | grep CPUS
CPUS          72.0      58.0
```

Cincuenta y ocho de setenta y dos usadas antes de una campaña que triplica el tráfico: eso no es un problema técnico, es un problema de planificación, y solo se ve mirándolo. La regla que lo evita es sencilla y casi nunca está: revisar la cuota frente al pico previsto forma parte de la preparación de cualquier evento de carga, junto con la prueba de carga misma.

Y una alerta que rinde más que muchas otras: la cuota consumida por encima del 80 %, que en Cloud Monitoring existe como métrica y avisa antes de que el despliegue falle en vez de después.

4. La facturación es otro eje, y la exportación no es retroactiva

Aquí está la asimetría estructural de esta clase, la que no tiene equivalente en las dos plataformas anteriores.

AWS	la cuenta pagadora está DENTRO de la organización
Azure	la suscripción ES la frontera de facturación
Google	la cuenta de facturación está FUERA de la jerarquía
	y se VINCULA a los proyectos

Un proyecto puede moverse de carpeta sin cambiar de cuenta de facturación, y varias organizaciones pueden compartir una. La consecuencia práctica es que la estructura de costos y la estructura de gobierno son dos diseños distintos y hay que hacer los dos.

Y hay una operación con un efecto que sorprende: desvincular la facturación de un proyecto no lo borra, pero detiene sus recursos. Las máquinas se apagan, los servicios gestionados dejan de responder. Es a la vez un mecanismo de contención de emergencia y una forma muy eficaz de tumbar producción sin darse cuenta de lo que se estaba haciendo. Merece un bloqueo organizativo y figurar en la lista de operaciones peligrosas del equipo.

La exportación de facturación a BigQuery es el mecanismo real de atribución, y tiene una propiedad que hay que conocer antes y no después:

```
$ gcloud beta billing accounts describe $CUENTA_FACTURACION
# la exportación se configura en la consola de facturación → Exportación de BigQuery

la exportación NO es retroactiva
lo que no se exportó mientras ocurría, no se puede recuperar después
```

Activarla el primer día cuesta cinco minutos y una tabla de BigQuery; activarla el tercer mes deja dos meses sin detalle, con solo los informes agregados de la consola. Es exactamente el mismo error de la clase 045 con los registros de recurso apagados: una señal que no se recoge mientras ocurre no existe.

Lo mismo vale para las etiquetas. La exportación atribuye costo por etiqueta, y solo desde el momento en que la etiqueta estaba puesta:

```
SELECT
  (SELECT value FROM UNNEST(labels) WHERE key = 'sistema') AS sistema,
  (SELECT value FROM UNNEST(labels) WHERE key = 'entorno') AS entorno,
  ROUND(SUM(cost), 2) AS costo
FROM `cls-facturacion.billing.gcp_billing_export_v1_XXXXXX`
WHERE DATE(usage_start_time) BETWEEN '2026-07-01' AND '2026-07-31'
GROUP BY sistema, entorno
ORDER BY costo DESC
```

Un recurso sin etiqueta aparece con sistema nulo, y esa fila —«coste no atribuible»— es la métrica de gobierno que importa: mientras no sea cero, hay gasto del que nadie responde. Es el mismo indicador que la clase 025 definió para AWS, con otra consulta.

Y sobre los presupuestos, una precisión que evita una expectativa falsa:

un presupuesto NOTIFICA; no limita el gasto

Exactamente igual que en AWS. Limitar de verdad exige automatizar una reacción —notificación a Pub/Sub y una función que actúe— y la única acción realmente contundente es desvincular la facturación, que apaga el proyecto entero. Es una opción legítima para un entorno de pruebas y no lo es para producción, así que el control efectivo en producción sigue siendo el de siempre: alertar pronto, atribuir bien y revisar.

5. Políticas de organización: el gobierno que no es IAM

Es la tercera vez que aparece el mismo patrón, así que ya se puede enunciar como regla general del programa:

AWS	política de control de servicios (SCP)
Azure	Azure Policy
Google	política de organización

las tres: acotan CONFIGURACIONES, no identidades
se heredan por la jerarquía
producen un error DISTINTO al de falta de permiso
y no arreglan lo que ya existe

Hay dos formas de restricción:

booleana	se aplica o no constraints/storage.publicAccessPrevention constraints/compute.requireOsLogin constraints/iam.disableServiceAccountKeyCreation
de lista	permite o deniega valores concretos constraints/compute.vmExternalIpAccess constraints/gcp.resourceLocations constraints/iam.allowedPolicyMemberDomains

La que más rinde y conviene poner el primer día es iam.disableServiceAccountKeyCreation. Las claves de cuenta de servicio son ficheros JSON con credenciales de larga duración, y son la fuga de credenciales más habitual de Google Cloud: acaban en repositorios, en portátiles y en imágenes de contenedor. La clase 050 desarrolla la alternativa; la política que impide crearlas es lo que hace que la alternativa se use de verdad.

```
$ gcloud resource-manager org-policies enable-enforce \
  constraints/iam.disableServiceAccountKeyCreation --organization $ORG_ID
$ gcloud resource-manager org-policies enable-enforce \
  constraints/storage.publicAccessPrevention --organization $ORG_ID
```

Y dos comportamientos que conviene tener claros antes de asignarlas:

La herencia se puede romper deliberadamente. Una política definida en un proyecto puede sustituir a la de la carpeta si la restricción lo permite. Eso hace posible la excepción legítima —un proyecto que sí necesita un contenedor público para recursos estáticos— y también hace posible la excepción silenciosa. La comprobación es la misma que en la clase 046: buscar dónde se ha roto la herencia y exigir que cada caso tenga motivo escrito.

```
$ gcloud asset search-all-resources --scope organizations/$ORG_ID \
  --asset-types cloudresourcemanager.googleapis.com/Project \
```

```
--format "value(name)" | while read p; do
gcloud resource-manager org-policies describe \
  constraints/storage.publicAccessPrevention --project "$p" 2>/dev/null \
  | grep -q "enforce: false" && echo "excepción en $p"
done
```

No limpia lo existente. Activar `publicAccessPrevention` impide abrir nuevos buckets al público y no cierra los que ya lo están. Es literalmente la lección de la clase 046 en otro proveedor, lo que confirma la parte de la hipótesis que decía que los contratos se conservan: la secuencia correcta vuelve a ser inventariar, corregir y después imponer.

Y la prueba negativa, que es la única evidencia aceptable:

```
$ gsutil iam ch allUsers:objectViewer gs://cls-tienda-publico
AccessDeniedException: 412 Request violates constraint
constraints/storage.publicAccessPrevention
```

✓

El código de error importa por lo mismo que en las clases 038 y 046: no dice que falte un permiso. Añadir roles no habría cambiado nada, y saber leerlo es lo que evita una hora buscando en el sistema equivocado.

Ejemplo trabajado

CloudShop abre su tercera plataforma. El equipo llega con el contrato de once filas de la clase 048 y lo aplica en dos días — y luego encuentra tres cosas que ese contrato no cubría, todas propias de Google Cloud.

Lo que se reutilizó sin discusión (y confirma la primera parte de la hipótesis):

la identidad precede a la jerarquía	organización desde dominio verificado
la jerarquía hereda y el permiso suma	carpetas y proyectos
el gobierno acota configuraciones	políticas de organización
la frontera de aislamiento es la de cuotas	proyecto
etiquetas obligatorias para atribuir costo	labels
inventariar → corregir → imponer	misma secuencia

Seis de las once filas quedaron resueltas en el primer día, sin aprender nada nuevo salvo el nombre.

Sorpresa 1 — la traducción «suscripción → proyecto» sale cara en una prueba de carga.

Siguiendo el modelo de Azure, se crean tres proyectos: `cls-dev`, `cls-stage`, `cls-prod`, con todo dentro. La primera prueba de carga previa a campaña falla al escalar:

```
$ gcloud compute instances create ... --zone europe-west1-b
ERROR: Quota 'CPUS' exceeded. Limit: 72.0 in region europe-west1.
$ gcloud compute regions describe europe-west1 \
  --format="value(quotas.filter(metric:CPUS).usage)"
71.0
```

Las cuotas son por proyecto, y en `cls-prod` convivían la tienda, el procesamiento por lotes y el entorno de análisis. La solicitud de aumento tardó tres días, con la campaña a cinco.

Se rehace la estructura con el idioma correcto de la plataforma:

proyectos	3	19
bolsas de CPU independientes por región	1	7
API habilitadas en el proyecto de tienda	61	14
aumento de cuota necesario	sí	no
borrado de un entorno de pruebas	inventario a mano	<code>gcloud projects delete</code>

La fila de las API es un efecto secundario que nadie buscaba: al separar, cada proyecto solo habilita lo que usa, y sesenta y una API encendidas «por si acaso» pasaron a catorce declaradas.

Sorpresa 2 — dos meses de gasto sin poder atribuirlo.

A los dos meses, finanzas pide el costo por sistema. La consola da totales por proyecto y por servicio, y no por etiqueta.

exportación de facturación a BigQuery	activada el día 61
datos disponibles	desde el día 61
datos de los días 1 al 60	solo agregados, sin detalle

La exportación no es retroactiva. Y las etiquetas se habían añadido en la semana 4, así que incluso desde el día 61 una parte del gasto —recursos creados antes y nunca modificados— seguía sin sistema:

```
SELECT COUNT(*) AS lineas, ROUND(SUM(cost),2) AS costo
FROM `cls-facturacion.billing.gcp_billing_export_v1_XXXXXX`
```

```
WHERE (SELECT value FROM UNNEST(labels) WHERE key='sistema') IS NULL

costo no atribuible                2.140 USD/mes                0
etiquetas obligatorias            por convención            por política y en plantilla
exportación de facturación        día 61                    día 1 en la organización nueva
```

La medida de fondo no fue etiquetar: fue hacer de «costo no atribuible = 0» un indicador con dueño, revisado cada semana. Una convención se erosiona; un número que alguien tiene que explicar, no.

Sorpresa 3 — cuatro proyectos que nadie sabía que existían.

Una revisión de la cuenta de facturación encuentra gasto en proyectos que no están en la organización:

```
$ gcloud beta billing projects list --billing-account $CUENTA \
  --format="value(projectId)" > facturados.txt
$ gcloud asset search-all-resources --scope organizations/$ORG_ID \
  --asset-types cloudresourcecmanager.googleapis.com/Project \
  --format="value(displayName)" > en_organizacion.txt
$ comm -23 <(sort facturados.txt) <(sort en_organizacion.txt)
cloudshop-poc-ml
cloudshop-demo-cliente
prueba-integracion-2025
sandbox-jm
```

Cuatro proyectos creados con cuentas personales, vinculados a la facturación de la empresa. Sin políticas heredadas, sin registro central, sin aparecer en ningún panel de seguridad. Uno de ellos, cloudshop-demo-cliente, tenía un bucket público con datos de prueba que resultaron ser un volcado real anonimizado a medias.

proyectos fuera de la organización	4	0
quién puede crear proyectos	cualquiera	grupo plataforma
dominios permitidos en IAM	cualquiera	solo el propio
buckets públicos	1	0
comprobación de proyectos no gobernados	ninguna	semanal, automática

Y la prueba negativa que cierra el caso:

```
$ gsutil iam ch allUsers:objectViewer gs://cls-prueba
AccessDeniedException: 412 Request violates constraint
constraints/storage.publicAccessPrevention ✓
$ gcloud projects add-iam-policy-binding cls-tienda-prod-euw1-01 \
  --member "user:externo@gmail.com" --role roles/viewer
ERROR: Request violates constraint constraints/iam.allowedPolicyMemberDomains ✓
```

Resumen de la puesta en marcha:

proyectos	3	19
proyectos fuera de la organización	4	0
API habilitadas en el proyecto principal	61	14
aumentos de cuota necesarios	1	0
costo no atribuible	2.140 USD/mes	0
exportación de facturación	día 61	día 1
políticas de organización aplicadas	0	6
controles con prueba negativa	0	6

La lección que esta clase traslada al resto de la parte 04: seis de los once contratos de la clase 048 se reutilizaron el primer día, y las tres sorpresas fueron todas de la misma familia — cosas que en las otras plataformas eran caras y aquí son gratis, y que por eso se usan de otra manera. El proyecto barato cambia la granularidad, la creación libre de proyectos abre un agujero de gobierno que no existía, y la facturación fuera de la jerarquía obliga a diseñar dos estructuras en vez de una. Ninguna de las tres es una carencia de la plataforma: son consecuencias de una decisión de diseño distinta, y confundirlas con carencias es lo que produce una arquitectura que pelea contra su proveedor.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/049-organizacion-folders-proyectos-billing-y-cuotas/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.

2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa jerarquía-gcp en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye jerarquía-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Una prueba de carga falla por cuota con margen aparente en la organización	Las cuotas son por proyecto y por región, y varios servicios compartían proyecto	Reparte en más proyectos —son baratos— y revisa la cuota frente al pico previsto antes de cualquier evento de carga.
La automatización deja de funcionar tras renombrar un proyecto	Se usó el nombre en vez del identificador, que es el inmutable	Usa siempre el identificador de proyecto y fija una convención de nombres con prefijo, porque el identificador es único en todo Google Cloud.
No se puede atribuir el gasto de los primeros meses	La exportación de facturación a BigQuery no es retroactiva y las etiquetas se pusieron tarde	Actívala el primer día y convierte «costo no atribuible = 0» en un indicador con responsable.
Aparecen proyectos con gasto que no están en la organización	Cualquiera con una cuenta de Google puede crear proyectos fuera de ella	Restringe la creación a un grupo, aplica iam.allowedPolicyMemberDomains y compara periódicamente los proyectos facturados con los gobernados.
Un despliegue falla con SERVICEDISABLED de forma intermitente	La API se habilita en la misma ejecución que la usa y la propagación tarda	Habilita las API en un paso previo e independiente del despliegue que las consume.
Se activa una política de organización y siguen existiendo recursos que la incumplen	Las políticas acotan configuraciones nuevas y no corrigen lo existente	Inventaria, corrige y después impón; y busca dónde se ha roto la herencia con una excepción no documentada.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿En qué se parece un proyecto a una suscripción de Azure y en qué cambia el diseño el hecho de que sea barato?
2. ¿Cuál de los tres identificadores de un proyecto debe usar la automatización, y qué implica que sea único globalmente?
3. ¿Por qué la granularidad de proyectos es una decisión de capacidad y no solo de orden?
4. ¿Qué se pierde por activar la exportación de facturación en el mes tres en vez de en el día uno?
5. ¿Qué distingue una política de organización de una concesión de IAM, y cómo lo confirma el mensaje de error?

Referencias

- Google Cloud (2025). Resource hierarchy — organización, carpetas, proyectos y herencia.
<<https://cloud.google.com/resource-manager/docs/cloud-platform-resource-hierarchy>>
- Google Cloud (2025). Creating and managing projects — identificador, nombre, número y borrado con retención.
<<https://cloud.google.com/resource-manager/docs/creating-managing-projects>>
- Google Cloud (2025). Working with quotas — cuotas de tasa y de asignación, por proyecto y por región.
<<https://cloud.google.com/docs/quotas/overview>>
- Google Cloud (2025). Export Cloud Billing data to BigQuery — configuración, esquema y ausencia de retroactividad.
<<https://cloud.google.com/billing/docs/how-to/export-data-bigquery>>
- Google Cloud (2025). Organization policy constraints — restricciones booleanas y de lista, herencia y excepciones.
<<https://cloud.google.com/resource-manager/docs/organization-policy/org-policy-constraints>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 04 en PDF · Recorrido de Google Cloud en PDF · Manual integral

Anterior	Índice	Siguiente
← 048 · Proyecto: aplicación de tres capas en Azure	Parte 04 · Programa	050 · IAM, service accounts y Workload Identity Federation →

Evaluación — 049 Organización, folders, proyectos, billing y cuotas

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega jerarquía-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.

- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

050 — IAM, service accounts y Workload Identity Federation

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: iam · Estado: EXECUTABLECORE

Propósito

Dominar el modelo de identidad de Google Cloud, cuya pieza central no tiene equivalente exacto en las otras dos plataformas: la cuenta de servicio es a la vez una identidad y un recurso, así que hay dos preguntas de permiso en vez de una —qué puede hacer y quién puede usarla—. De ahí sale el camino de escalada más habitual de la plataforma, la fuga de credenciales más repetida y la razón por la que aquí el privilegio mínimo se revisa con datos de uso en vez de con criterio.

Resultados de aprendizaje

Al finalizar podrás:

1. Distinguir los permisos de una cuenta de servicio de los permisos para suplantarla, y detectar la escalada que produce la segunda.
2. Sustituir claves de cuenta de servicio por identidad adjunta, suplantación y federación de identidad de carga de trabajo.
3. Acotar una federación con una condición de atributo y demostrar con una prueba negativa que la acotación funciona.
4. Aplicar políticas de denegación de IAM y condiciones con caducidad, y saber en qué orden se evalúa todo.
5. Reducir privilegios a partir del uso observado en vez de por estimación.

Conceptos centrales

Concepto	Comprensión verificable
cuenta de servicio	Identidad y recurso a la vez. Tiene permisos propios sobre otros recursos y una política propia que dice quién puede usarla. Confundir las dos caras es la causa de la escalada más común de la plataforma.
suplantación	Obtener un testigo de corta duración de una cuenta de servicio. Sustituye a las claves para personas y para automatización, y deja rastro de quién suplantó a quién.
clave de cuenta de servicio	Fichero JSON con credenciales de larga duración. Es la fuga más frecuente de Google Cloud: acaba en repositorios, portátiles e imágenes de contenedor.
rol básico	Owner, Editor y Viewer. Editor permite modificar casi todo en el proyecto, y es lo que las cuentas de servicio por defecto reciben automáticamente.
condición de atributo	Filtro sobre las afirmaciones del testigo externo en una federación. Sin ella, cualquier repositorio del proveedor externo puede obtener credenciales.
política de denegación de IAM	Regla que quita permisos y se evalúa antes que las concesiones. Es lo que resta en un modelo aditivo, y convive con las políticas de organización sin sustituirlas.

Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. La cuenta de servicio tiene dos caras y solo se mira una

En AWS un rol se asume; en Azure una identidad administrada se asocia a un recurso. En Google Cloud la cuenta de servicio es las dos cosas a la vez, y de ahí sale casi todo lo interesante de esta clase:

cara 1 · es una IDENTIDAD
 tiene una dirección: `despliegue@cls-tienda-prod.iam.gserviceaccount.com`
 tiene roles sobre otros recursos: qué PUEDE HACER

cara 2 · es un RECURSO
 tiene su propia política de IAM: quién puede USARLA

La revisión de permisos habitual mira la primera cara —qué roles tiene cada persona— y se salta la segunda. Y la segunda es la que concede de verdad:

```
$ gcloud projects get-iam-policy cls-tienda-prod-euw1-01 \
  --flatten="bindings[].members" --filter="bindings.members:ana@cloudshop.example" \
  --format="value(bindings.role)"
roles/viewer
```

Parece que Ana solo puede leer. Pero:

```
$ gcloud iam service-accounts get-iam-policy \
  despliegue@cls-tienda-prod-euw1-01.iam.gserviceaccount.com \
  --format="value(bindings.role,bindings.members)"
roles/iam.serviceAccountTokenCreator user:ana@cloudshop.example

$ gcloud storage ls --impersonate-service-account \
  despliegue@cls-tienda-prod-euw1-01.iam.gserviceaccount.com
# lista todo lo que puede la cuenta de despliegue, que tiene roles/editor
```

Ana tiene, en la práctica, los permisos de la cuenta de despliegue. Su rol propio es irrelevante. Los dos permisos que producen esto:

roles/iam.serviceAccountTokenCreator	genera testigos: acceso completo a la cuenta
roles/iam.serviceAccountUser	permite ADJUNTARLA a un recurso nuevo
	→ crear una máquina con esa cuenta y
	ejecutar código con sus permisos

El segundo es más sutil y no menos potente: quien puede crear una máquina virtual y adjuntarle una cuenta de servicio privilegiada puede ejecutar lo que quiera con esos permisos. Por eso serviceAccountUser sobre una cuenta con Editor equivale a Editor.

La regla operativa que se deduce, y que cambia cómo se audita:

```
el permiso efectivo de una persona es la UNIÓN de:
  sus propios roles
  + los roles de TODA cuenta de servicio que pueda suplantar o adjuntar
  + (recursivamente) las que esas cuentas puedan suplantar a su vez
```

La palabra «recursivamente» no es teórica: una cadena de dos saltos —Ana suplanta a la cuenta A, que puede suplantar a la cuenta B, que es Owner— es un camino real y ninguna revisión que mire una sola tabla lo encuentra. La herramienta que lo resuelve es el analizador de políticas, que responde a la pregunta inversa: quién puede llegar a este recurso, por cualquier camino.

```
$ gcloud asset analyze-iam-policy --organization=$ORG_ID \
  --analyze-service-account-impersonation \
  --identity="user:ana@cloudshop.example"
```

2. Las cuentas por defecto ya vienen con Editor

Este es el hecho más importante de la clase para una organización recién abierta, porque el riesgo está puesto de fábrica.

Al habilitar Compute Engine o App Engine, el proyecto recibe cuentas de servicio por defecto y —salvo que se impida— se les concede automáticamente roles/editor sobre el proyecto:

```
$ gcloud projects get-iam-policy cls-tienda-prod-euw1-01 \
  --flatten="bindings[].members" --filter="bindings.role:roles/editor" \
  --format="value(bindings.members)"
serviceAccount:418293047512-compute@developer.gserviceaccount.com
```

Y esa cuenta es la que se adjunta por omisión a cualquier máquina virtual creada sin especificar otra. Es decir:

```
cualquier máquina creada sin pensar
  → ejecuta con una identidad que puede modificar casi todo el proyecto
  → una dependencia comprometida en esa máquina hereda ese poder
  → y el servicio de metadatos entrega el testigo sin pedir nada
```

La comprobación desde dentro de la máquina, que conviene hacer una vez para entender la magnitud:

```
$ curl -s -H "Metadata-Flavor: Google" \
  "http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/default/email"
418293047512-compute@developer.gserviceaccount.com
```

Las dos medidas, en este orden:

```
# 1. que las cuentas por defecto dejen de recibir Editor automáticamente
$ gcloud resource-manager org-policies enable-enforce \
  constraints/iam.automaticIamGrantsForDefaultServiceAccounts --organization $ORG_ID
```

```
# 2. una cuenta dedicada por carga de trabajo, con los roles mínimos
$ gcloud iam service-accounts create sa-tienda-web --project cls-tienda-prod-euw1-01
$ gcloud projects add-iam-policy-binding cls-tienda-prod-euw1-01 \
  --member "serviceAccount:sa-tienda-web@cls-tienda-prod-euw1-01.iam.gserviceaccount.com" \
  --role roles/secretmanager.secretAccessor --condition=None
```

Y hay una complicación heredada que confunde a quien depura permisos en máquinas antiguas: los ámbitos de acceso. Son un mecanismo previo a IAM que sigue existiendo en Compute Engine, y el permiso efectivo de una máquina es la intersección de ambos:

permiso efectivo = roles de la cuenta de servicio $\cap$ ámbitos de la máquina

Eso produce el desconcierto clásico: la cuenta tiene el rol, la llamada falla, y añadir más roles no cambia nada porque el ámbito no lo incluye. La guía vigente es fijar el ámbito en cloud-platform y controlar exclusivamente con IAM, que es donde hay granularidad y auditoría:

```
$ gcloud compute instances create web-01 --zone europe-west1-b \
  --service-account sa-tienda-web@cls-tienda-prod-euw1-01.iam.gserviceaccount.com \
  --scopes cloud-platform
```

Y los roles básicos merecen una frase propia. Editor incluye modificar prácticamente cualquier recurso del proyecto; Owner añade gestionar los permisos, que es lo que convierte cualquier acceso en permanente. Ninguno de los tres debería aparecer en una concesión nueva. Su equivalente correcto son los roles predefinidos —hay cientos, muy específicos— y, cuando ninguno encaja, un rol personalizado:

```
$ gcloud iam roles create tiendaOperador --organization $ORG_ID \
  --permissions run.services.get,run.services.update,logging.logEntries.list \
  --stage GA
```

3. Las claves: por qué desaparecen y por qué no

Una clave de cuenta de servicio es un fichero JSON con una credencial que no caduca. Es la fuga de credenciales más frecuente de Google Cloud, y no por descuido excepcional: es que resulta cómoda.

```
{ "type": "service_account", "project_id": "cls-tienda-prod-euw1-01",
  "private_key_id": "...", "private_key": "-----BEGIN PRIVATE KEY-----\n...",
  "client_email": "despliegue@cls-tienda-prod-euw1-01.iam.gserviceaccount.com" }
```

Cuatro sitios donde acaban, en orden de frecuencia: un repositorio, la máquina de alguien, una imagen de contenedor y un gestor de secretos de otro sistema desde el que se copia a mano.

Las cuatro alternativas, por caso de uso, ordenadas de mejor a peor:

carga dentro de Google Cloud	cuenta de servicio ADJUNTA al recurso no hay credencial: el servicio de metadatos entrega un testigo corto
persona	suplantación con el propio inicio de sesión gcloud --impersonate-service-account
CI externo o nube ajena	federación de identidad de carga de trabajo
caso sin alternativa	clave, con caducidad y rotación automatizada y con la justificación escrita

La suplantación para personas cambia además lo que dice la auditoría, que es la mitad del valor:

con clave	el registro dice: actuó despliegue@... quién la usó: se desconoce
con suplantación	el registro dice: ana@cloudshop.example actuó COMO despliegue@...

La primera forma hace imposible responder «quién hizo esto» durante un incidente. La segunda lo responde sola.

```
$ gcloud config set auth/impersonate_service_account \
  despliegue@cls-tienda-prod-euw1-01.iam.gserviceaccount.com
$ gcloud storage ls gs://cls-tienda-facturas
```

Y el interruptor que hace que todo lo anterior se cumpla de verdad, en lugar de quedarse en una recomendación:

```
$ gcloud resource-manager org-policies enable-enforce \
  constraints/iam.disableServiceAccountKeyCreation --organization $ORG_ID
```

Con él activo, la vía cómoda deja de existir y los equipos adoptan las otras tres. Sin él, siempre habrá un caso urgente que justifique una clave más. Es el mismo argumento de la clase 046 sobre por qué una directiva vale más que una norma: el control que no se puede saltar no depende de la disciplina de nadie.

Y para las claves que ya están en circulación, el inventario primero y la prueba negativa después:

```
$ gcloud asset search-all-resources --scope organizations/$ORG_ID \
  --asset-types iam.googleapis.com/ServiceAccountKey \
  --query "NOT name:*/keys/system-managed*" --format="value(name)" | wc -l
14
```

Las gestionadas por el sistema se excluyen del recuento porque son internas y rotan solas. Las catorce restantes son las creadas por personas, y cada una necesita un destino: sustituirla, o quedar documentada con responsable y fecha.

4. Federación: la tercera vez que aparece la misma condición

La federación de identidad de carga de trabajo permite que un sistema externo —una canalización de GitHub, una carga en otra nube, un servidor propio— obtenga credenciales sin ninguna clave. Es el mismo contrato de las clases 026 y 038, así que aquí interesa lo que cambia y lo que no.

Lo que no cambia: la pieza crítica sigue siendo acotar quién puede usarla.

```
$ gcloud iam workload-identity-pools create cls-ci --location global \
  --display-name "CI de CloudShop"

$ gcloud iam workload-identity-pools providers create-oidc github \
  --workload-identity-pool cls-ci --location global \
  --issuer-uri "https://token.actions.githubusercontent.com" \
  --attribute-mapping "google.subject=assertion.sub,\
attribute.repository=assertion.repository,\
attribute.ref=assertion.ref" \
  --attribute-condition "assertion.repository == 'cloudshop/tienda' && \
assertion.ref == 'refs/heads/main'"
```

Sin `--attribute-condition`, cualquier repositorio de GitHub del mundo puede obtener credenciales de tu organización. Es exactamente el mismo fallo que en AWS con un sub sin acotar y en Azure con un subject vacío, y es la tercera confirmación del mismo contrato: la parte peligrosa de una federación nunca es el emisor, es el sujeto.

Lo que sí cambia es la forma de conceder, que aquí es más expresiva. En vez de asociar la federación a una única identidad, se conceden roles a un conjunto de principales:

```
$ gcloud iam service-accounts add-iam-policy-binding \
  despliegue@cls-tienda-prod-euw1-01.iam.gserviceaccount.com \
  --role roles/iam.workloadIdentityUser \
  --member "principalSet://iam.googleapis.com/projects/$NUM/locations/global/\
workloadIdentityPools/cls-ci/attribute.repository/cloudshop/tienda"
```

Ese formato permite conceder por atributo —todo lo que venga de un repositorio, o de una rama— sin enumerar identidades una a una. Es cómodo y es la misma cuerda de la que colgarse: un `principalSet` con un atributo demasiado amplio concede a más de lo previsto.

Y la prueba negativa, que es la única evidencia aceptable y ya se ha ejecutado en tres plataformas:

desde otro repositorio	permiso denegado	✓
desde cloudshop/tienda, rama de trabajo	permiso denegado	✓
desde cloudshop/tienda, rama main	testigo emitido	✓

Dos mecanismos más completan el modelo, y conviene situarlos:

Condiciones en las concesiones. Una concesión puede llevar una expresión que la limite en el tiempo o por nombre de recurso. Es el privilegio mínimo en el tiempo de la clase 038, más simple que la activación con aprobación de allí y suficiente para un acceso temporal:

```
$ gcloud projects add-iam-policy-binding cls-tienda-prod-euw1-01 \
  --member "user:ana@cloudshop.example" --role roles/run.admin \
  --condition "expression=request.time < timestamp('2026-08-15T00:00:00Z'),\
title=incidente-4821,description=acceso temporal"
```

La concesión caduca sola. Eso elimina la deuda de accesos que nadie retira, que es de donde salen la mitad de los permisos excesivos de cualquier organización con dos años de vida.

Políticas de denegación. Se evalúan antes que las concesiones y ganan siempre, y se adjuntan a la organización, la carpeta o el proyecto:

```
orden de evaluación
1. políticas de denegación ¿hay una regla que quite este permiso? → fin
2. concesiones ¿hay alguna que lo dé? → permitido
3. políticas de organización ¿la configuración resultante está permitida?
```

Son la pieza que faltaba para acotar un rol amplio sin rehacerlo: negar `iam.serviceAccounts.getAccessToken` sobre las cuentas de producción a todo el mundo salvo a un grupo, por ejemplo. Y conviven con las políticas de organización sin sustituirlas, porque responden preguntas distintas: la denegación dice qué permisos no tienes; la política de organización dice qué configuraciones no son válidas, tengas el permiso que tengas.

5. Reducir privilegios con datos en vez de con criterio

Todas las plataformas del programa han pedido privilegio mínimo y ninguna ha ofrecido hasta ahora una forma no subjetiva de conseguirlo. Aquí sí la hay, y merece ser el cierre de la clase porque cambia la conversación.

El recomendador observa el uso real de los últimos 90 días y propone el rol ajustado:

```
$ gcloud recommender recommendations list \
  --project cls-tienda-prod-euw1-01 --location global \
  --recommender google.iam.policy.Recommender \
  --format "table(content.overview.member, content.overview.removedRole,
                  content.overview.addedRole)"
```

miembro	rol retirado	rol propuesto
sa-tienda-web@...	roles/editor	roles/run.invoker roles/secretmanager.secretAccessor
sa-informes@...	roles/editor	roles/bigquery.dataViewer
user:carlos@cloudshop.example	roles/owner	roles/viewer

Esto convierte una discusión —«¿de verdad necesitas Editor?»— en un dato: durante 90 días esa identidad usó tres permisos de los cientos que tenía. Y tiene dos límites que hay que decir en voz alta para no crear una falsa confianza:

1. una acción que solo ocurre una vez al trimestre no aparece en 90 días
→ aplicar la recomendación puede romper el cierre anual
2. mide lo USADO, no lo NECESARIO
→ un permiso usado por error sigue apareciendo como usado

La forma responsable de aplicarlo es por pasos y con vuelta atrás preparada:

1. aplicar en no producción y observar dos semanas
2. en producción, empezar por identidades de carga de trabajo (su comportamiento es más predecible que el de las personas)
3. conservar el permiso para procesos periódicos conocidos aunque no aparezcan en la ventana
4. registrar los errores de permiso en un panel: son la señal de que se recortó de más, y aparecen en minutos

El punto 4 es el que hace segura la operación. Un `PERMISSIONDENIED` es visible, inmediato y reversible; un permiso de más es invisible durante años. La asimetría favorece recortar, y esa es la razón por la que conviene hacerlo con datos y no esperar a tener certeza.

Y para diagnosticar un acceso denegado concreto, el orden que acota el problema, con la misma estructura de las clases 038 y 026:

```
# 1. ¿por qué se deniega ESTA llamada a ESTE recurso para ESTA identidad?
$ gcloud policy-intelligence troubleshoot-policy iam \
  --principal-email sa-tienda-web@cls-tienda-prod-euw1-01.iam.gserviceaccount.com \
  --resource-name //storage.googleapis.com/projects/_/buckets/cls-tienda-facturas \
  --permission storage.objects.get
```

La respuesta indica si falta la concesión, si hay una política de denegación que gana o si una condición no se cumple. Y si nada de eso explica el fallo, el error no es de IAM: es de una política de organización, y el mensaje lo dirá con otro texto. Tres plataformas, tres vocabularios y la misma pregunta: ¿en qué sistema está la regla que me impide esto?

Ejemplo trabajado

CloudShop configura la identidad de su plataforma en Google Cloud. Llega con el contrato de las clases 026 y 038 y lo aplica en un día. Después aparecen cuatro cosas que ese contrato no cubría, y tres de ellas son la misma pieza vista desde ángulos distintos: la cuenta de servicio como recurso.

Hallazgo 1 — todas las máquinas ejecutan con Editor.

La revisión inicial de permisos sobre el proyecto de tienda:

```
$ gcloud projects get-iam-policy cls-tienda-prod-euw1-01 \
  --flatten="bindings[].members" --filter="bindings.role:roles/editor" \
  --format="value(bindings.members)"
```

serviceAccount:418293047512-compute@developer.gserviceaccount.com

Seis máquinas y dos grupos de instancias usaban esa cuenta por omisión. Cualquier dependencia comprometida en cualquiera de ellas podía leer todos los secretos del proyecto, modificar la base de datos y crear recursos.

cuentas con roles/editor	2	0
cuentas de servicio dedicadas	0	7
roles por carga de trabajo	editor	3,1 roles de media
concesiones automáticas por defecto	activas	desactivadas por política
ámbito de acceso de las máquinas	heredado	cloud-platform + IAM

Hallazgo 2 — Ana era «Viewer» y podía desplegar en producción.

Una auditoría de accesos cruzada con el analizador de políticas:

```
$ gcloud asset analyze-iam-policy --organization=$ORG_ID \
  --analyze-service-account-impersonation \
  --identity="user:ana@cloudshop.example" --format="value(...)"
ana@cloudshop.example → despliegue@cls-tienda-prod (tokenCreator)
                        → despliegue tiene roles/editor
```

El rol propio de Ana era roles/viewer. Su permiso efectivo era Editor sobre producción, por un camino que ninguna tabla de roles de usuario mostraba. Y no era un caso aislado:

personas con acceso efectivo a producción	11	3
caminos de suplantación no previstos	7	0
concesiones de tokenCreator	11	3, con caducidad de 8 h
revisión de permisos	tabla de roles	analizador de políticas

La corrección de fondo no fue quitar permisos: fue cambiar qué se revisa. Una revisión que mira solo los roles directos no puede encontrar esto.

Hallazgo 3 — catorce claves, una en un repositorio público.

```
$ gcloud asset search-all-resources --scope organizations/$ORG_ID \
  --asset-types iam.googleapis.com/ServiceAccountKey \
  --query "NOT name:*/keys/system-managed*" --format="value(name)" | wc -l
14
```

Una de ellas apareció en un repositorio público de un antiguo becario, con la cuenta informes@, que tenía roles/bigquery.dataViewer sobre el conjunto de datos de ventas. Estuvo accesible cuatro meses.

La migración, por casos de uso y en este orden:

canalizaciones de CI	5 claves	federación con condición de atributo
cargas en Compute y Cloud Run	6 claves	cuenta de servicio adjunta
herramientas de personas	3 claves	suplantación con el propio acceso
claves restantes	14	0
creación de claves nuevas	permitida	bloqueada por política

Y la prueba negativa de la federación, la tercera vez que se ejecuta la misma prueba en este programa:

desde otro repositorio	permiso denegado	✓
desde cloudshop/tienda, rama de trabajo	permiso denegado	✓
desde cloudshop/tienda, rama main	testigo emitido	✓

Hallazgo 4 — el recomendador dice que sobra el 96 % de los permisos.

Tras 90 días de operación con las cuentas dedicadas ya creadas:

identidad	permisos concedidos	usados en 90 días
sa-tienda-web	412	7
sa-procesador	412	11
sa-informes	189	4

Las recomendaciones se aplican por pasos, empezando por las cuentas de carga de trabajo, y con el panel de errores de permiso vigilando:

permisos concedidos (suma)	1.013	34
errores PERMISSION_DENIED		
en las 2 semanas siguientes	—	3
de los cuales, recortes de más	—	2, restaurados en 20 min
el tercero	—	un proceso trimestral no visible en la ventana de 90 días

El tercero es el interesante y confirma el límite que había que declarar: un cierre trimestral no aparece en una ventana de noventa días. Se restauró su permiso con una condición que lo limita a los cinco primeros días de cada trimestre.

Resumen del modelo de identidad:

cuentas con rol básico	2	0
claves de cuenta de servicio	14	0
personas con acceso efectivo a producción	11	3
caminos de suplantación no previstos	7	0
permisos concedidos (suma de las tres cuentas)	1.013	34
concesiones con caducidad	0	6
prueba negativa de la federación	no	sí, tres casos

La lección que esta clase traslada al resto de la parte 04: el contrato de identidad de las partes 02 y 03 se reutilizó entero —sin secretos, federación acotada por sujeto, privilegio mínimo, prueba negativa— y aun así hubo cuatro hallazgos, porque la cuenta de servicio es un recurso además de una identidad y eso duplica las preguntas de permiso. Una auditoría que mire solo quién tiene qué rol es correcta y está incompleta: en Google Cloud hay que preguntar además a quién puede suplantar cada quien, y responderlo requiere una herramienta, no una hoja de cálculo.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/050-iam-service-accounts-y-workload-identity-federation/lab.py
```

El laboratorio selecciona el motor de práctica iam y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa identidad-gcp en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de acceso mínimo con prueba de denegación. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye identidad-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Una persona con rol de solo lectura despliega en producción	Tiene permiso para suplantar o adjuntar una cuenta de servicio privilegiada	Audita con el analizador de políticas incluyendo suplantación, y limita serviceAccountTokenCreator y serviceAccountUser con concesiones caducas.
Una máquina virtual puede modificar casi todo el proyecto	Usa la cuenta de servicio por defecto, que recibe roles/editor automáticamente	Desactiva las concesiones automáticas por política y crea una cuenta dedicada por carga con roles mínimos.
La cuenta tiene el rol correcto y la llamada sigue fallando en una máquina antigua	Los ámbitos de acceso de Compute Engine intersecan con IAM y no incluyen esa API	Fija el ámbito en cloud-platform y controla exclusivamente con roles de IAM.
Una credencial filtrada sigue siendo válida meses después	Es una clave de cuenta de servicio, que no caduca	Sustituye por identidad adjunta, suplantación o federación, y bloquea la creación de claves con una política de organización.
Un repositorio ajeno obtiene credenciales de la organización	El proveedor de identidad federada no tiene condición de atributo	Acota con --attribute-condition sobre repositorio y rama, y verifica con las tres pruebas negativas.
Aplicar las recomendaciones de privilegio rompe un proceso trimestral	El recomendador observa 90 días y un proceso trimestral puede no aparecer	Conserva los permisos de procesos periódicos conocidos, vigila los errores de permiso y restaura con una concesión condicionada a su ventana.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Cuáles son las dos caras de una cuenta de servicio y qué pregunta se salta una auditoría que solo mira los roles de las personas?
2. ¿Por qué serviceAccountUser sobre una cuenta con Editor equivale a tener Editor?
3. ¿Qué se concede automáticamente al habilitar Compute Engine y cómo se impide?
4. Enumera las cuatro alternativas a una clave de cuenta de servicio y en qué caso corresponde cada una.
5. ¿En qué orden se evalúan las políticas de denegación, las concesiones y las políticas de organización?

Referencias

- Google Cloud (2025). Service accounts overview — identidad y recurso, suplantación y adjunción.
<<https://cloud.google.com/iam/docs/service-account-overview>>
- Google Cloud (2025). Best practices for using service accounts — claves, alternativas y cuentas por defecto.
<<https://cloud.google.com/iam/docs/best-practices-service-accounts>>
- Google Cloud (2025). Workload Identity Federation — grupos, proveedores, mapeo y condición de atributo.
<<https://cloud.google.com/iam/docs/workload-identity-federation>>
- Google Cloud (2025). IAM deny policies — reglas de denegación y orden de evaluación.
<<https://cloud.google.com/iam/docs/deny-overview>>
- Google Cloud (2025). Role recommendations — reducción de privilegios a partir del uso observado y sus límites.
<<https://cloud.google.com/policy-intelligence/docs/role-recommendations-overview>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Anterior	Índice	Siguiente
← 049 · Organización, folders, proyectos, billing y cuotas	Parte 04 · Programa	051 · VPC global, subredes regionales, firewall y Cloud NAT →

Evaluación — 050 IAM, service accounts y Workload Identity Federation

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega identidad-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

051 — VPC global, subredes regionales, firewall y Cloud NAT

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

Propósito

Diseñar la red de Google Cloud partiendo del hecho que reordena todo lo aprendido en las clases 027 y 039: la VPC es global. Una sola red cubre todas las regiones, las subredes son regionales y dos máquinas en continentes distintos se hablan por dirección interna sin ningún emparejamiento. Eso hace desaparecer el diseño de concentrador y radios para el caso interno, y desplaza el trabajo a otros sitios: quién puede usar qué subred, cómo se etiqueta el destino de una regla de firewall, y por qué una máquina sin dirección externa aquí sí está aislada de verdad.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar qué implica que la VPC sea global y qué diseño de las partes anteriores deja de hacer falta.
2. Elegir entre VPC compartida, emparejamiento y red por proyecto según quién debe gobernar el firewall.
3. Escribir reglas de firewall dirigidas por cuenta de servicio en vez de por etiqueta, y justificar por qué.
4. Habilitar el acceso privado a las API de Google y distinguir su fallo del fallo silencioso de otras plataformas.
5. Dimensionar la asignación de puertos de Cloud NAT antes de que un servicio locuaz la agote.

Conceptos centrales

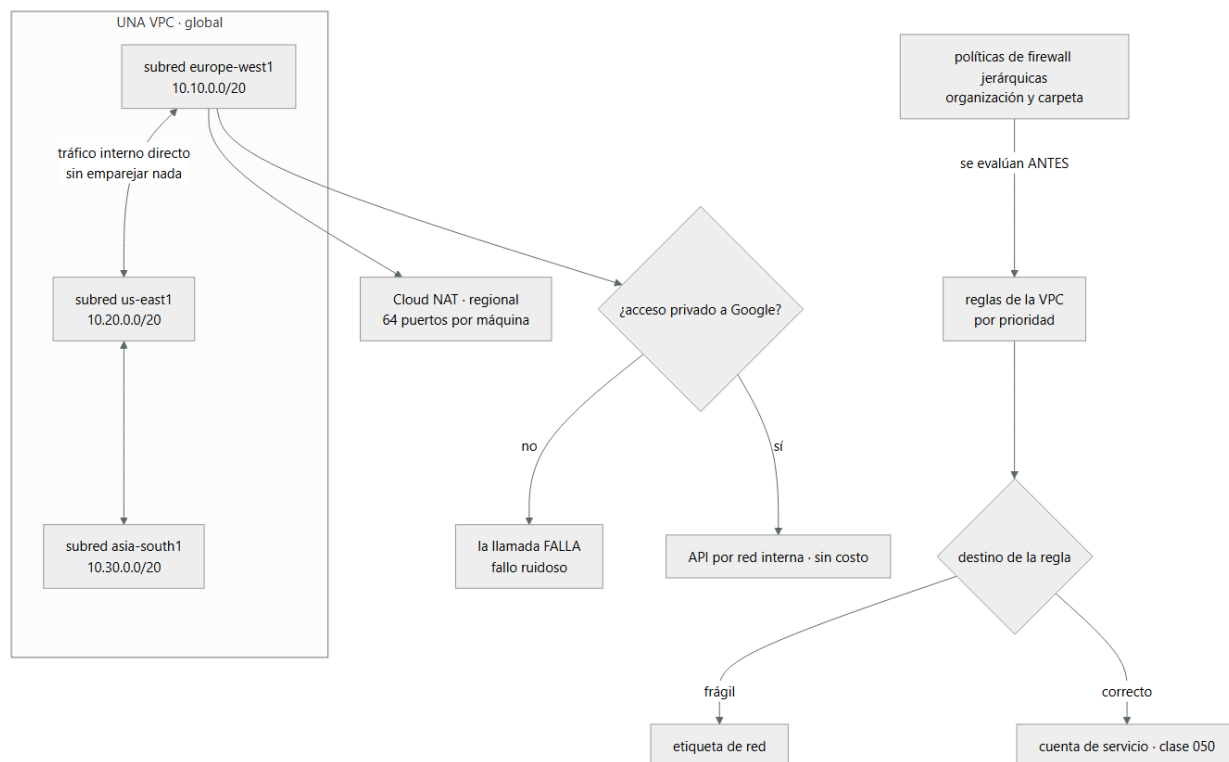
Concepto	Comprensión verificable
VPC global	Red que abarca todas las regiones. Las subredes son regionales y el enrutamiento entre ellas es interno y automático: no hay emparejamiento ni pasarela entre regiones de la misma VPC.
modo automático frente a personalizado	La red por defecto crea una subred en cada región con rangos predefinidos que coinciden con los de todo el mundo. Eso impide emparejar más adelante. La única opción defendible es el modo personalizado.
destino de regla de firewall	A qué máquinas aplica una regla: a todas, a las que tengan una etiqueta de red o a las que ejecuten con una cuenta de servicio. La etiqueta la puede poner cualquiera que edite la máquina; la cuenta de servicio, no.
VPC compartida	Un proyecto anfitrión posee la red y otros proyectos despliegan dentro de ella. No es un emparejamiento: es la misma red, con IAM decidiendo quién puede usar cada subred.
acceso privado a Google	Ajuste por subred que permite a una máquina sin dirección externa alcanzar las API de Google por la red interna. Sin él, la llamada no funciona — y eso es una buena noticia.
asignación de puertos de Cloud NAT	Puertos de traducción reservados por máquina, 64 por defecto. Es el mismo agotamiento de las clases 039 y 043, con un tercer mecanismo y la misma señal.

Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. La VPC es global, y eso borra un diseño entero

En AWS una VPC vive en una región y sus subredes en zonas. En Azure, una red virtual vive en una región y conectar dos exige emparejarlas. En Google Cloud:

la VPC es GLOBAL: abarca todas las regiones
la subred es REGIONAL: no zonal

Dos consecuencias inmediatas. La primera: dos máquinas en regiones distintas de la misma VPC se hablan por dirección interna sin configurar nada. No hay emparejamiento, no hay pasarela, no hay tabla de rutas que tocar. La segunda: una subred cubre todas las zonas de su región, así que la alta disponibilidad zonal no obliga a repartir direcciones entre subredes.

El efecto práctico sobre lo construido en la clase 039 es que un diseño entero deja de hacer falta:

Azure concentrador y radios, emparejamientos bidireccionales,
 allowForwardedTraffic, UDR por radio, tránsito facturado dos veces
Google una VPC, subredes por región, y ya está

Eso no significa que no haya nada que diseñar: significa que el trabajo se desplaza. Lo que en Azure se resolvía con topología, aquí se resuelve con IAM sobre la red, y esa es la parte que sorprende a quien llega esperando dibujar cajas.

El segundo hecho estructural es la red por defecto. Todo proyecto nuevo recibe una VPC en modo automático: una subred en cada región, con rangos predefinidos dentro de 10.128.0.0/9. Son los mismos rangos para todos los clientes de Google Cloud, así que:

dos proyectos con red automática NO se pueden emparejar: sus rangos coinciden
tampoco se puede emparejar con la red de un socio que usara la automática
y descubrirlo es tarde: hay que recrear la red y mover todo lo que hay dentro

La medida se toma antes de crear el primer proyecto, y es una política de organización de la clase 049:

```
$ gcloud resource-manager org-policies enable-enforce \
  constraints/compute.skipDefaultNetworkCreation --organization $ORG_ID
```

```
$ gcloud compute networks create vpc-cloudshop --subnet-mode custom
$ gcloud compute networks subnets create snet-tienda-euw1 \
  --network vpc-cloudshop --region europe-west1 --range 10.10.0.0/20 \
  --enable-private-ip-google-access --enable-flow-logs
```

Y el reparto de direcciones se planifica con la misma disciplina de las clases 027 y 039, con dos particularidades: Google reserva cuatro direcciones por subred en lugar de cinco, y una subred se puede ampliar en caliente —cambiar su máscara a una mayor sin recrearla—, lo que no significa que se pueda improvisar, porque reducirla sigue sin ser posible.

```
vpc-cloudshop (global)
  snet-tienda-euw1      10.10.0.0/20    europe-west1
  snet-datos-euw1       10.10.16.0/20   europe-west1
  snet-tienda-use1      10.20.0.0/20    us-east1
  rango secundario para contenedores 10.60.0.0/14    (parte 06)
```

La última línea conviene reservarla ya: los clústeres de Kubernetes en Google Cloud consumen rangos secundarios de la subred para pods y servicios, y son grandes. Descubrirlo en la parte 06 con el espacio ya repartido obliga a rehacer lo de arriba.

2. El firewall es de la red, y el destino correcto es una identidad

Las reglas de firewall en Google Cloud no cuelgan de la subred ni de la interfaz: cuelgan de la VPC, y se aplican a un conjunto de máquinas definido por el propio destino de la regla. Son con estado, tienen prioridad —número menor gana— y se evalúan hasta la primera coincidencia.

```
reglas implícitas, que no se pueden borrar
  entrada denegar todo          prioridad 65535
  salida  permitir todo         prioridad 65535
```

Comparado con las otras dos plataformas, esto ya es una diferencia notable: la entrada está denegada por defecto entre máquinas de la misma red, sin la regla AllowVnetInBound que en Azure dejaba hablar a todo con todo (clase 039). La red por defecto en modo automático sí trae una regla que permite el tráfico interno, y es otra razón para no usarla.

La decisión que de verdad importa es a quién aplica la regla, y hay tres formas:

Destino	Cómo se asigna	Quién puede cambiarlo
Todas las instancias de la red	—	—
Etiqueta de red	Cadena en la máquina	Cualquiera que pueda editar la máquina
Cuenta de servicio	La identidad con la que ejecuta	Solo quien tenga serviceAccountUser sobre ella

La fila del medio es la trampa. Una etiqueta de red es texto libre: quien tenga permiso para modificar una instancia puede añadirle permitir-ssh y quedar dentro del ámbito de la regla que abre el puerto 22. No hace falta tocar el firewall para abrir un puerto — basta con etiquetarse.

Dirigir por cuenta de servicio cierra ese camino y encadena con la clase 050: cambiar la identidad de una máquina exige permiso sobre esa cuenta de servicio, que es un permiso auditado y restringido.

```
$ gcloud compute firewall-rules create permitir-app-a-datos \
  --network vpc-cloudshop --direction INGRESS --priority 1000 --action ALLOW \
  --rules tcp:5432 \
  --source-service-accounts sa-tienda-web@cls-tienda-prod-euw1-01.iam.gserviceaccount.com \
  --target-service-accounts sa-datos@cls-datos-prod-euw1-01.iam.gserviceaccount.com
```

Esa regla dice exactamente lo que se quiere decir: el servicio web puede hablar con la base de datos por el 5432, sin mencionar direcciones ni depender de que nadie ponga la etiqueta correcta. Y sobrevive a que las máquinas cambien de dirección, de zona o de región.

Una limitación que hay que conocer: una regla no puede mezclar etiquetas y cuentas de servicio como origen y destino. La migración es por bloques, no gradual dentro de la misma regla.

Por encima de las reglas de la VPC están las políticas de firewall jerárquicas, definidas en la organización o en la carpeta y evaluadas antes:

```
orden de evaluación
  1. política jerárquica de la organización
```

2. política jerárquica de la carpeta
 3. reglas de la VPC
- cada nivel puede permitir, denegar o ceder al siguiente con `goto_next`

Es el mismo patrón de gobierno de las clases 046 y 049 aplicado a la red: una regla que ningún equipo puede saltarse, como denegar el 22 y el 3389 desde internet en toda la organización. Y con la misma advertencia de siempre: no cierra lo que ya está abierto por otra vía, solo impide que las reglas de abajo lo permitan.

Y el registro de reglas de firewall, que conviene activar en las que importan aunque tenga costo, porque responde la pregunta que aparece en todo incidente de conectividad —qué regla decidió— sin tener que deducirla:

```
$ gcloud compute firewall-rules update permitir-app-a-datos --enable-logging
```

3. VPC compartida: la red la gobierna quien debe, no quien despliega

Con una VPC global y proyectos baratos (clase 049) aparece una pregunta que en las otras plataformas no se planteaba así: si cada equipo tiene sus propios proyectos, ¿cada uno tiene su propia red?

Las tres respuestas posibles y cuándo vale cada una:

	Cómo funciona	Cuándo
Una red por proyecto	Cada proyecto con su VPC	Aislamiento fuerte, sin tráfico interno entre equipos
Emparejamiento de VPC	Redes distintas conectadas	Dos organizaciones, o un socio
VPC compartida	Un proyecto anfitrión posee la red; los demás despliegan en ella	El caso normal dentro de una organización

La VPC compartida no es un emparejamiento: es la misma red, usada desde varios proyectos. El proyecto anfitrión posee las subredes, las reglas de firewall y las rutas; los proyectos de servicio crean máquinas, balanceadores y clústeres dentro de esas subredes. Y el reparto se hace con IAM:

```
$ gcloud compute shared-vpc enable cls-red-prod
$ gcloud compute shared-vpc associated-projects add cls-tienda-prod-euw1 \
  --host-project cls-red-prod

# el equipo de tienda solo puede usar SU subred, no todas
$ gcloud compute networks subnets add-iam-policy-binding snet-tienda-euw1 \
  --region europe-west1 --project cls-red-prod \
  --member "group:equipo-tienda@cloudshop.example" \
  --role roles/compute.networkUser
```

Esa última concesión es la pieza elegante del modelo: el permiso de red se concede por subred, así que un equipo puede desplegar en la suya y no en la de datos, sin que nadie tenga que dibujar una topología para conseguirlo. Lo que en Azure se resolvía separando redes y emparejándolas, aquí se resuelve con una concesión.

La contrapartida hay que aceptarla porque define el modelo operativo: las reglas de firewall las gestiona el proyecto anfitrión. Un equipo que necesita abrir un puerto no puede hacerlo solo. Eso es exactamente lo que se quiere en una organización con requisitos de seguridad, y es un cuello de botella si el equipo de red tarda días en atender una petición. La respuesta razonable no es repartir el permiso, sino automatizar la petición: las reglas se declaran en código en el repositorio del proyecto anfitrión y se aprueban por revisión, que es el flujo de la clase 059.

El emparejamiento de VPC queda entonces para lo que de verdad lo necesita —otra organización, un proveedor— y trae las restricciones conocidas:

no es transitivo	$A \leftrightarrow B \leftrightarrow C$ no da $A \leftrightarrow C$
rangos sin solapar	por eso importaba no usar la red automática
cuota de rutas	las rutas aprendidas cuentan contra un límite
firewall independiente	cada lado decide qué acepta

Y para el caso de muchas redes que sí necesitan hablarse entre sí, existe el centro de conectividad de red, que resuelve la transitividad con un concentrador gestionado — el mismo problema de la clase 039 con una pieza distinta. Conviene saber que existe y no llegar a él por costumbre: con una VPC global bien diseñada, la mayoría de organizaciones no lo necesitan.

4. Salir a internet y llegar a Google: dos caminos distintos

Aquí está la asimetría propia de esta plataforma, y va en la dirección contraria a la de Azure.

Una máquina sin dirección IP externa no alcanza internet. La ruta por defecto hacia la pasarela de internet existe en la tabla, y sin dirección externa y sin NAT el paquete no llega a ningún sitio. Comparado con lo aprendido en la clase 039:

Azure	existía una traducción implícita: la "subred privada" salía igual
Google	no hay traducción implícita: sin dirección externa y sin NAT, no sale

Es decir: el aislamiento es el estado por defecto, y es la buena noticia de esta clase. Lo que hay que declarar aquí es lo contrario que allí — no cómo cerrar la salida, sino cómo abrirla cuando hace falta.

Cloud NAT es esa salida, y es regional y sin instancias que gestionar:

```
$ gcloud compute routers create rt-euw1 --network vpc-cloudshop --region europe-west1
$ gcloud compute routers nats create nat-euw1 --router rt-euw1 --region europe-west1 \
  --nat-all-subnet-ip-ranges --auto-allocate-nat-external-ips \
  --enable-dynamic-port-allocation --min-ports-per-vm 64 --max-ports-per-vm 2048 \
  --enable-logging --log-filter ERRORS_ONLY
```

Dos parámetros de esa orden merecen atención. El primero es el precio, que tiene la misma forma que en las otras dos plataformas y por tanto la misma lección:

~0,044 USD/h por la pasarela + ~0,045 USD por GB procesado

El segundo es la asignación de puertos: por defecto, 64 por máquina. Un servicio que abre muchas conexiones cortas al mismo destino los agota, y el síntoma es el mismo de las clases 039 y 043 —tiempos de espera intermitentes hacia un destino concreto mientras el resto funciona—. Es la tercera vez que aparece este problema en el programa, con un tercer mecanismo:

AWS	puertos del NAT gateway, gestionados por el servicio
Azure	64.000 repartidos entre instancias, o NAT Gateway bajo demanda
Google	64 por máquina por defecto, ampliables y con asignación dinámica

La asignación dinámica es la respuesta correcta: reparte entre un mínimo y un máximo según la necesidad, en vez de reservar por adelantado. Y el registro con ERRORSONLY da la señal exacta —OUTOFRESOURCES— en vez de dejar deducirlo desde la aplicación.

Llegar a las API de Google es otro camino y no pasa por el NAT. Cloud Storage, BigQuery, Secret Manager y el resto viven en direcciones públicas, así que una máquina sin dirección externa no las alcanza — salvo que la subred tenga habilitado el acceso privado a Google:

```
$ gcloud compute networks subnets update snet-datos-euw1 --region europe-west1 \
  --enable-private-ip-google-access
```

Es un ajuste de subred, gratuito, y evita pagar el NAT por ese tráfico, que suele ser la mayor parte. La aritmética es la misma de la clase 027 y vuelve a salir a cuenta:

2,4 TB/mes hacia Cloud Storage por Cloud NAT	$2.400 \times 0,045 = 108$ USD/mes
los mismos 2,4 TB con acceso privado a Google	0 USD/mes

Y lo más valioso para quien viene de la clase 039: cuando falta, falla. Sin acceso privado a Google, la llamada agota su tiempo de espera y aparece en el registro. No hay un camino público alternativo por el que funcione mientras el diagrama afirma que es privado. Comparar los dos fallos es la lección portable:

Azure sin zona DNS privada	funciona por el camino público	→ fallo SILENCIOSO
Google sin acceso privado	no funciona	→ fallo RUIDOSO

Un fallo ruidoso se arregla en diez minutos. Uno silencioso dura cuatro meses.

Para el resto de casos —una API concreta con una dirección interna elegida, o un servicio de un tercero— está Private Service Connect, que crea un extremo con IP interna propia y su entrada de DNS. Es el análogo del punto de conexión privado de Azure y del endpoint de interfaz de AWS, y trae de vuelta la parte de DNS: si el nombre no resuelve al extremo, el tráfico busca el camino de siempre. La comprobación es la misma que se hizo en la clase 039, y hay que hacerla igual.

5. Diagnóstico y evidencia de una red que no se ve

Con la topología reducida a una VPC, el diagnóstico se apoya menos en el diagrama y más en tres herramientas que conviene conocer antes del incidente.

Pruebas de conectividad, que simulan un paquete sin enviarlo y dicen qué lo permitiría o lo bloquearía:

```
$ gcloud network-management connectivity-tests create prueba-app-datos \
  --source-instance web-01 --destination-instance db-01 \
  --destination-port 5432 --protocol TCP
$ gcloud network-management connectivity-tests describe prueba-app-datos \
  --format="value(reachabilityDetails.result)"
REACHABLE
```

Es la respuesta a «¿está abierto?» sin tener que abrir una sesión en ninguna máquina, y funciona igual antes de desplegar que durante una caída. Que además se puedan guardar y reejecutar la convierte en el equivalente de red del guion de pruebas negativas de la clase 046:

desde la subred de tienda a la base de datos por 5432	REACHABLE	✓
desde la subred de tienda a la base de datos por 22	UNREACHABLE	✓
desde internet a cualquier máquina por 22	UNREACHABLE	✓

Registros de flujo de VPC, activados por subred, con muestreo configurable. Responden a quién habló con quién y cuánto, que es la pregunta de la investigación posterior. Su costo depende del muestreo, así que la decisión es la misma de la clase 045: se activan al 100 % donde hay datos sensibles y con muestreo bajo donde el volumen es alto.

Espejo de paquetes, para cuando hace falta el contenido y no el resumen. Es caro y es la única forma de responder ciertas preguntas; conviene tenerlo previsto y apagado.

Y el orden de diagnóstico, que con tres plataformas ya se puede escribir como método y no como receta de proveedor:

1. ¿existe camino? prueba de conectividad · ruta y firewall
2. ¿resuelve el nombre? la respuesta debe ser una dirección interna
3. ¿tiene permiso? IAM, y si no, política de organización (049, 050)
4. ¿lo permite el destino? firewall del otro lado, o su propia configuración

Los cuatro pasos son idénticos en AWS, Azure y Google Cloud. Lo único que cambia es el nombre del comando y el texto del error — y esa es exactamente la afirmación que la clase 048 dejó por comprobar.

Ejemplo trabajado

CloudShop lleva a Google Cloud la red que diseñó en la clase 039. El primer día descubre que la mitad del diseño sobra, y las tres semanas siguientes descubre dónde estaba el trabajo de verdad.

Lo que desapareció el primer día.

El diseño de Azure tenía un concentrador, tres radios, seis emparejamientos y una tabla de rutas por radio. Al trasladarlo:

redes virtuales / VPC	4	1
emparejamientos	6	0
tablas de rutas definidas por el usuario	3	0
costo mensual de tránsito entre radios	50,40 USD	0 USD
latencia entre servicios de regiones distintas	vía concentrador	directa

Los 50,40 USD al mes de la clase 039 —de los cuales 32,40 eran el precio de inspeccionar— desaparecen para el tráfico interno. Lo que no desaparece es la necesidad de inspeccionar: se traslada a las reglas de firewall y a las políticas jerárquicas, que no cuestan por GB.

Incidente 1 — no se puede emparejar con la red del socio logístico.

```
$ gcloud compute networks peerings create socio --network default \
  --peer-project socio-logistico --peer-network default
ERROR: IP range 10.128.0.0/9 overlaps with peer network range
```

Ambas partes usaban la red por defecto en modo automático, con los mismos rangos. No hay ajuste que lo resuelva: hay que recrear la red y mover lo que hay dentro.

modo de la red	automático	personalizado
rangos	10.128.0.0/9	10.10.0.0/16 planificado
redes por defecto en proyectos nuevos	se crean	bloqueadas por política
máquinas que hubo que recrear	9	—
tiempo de la migración	2 días	—

Dos días de trabajo por una decisión que costaba un minuto antes de empezar.

Incidente 2 — un puerto 22 abierto sin tocar el firewall.

Una revisión encuentra una máquina de producción accesible por SSH desde una subred de desarrollo.

```
$ gcloud compute instances describe api-03 --zone europe-west1-b \
  --format="value(tags.items)"
permitir-ssh-dev
```

Nadie modificó ninguna regla: alguien añadió una etiqueta a la máquina y quedó dentro del ámbito de una regla existente. El permiso necesario para hacerlo era el de editar instancias, que tenían nueve personas.

reglas dirigidas por etiqueta de red	14	0
reglas dirigidas por cuenta de servicio	0	14
quién puede cambiar el ámbito de una regla	9 personas	solo con permiso sobre la cuenta (050)
política jerárquica que niega 22 y 3389 desde internet	ninguna	en la organización

Y la prueba negativa correspondiente:

prueba de conectividad: internet → api-03 : 22	UNREACHABLE	✓
prueba de conectividad: dev → api-03 : 22	UNREACHABLE	✓
prueba de conectividad: tienda → datos : 5432	REACHABLE	✓

Incidente 3 — el procesador no puede leer de Cloud Storage, y se arregla en once minutos.

Al quitar las direcciones externas de las máquinas de proceso, el trabajo nocturno falla:

```
google.api_core.exceptions.RetryError: Deadline of 60.0s exceeded
while calling storage.googleapis.com

$ gcloud compute networks subnets describe snet-datos-euw1 --region europe-west1 \
  --format="value(privateIpGoogleAccess)"
False
$ gcloud compute networks subnets update snet-datos-euw1 --region europe-west1 \
  --enable-private-ip-google-access
```

Once minutos desde el fallo hasta la corrección, porque el fallo fue ruidoso. El equipo anota la comparación con lo ocurrido en Azure, donde el mismo error conceptual —el camino privado no configurado— tardó cuatro meses en detectarse porque todo seguía funcionando por el camino público.

Y el efecto en la factura:

tráfico hacia API de Google por NAT	2,4 TB/mes	0
costo de ese tráfico	108 USD/mes	0
tráfico restante por NAT	0,8 TB/mes	0,8 TB/mes
costo total de Cloud NAT	176 USD/mes	68 USD/mes

Incidente 4 — tiempos de espera hacia la pasarela de pago en los picos.

El mismo síntoma de la clase 043, con otro mecanismo debajo:

```
$ gcloud logging read 'resource.type="nat_gateway" AND jsonPayload.allocation_status="DROPPED"' \
  --limit 5 --format="value(jsonPayload.reason)"
OUT_OF_RESOURCES
```

Sesenta y cuatro puertos por máquina, y un cliente HTTP que abría conexión por petición.

asignación de puertos	64 fijos	64-2048 dinámica
cliente HTTP	uno por petición	uno por proceso
descartes por falta de recursos	1.412/hora	0
fallos hacia la pasarela	2,7 %	0,0 %

Dos correcciones a la vez, y la segunda es la que de verdad importa: el mismo defecto de código ha producido el mismo incidente en tres plataformas distintas, con tres mecanismos de traducción diferentes. Es la definición operativa de un problema portable.

Incidente 5 — cuatro redes en la sombra.

Con proyectos baratos, tres equipos habían creado sus propias VPC para no depender del equipo de red:

```
$ gcloud asset search-all-resources --scope organizations/$ORG_ID \
  --asset-types compute.googleapis.com/Network --format="value(name)" | wc -l
5
```

Cinco redes donde debía haber una, sin conectividad entre ellas y con reglas de firewall que nadie revisaba. Se migra a VPC compartida:

VPC en la organización	5	1
------------------------	---	---

quién gestiona las reglas de firewall	cada equipo	proyecto anfitrión, por revisión de código
permiso de red	completo por proyecto	por SUBRED, con roles/compute.networkUser
tiempo medio de una petición de regla	—	4 h (revisión + despliegue)
reglas de firewall sin revisar	31	0

La fila del tiempo medio es la que se negoció explícitamente: cuatro horas es aceptable y cuatro días no lo sería. Sin ese compromiso, los equipos vuelven a crear redes propias.

Resumen de la red:

redes	4	1
emparejamientos	6	0
costo de tránsito interno	50,40 USD/mes	0
costo de salida (NAT)	68,85 USD/mes	68 USD/mes
reglas dirigidas por identidad	—	14 de 14
fallo del camino privado	silencioso	ruidoso
controles con prueba de conectividad guardada	0	6

La lección que esta clase traslada al resto de la parte 04: la VPC global elimina un diseño entero y no elimina el trabajo — lo mueve de la topología a la identidad. Y la comparación entre el fallo silencioso de Azure y el ruidoso de Google Cloud es la más valiosa de la clase: entre dos plataformas que fallan, es mejor la que falla ruidosamente, y esa propiedad casi nunca aparece en una tabla comparativa de servicios.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/051-vpc-global-subredes-regionales-firewall-y-cloud-nat/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa vpc-gcp en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye vpc-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
No se puede emparejar con la red de otra organización	Ambas usan la red por defecto en modo automático, con rangos idénticos	Bloquea la creación de la red por defecto con una política de organización y crea siempre VPC en modo personalizado con rangos planificados.
Un puerto queda abierto en producción sin que nadie modifique el firewall	La regla se dirige por etiqueta de red y cualquiera que edite la máquina puede ponérsela	Dirige las reglas por cuenta de servicio y añade una política de firewall jerárquica que niegue los puertos de administración desde internet.
Una máquina sin dirección externa no alcanza Cloud Storage	La subred no tiene habilitado el acceso privado a Google	Habilítalo en la subred: es gratuito y además evita pagar Cloud NAT por ese tráfico.
Tiempos de espera intermitentes hacia un destino externo concreto	Se agotan los 64 puertos por máquina que Cloud NAT asigna por defecto	Activa la asignación dinámica de puertos, revisa el registro con ERRORONLY y reutiliza el cliente HTTP en la aplicación.
Aparecen varias VPC creadas por equipos distintos sin conectividad entre ellas	Los proyectos son baratos y crear una red propia es más rápido que pedir una regla	VPC compartida con permiso por subred, y un compromiso de tiempo de respuesta para las peticiones de firewall declaradas en código.
El espacio de direcciones no da para desplegar un clúster de Kubernetes	No se reservaron rangos secundarios para pods y servicios, que son grandes	Reserva los rangos secundarios al planificar la subred, antes de necesitarlos.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué implica que la VPC sea global y qué parte del diseño de la clase 039 deja de ser necesaria?
2. ¿Por qué dirigir una regla de firewall por cuenta de servicio es más seguro que por etiqueta de red?
3. ¿Qué diferencia hay entre VPC compartida y emparejamiento, y qué gobierna cada una?
4. Una máquina sin IP externa no alcanza una API de Google. ¿Qué falta y por qué este fallo es preferible al equivalente de Azure?
5. ¿Cuál es la tercera versión del agotamiento de puertos de traducción que aparece en el programa y cómo se corrige aquí?

Referencias

- Google Cloud (2025). VPC network overview — alcance global, subredes regionales y rutas.
<<https://cloud.google.com/vpc/docs/vpc>>
- Google Cloud (2025). VPC firewall rules — prioridad, reglas implícitas y destinos por etiqueta o cuenta de servicio.
<<https://cloud.google.com/firewall/docs/firewalls>>
- Google Cloud (2025). Shared VPC overview — proyecto anfitrión, proyectos de servicio y permiso por subred.
<<https://cloud.google.com/vpc/docs/shared-vpc>>
- Google Cloud (2025). Private Google Access — acceso a las API desde máquinas sin dirección externa.
<<https://cloud.google.com/vpc/docs/private-google-access>>
- Google Cloud (2025). Cloud NAT overview — asignación de puertos, asignación dinámica y registro.
<<https://cloud.google.com/nat/docs/overview>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Anterior	Índice	Siguiente
← 050 · IAM, service accounts y Workload Identity Federation	Parte 04 · Programa	052 · Compute Engine, managed instance groups y load balancing →

Evaluación — 051 VPC global, subredes regionales, firewall y Cloud NAT

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega vpc-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

052 — Compute Engine, managed instance groups y load balancing

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: compute · Estado: EXECUTABLECORE

Propósito

Dimensionar cómputo y reparto de carga en Google Cloud, donde tres cosas funcionan al revés de lo aprendido: el descuento por uso sostenido se aplica solo, sin comprar nada; la máquina se puede definir con el número exacto de vCPU y de memoria que hace falta; y el balanceador global tiene una sola dirección IP para todo el planeta, así que la conmutación entre regiones no depende del DNS ni de su TTL. A cambio, reaparece por tercera vez el problema de los dos límites de disco y aparece una forma nueva de tumbar un servicio: la reparación automática.

Resultados de aprendizaje

Al finalizar podrás:

1. Elegir tipo de máquina, incluido uno personalizado, a partir del uso medido en vez del catálogo.
2. Calcular el costo real considerando el descuento automático por uso sostenido antes de comprometer nada.
3. Separar la comprobación de estado del balanceador de la de reparación automática, y explicar qué rompe mezclarlas.
4. Configurar un balanceador global con modo de equilibrio por capacidad y medir su conmutación entre regiones.
5. Decidir entre disco zonal y disco regional a partir del RTO exigido ante la caída de una zona.

Conceptos centrales

Concepto	Comprensión verificable
tipo de máquina personalizado	Definición con el número exacto de vCPU y de memoria, dentro de unas proporciones permitidas. Elimina el redondeo al escalón siguiente, que en las otras plataformas es obligatorio.
descuento por uso sostenido	Reducción automática y sin compromiso que crece con el porcentaje del mes que la máquina está encendida, hasta cerca del 30 %. No hay que comprar nada ni pedir nada.
migración en vivo	Traslado transparente de una máquina en ejecución durante el mantenimiento del anfitrión. Hace innecesario el concepto de dominio de actualización de la clase 040 — salvo en excedente y algunos tipos, que sí terminan.
plantilla de instancia	Definición inmutable de cómo es una instancia. Cambiar algo obliga a crear otra plantilla y ejecutar una actualización progresiva, lo que hace que cada cambio quede versionado.
reparación automática	Sustitución de instancias que fallan su comprobación de estado. Con una comprobación que depende de servicios externos, convierte una caída ajena en una caída propia.
modo de equilibrio por capacidad	Declaración de cuántas peticiones por segundo aguanta cada instancia. El balanceador global desborda a la región siguiente al superarla: así conmuta sin tocar el DNS.
disco persistente regional	Disco replicado de forma síncrona entre dos zonas. Permite reconectar el volumen en la otra zona en minutos, sin restaurar nada.

Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

■ Flujo de razonamiento



La nomenclatura es la más simple de las tres plataformas y no esconde letras con significado:

E2	propósito general económico; en tamaños pequeños comparte núcleo
N2 / N2D	propósito general, Intel y AMD
C3 / C4	cómputo, la generación con mejor rendimiento por núcleo
M3	memoria extrema
A3 / G2	aceleradores
T2D	rendimiento por precio para cargas escalables

Y una capacidad que ninguna de las otras dos ofrece: el tipo personalizado. Si el servicio usa 6 vCPU y 50 GB, no hay que comprar el escalón de 16 vCPU y 64 GB para llegar a la memoria:

```
$ gcloud compute instances create app-01 --zone europe-west1-b \
  --custom-cpu 8 --custom-memory 52GB --custom-vm-type n2
```

Las proporciones tienen límites —hay un mínimo y un máximo de memoria por vCPU—, y dentro de ellos el ajuste es exacto. En una flota mediana, dejar de pagar el redondeo es de los ahorros más grandes y menos laboriosos que existen: no cambia la arquitectura, solo el tamaño.

El segundo hecho cambia por completo la conversación sobre compromisos. En AWS y en Azure, el precio bajo exige comprar por adelantado. Aquí hay un descuento automático por uso sostenido:

porcentaje del mes encendida	descuento aproximado
25 %	~10 %
50 %	~20 %
100 %	~30 %

No se solicita, no se compra y no se puede olvidar. Una máquina encendida todo el mes ya está pagando cerca de un 30 % menos que su precio de lista, sin que nadie haya hecho nada. La consecuencia práctica al comparar costos entre proveedores es que el precio de lista de Google Cloud no es el precio que se paga, y compararlo con el precio de lista de otro proveedor sobreestima el gasto.

Encima de eso están los compromisos de uso, en dos formas:

por recurso	familia y región concretas, 1 o 3 años
	descuento mayor, flexibilidad menor
por gasto	un importe por hora comprometido, aplicable a varias familias
	descuento algo menor, mucha más flexibilidad

El orden correcto de decisión es el mismo que la clase 028 estableció y aquí tiene un escalón más: medir, ajustar el tamaño, dejar que el descuento automático haga su parte, y solo entonces comprometer lo que quede estable. Comprometerse antes de ajustar el tamaño es comprometerse con el desperdicio.

Y sobre el excedente, la tercera medición del mismo dato del programa:

AWS	~120 s de aviso
Azure	~30 s
Google	~30 s

Treinta segundos, otra vez, dan para abortar y no para drenar. El aviso llega por el servicio de metadatos y por una señal del sistema operativo, así que se puede atender desde un script de apagado:

```
$ curl -s -H "Metadata-Flavor: Google" \
  "http://metadata.google.internal/computeMetadata/v1/instance/preempted"
TRUE
```

Y un detalle que evita una sorpresa: las máquinas de excedente no migran en vivo, terminan. La migración en vivo —que en el resto de casos hace que un mantenimiento del anfitrión no reinicie nada, y por eso aquí no existe el concepto de dominio de actualización de la clase 040— no aplica al excedente ni a algunos tipos con acelerador.

2. Discos: los dos límites, por tercera vez, y uno que sí replica

El disco persistente está conectado por red, no al anfitrión, y su rendimiento depende de dos cosas a la vez:

del TAMAÑO del disco	pd-ssd: ~30 IOPS por GB
del NÚMERO DE vCPU	cada tamaño de máquina tiene su tope

Es el mismo patrón de la clase 040 con otra aritmética, y es la tercera vez que aparece en el programa:

AWS	el volumen y la instancia tienen topes separados
Azure	el disco y la máquina tienen topes separados
Google	el disco escala por GB y la máquina por vCPU

La consecuencia es idéntica: el caudal real es el mínimo de los dos, y ampliar el disco no mueve la cifra si el techo era el otro.

pd-ssd de 1.024 GB	~30.000 IOPS de lectura
n2-standard-2 (2 vCPU)	~15.000 IOPS de lectura
→ real: 15.000	
cambiar a n2-standard-8	~25.000 IOPS → real: 25.000

Los tipos disponibles y para qué sirve cada uno:

pd-standard	disco duro; archivos y arranques que no importan
pd-balanced	valor por defecto razonable para casi todo
pd-ssd	latencia baja y IOPS por GB
pd-extreme	IOPS aprovisionadas de forma independiente
Hyperdisk	familia moderna: tamaño, IOPS y caudal se fijan por separado
SSD local	efímero, muy rápido, SE PIERDE al detener la máquina

La última línea repite la lección de la clase 040: es la versión de Google Cloud del disco temporal, con la misma trampa. Sirve para caché, archivos temporales e índices reconstruibles, y no para datos.

Y aquí está la capacidad que no tiene equivalente directo en las otras dos plataformas: el disco persistente regional, replicado de forma síncrona entre dos zonas de la misma región.

```
$ gcloud compute disks create datos-01 --size 500GB --type pd-balanced \
  --region europe-west1 --replica-zones europe-west1-b,europe-west1-c
```

Lo que compra es un RTO distinto ante la caída de una zona:

disco zonal	la zona cae → el disco no está → restaurar desde instantánea RTO: el tiempo de restaurar y arrancar
disco regional	la zona cae → conectar el disco a una máquina de la otra zona RTO: minutos, sin restaurar nada

El precio es aproximadamente el doble por GB y la escritura paga la latencia de confirmar en dos zonas, así que no es la opción por defecto: es la opción para el volumen cuya pérdida de disponibilidad define el RTO del servicio — típicamente el de una base de datos que no está en un servicio gestionado.

Un apunte sobre instantáneas que ahorra trabajo: son incrementales y globales, así que una instantánea tomada en Europa se restaura en Estados Unidos sin copiarla explícitamente. Eso convierte la recuperación entre regiones en una operación sencilla, y hace que la política de instantáneas programadas sea una de las cosas más rentables de configurar el primer día.

3. Grupos gestionados: la plantilla inmutable y la reparación que hace daño

Un grupo de instancias gestionado se define con una plantilla inmutable. No se edita: para cambiar algo se crea otra y se ejecuta una actualización progresiva. Eso obliga a que cada cambio quede versionado, que es exactamente lo que se quiere:

```
$ gcloud compute instance-templates create tpl-tienda-v8 \
  --machine-type n2-standard-4 --image-family cls-tienda --image-project cls-imagenes \
  --service-account sa-tienda-web@cls-tienda-prod-euw1-01.iam.gserviceaccount.com \
  --scopes cloud-platform --no-address

$ gcloud compute instance-groups managed rolling-action start-update mig-tienda \
  --region europe-west1 --version template=tpl-tienda-v8 \
  --max-surge 2 --max-unavailable 0
```

--max-unavailable 0 es la elección por defecto correcta: se crean instancias nuevas antes de retirar las viejas, así que la capacidad nunca baja durante el despliegue. Y el canario está integrado, sin infraestructura adicional:

```
$ gcloud compute instance-groups managed rolling-action start-update mig-tienda \
  --region europe-west1 \
  --version template=tpl-tienda-v7 \
  --canary-version template=tpl-tienda-v8,target-size=10%
```

El grupo debe ser regional, no zonal: reparte automáticamente entre las zonas de la región y sobrevive a la caída de una. Un grupo zonal es un punto único de fallo con nombre de grupo.

Y ahora la parte que produce el incidente más caro de esta clase. Un grupo gestionado usa comprobaciones de estado para dos cosas distintas, y la tentación es usar la misma:

comprobación del BALANCEADOR	decide si la instancia recibe tráfico debe ser exigente: si no puede servir, que salga de rotación
comprobación de REPARACIÓN	decide si la instancia se DESTRUYE y se crea otra

Si la comprobación de reparación consulta las dependencias —base de datos, caché, otro servicio—, entonces una caída ajena hace que todas las instancias fallen a la vez, y el grupo las recrea todas a la vez. El resultado es que un fallo externo de cuarenta segundos se convierte en una caída propia de varios minutos: mientras las instancias nuevas arrancan, no hay ninguna sirviendo, y las nuevas fallan la misma comprobación en cuanto arrancan.

La regla que lo evita:

```
reparación → comprueba si el PROCESO está vivo, y nada más
              /healthz, sin dependencias, con retardo inicial generoso
balanceador → comprueba si la instancia PUEDE SERVIR
              /readyz, con dependencias, umbral corto
```

Es la distinción entre vivacidad y disponibilidad que la clase 029 introdujo, y aquí tiene una consecuencia destructiva si se ignora, porque el mecanismo no retira de rotación: borra la máquina.

```
$ gcloud compute health-checks create http hc-reparacion \
  --request-path /healthz --check-interval 30s --timeout 10s \
  --unhealthy-threshold 5 --healthy-threshold 2
$ gcloud compute instance-groups managed update mig-tienda --region europe-west1 \
  --health-check hc-reparacion --initial-delay 300
```

El retardo inicial de 300 segundos es la otra mitad: sin él, una instancia que tarda cuatro minutos en calentarse se destruye antes de llegar a servir, y el grupo entra en un ciclo de creación y destrucción que no converge nunca.

Y sobre el autoescalado, los cuatro parámetros de la clase 029 siguen valiendo. Lo que añade Google Cloud es un control de reducción que evita el otro extremo:

```
$ gcloud compute instance-groups managed set-autoscaling mig-tienda \
  --region europe-west1 --min-num-replicas 3 --max-num-replicas 30 \
  --target-cpu-utilization 0.6 --cool-down-period 90 \
  --scale-in-control max-scaled-in-replicas=3,time-window=600
```

Esa última línea limita cuántas instancias pueden retirarse en una ventana. Es la respuesta al comportamiento que la clase 029 describía: escalar hacia arriba es barato y escalar hacia abajo demasiado rápido deja al servicio sin margen justo cuando vuelve la carga.

4. Una sola dirección IP para el mundo

El balanceador externo global de aplicación es la pieza donde Google Cloud se separa más de las otras dos plataformas, y conviene entender por qué funciona como funciona.

```
una única dirección IP anycast, anunciada desde todo el borde de Google
el usuario conecta al borde más cercano
el borde decide a qué región enviar
```

Comparado con lo aprendido en la clase 040:

	Ámbito	Conmutación entre regiones
Traffic Manager (Azure)	DNS	La caché del cliente: minutos
Front Door (Azure)	Anycast	Segundos
Balanceador global (Google)	Anycast	Segundos, y por capacidad

La última columna esconde el mecanismo interesante. El reparto entre regiones no es una lista de prioridades: es capacidad declarada.

```
$ gcloud compute backend-services add-backend bs-tienda --global \
  --instance-group mig-tienda --instance-group-region europe-west1 \
  --balancing-mode RATE --max-rate-per-instance 120 --capacity-scaler 1.0
$ gcloud compute backend-services add-backend bs-tienda --global \
  --instance-group mig-tienda-use --instance-group-region us-east1 \
  --balancing-mode RATE --max-rate-per-instance 120 --capacity-scaler 1.0
```

Con eso, el balanceador envía a la región más cercana mientras quepa, y desborda a la siguiente cuando se supera la capacidad declarada o cuando las comprobaciones de estado la marcan como no disponible. No hay DNS de por medio, así que el TTL no interviene y la cola larga de clientes que seguían yendo a la región caída —los nueve minutos de la clase 040— no existe.

Y el capacity-scaler es una palanca operativa que conviene conocer: ponerlo a 0 drena una región sin borrar nada, lo que convierte un mantenimiento regional en una operación de una línea y reversible.

La familia completa, para elegir sin confundirse:

```
global externo de aplicación    HTTP(S), anycast, multirregión, con Cloud Armor
regional externo de aplicación  HTTP(S) en una región; requisitos de residencia
```

interno de aplicación	HTTP(S) dentro de la VPC
pasarela de red externo	capa 4, conserva la IP de origen
interno de paso	capa 4 interno, para dispositivos y bases de datos

Y dos piezas que completan el borde:

Cloud Armor aplica reglas de WAF y protección contra denegación de servicio en el borde, antes de llegar a la región. La disciplina de la clase 035 se traslada entera: primero medir en modo vista previa, luego bloquear. Una regla desplegada directamente en modo bloqueo contra tráfico real es la forma habitual de cortar a clientes legítimos.

```
$ gcloud compute security-policies rules create 1000 --security-policy pol-tienda \
  --expression "evaluatePreconfiguredExpr('sqli-v33-stable')" --action deny-403 \
  --preview
```

Cloud CDN se activa como una propiedad del servicio de backend, no como un producto aparte. Un solo interruptor pone el contenido estático en el borde, y su efecto sobre el costo de salida es inmediato — la partida que la clase 025 identificó como la que más sorprende.

Y la comprobación de estado, otra vez: la del balanceador es la exigente, y debe ejercitar las dependencias. Un / que devuelve 200 con la base de datos caída mantiene en rotación una región entera que no puede servir, y el desbordamiento por capacidad no se activa porque, desde fuera, esa región parece sana.

5. Elegir el tamaño con datos, no con la talla anterior

Con el descuento automático, el tipo personalizado y el recomendador, Google Cloud permite cerrar el bucle de dimensionamiento con datos en las tres decisiones. Merece un método corto porque es lo que se lleva a la clase 060.

```
$ gcloud recommender recommendations list --project cls-tienda-prod-euw1-01 \
  --location europe-west1-b --recommender google.compute.instance.MachineTypeRecommender \
  --format "table(content.overview.resourceName, content.overview.recommendedMachineType)"
```

Las recomendaciones de tamaño se basan en el uso de las últimas semanas, y tienen los mismos dos límites que las de permisos de la clase 050: no ven lo que ocurre una vez al trimestre, y miden lo usado y no lo necesario. La forma responsable de aplicarlas es la misma —empezar por lo predecible, observar, y tener a mano la vuelta atrás— con una ventaja: cambiar el tipo de máquina de un grupo gestionado es crear otra plantilla y hacer una actualización progresiva, que ya está ensayada.

El orden completo, que evita el error de comprometer desperdicio:

1. medir uso real de CPU y memoria durante al menos dos semanas, incluyendo el pico estacional si lo hay
2. ajustar el tamaño, usando tipo personalizado si el escalón no encaja
3. dejar que el descuento por uso sostenido se aplique solo
4. comprometer solo la parte que llevará un año encendida
5. mover a excedente lo que tolere 30 segundos de aviso

Y una comparación que conviene tener escrita para cuando alguien proponga migrar por precio:

máquina encendida todo el mes	100 %	~70 % (uso sostenido)
+ tamaño ajustado con tipo personalizado		~55 %
+ compromiso de un año sobre la base		~40 %
+ excedente para la capacidad de pico		~30 %

Cada línea es acumulativa y ninguna cambia la arquitectura. Es el mismo argumento de la clase 028 —la elección de capacidad es una decisión medible— con una plataforma que regala el primer escalón y permite el segundo con precisión.

Y el cierre que conecta con lo que viene: todo esto es cómputo gestionado por ti. Las clases 055 y 060 plantearán la pregunta que hace obsoleta buena parte de este ajuste — si el servicio puede correr en una plataforma que escala a cero, el trabajo de dimensionar deja de existir. Conviene medir el cómputo bien antes de decidir si merece la pena seguir teniéndolo.

Ejemplo trabajado

CloudShop traslada su capa de cómputo a Google Cloud. La primera medición trae una sorpresa agradable y las cuatro siguientes son problemas — uno de ellos, el mismo que ya había aparecido dos veces en el programa.

Sorpresa inicial — el descuento ya estaba aplicado.

La estimación previa se hizo con precios de lista y salió más cara que Azure. La primera factura real:

6 × n2-standard-4, mes completo	1.128 USD	790 USD (-30 % por uso sostenido)
---------------------------------	-----------	--------------------------------------

Nadie compró nada. El equipo corrige el método de estimación: comparar precios de lista entre proveedores con modelos de descuento distintos produce conclusiones falsas, y hay que comparar el precio efectivo de una carga concreta.

Incidente 1 — el disco de 1 TB que rinde la mitad.

La base de datos autogestionada de informes no pasa de 15.000 IOPS con un disco que debería dar 30.000.

```
$ gcloud compute disks describe datos-informes --zone europe-west1-b \
  --format="value(sizeGb,type)"
1024    pd-ssd
$ gcloud compute instances describe informes-01 --zone europe-west1-b \
  --format="value(machineType.basename())"
n2-standard-2
```

Es la tercera vez que el programa encuentra este problema: en AWS con el volumen y la instancia, en Azure con el disco y el tamaño de máquina, y aquí con los GB del disco y las vCPU de la máquina.

máquina	n2-standard-2 (15.000)	n2-standard-8 (25.000)
disco	pd-ssd 1.024 GB	pd-ssd 1.024 GB
IOPS reales	15.000	25.000
duración del informe nocturno	94 min	41 min

Incidente 2 — 16 vCPU para usar 6.

El servicio de catálogo estaba en n2-standard-16 porque necesitaba memoria, no CPU:

uso medido	CPU 34 % de 16 vCPU	memoria 51 GB de 64
tipo	n2-standard-16	n2-custom-8-53248
vCPU	16	8
memoria	64 GB	52 GB
costo mensual por instancia	~395 USD	~232 USD
instancias	4	4
costo mensual del servicio	1.580 USD	928 USD

Seiscientos cincuenta dólares al mes por no redondear al escalón siguiente. La arquitectura no cambió.

Incidente 3 — una caída de 40 segundos que duró nueve minutos.

La base de datos gestionada conmutó entre zonas —40 segundos—. El servicio estuvo caído nueve minutos.

```
$ gcloud compute operations list --filter="operationType:compute.instances.insert" \
  --format="value(insertTime,targetLink.basename())" | head
10:14:22  tienda-a7f2
10:14:22  tienda-c91e
10:14:23  tienda-b03d
10:14:23  tienda-4ae8
```

Las cuatro instancias se recrearon a la vez. La comprobación de reparación automática consultaba /readyz, que verifica la base de datos: al conmutar esta, las cuatro instancias fallaron a la vez y el grupo las destruyó a la vez. Las nuevas tardaron cuatro minutos en arrancar y volvieron a fallar la misma comprobación.

comprobación de reparación	/readyz (dependencias)	/healthz (proceso vivo)
umbral de fallo	2 fallos	5 fallos
intervalo	10 s	30 s
retardo inicial	0 s	300 s
comprobación del balanceador	/readyz	/readyz ← se queda
duración del episodio equivalente	9 min	41 s

La comprobación del balanceador no cambió: estaba bien. Lo que estaba mal era usar la misma para decidir si destruir una máquina.

Incidente 4 — el simulacro de región, comparado con la clase 040.

Se drena europe-west1 poniendo su escalador de capacidad a cero:

```
$ gcloud compute backend-services update-backend bs-tienda --global \
  --instance-group mig-tienda --instance-group-region europe-west1 \
  --capacity-scaler 0.0
```

mecanismo	Traffic Manager / Front Door	anycast + capacidad
conmutación observada	9 min / 35 s	2,4 s

errores vistos por el cliente	algunos / ninguno tras reintento	ninguno
direcciones IP implicadas	2	1
reversión	cambiar destinos	capacity-scaler 1.0

Dos coma cuatro segundos y una sola dirección IP. Y la operación inversa es la misma línea con otro valor, lo que la convierte en un procedimiento de mantenimiento y no en un plan de contingencia.

Incidente 5 — una zona cae y el disco estaba en ella.

El simulacro de zona deja fuera la máquina que aloja el servicio de búsqueda, con su índice en un disco zonal:

recuperación	restaurar instantánea	conectar en la otra zona
RTO medido	47 min	4 min
pérdida de datos	hasta 6 h	ninguna
costo mensual del volumen	42 USD	84 USD

Cuarenta y dos dólares al mes por bajar el RTO de 47 a 4 minutos, solo para el volumen cuya disponibilidad define el RTO del servicio. El resto de discos se quedan zonales con instantáneas programadas.

Resumen de la capa de cómputo:

tipo de las máquinas de catálogo	n2-standard-16	n2-custom-8-53248
IOPS reales del volumen de informes	15.000	25.000
duración del informe nocturno	94 min	41 min
episodio por conmutación de base de datos	9 min	41 s
conmutación entre regiones	—	2,4 s
RTO ante caída de zona del índice	47 min	4 min
costo mensual de cómputo	2.370 USD	1.718 USD

La lección que esta clase traslada al resto de la parte 04: dos de los cinco problemas eran conocidos —los dos límites de disco y la comprobación de estado que confunde vivacidad con disponibilidad— y aparecieron igual, porque son propiedades del problema y no del proveedor. El único mecanismo genuinamente nuevo, la reparación automática, hizo más daño que ninguno: es la primera pieza del programa que responde a un fallo destruyendo infraestructura sana, y por eso su comprobación tiene que ser la más conservadora de todo el sistema.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/052-compute-engine-managed-instance-groups-y-load-balancing/lab.py
```

El laboratorio selecciona el motor de práctica compute y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa servicio-elastico-gcp en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una selección de capacidad justificada y observable. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye servicio-elastico-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.

- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Se amplía el disco y las IOPS no cambian	El tope efectivo lo pone el número de vCPU de la máquina, no el tamaño del disco	Consulta ambos límites: el caudal real es el mínimo, y se corrige subiendo de tipo de máquina.
Un fallo externo de segundos se convierte en una caída propia de minutos	La comprobación de reparación automática consulta dependencias, así que todas las instancias fallan y se destruyen a la vez	Reparación con <code>/healthz</code> sin dependencias, umbral alto y retardo inicial; balanceador con <code>/readyz</code> .
Un grupo gestionado crea y destruye instancias sin converger	El retardo inicial es menor que el tiempo de arranque del servicio	Fija <code>--initial-delay</code> por encima del arranque real medido, con margen.
La comparación de costos con otro proveedor sale muy desfavorable	Se compararon precios de lista y el descuento por uso sostenido se aplica solo	Compara el precio efectivo de una carga concreta, con descuentos y tamaños ajustados en ambos lados.
Se paga una máquina grande para llegar a la memoria necesaria	Se eligió un tipo predefinido y el escalón siguiente duplica también la CPU	Usa un tipo personalizado con las vCPU y la memoria medidas, dentro de las proporciones permitidas.
Tras la caída de una zona hay que restaurar para recuperar un volumen	El disco era zonal y su disponibilidad definía el RTO del servicio	Usa disco persistente regional solo para ese volumen y deja el resto zonal con instantáneas programadas.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué descuento se aplica sin comprar nada y cómo cambia la forma correcta de comparar costos entre proveedores?
2. ¿Qué dos factores determinan las IOPS de un disco persistente, y en qué se parece a lo visto en las clases 028 y 040?
3. ¿Por qué la comprobación de reparación automática debe ser distinta de la del balanceador, y qué ocurre si son la misma?
4. ¿Cómo conmuta entre regiones un balanceador global sin depender del DNS, y qué papel tiene la capacidad declarada?
5. ¿En qué caso concreto compensa un disco persistente regional y cómo se justifica su costo?

Referencias

- Google Cloud (2025). Machine families resource and comparison guide — familias, tipos personalizados y proporciones. <<https://cloud.google.com/compute/docs/machine-resource>>
- Google Cloud (2025). Sustained use discounts — descuento automático y su interacción con los compromisos. <<https://cloud.google.com/compute/docs/sustained-use-discounts>>

- Google Cloud (2025). Block storage performance — IOPS por GB y límites por número de vCPU. <<https://cloud.google.com/compute/docs/disks/performance>>
- Google Cloud (2025). Autohealing instances in MIGs — comprobación de reparación, retardo inicial y diferencias con la del balanceador. <<https://cloud.google.com/compute/docs/instance-groups/autohealing-instances-in-migs>>
- Google Cloud (2025). External Application Load Balancer overview — anycast, modos de equilibrio y escalador de capacidad. <<https://cloud.google.com/load-balancing/docs/https>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 04 en PDF · Recorrido de Google Cloud en PDF · Manual integral

Anterior	Índice	Siguiente
← 051 · VPC global, subredes regionales, firewall y Cloud NAT	Parte 04 · Programa	053 · Cloud Storage, clases, lifecycle y replicación →

Evaluación — 052 Compute Engine, managed instance groups y load balancing

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega servicio-elastico-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

053 — Cloud Storage, clases, lifecycle y replicación

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: storage · Estado: EXECUTABLECORE

Propósito

Operar el almacenamiento de objetos de Google Cloud, donde dos de las trampas que costaron caro en las clases 030 y 041 sencillamente no existen —la región del par replicado se elige, y el nivel más frío se lee al instante— y aparece una dimensión que ninguna de las dos plataformas anteriores obligaba a considerar: el número de objetos cuesta dinero además del volumen, y una regla de ciclo de vida mal escrita puede gastar más en operaciones de lo que ahorra en almacenamiento.

Resultados de aprendizaje

Al finalizar podrás:

1. Elegir ubicación —región, birregión o multirregión— a partir de residencia de datos y RPO exigido.
2. Calcular el ahorro de un cambio de clase incluyendo el costo por operación y el número de objetos.
3. Configurar acceso uniforme y prevención de acceso público, con su prueba negativa.
4. Emitir URL firmadas sin ninguna clave, mediante suplantación y firma delegada.
5. Verificar la recuperación de objetos y de buckets borrados en vez de suponerla.

Conceptos centrales

Concepto	Comprensión verificable
ubicación del bucket	Región, birregión —dos regiones que eliges tú— o multirregión. Es lo que en la clase 041 no se podía elegir, y aquí sí.
clase de almacenamiento	Propiedad de cada objeto. Las cuatro clases tienen acceso en milisegundos: no hay rehidratación. Lo que sube al enfriar no es el tiempo, es el precio de leer y de operar.
operación de clase A y de clase B	Escrituras y listados frente a lecturas. Su precio sube al enfriar la clase, hasta diez veces, así que mover millones de objetos pequeños puede costar más que guardarlos.
acceso uniforme a nivel de bucket	Modo en el que solo IAM decide el acceso. El modo detallado mantiene además listas de control por objeto, invisibles en la política del bucket.
Autoclass	Cambio de clase automático por objeto según su acceso real, sin cargos por recuperación, a cambio de una tarifa de gestión. Es la respuesta cuando el patrón de acceso no se puede predecir.
eliminación temporal	Retención por defecto de los objetos borrados y del bucket, durante siete días. Cubre de un golpe el hueco que en la clase 041 exigía dos interruptores distintos.

Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. La ubicación se elige, y eso resuelve un problema de la clase 041

Un bucket tiene un nombre global —como el identificador de proyecto de la clase 049, único entre todos los clientes— y una ubicación que se fija al crearlo y no cambia:

	Qué replica	Cuándo
Región	Dentro de una región, entre zonas	Residencia estricta y menor costo
Birregión	Entre dos regiones que eliges	Recuperación regional con control del destino
Multirregión	Dentro de un continente	Lectura desde muchos sitios, sin elegir dónde

La fila del medio merece detenerse, porque responde a un problema concreto que la clase 041 dejó sin solución elegante. Allí, la redundancia geográfica de Azure replicaba a una región emparejada asignada por Microsoft, y cuando ese destino caía fuera de la jurisdicción aprobada había que renunciar a la redundancia geográfica y montar una replicación explícita.

Aquí la birregión se declara:

```
$ gcloud storage buckets create gs://cls-facturas \
  --placement europe-west1,europe-west4 \
  --default-storage-class STANDARD --uniform-bucket-level-access \
  --enable-autoclass
```

Dos regiones elegidas, ambas dentro del espacio aprobado, con una sola URL y una sola API. Y si el RPO importa, la replicación turbo lo convierte en un compromiso medible en vez de en una aspiración:

```
replicación birregional por defecto    asíncrona, sin RPO comprometido
replicación turbo                      objetivo de 15 minutos, con acuerdo
```

Esa segunda línea es el contraste directo con lastSyncTime de la clase 041: allí el RPO era un número que había que medir y que nadie garantizaba; aquí hay un compromiso, a cambio de un precio.

Un detalle de ingeniería que conviene entender antes de elegir multirregión por comodidad: la replicación no es síncrona, así que una escritura confirmada en una región puede no estar en la otra durante un intervalo. La consistencia de una lectura tras escritura sí está garantizada para el objeto —Google Cloud Storage es fuertemente consistente en lectura tras escritura y en listados— y lo que no está garantizado es que la copia de la otra región esté al día en ese instante. Diseñar suponiendo lo contrario produce el mismo tipo de sorpresa que la conmutación de la clase 041.

Y hay una asimetría de costos que decide más de lo que parece: multirregión cuesta más por GB almacenado y, según la operación, más por operación. Para datos que se leen desde una sola región, pagar la multirregión es pagar disponibilidad que nadie usa. El criterio es el de siempre: la ubicación se elige por el requisito de residencia y de recuperación, no por el que suena mejor.

2. Enfriar cuesta operaciones, no tiempo

Aquí está la diferencia que más cambia el diseño respecto a las dos plataformas anteriores. Las cuatro clases se leen en milisegundos:

Clase	Almacenamiento aprox.	Permanencia mínima	Recuperación por GB	¿Hay que rehidratar?
Standard	0,020 USD/GB/mes	—	—	—
Nearline	0,010	30 días	0,01	No

Clase	Almacenamiento aprox.	Permanencia mínima	Recuperación por GB	¿Hay que rehidratar?
Coldline	0,004	90 días	0,02	No
Archive	0,0012	365 días	0,05	No

Comparado con lo aprendido:

Azure archivo hasta 15 h de rehidratación → incompatible con casi cualquier RTO
 AWS Glacier minutos a horas según el nivel
 Google Archive milisegundos, con cargo por lectura

Eso elimina la frontera que la clase 041 estableció —«archivo es para lo que hay que guardar, no para lo que hay que poder leer»— y la sustituye por otra: archivo es para lo que casi nunca se lee, porque leerlo cuesta.

Y aparece la dimensión nueva, la que ninguna de las dos plataformas anteriores obligaba a mirar: el precio por operación sube al enfriar la clase.

Standard	~0,005 USD por 1.000	
Nearline	~0,010	
Coldline	~0,020	
Archive	~0,050	← diez veces la de Standard

Una regla de ciclo de vida que mueve millones de objetos pequeños a Archive ejecuta una operación de clase A por objeto:

4,2 millones de objetos a Archive
 $4.200.000 / 1.000 \times 0,05 = 210$ USD, de una vez

y si esos objetos suman 300 GB:
 ahorro mensual = $300 \times (0,020 - 0,0012) = 5,64$ USD/mes
 → la operación tarda 37 MESES en amortizarse

Y eso sin contar que cada lectura posterior cuesta 0,05 USD por GB. La operación destruye valor, y en las plataformas anteriores el mismo razonamiento no aparecía porque el precio por operación no cambiaba con la clase.

La corrección es una condición de tamaño en la regla, para que el ciclo de vida ignore lo pequeño:

```
{
  "lifecycle": {"rule": [
    {"action": {"type": "SetStorageClass", "storageClass": "NEARLINE"},
      "condition": {"age": 30, "matchesPrefix": ["facturas/"],
        "matchesStorageClass": ["STANDARD"]}},
    {"action": {"type": "SetStorageClass", "storageClass": "ARCHIVE"},
      "condition": {"age": 365, "matchesStorageClass": ["NEARLINE"]}},
    {"action": {"type": "Delete"},
      "condition": {"daysSinceNoncurrentTime": 90, "isLive": false}}
  ]}
}
```

La última regla es la que ya ha aparecido en las clases 030 y 041 y aquí es la cuarta: con versionado activo, borrar no libera nada. Las versiones no actuales siguen facturando hasta que una regla las retire, y la factura no baja tras una limpieza si nadie escribió esa condición.

Y cuando el patrón de acceso no se puede predecir —que es el caso honesto en la mayoría de conjuntos de datos—, está Autoclass:

mueve cada objeto de clase según SU acceso real
 no cobra recuperación al leer un objeto frío
 no cobra la operación de transición
 cobra una tarifa de gestión por objeto y mes

Eso elimina de un golpe todo el cálculo de punto de equilibrio de las clases 030 y 041. La decisión pasa a ser una comparación simple: la tarifa de gestión frente al ahorro esperado. Para un bucket con acceso irregular, casi siempre gana Autoclass; para uno con un patrón claro y estable, una regla explícita sale más barata.

3. Acceso: uniforme, sin claves y sin URL eternas

Google Cloud Storage tiene dos modelos de control de acceso y uno de ellos es un problema heredado:

acceso uniforme a nivel de bucket	solo IAM decide. Auditable.
acceso detallado	IAM + listas de control POR OBJETO

El segundo es la razón por la que un bucket puede estar exponiendo un objeto concreto mientras su política de IAM parece impecable: la lista de control vive en el objeto, no en el bucket, y revisar la política no la muestra. Encontrar todos los objetos con una lista permisiva exige recorrerlos uno a uno.

```
$ gcloud storage buckets update gs://cls-facturas --uniform-bucket-level-access
```

Una vez activado, las listas por objeto dejan de tener efecto. Y por encima, la política de organización de la clase 049 impide que nadie vuelva a abrirlo:

```
$ gcloud resource-manager org-policies enable-enforce \
  constraints/storage.publicAccessPrevention --organization $ORG_ID
$ gcloud storage buckets add-iam-policy-binding gs://cls-facturas \
  --member allUsers --role roles/storage.objectViewer
ERROR: 412 Request violates constraint storage.publicAccessPrevention    ✓
```

Es la misma prueba negativa de la clase 046, ejecutada por segunda vez con otro vocabulario y el mismo valor: el mensaje no habla de permisos, así que añadir roles no habría cambiado nada.

Y sobre el acceso temporal a un objeto concreto, la URL firmada tiene el mismo problema estructural que la SAS de la clase 041:

una URL firmada NO se puede revocar
solo caduca

La diferencia está en con qué se firma. La vía cómoda es una clave HMAC o una clave de cuenta de servicio, y las dos son exactamente lo que la clase 050 acaba de eliminar. La vía correcta no crea ninguna clave: la firma la produce la propia API de IAM, previa suplantación.

```
$ gcloud storage sign-url gs://cls-facturas/2026-07.pdf \
  --duration 15m \
  --impersonate-service-account firmador@cls-tienda-prod-euw1-01.iam.gserviceaccount.com
```

Eso exige que quien firma tenga roles/iam.serviceAccountTokenCreator sobre la cuenta firmadora, lo que a su vez es auditable y caduca. Y la duración corta es la única protección real: si no se puede revocar, que dure quince minutos y no siete días.

Tres piezas más completan el control, y cada una responde a un requisito distinto:

política de retención + bloqueo	WORM: ni el propietario puede borrar antes el bloqueo es IRREVERSIBLE, como en la clase 041
retención por objeto	plazo distinto para objetos distintos
clave gestionada por el cliente	control y revocación, con la responsabilidad descrita en la clase 046

Y el que quien pague sea el lector —el bucket con pago por solicitante— resuelve un caso concreto que aparece al compartir datos: publicar un conjunto grande sin pagar la salida de quien lo descargue. Es una decisión de costo con nombre, y conviene conocerla antes de que alguien proponga copiar el conjunto a otro sitio para evitar el gasto.

4. Borrar y recuperar: el hueco de la clase 041 aquí está cubierto

La clase 041 dejó el incidente más caro de la parte 03: la eliminación temporal de blob estaba activa, la de contenedor no, y un contenedor borrado se llevó 412 GiB sin recuperación posible. Eran dos interruptores independientes y solo uno estaba puesto.

Aquí ese hueco no existe de la misma forma. La eliminación temporal de Cloud Storage está activada por defecto con siete días de retención y cubre las dos cosas a la vez:

```
$ gcloud storage buckets describe gs://cls-facturas \
  --format="value(soft_delete_policy.retentionDurationSeconds)"
604800

objetos borrados      recuperables durante la retención
BUCKET borrado       recuperable durante la retención, con su contenido
```

La restauración de un bucket entero:

```
$ gcloud storage buckets list --soft-deleted --format="value(name,generation)"
cls-pruebas 1753992014882
$ gcloud storage buckets restore gs://cls-pruebas --generation 1753992014882
```

Eso no elimina la obligación de probarlo. Es exactamente el mismo argumento de la clase 041: la única forma honesta de saber si una recuperación funciona es borrar algo a propósito y recuperarlo, y hacerlo periódicamente, porque la retención se puede reducir y alguien puede hacerlo.

Los mecanismos completos y qué cubre cada uno, que siguen siendo varios y no uno:

eliminación temporal	borrado accidental de objetos y del bucket, 7 días
versionado	sobrescrituras: conserva la versión anterior
retención + bloqueo	borrado deliberado, incluso por el propietario
instantáneas / copias	corrupción lógica propagada por la aplicación

La cuarta fila es la que ninguna de las tres primeras cubre y la que más se olvida: si la aplicación escribe datos corruptos, el versionado guarda la versión buena y la eliminación temporal no interviene, pero nadie avisa, y cuando se descubre puede haber pasado la ventana. La defensa es la de la clase 030: una copia con ciclo de vida propio, en otro proyecto, con permisos distintos.

Y una decisión de gobierno que conviene tomar pronto: el proyecto que contiene las copias no debe ser administrable por quien administra el original. Es la diferencia entre una copia de seguridad y una segunda carpeta.

La comparación final con las dos plataformas anteriores, que es lo que se lleva a la clase 060:

protección por defecto	ninguna	ninguna	eliminación temporal
interruptores necesarios	3	5	2
región del par replicado	se elige	asignada	se elige
lectura del nivel más frío	minutos-horas	hasta 15 h	milisegundos
costo por operación al enfriar	estable	estable	hasta ×10

La última fila es la única en la que Google Cloud añade una trampa que las otras no tenían. Las cuatro primeras van en la otra dirección, y confirman la parte de la hipótesis de la clase 048 que decía que las excepciones serían otras: no se repite ninguna de las de Azure, y aparece una nueva que hay que aprender.

5. Lo que cuesta de verdad: salida, operaciones y objetos pequeños

La factura de almacenamiento tiene cuatro partidas y la primera casi nunca es la mayor:

almacenamiento	GB por mes, según clase y ubicación
operaciones	clase A y clase B, según clase
recuperación	GB leídos desde clases frías
salida de red	GB que salen de Google Cloud

La salida de red es la que sorprende, igual que en las dos plataformas anteriores, y tiene aquí una respuesta directa: casi todo el tráfico interno no sale. Un servicio en Compute Engine o Cloud Run leyendo de un bucket de la misma región no genera salida, y el acceso privado a Google de la clase 051 asegura además que no pase por Cloud NAT. Las dos decisiones juntas eliminan la partida entera para el tráfico interno.

Para el tráfico hacia usuarios, la palanca es Cloud CDN delante del bucket, que la clase 052 dejó a un interruptor de distancia. El cálculo es el de siempre: cuánto del contenido es cacheable y qué proporción de peticiones deja de llegar al origen.

Y la partida de operaciones merece una regla propia, porque es donde esta plataforma se comporta distinto:

un objeto de 2 KB en Archive	
almacenamiento	0,0000024 USD/mes
una lectura	0,0000004 USD de operación + 0,0000001 de recuperación
→ el objeto cuesta más leerlo que guardarlo un año	

Eso lleva a un principio de diseño que no aparecía en las clases 030 ni 041: agrupar objetos pequeños antes de enfriarlos. Un millón de registros de 2 KB en un solo archivo comprimido cuesta una operación en lugar de un millón, y el ahorro es de dos órdenes de magnitud. La composición de objetos y los formatos columnares existen para eso, y la decisión se toma al escribir, no al archivar.

El método completo para decidir una regla de ciclo de vida, que es el entregable de esta clase:

1. ¿cuántos objetos y de qué tamaño medio?
si el tamaño medio es de pocos KB, agrupar antes de nada
2. ¿qué proporción se relee al mes?
por encima del punto de equilibrio, enfriar cuesta dinero
3. ¿se puede predecir el acceso?
si no, Autoclass y se acabó el cálculo
4. ¿cuánto cuesta la transición?
objetos × precio de operación de la clase destino, de una vez
5. ¿en cuántos meses se amortiza?
si la respuesta es más de doce, no se hace

El paso 5 es el que falta en casi todas las políticas de ciclo de vida que existen, y es el único que convierte la decisión en algo defendible.

Ejemplo trabajado

CloudShop lleva su almacenamiento a Google Cloud con la línea base de la clase 041 en la mano. Dos de los cinco problemas de aquella clase no se reproducen, uno se reproduce igual, y aparece uno nuevo que cuesta 210 dólares en una tarde.

Lo que no se reprodujo.

la región del par replicado	en Azure se asignaba; aquí se eligió europe-west1 + europe-west4, ambas dentro de la jurisdicción aprobada
borrado de un contenedor	en Azure se perdieron 412 GiB por tener un interruptor de dos; aquí la eliminación temporal viene activa y cubre las dos cosas

Ambas se verificaron en vez de darse por buenas:

```
$ gcloud storage buckets delete gs://cls-prueba-recuperacion
$ gcloud storage buckets list --soft-deleted --format="value(name,generation)"
cls-prueba-recuperacion 1753992014882
$ gcloud storage buckets restore gs://cls-prueba-recuperacion --generation 1753992014882
$ gcloud storage ls gs://cls-prueba-recuperacion | wc -l
50
```

✓

Incidente 1 — la regla de ciclo de vida que destruyó valor.

Se aplicó la política de la clase 041 traducida: a los 365 días, todo a Archive. El bucket de eventos contenía 4,2 millones de objetos de 71 KB de media.

```
$ gcloud storage ls -l gs://cls-eventos/** | tail -1
TOTAL: 4203118 objects, 298471029248 bytes (278 GiB)
```

costo de la transición

$4.203.118 / 1.000 \times 0,05 \text{ USD} = 210,16 \text{ USD}$, cobrados de una vez

ahorro mensual

$278 \text{ GiB} \times (0,020 - 0,0012) = 5,22 \text{ USD/mes}$

amortización 40 meses

Y el golpe real llegó al mes siguiente: un análisis releyó el 12 % del conjunto.

recuperación $33 \text{ GiB} \times 0,05 = 1,65 \text{ USD}$

operaciones $504.000 \text{ lecturas} / 1.000 \times 0,05 = 25,20 \text{ USD}$

total del análisis, que en Standard habría costado ~2,50 USD: 26,85 USD

La corrección tiene dos partes, y la segunda es la que dura:

regla de ciclo de vida	todo a Archive a 365 d	solo objetos > 1 MB
objetos pequeños	4,1 M	agrupados al escribir: un archivo diario
objetos tras agrupar	4,2 M	31.400
clase del conjunto agrupado	Archive	Coldline
costo de la siguiente transición	210 USD	1,57 USD
costo del mismo análisis	26,85 USD	2,90 USD

Incidente 2 — un objeto público con la política del bucket impecable.

Una revisión externa encuentra una factura accesible sin autenticar. La política del bucket no concede nada a allUsers.

```
$ gcloud storage buckets describe gs://cls-facturas \
  --format="value(uniform_bucket_level_access.enabled)"
False
$ gcloud storage objects describe gs://cls-facturas/2025-11.pdf --format="value(acl)"
[{'entity': 'allUsers', 'role': 'READER'}]
```

Una lista de control en el objeto, puesta dos años atrás por una herramienta de migración. La política del bucket nunca la mostró.

modelo de acceso	detallado	uniforme
objetos con lista de control permisiva	3	0
prevención de acceso público	no aplicada	política de organización
prueba negativa	no	sí, ejecutada

Incidente 3 — la factura que no baja. Cuarta vez.

datos visibles	412 GiB
facturado	1.108 GiB

Versionado activo y ninguna regla sobre versiones no actuales. Es el mismo problema del marcador de borrado de la clase 030 y de las versiones de la clase 041, con un tercer nombre.

facturado	1.108 GiB	430 GiB
costo mensual del bucket	22,16 USD	8,60 USD

La nota que el equipo añade a su lista de comprobación de plataforma nueva: si hay versionado, tiene que haber una regla que retire versiones, y hay que verificar que el facturado coincide con lo visible.

Incidente 4 — una URL firmada de siete días y la clave con la que se firmaba.

```
$ gcloud storage hmac list --project cls-tienda-prod-euw1-01 --format="value(accessId,state)"
G00G1E... ACTIVE
```

La aplicación móvil pedía al backend una URL firmada de siete días, y el backend firmaba con una clave HMAC guardada en su configuración. Dos problemas encadenados: la clave es una credencial de larga duración —justo lo que la clase 050 eliminó— y la URL no se puede revocar.

firma	clave HMAC en config	API de IAM, sin clave
duración de la URL	7 días	15 minutos
revocación posible	no	irrelevante: caduca sola
claves HMAC activas	1	0

Resumen del almacenamiento:

objetos en el bucket de eventos	4,2 M	31.400
costo de una transición de clase	210 USD	1,57 USD
costo de un análisis del 12 %	26,85 USD	2,90 USD
facturado frente a visible	1.108/412 GiB	430/412 GiB
objetos públicos	3	0
claves de firma de larga duración	1	0
duración de las URL firmadas	7 días	15 minutos
recuperación de bucket probada	no	sí, mensual
costo mensual de almacenamiento	64,80 USD	28,40 USD

La lección que esta clase traslada al resto de la parte 04: dos trampas de la clase 041 no se repitieron y una tercera —la factura que no baja con versionado— apareció por cuarta vez, lo que la confirma como propiedad del problema y no del proveedor. Y la trampa nueva tiene una forma que conviene reconocer para el futuro: una plataforma que abarata el almacenamiento y encarece la operación cambia el punto de equilibrio de sitio, y una política copiada de otro proveedor puede costar más de lo que ahorra sin que nada falle.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/053-cloud-storage-clases-lifecycle-y-replicacion/lab.py
```

El laboratorio selecciona el motor de práctica storage y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa bucket-gobernado-gcp en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una política de durabilidad, acceso, retención y costo. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye bucket-gobernado-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Una regla de ciclo de vida dispara la factura en vez de reducirla	Mueve millones de objetos pequeños a una clase cuyo precio por operación es hasta diez veces mayor	Añade condición de tamaño mínimo, agrupa los objetos pequeños al escribirlos y calcula en cuántos meses se amortiza la transición.
Un objeto es público y la política del bucket no concede nada a nadie	El bucket está en modo de acceso detallado y el permiso está en una lista de control del objeto	Activa el acceso uniforme a nivel de bucket y la prevención de acceso público, y verifica con la prueba negativa.
Se libera espacio y el facturado no baja	Con versionado activo, borrar conserva la versión no actual	Añade una regla sobre daysSinceNoncurrentTime y comprueba que el volumen facturado converge con el visible.
Una URL firmada filtrada sigue siendo válida durante días	No se puede revocar y se emitió con una duración larga, firmada con una clave de larga duración	Firma mediante suplantación con la API de IAM, sin claves, y emite duraciones de minutos.
Leer un conjunto archivado cuesta más que haberlo dejado en Standard	El costo de recuperación y el de operación por objeto superan el ahorro de almacenamiento	Calcula la proporción releída al mes; si no se puede predecir, usa Autoclass, que no cobra recuperación.
Se elige multirregión por comodidad y el costo por GB sube	La ubicación se decidió por preferencia y no por requisito de residencia o recuperación	Elige región para lectura local, birregión con destino propio para recuperación y multirregión solo cuando la lectura es realmente global.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué problema concreto de la clase 041 resuelve poder elegir las dos regiones de una birregión?
2. ¿Por qué aquí no hay rehidratación y en qué cambia eso el criterio para usar la clase más fría?
3. Calcula si compensa mover 3 millones de objetos de 40 KB a Archive, y explica qué dato falta si no puedes.
4. Un objeto es público con la política del bucket vacía. ¿Dónde está el permiso y cómo se elimina esa clase entera de problemas?
5. ¿Cómo se emite una URL firmada sin que exista ninguna clave, y por qué la duración corta es la única protección real?

Referencias

- Google Cloud (2025). Bucket locations — región, birregión con destino elegido, multirregión y replicación turbo.
<<https://cloud.google.com/storage/docs/locations>>
- Google Cloud (2025). Storage classes — permanencia mínima, recuperación y precios por operación.
<<https://cloud.google.com/storage/docs/storage-classes>>
- Google Cloud (2025). Object Lifecycle Management — condiciones, versiones no actuales y transiciones.
<<https://cloud.google.com/storage/docs/lifecycle>>
- Google Cloud (2025). Uniform bucket-level access — IAM frente a listas de control por objeto.
<<https://cloud.google.com/storage/docs/uniform-bucket-level-access>>
- Google Cloud (2025). Soft delete — retención de objetos y de buckets, y restauración.
<<https://cloud.google.com/storage/docs/soft-delete>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 04 en PDF · Recorrido de Google Cloud en PDF · Manual integral

Anterior	Índice	Siguiente
← 052 · Compute Engine, managed instance groups y load balancing	Parte 04 · Programa	054 · Cloud SQL, Spanner, Firestore y Memorystore →

Evaluación — 053 Cloud Storage, clases, lifecycle y replicación

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega bucket-gobernado-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

054 — Cloud SQL, Spanner, Firestore y Memorystore

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: data · Estado: EXECUTABLECORE

Propósito

Elegir motor de datos en Google Cloud con el criterio de la clase 031 —los patrones de acceso primero— y con tres precisiones que solo aparecen aquí: llegar a Cloud SQL por red privada exige reservar un rango de direcciones que nadie planificó, Spanner cuesta lo que cuesta y solo se justifica por lo que únicamente él da, y en Firestore el precio de una consulta es el número de documentos que devuelve, así que un panel mal escrito factura más que la base de datos entera.

Resultados de aprendizaje

Al finalizar podrás:

1. Conectar a Cloud SQL por red privada sabiendo qué rango hay que reservar y por qué el proxy autenticado es la vía correcta.
2. Decidir entre Cloud SQL, Spanner y Firestore con una justificación de costo y de patrón de acceso, no de catálogo.
3. Diseñar una clave primaria que no concentre las escrituras, en el motor que sea.
4. Calcular el costo de una consulta de Firestore a partir del tamaño del resultado.
5. Anticipar las decisiones irreversibles de esta clase, empezando por el almacenamiento que crece y no baja.

Conceptos centrales

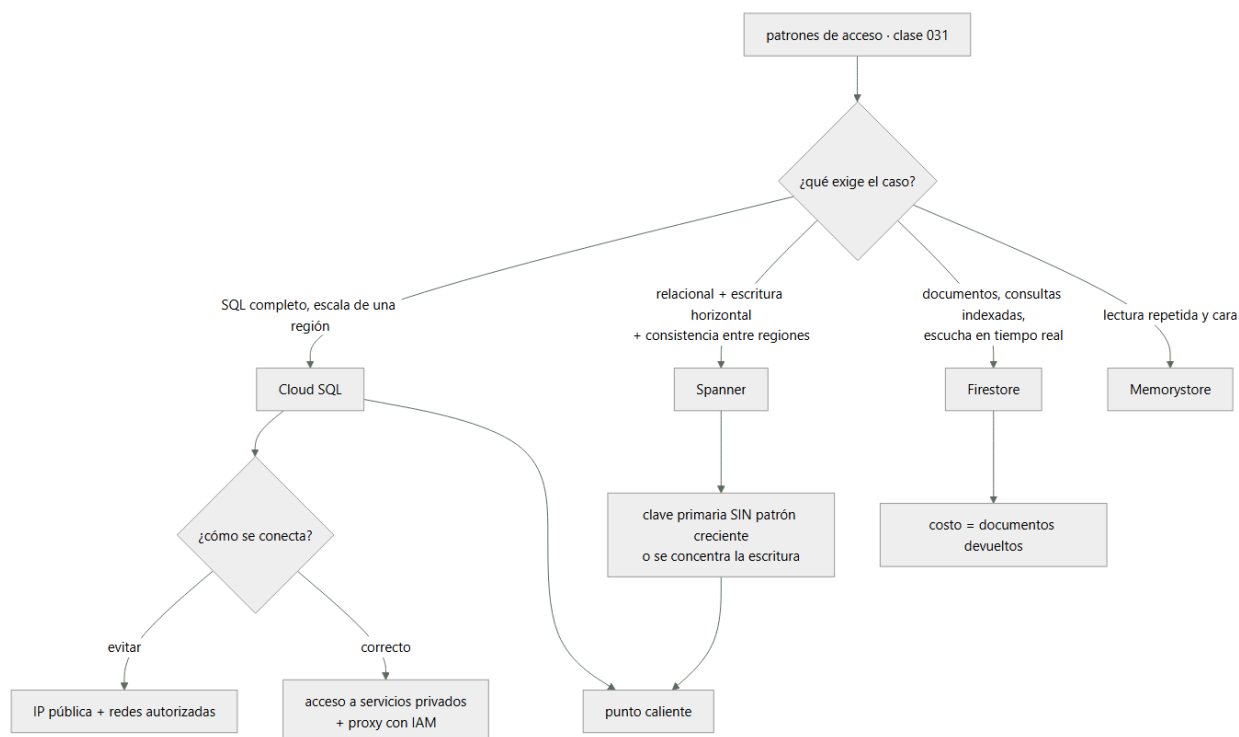
Concepto	Comprensión verificable
acceso a servicios privados	Emparejamiento con una red gestionada por Google que permite alcanzar Cloud SQL por dirección interna. Consume un rango reservado de tu espacio, que hay que planificar en la clase 051.
proxy de autenticación de Cloud SQL	Conector que autentica con IAM y cifra el tránsito sin gestionar certificados ni listas de direcciones autorizadas. Es la vía recomendada y la que evita exponer una IP pública.
punto caliente	Concentración de escrituras en una porción del espacio de claves. Una clave primaria creciente —una marca de tiempo, un contador— lo garantiza, y limita el rendimiento por mucho que se añada capacidad.
consistencia externa	Garantía de Spanner: las transacciones se ven en un orden global coherente incluso entre regiones. Es lo único que no se puede construir encima de otro motor, y es lo que se está pagando.
lectura de documento	Unidad de facturación de Firestore. Una consulta que devuelve mil documentos cuesta mil lecturas: el precio es el tamaño del resultado, no la complejidad.
crecimiento automático de almacenamiento	Cloud SQL amplía el disco solo cuando hace falta y no lo reduce nunca. Un crecimiento accidental fija un costo mensual hasta que se migre la instancia.

Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Llegar a Cloud SQL sin exponerlo: el rango que nadie reservó

Una instancia de Cloud SQL no vive en tu VPC. Vive en una red gestionada por Google, y eso deja tres formas de llegar a ella, con consecuencias muy distintas:

IP pública + redes autorizadas	el tráfico sale de tu red y vuelve la lista de direcciones hay que mantenerla y una IP dinámica la rompe
acceso a servicios privados	emparejamiento con la red de Google
Private Service Connect	dirección interna, tráfico que no sale extremo con IP interna elegida por ti el modelo de la clase 051

La primera es la que sale por defecto y la que hay que evitar. La segunda es la habitual, y tiene un requisito que se descubre a mitad del despliegue: consume un rango de tu espacio de direcciones, que hay que reservar explícitamente.

```

$ gcloud compute addresses create rango-servicios-google \
  --global --purpose VPC_PEERING --prefix-length 20 \
  --network vpc-cloudshop --addresses 10.90.0.0
$ gcloud services vpc-peerings connect --service servicenetworking.googleapis.com \
  --ranges rango-servicios-google --network vpc-cloudshop
  
```

Un /20 para empezar, y conviene que sea holgado: el rango se comparte entre todos los servicios gestionados que usen este mecanismo —Cloud SQL, Memorystore, algunas versiones de otros productos— y ampliarlo después obliga a recrear el emparejamiento. Es exactamente el tipo de reserva que la clase 051 pedía hacer al planificar la subred, junto con los rangos secundarios de Kubernetes.

Y sobre cómo se autentica la aplicación, la vía correcta elimina dos gestiones a la vez:

```

$ ./cloud-sql-proxy --private-ip --auto-iam-authn \
  cls-datos-prod-euw1-01:europa-west1:pg-pedidos

--private-ip      usa la dirección interna, no la pública
--auto-iam-authn  autentica con la identidad de la carga (clase 050)
                  → no hay contraseña de base de datos que guardar ni rotar
  
```

La segunda línea cierra el círculo que empezó en la clase 026 y siguió en la 038: el motor de datos deja de tener una credencial propia. El usuario de base de datos se corresponde con una cuenta de servicio, y quitarle el acceso es quitarle un rol.

Dos comportamientos de la alta disponibilidad que hay que anticipar:

La conmutación produce errores transitorios. Una instancia con alta disponibilidad mantiene una réplica en espera en otra zona, y la conmutación tarda alrededor de un minuto. Durante ese minuto, las conexiones fallan. Es la cuarta vez que este hecho aparece en el programa —la clase 048 lo descubrió midiendo 1.104 peticiones perdidas con las alarmas en verde— y merece enunciarse como propiedad general: en cualquier base de datos gestionada, la conmutación es parte del funcionamiento normal y el cliente tiene que reintentar. Ninguna cantidad de infraestructura sustituye ese reintento.

El mantenimiento reinicia. A diferencia de Compute Engine, que migra en vivo (clase 052), los servicios gestionados de datos aplican mantenimiento con un reinicio dentro de una ventana declarada. Elegir esa ventana y declararla como parte del contrato del servicio es trabajo de diseño, no un trámite:

```
$ gcloud sql instances patch pg-pedidos \
  --maintenance-window-day SUN --maintenance-window-hour 3 \
  --maintenance-release-channel production
```

Y una puerta de un solo sentido con precio mensual: el almacenamiento crece automáticamente y no se reduce nunca. Una tabla de registro descontrolada que lleve el disco de 500 GB a 4 TB deja ese costo fijado hasta que alguien migre los datos a una instancia nueva. Conviene poner un límite al crecimiento automático y una alerta sobre el uso, en vez de confiar en que nada crezca.

2. Spanner: qué se está comprando y qué exige a cambio

Spanner es un motor relacional que escala horizontalmente la escritura y mantiene consistencia externa entre regiones. Es la única pieza del programa que ofrece las tres cosas a la vez, y la conversación honesta empieza por el precio:

```
~0,90 USD por nodo y hora en configuración regional
→ ~657 USD por nodo y mes, más almacenamiento
una configuración de producción razonable son 3 nodos: ~1.970 USD/mes
```

Comparado con una instancia de Cloud SQL equivalente en capacidad para una carga de una región:

Cloud SQL 4 vCPU / 16 GB con alta disponibilidad	~320 USD/mes
Spanner, 3 nodos	~1.970 USD/mes

Seis veces. Eso no descalifica a Spanner: sitúa la pregunta. ¿Qué hace este caso que solo Spanner resuelve?

```
sí lo justifica
  escritura que no cabe en una instancia vertical, con esquema relacional
  consistencia fuerte entre regiones, sin ventana de replicación
  crecimiento sin ventanas de mantenimiento ni fragmentación manual
```

```
no lo justifica
  "queremos que escale" sin una cifra que lo respalde
  volumen que cabe en una instancia grande de Cloud SQL
  necesidad de extensiones o funciones específicas de PostgreSQL
```

Y si se elige, exige una disciplina de esquema que no es opcional. La clave primaria decide el rendimiento, y una clave creciente lo destruye:

```
-- concentra TODAS las escrituras en la última división
CREATE TABLE pedidos (
  creado TIMESTAMP NOT NULL,
  pedido_id STRING(36) NOT NULL,
) PRIMARY KEY (creado, pedido_id);

-- reparte: el identificador aleatorio distribuye por el espacio de claves
CREATE TABLE pedidos (
  pedido_id STRING(36) NOT NULL,    -- UUID v4
  creado TIMESTAMP NOT NULL,
) PRIMARY KEY (pedido_id);
```

El mecanismo del fallo es el mismo que la clase 042 describió para Cosmos DB y la 031 para DynamoDB, con una diferencia importante: aquí no hay un límite duro que produzca un error. La escritura no falla; simplemente no pasa de un techo, y añadir nodos no lo mueve porque el trabajo sigue cayendo en la misma división. El síntoma es una gráfica

de rendimiento plana con la CPU de un nodo alta y la de los demás baja.

Es la tercera aparición del mismo problema en el programa, con tres formas de manifestarse:

DynamoDB	partición caliente: limitación
Cosmos DB	partición lógica llena: error 403, la escritura FALLA
Spanner	división caliente: techo de rendimiento, sin ningún error

La versión de Spanner es la más difícil de diagnosticar precisamente porque no falla nada.

Y la herramienta de localidad propia del motor, las tablas intercaladas, guarda las filas hijas físicamente junto a la padre:

```
CREATE TABLE lineas_pedido (  
  pedido_id STRING(36) NOT NULL,  
  linea_id INT64 NOT NULL,  
) PRIMARY KEY (pedido_id, linea_id),  
  INTERLEAVE IN PARENT pedidos ON DELETE CASCADE;
```

Leer un pedido con todas sus líneas pasa a ser una sola lectura local en vez de una unión distribuida. Es la misma idea que la clave de partición compartida de la clase 031, expresada en el esquema.

3. Firestore: el precio es el tamaño del resultado

Firestore es una base de datos de documentos con dos propiedades que definen cómo se diseña encima de ella.

Toda consulta necesita un índice. Los de un solo campo se crean solos; los compuestos se declaran. Y una consulta sin índice no se ejecuta lentamente: falla, con un mensaje que incluye el enlace para crear el que falta.

FAILED_PRECONDITION: The query requires an index.

Eso es una buena noticia por la misma razón que lo era el fallo ruidoso de la clase 051: no existe la consulta que funciona en desarrollo con mil documentos y tumba producción con diez millones, porque no llega a ejecutarse sin índice. El coste de esta propiedad es que el conjunto de consultas posibles se declara por adelantado, así que un patrón de acceso nuevo exige un despliegue de índices.

El precio es el número de documentos. No hay unidades de solicitud ni horas de instancia:

lecturas	~0,03 USD por 100.000 documentos
escrituras	~0,09 USD por 100.000
borrados	~0,01 USD por 100.000

La consecuencia práctica cambia el diseño de la aplicación más que cualquier ajuste de infraestructura:

una consulta que devuelve 40.000 documentos cuesta 40.000 lecturas
aunque la aplicación solo muestre un total

Un panel que calcula «pedidos de hoy» leyendo todos los pedidos de hoy paga por cada uno, cada vez que alguien abre la página. La aritmética se hace grande deprisa:

40.000 documentos por carga × 12.000 cargas al día × 30 días
= 14.400 millones de lecturas al mes
× 0,03 / 100.000 = 4.320 USD/mes por un contador

Las dos correcciones son de modelo de datos, no de configuración:

1. consultas de agregación (COUNT, SUM) – se facturan por índice leído, no por documento devuelto
2. contadores materializados: un documento que la escritura actualiza y el panel lee de uno en uno

La segunda trae de vuelta el problema de los puntos calientes en su forma documental: un documento admite del orden de una escritura por segundo de forma sostenida. Un contador global actualizado por cada pedido se convierte en el cuello de botella de todo el sistema. La solución conocida es repartirlo:

contador repartido en N documentos, elegido al azar al escribir
lectura = suma de los N
con N = 10, se admiten ~10 escrituras por segundo

Y los modos conviene aclararlos porque el nombre confunde: Firestore tiene modo nativo y modo Datastore, la elección se hace al crear la base de datos y no se cambia. El nativo es el que trae escuchas en tiempo real y bibliotecas para cliente móvil; el modo Datastore existe por compatibilidad. Para todo lo nuevo, nativo.

Sobre Memorystore basta con situarlo, porque las lecciones de la clase 042 se aplican sin cambios: nivel básico sin réplica y sin acuerdo de servicio —no para producción—, política de expulsión que debe considerar todas las claves, TTL siempre, y una sola conexión reutilizada por proceso. Lo único propio es que se conecta por el mismo mecanismo de acceso a servicios privados que Cloud SQL, así que consume del mismo rango reservado.

4. Elegir con una tabla y defenderlo con un número

La discusión sobre qué motor usar se repite en cada equipo hasta que alguien la escribe. Escrita, cabe aquí:

Pregunta	Cloud SQL	Spanner	Firestore
¿SQL completo y extensiones?	Sí	Subconjunto	No
¿Transacciones entre entidades?	Sí	Sí, entre regiones	Sí, acotadas
¿Escritura que escala horizontalmente?	No	Sí	Sí
¿Consultas no previstas?	Sí	Sí	No: hacen falta índices
¿Escucha en tiempo real?	No	No	Sí
Se paga por	Instancia y hora	Nodo y hora	Documento
Costo de partida	320 USD/mes	1.970 USD/mes	Casi cero

La última fila engaña si se lee sola. Firestore empieza casi gratis y su costo crece con el uso de la aplicación, no con el tamaño de los datos: una aplicación con pocos datos y muchas lecturas puede costar más que una instancia de Cloud SQL con terabytes. Al revés, Cloud SQL cobra la instancia esté ociosa o no.

La forma defendible de decidir es estimar el costo de la carga real en los candidatos, con las cifras que se conocen:

para CloudShop, catálogo con 4M de productos y 30M de lecturas al mes:

Cloud SQL 4 vCPU con alta disponibilidad	~320 USD/mes
Firestore: 30M lecturas × 0,03/100.000	~9 USD/mes
+ escrituras y almacenamiento	~40 USD/mes
→ Firestore, con diferencia	

para pedidos, con transacciones entre pedido, stock y pago:

Firestore obliga a rediseñar la transacción
Cloud SQL lo resuelve sin cambiar el modelo
→ Cloud SQL

Ese ejercicio de dos párrafos evita la discusión de dos semanas, y deja escrito por qué. Lo que hay que exigir a cualquier propuesta es exactamente eso: una cifra por candidato para la carga real, no una preferencia.

Y las tres decisiones de esta clase que no se deshacen editando una configuración, que conviene señalar antes de tomarlas:

1. la clave primaria de una tabla de Spanner
cambiarla es crear otra tabla y migrar
2. el modo de Firestore, nativo o Datastore
se elige al crear la base de datos
3. el almacenamiento de Cloud SQL, que crece y no baja
revertirlo es volcar y restaurar en una instancia nueva

Las tres tienen la misma forma que las de la clase 042 y la misma respuesta: se toman con la evidencia delante, se documentan como decisión con su alternativa descartada, y se protegen con un límite o una alerta que avise antes de que sea tarde.

5. Diagnosticar por el límite que no es la CPU

La clase 042 cerró con una afirmación que aquí se confirma con otros tres motores: el recurso que se agota primero casi nunca es la CPU, y el panel por defecto muestra la CPU.

Lo que hay que mirar en cada uno:

Cloud SQL	
conexiones activas frente al máximo	un agotamiento de conexiones parece lentitud del motor
retraso de la réplica de lectura	informes leyendo datos viejos
IOPS del disco	ligadas al TAMAÑO del disco,

espera de bloqueos	igual que en la clase 052 una transacción larga bloquea a todas
Spanner	
utilización de CPU POR DIVISIÓN	la media engaña; el punto caliente es una división al 100 % y su percentil, no la media
latencia por operación	
Firestore	
lecturas por operación	el indicador de costo y de diseño
contención en documentos	escrituras al mismo documento

La fila de las IOPS de Cloud SQL merece subrayarse porque es la cuarta aparición del mismo patrón: el rendimiento del disco depende del tamaño del disco, así que una instancia con un disco pequeño tiene un techo de E/S bajo aunque le sobre CPU. Ampliar el disco lo sube — y como el almacenamiento no baja, esa ampliación es permanente.

El orden de diagnóstico que sirve para los tres, y que ya es el mismo de las partes 02 y 03:

1. ¿qué límite está al 100 %? conexiones, E/S, bloqueos, una división
2. ¿desde cuándo? percentil de latencia en serie temporal
3. ¿qué cambió en esa ventana? despliegue, esquema, índice, volumen
4. ¿qué consulta concreta? Query Insights o el registro de consultas lentas

Y una recomendación de operación que evita la mitad de los incidentes de esta clase: medir el costo de las consultas en el mismo sitio donde se mide su latencia. En Firestore eso significa registrar cuántos documentos devolvió cada consulta; en Cloud SQL, cuántas filas examinó frente a cuántas devolvió. Las dos cifras detectan el mismo problema —una consulta que lee mucho para devolver poco— y lo detectan el día del despliegue en lugar de en la factura del mes siguiente. Es el mismo principio de la cabecera de costo por consulta de la clase 042: lo que tiene número se discute; lo que no, se opina.

Ejemplo trabajado

CloudShop reparte su capa de datos en Google Cloud. La decisión inicial es razonable, el despliegue falla por una razón de red, y después aparecen cuatro problemas de los cuales dos ya se habían visto en otras plataformas.

Decisión inicial, documentada con cifras:

pedidos	Cloud SQL PostgreSQL, alta disponibilidad	transacciones entre entidades
catálogo	Firestore modo nativo	30M lecturas/mes, sin uniones
sesiones	Memorystore Redis estándar	caché con TTL
inventario	pendiente de decidir	se propone Spanner

Incidente 1 — la aplicación no puede conectar, y la solución rápida saca el tráfico de la VPC.

```
$ gcloud sql instances describe pg-pedidos --format="value(ipAddresses[].type)"
PRIMARY
```

Solo IP pública. Para conectar en privado hacía falta acceso a servicios privados, y este exige un rango reservado que no estaba en el plan de direcciones de la clase 051. La solución de urgencia fue autorizar el rango de salida del NAT, lo que significaba que el tráfico salía de la red y volvía.

camino del tráfico	sale por NAT y vuelve	dirección interna
rango reservado para servicios	ninguno	10.90.0.0/20
autenticación	contraseña en secreto	IAM con el proxy
contraseñas de base de datos	2	0
IP pública de la instancia	activa	desactivada

Y la prueba negativa:

```
$ psql "host=34.78.x.x dbname=pedidos" -c "select 1"
psql: error: connection to server ... failed: Connection timed out ✓
```

Incidente 2 — la conmutación de alta disponibilidad, por cuarta vez en el programa.

simulacro de conmutación	
duración	62 s
errores devueltos al cliente	847
pedidos no registrados	31

El mismo hallazgo de la clase 048, con otro motor y otros códigos de error. El equipo ya tenía la corrección escrita del capstone anterior y la aplicó directamente:

reintento de errores transitorios	no	sí
-----------------------------------	----	----

tiempo de espera de conexión	30 s (largo)	5 s + reintento
errores en la conmutación repetida	847	0

Es la primera vez en el programa que un problema conocido se corrige antes de causar daño en la plataforma nueva. Ese es el valor concreto del contrato portable de la clase 048.

Incidente 3 — el disco que creció a 4 TB y no baja.

Una tabla de auditoría sin política de retención creció durante siete semanas.

```
$ gcloud sql instances describe pg-pedidos \
  --format="value(settings.dataDiskSizeGb,settings.storageAutoResize)"
4096 True
$ psql -c "SELECT pg_size_pretty(pg_total_relation_size('auditoria'))"
3312 GB
```

Se purgó la tabla y el disco siguió en 4 TB, porque el crecimiento automático no se revierte.

disco	4.096 GB	500 GB (instancia nueva)
costo mensual del almacenamiento	~697 USD	~85 USD
límite de crecimiento automático	ninguno	800 GB
alerta de uso de disco	ninguna	> 70 %
retención de la tabla de auditoría	ninguna	90 días, exportada a Cloud Storage (clase 053)
migración necesaria	—	volcado y restauración, 4 h

Seiscientos doce dólares al mes y cuatro horas de migración por un límite que se configura en un minuto.

Incidente 4 — Spanner sí, pero no como se propuso.

La propuesta era «inventario en Spanner porque escala». La evaluación con cifras:

volumen de inventario	18 GB
escrituras en el pico	1.400 por segundo
requisito real	stock coherente entre tres regiones, sin vender dos veces la última unidad

El volumen cabía en Cloud SQL de sobra; lo que no cabía era el requisito de consistencia entre regiones sin ventana de replicación. Eso sí es lo que solo Spanner da, y quedó escrito así en la decisión.

Pero la primera implementación no llegaba a 1.200 escrituras por segundo con tres nodos:

```
PRIMARY KEY (actualizado, sku) -- marca de tiempo primero
utilización media de CPU      38 %
utilización de la división caliente 99 %
```

Ninguna escritura falló: simplemente había un techo. Es la tercera versión del mismo problema del programa y la más difícil de ver, porque no produce ningún error.

clave primaria	(actualizado, sku)	(sku) — reparte por hash
tablas relacionadas	tablas aparte	intercaladas en el padre
escrituras por segundo	1.180	6.400
nodos	3	3
lectura de un SKU con su historial	2 consultas	1 lectura local

El rendimiento se multiplicó por cinco sin añadir capacidad. Es la misma lección de la clase 042 con otro motor: el precio de un modelo de datos equivocado no se paga con infraestructura.

Incidente 5 — un panel que costaba más que la base de datos.

factura de Firestore, mes 2		
lecturas	14.380 millones	4.314 USD
escrituras	41 millones	37 USD
almacenamiento		12 USD

El panel de operaciones cargaba «pedidos de hoy» leyendo todos los documentos del día para contarlos, en cada carga de página.

cálculo del total	leer 40.000 docs	consulta de agregación
contador de pedidos	—	documento repartido en 10
lecturas mensuales	14.380 millones	62 millones
costo mensual de Firestore	4.363 USD	31 USD

Y se añadió la medida que evita la repetición: el número de documentos devueltos se registra en cada consulta, junto a su latencia, y aparece en el mismo panel.

Resumen de la capa de datos:

	público	privado
camino a Cloud SQL		
contraseñas de base de datos	2	0
errores en conmutación de alta disponibilidad	847	0
almacenamiento de Cloud SQL	4.096 GB	500 GB
escrituras por segundo en Spanner	1.180	6.400
lecturas mensuales de Firestore	14.380 M	62 M
costo mensual de la capa de datos	7.190 USD	2.470 USD

La lección que esta clase traslada al resto de la parte 04: de los cinco problemas, dos eran conocidos —la conmutación que exige reintento y la clave que concentra escrituras— y uno de ellos se corrigió antes de causar daño porque estaba escrito en el contrato de la clase 048. Los otros tres eran propios de la plataforma, y los tres tenían la misma forma: un valor por defecto cómodo que fija un costo o un límite difícil de revertir. La IP pública, el crecimiento automático de disco y el precio por documento no fallan; simplemente cobran.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/054-cloud-sql-spanner-firestore-y-memorystore/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa matriz-datos-gcp en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye matriz-datos-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
La aplicación no puede conectar a Cloud SQL por dirección interna	El acceso a servicios privados exige un rango reservado del propio espacio de direcciones	Reserva un rango holgado al planificar la red y conecta con el proxy autenticado por IAM, sin IP pública.
Se purga una tabla enorme y el costo de almacenamiento no baja	El crecimiento automático de Cloud SQL amplía el disco y nunca lo reduce	Pon un límite al crecimiento y una alerta de uso; revertir exige volcar y restaurar en una instancia nueva.
Spanner no supera un techo de escrituras pese a tener nodos libres	La clave primaria es creciente y concentra las escrituras en una división	Usa una clave que reparta —identificador aleatorio o secuencia invertida— e intercala las tablas hijas.
La factura de Firestore es mayor que la de la base de datos relacional	El precio es el número de documentos devueltos y un panel los leía todos para contar	Usa consultas de agregación o contadores materializados repartidos, y registra los documentos devueltos junto a la latencia.
Una consulta nueva falla en producción con FAILEDPRECONDITION	Firestore exige un índice para cada consulta y el compuesto no estaba declarado	Declara los índices como parte del despliegue; el fallo es ruidoso y es preferible a una consulta que degrada en silencio.
Se pierden peticiones durante un mantenimiento planificado	Los servicios gestionados de datos reinician en su ventana, a diferencia de la migración en vivo de Compute Engine	Declara la ventana, reintenta los errores transitorios y prueba la conmutación como parte del simulacro.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué hay que reservar para llegar a Cloud SQL por red privada, y en qué clase debía haberse planificado?
2. ¿Qué exige un caso para justificar el sobre coste de Spanner frente a Cloud SQL?
3. ¿Cómo se manifiesta un punto caliente en Spanner y por qué es más difícil de diagnosticar que en Cosmos DB?
4. Calcula el costo mensual de un panel que lee 25.000 documentos y se carga 8.000 veces al día.
5. ¿Cuál es la cuarta aparición en el programa del problema de los errores transitorios, y qué lo corrige?

Referencias

- Google Cloud (2025). Configure private IP for Cloud SQL — acceso a servicios privados y rango reservado. <<https://cloud.google.com/sql/docs/postgres/configure-private-ip>>
- Google Cloud (2025). About the Cloud SQL Auth Proxy — autenticación con IAM y conexión cifrada. <<https://cloud.google.com/sql/docs/postgres/sql-proxy>>
- Google Cloud (2025). Schema design best practices in Spanner — claves primarias, puntos calientes y tablas intercaladas. <<https://cloud.google.com/spanner/docs/schema-design>>
- Google Cloud (2025). Understand Firestore billing — lecturas, escrituras y costo de las consultas. <<https://cloud.google.com/firestore/docs/billing-example>>
- Google Cloud (2025). Distributed counters in Firestore — límite de escritura por documento y contadores repartidos. <<https://cloud.google.com/firestore/docs/solutions/counters>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 04 en PDF · Recorrido de Google Cloud en PDF · Manual integral

Anterior	Índice	Siguiente
← 053 · Cloud Storage, clases, lifecycle y replicación	Parte 04 · Programa	055 · Cloud Run, Cloud Functions y API Gateway →

Evaluación — 054 Cloud SQL, Spanner, Firestore y Memorystore

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega matriz-datos-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

055 — Cloud Run, Cloud Functions y API Gateway

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: serverless · Estado: EXECUTABLECORE

Propósito

Ejecutar aplicaciones sin gestionar servidores en Google Cloud, donde la pieza central se comporta de una forma que rompe el hábito adquirido en las clases 032 y 043: una instancia de Cloud Run atiende muchas peticiones a la vez. Esa sola propiedad cambia el costo por un factor cercano a diez, cambia lo que el código puede suponer sobre su propio aislamiento, y hace que el ajuste por defecto —la CPU solo se asigna durante la petición— produzca el fallo más desconcertante de la clase: trabajo que se planifica y nunca se ejecuta.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar el efecto de la concurrencia por instancia sobre el costo y sobre los requisitos del código.
2. Diagnosticar por qué desaparecen registros, métricas o tareas en segundo plano según la asignación de CPU.
3. Desplegar revisiones con reparto de tráfico y una URL propia para probar antes de dirigir usuarios.
4. Autenticar llamadas entre servicios con testigos de identidad en vez de dejarlos públicos.
5. Elegir entre salida directa a la VPC y conector, y entre servicio, trabajo y pasarela de API.

Conceptos centrales

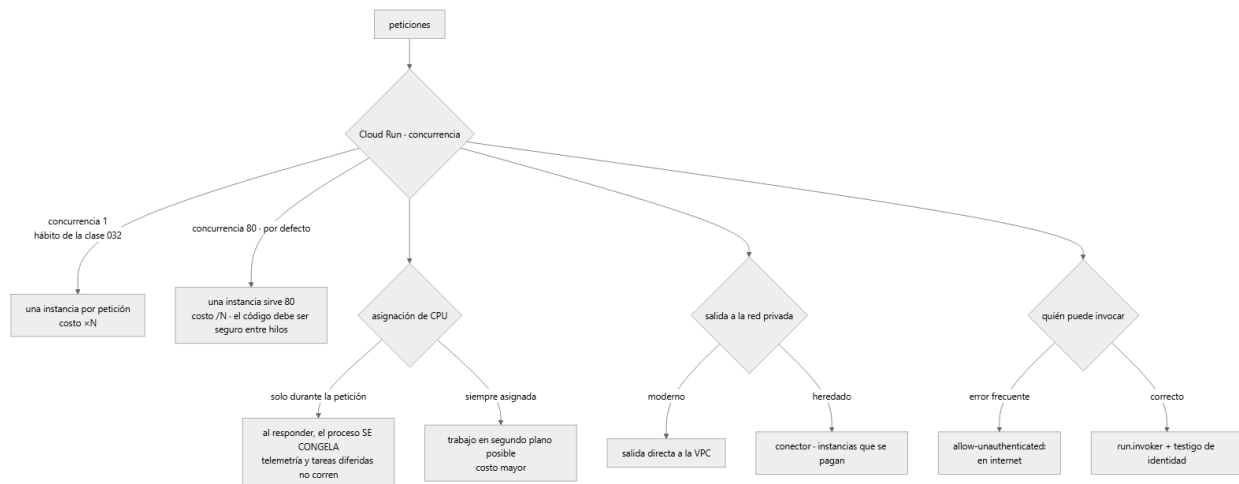
Concepto	Comprensión verificable
concurrencia por instancia	Número de peticiones que una misma instancia atiende a la vez, 80 por defecto. Es lo que separa el modelo de Cloud Run del de una función por petición, y lo que divide el costo.
asignación de CPU	Por defecto solo durante la petición. Fuera de ella el proceso está congelado: lo que se programó para «después de responder» no se ejecuta hasta la petición siguiente, si la hay.
instancias mínimas	Instancias mantenidas vivas para evitar el arranque en frío. Convierten la latencia del arranque en un costo fijo, con el mismo criterio de la clase 032.
revisión con etiqueta	Versión inmutable con URL propia. Permite probar una revisión con tráfico real dirigido a mano antes de asignarle un porcentaje del tráfico general.
testigo de identidad	Credencial que un servicio presenta a otro para demostrar quién es. Con roles/run.invoker, sustituye a dejar el servicio abierto a internet.
trabajo de Cloud Run	Ejecución por lotes con tareas paralelas y reintentos, sin servidor HTTP. Es la respuesta al proceso largo que en la clase 043 chocaba con un tope de diez minutos.

Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Una instancia atiende ochenta peticiones, y eso cambia la aritmética

La clase 032 estableció que en serverless la concurrencia es el recurso que se agota, con un modelo donde cada petición ocupa un entorno de ejecución. La clase 043 mantuvo esa forma. Cloud Run funciona al revés:

modelo de función por petición 1 petición → 1 entorno ocupado
Cloud Run N peticiones → 1 instancia, hasta la concurrencia

La consecuencia inmediata es de costo, y es grande. Con 10 millones de peticiones al mes de 200 ms, 1 vCPU y 512 MiB:

concurrencia 1	
10.000.000 × 0,2 s = 2.000.000 vCPU-s × 0,000024 =	48,00 USD
memoria	+ 2,50
peticiones 10 M × 0,40/millón	+ 4,00
	54,50 USD/mes

concurrencia 80 (con empaquetado imperfecto y algo de tiempo ocioso)
~8,90 USD/mes

Seis veces menos por un parámetro. Y el error que produce la cifra alta es exactamente el que comete quien llega de las clases 032 o 043: poner la concurrencia a 1 «para que cada petición esté aislada», que es como funcionaba allí.

Pero el aislamiento que se pierde es real y hay que entenderlo, porque la concurrencia impone dos requisitos al código:

1. tiene que ser seguro entre peticiones simultáneas
una variable global mutable la comparten 80 peticiones a la vez
2. su consumo de memoria y de conexiones se multiplica por la concurrencia
80 peticiones × una conexión a la base de datos cada una = 80 conexiones por instancia

La segunda es la que produce el incidente sutil: un servicio con 20 instancias y concurrencia 80 puede abrir 1.600 conexiones a una base de datos que admite 200. La corrección es un grupo de conexiones acotado por instancia, dimensionado como máximo de conexiones / instancias máximas, y no como «una por petición».

La primera produce algo peor: si un objeto global guarda estado de la petición en curso —un cliente con una cabecera de usuario, un contexto que se reasigna—, dos peticiones simultáneas se pisan y un usuario ve datos de otro. No hay error, no hay traza: hay una respuesta incorrecta. Es el tipo de fallo que aparece solo bajo carga y desaparece al reproducirlo.

La forma de elegir el valor no es dejar el que viene:

servicio con trabajo ligado a E/S (espera a la base de datos, a otra API)
→ concurrencia alta: la instancia está esperando, no calculando
servicio con trabajo intensivo de CPU (imágenes, cifrado, compresión)
→ concurrencia baja: las peticiones compiten por el mismo núcleo
servicio con memoria proporcional a la petición
→ concurrencia limitada por la memoria de la instancia

Y se comprueba midiendo, no razonando: se sube la concurrencia y se observa el percentil 95. Cuando empieza a subir, se ha pasado el punto donde las peticiones compiten.

2. La CPU solo durante la petición: el trabajo que nunca se ejecuta

Este es el fallo más desconcertante de la clase, porque no produce ningún error y desaparece cuando se investiga.

Por defecto, Cloud Run asigna CPU únicamente mientras se procesa una petición. En cuanto el servicio responde, el proceso se congela: sigue en memoria y no ejecuta nada.

```
recibe petición → CPU asignada → procesa → responde → CPU RETIRADA
                                     el proceso se congela
```

Todo lo que el código haga «después de responder» queda pendiente hasta la petición siguiente:

volcado de telemetría por lotes	los registros no aparecen, o aparecen tarde y atribuidos a otra petición
métricas acumuladas en memoria	se pierden si la instancia se retira
tareas diferidas tras responder	no se ejecutan
mantenimiento del grupo de conexiones	no se ejecuta: conexiones muertas
reintentos programados	no se ejecutan

El síntoma que llega al equipo suele ser «faltan registros» o «la última petición de cada rato falla con conexión cerrada», y ninguno de los dos apunta a la causa.

Hay dos respuestas, y la elección tiene precio:

```
# opción A: hacer el trabajo ANTES de responder
#   correcto, y añade latencia a la petición

# opción B: CPU siempre asignada
$ gcloud run deploy svc-tienda --no-cpu-throttling --min-instances 1
```

La opción B cobra la CPU también fuera de la petición, a una tarifa menor, y es la única que permite trabajo real en segundo plano. Es obligatoria si el servicio mantiene una conexión persistente, consume de una suscripción o procesa en un hilo propio.

La regla de decisión, corta:

```
servicio HTTP puro, sin nada que hacer entre peticiones    por defecto
cualquier cosa que deba ocurrir sin una petición en curso  CPU siempre asignada
```

Y sobre el arranque en frío, el método de la clase 032 se aplica sin cambios —comparar con el percentil del SLO, no con la media— y las palancas aquí son tres:

```
$ gcloud run deploy svc-tienda \
  --min-instances 2 \           # instancias vivas: latencia → costo fijo
  --cpu-boost \                 # más CPU durante el arranque
  --execution-environment gen2
```

La imagen del contenedor también pesa: una imagen de 900 MB arranca claramente más despacio que una de 80 MB, y esa es una optimización que no cuesta dinero, solo un Dockerfile con varias etapas.

Y un tercer modo de ejecución que resuelve el problema con el que chocó la clase 043 —el proceso de catorce minutos contra un tope de diez—: los trabajos de Cloud Run.

```
$ gcloud run jobs create cierre-mensual --image $IMAGEN \
  --tasks 12 --parallelism 4 --max-retries 3 --task-timeout 3600
$ gcloud run jobs execute cierre-mensual --wait
```

No hay servidor HTTP, hay tareas con índice, paralelismo, reintentos y un tiempo límite de horas. Es la forma correcta de ejecutar un cierre, una migración o una reindexación, y evita la deformación de convertir un proceso por lotes en un servicio web que se llama a sí mismo.

3. Revisiones, reparto de tráfico y quién puede invocar

Cada despliegue crea una revisión inmutable, y el tráfico se dirige por porcentaje o por etiqueta:

```
$ gcloud run deploy svc-tienda --image $IMAGEN --tag rc8 --no-traffic
# → https://rc8---svc-tienda-xxxx.a.run.app : URL propia, 0 % del tráfico

$ gcloud run services update-traffic svc-tienda --to-tags rc8=10
$ gcloud run services update-traffic svc-tienda --to-revisions svc-tienda-rc8=100
$ gcloud run services update-traffic svc-tienda --to-revisions svc-tienda-rc7=100 # vuelta atrás
```

La etiqueta con `--no-traffic` es la pieza que faltaba en los espacios de la clase 043: una URL real, con tráfico real dirigido a mano, sin que ningún usuario llegue a ella por accidente. El canario posterior por porcentaje es el mismo mecanismo de las revisiones de Container Apps, y con la misma condición: hace falta una señal por revisión para que repartir 90/10 signifique algo. La clase 057 monta esa parte.

La vuelta atrás es un cambio de porcentaje, así que dura segundos. Es la misma propiedad que hacía valiosos los espacios de App Service, sin el problema de los ajustes que viajan.

Y ahora la decisión de seguridad que más veces está mal en despliegues nuevos:

```
$ gcloud run deploy svc-pedidos --allow-unauthenticated
```

Esa opción publica el servicio en internet, sin autenticación, con una URL adivinable por su patrón. Es correcta para el frontal público y es un error para todo lo demás: un servicio interno desplegado así es una API abierta que ningún firewall protege, porque no está en la VPC.

El patrón correcto para llamadas entre servicios:

```
$ gcloud run deploy svc-pedidos --no-allow-unauthenticated \
  --service-account sa-pedidos@cls-tienda-prod-euw1-01.iam.gserviceaccount.com
$ gcloud run services add-iam-policy-binding svc-pedidos \
  --member "serviceAccount:sa-tienda@cls-tienda-prod-euw1-01.iam.gserviceaccount.com" \
  --role roles/run.invoker
```

Y el llamante presenta un testigo de identidad obtenido de su propia cuenta de servicio, sin ninguna credencial almacenada:

```
import google.auth.transport.requests, google.oauth2.id_token

destino = "https://svc-pedidos-xxxx.a.run.app"
testigo = google.oauth2.id_token.fetch_id_token(
    google.auth.transport.requests.Request(), destino)
respuesta = requests.post(f"{destino}/pedidos", json=datos,
    headers={"Authorization": f"Bearer {testigo}"})
```

La prueba negativa correspondiente es de una línea y debería estar en el guion de verificación:

```
$ curl -s -o /dev/null -w '%{http_code}\n' https://svc-pedidos-xxxx.a.run.app/pedidos
403 ✓
```

Para la salida hacia la red privada —alcanzar Cloud SQL o Memorystore de la clase 054— hay dos mecanismos y uno está obsoleto:

```
salida directa a la VPC      la instancia obtiene una dirección de tu subred
                             sin infraestructura intermedia que pagar, escala mejor
conector de acceso serverless máquinas gestionadas que hacen de puente
                             se pagan por hora y son un cuello de botella propio

$ gcloud run deploy svc-pedidos --network vpc-cloudshop \
  --subnet snet-serverless-euw1 --vpc-egress private-ranges-only
```

`private-ranges-only` envía por la VPC solo lo dirigido a rangos privados y deja salir el resto por el camino de siempre; `all-traffic` lo manda todo, que es lo que hace falta si la salida debe pasar por Cloud NAT para tener una dirección de origen estable. La subred necesita espacio suficiente: es otra reserva que la clase 051 debía haber previsto.

4. Funciones y pasarelas: menos productos de los que parece

Dos aclaraciones ahorran una discusión de arquitectura entera.

Las funciones de segunda generación son Cloud Run. Se ejecutan sobre la misma plataforma, con el mismo modelo de concurrencia, la misma asignación de CPU y las mismas revisiones. Lo que cambia es el empaquetado: en vez de una imagen de contenedor se entrega código fuente y la plataforma construye la imagen.

función	despliegas código; la plataforma construye
	disparadores integrados de eventos
servicio	despliegas una imagen; controlas el entorno completo

La consecuencia práctica es que la elección es de empaquetado, no de capacidades, y que todo lo aprendido sobre concurrencia y CPU se aplica igual a las dos. Y que migrar de una a otra es cambiar cómo se despliega, no rehacer la aplicación.

Los disparadores de eventos son la parte propia, y conectan con la clase 056:

```
$ gcloud functions deploy indexar-factura --gen2 --runtime python312 \
--trigger-event-filters="type=google.cloud.storage.object.v1.finalized" \
--trigger-event-filters="bucket=cls-facturas" \
--run-service-account sa-indexador@cls-tienda-prod-euw1-01.iam.gserviceaccount.com
```

La pasarela de API hace falta menos veces de las que se cree. Hay tres productos y conviene situarlos antes de elegir:

	Qué aporta	Cuándo
Balanceador global + Cloud Armor	Enrutamiento, TLS, WAF, CDN, una IP global	La mayoría de los casos
API Gateway	Contrato OpenAPI, claves de API, autenticación por API	Exponer API a terceros con contrato
Apigee	Gestión completa: monetización, portal, cuotas por cliente	Programa de API como producto

La primera fila cubre lo que la mayoría de los equipos busca cuando dice «necesitamos una pasarela»: un punto de entrada, TLS, reglas de seguridad y reparto por ruta. Eso ya está montado desde la clase 052 y no añade un salto más en la ruta de la petición.

API Gateway aporta algo distinto y concreto: un contrato declarado que valida las peticiones y gestiona claves para consumidores externos. Si no hay terceros ni contrato que imponer, añade latencia y una pieza más que operar.

Y Apigee resuelve un problema de negocio —API como producto, con planes y facturación— cuyo precio solo se justifica si ese negocio existe.

La regla que evita el sobrediseño: añade una pasarela cuando puedas nombrar la función concreta que hace y que no hace el balanceador. Si no se puede nombrar, no hace falta.

5. Qué cuesta y qué se mide

El modelo de precio de Cloud Run tiene tres componentes y una decisión que los domina:

```
CPU          ~0,000024 USD por vCPU-segundo mientras hay asignación
memoria      ~0,000025 USD por GiB-segundo
peticiones  ~0,40 USD por millón
```

Y la decisión que los domina es la concurrencia, porque divide los dos primeros. Las palancas, en orden de efecto:

1. concurrencia divide el costo por instancia-segundo
2. tamaño de la instancia 1 vCPU y 512 MiB suele bastar; medir antes de subir
3. instancias mínimas cuestan cuando no hay tráfico: solo donde el SLO lo exige
4. asignación de CPU «siempre» cuesta más: solo donde haga falta
5. tamaño de la imagen no cuesta dinero directamente; acorta el arranque

Y una comparación honesta con lo construido en las clases 032 y 043, para el mismo servicio y el mismo volumen:

función por petición (clase 032)	~54 USD	sí
plan de App Service con instancias fijas	~124 USD	no
Cloud Run, concurrencia 80, sin mínimos	~9 USD	sí
Cloud Run, concurrencia 80, 2 mínimas	~38 USD	no

La última fila es la que suele elegirse en producción: elimina el arranque en frío y sigue costando menos de un tercio que un plan con instancias fijas. Y la tercera es la que hace que un entorno de pruebas o un servicio poco usado cueste prácticamente nada.

Lo que hay que medir, y que no es lo mismo que en las plataformas anteriores:

utilización de la concurrencia	peticiones simultáneas frente al máximo si nunca se acerca, la concurrencia sobra y hay instancias de más
instancias activas	la curva que explica la factura
tiempo de arranque de instancia	el que decide si hacen falta mínimas
peticiones con arranque en frío	proporción, no media
conexiones abiertas por instancia	el número que hay que multiplicar por el máximo de instancias

La última merece una alerta propia, porque su fallo no ocurre en el servicio sino en la base de datos, y llega justo en el pico: el máximo de instancias multiplicado por las conexiones por instancia debe caber en el límite del motor. Es una cuenta de una línea que casi nunca se hace y que la clase 054 dejó preparada.

```
máximo de instancias (100) × conexiones por instancia (10) = 1.000
límite de la instancia de Cloud SQL = 200
→ el servicio puede tumbar su propia base de datos al escalar
```

La corrección es acotar las dos cifras a la vez —max-instances y el tamaño del grupo de conexiones—, y no solo una.

Ejemplo trabajado

CloudShop despliega su capa de aplicación en Cloud Run. La traducción desde las clases 032 y 043 funciona a la primera y cuesta seis veces de más; los cuatro problemas siguientes son consecuencias del modelo que nadie había tenido que considerar antes.

Despliegue inicial, traducido literalmente:

```
svc-tienda      concurrencia 1 · "para que cada petición esté aislada"
svc-pedidos     concurrencia 1 · --allow-unauthenticated
svc-informes    concurrencia 1 · proceso de cierre partido en trozos de 9 min
conector de acceso serverless para llegar a Cloud SQL
```

Problema 1 — la factura del primer mes.

svc-tienda	(10,2 M peticiones)	54,80 USD	8,90 USD
svc-pedidos	(3,1 M)	17,40 USD	3,20 USD
svc-informes	(0,4 M)	9,10 USD	2,10 USD
		81,30 USD	14,20 USD

El cambio fue un parámetro. Y la comprobación de que la concurrencia elegida es correcta se hizo midiendo, no razonando:

concurrencia	p95	instancias activas en el pico
1	118 ms	210
40	121 ms	7
80	124 ms	4
200	347 ms	2

← aquí las peticiones ya compiten

Se fija en 80. El percentil apenas se mueve y las instancias caen de 210 a 4.

Problema 2 — faltan registros y las conexiones mueren.

Dos síntomas aparentemente sin relación:

los registros de una petición aparecen atribuidos a la SIGUIENTE
la primera petición tras un rato de silencio falla con "connection reset"

Ambos tienen la misma causa: la CPU solo se asigna durante la petición, así que el volcado de telemetría por lotes y el mantenimiento del grupo de conexiones quedaban congelados hasta la petición siguiente.

asignación de CPU	solo en petición	siempre asignada
instancias mínimas	0	1
volcado de telemetría	por lotes	síncrono antes de responder
registros mal atribuidos	~7 %	0
fallos de primera petición	1 de 12	0
costo adicional	—	+11 USD/mes

Once dólares al mes por eliminar dos clases de fallo que consumieron una semana de investigación entre los dos.

Problema 3 — un cliente ve el pedido de otro.

Un cliente reporta ver, durante un segundo, datos que no eran suyos. Ocurre menos de una vez cada diez mil peticiones y solo en horas punta.

```
# el cliente HTTP era global y guardaba la cabecera de la petición en curso
cliente = ApiClient()

def manejar(peticion):
    cliente.headers["X-Usuario"] = peticion.usuario      # ← compartido entre 80
    return cliente.get("/perfil")
```

Con concurrencia 1 esto era correcto. Con concurrencia 80, ochenta peticiones comparten ese objeto y se pisan la cabecera.

estado por petición	en objeto global	pasado como argumento
pruebas de concurrencia	ninguna	carga con 200 simultáneas y verificación de aislamiento
incidencias reportadas	3	0

La medida que evita la repetición no es la corrección: es la prueba de carga con verificación de aislamiento, que comprueba que cada respuesta corresponde a su petición. Sin ella, el mismo defecto vuelve con el siguiente objeto

global.

Problema 4 — la API interna estaba en internet.

```
$ curl -s -o /dev/null -w '%{http_code}\n' https://svc-pedidos-xxxx.a.run.app/pedidos
200
```

Desplegado con `--allow-unauthenticated`, sin ninguna red que lo protegiera, porque Cloud Run no está en la VPC.

servicios públicos	3 de 3	1 de 3
autenticación entre servicios	ninguna	testigo de identidad
permiso de invocación	allUsers	roles/run.invoker
		a cuentas concretas
prueba negativa	no	sí, 403 sin testigo

Problema 5 — el conector y el cierre mensual.

Dos correcciones menores con efecto medible:

salida a la red privada	conector (2 instancias)	salida directa
costo del conector	~74 USD/mes	0
cierre mensual	7 llamadas HTTP encadenadas con estado en un bucket	trabajo de Cloud Run
		12 tareas, 4 en paralelo
duración del cierre	52 min	14 min
fallos parciales sin reintento	sí, a mano	reintento automático

El cierre mensual era el proceso que en la clase 043 había obligado a usar Durable Functions por el tope de diez minutos. Aquí el mecanismo correcto existe de fábrica y además paraleliza.

Resumen de la capa de aplicación:

conurrencia por instancia	1	80
instancias activas en el pico	210	4
servicios expuestos a internet	3 de 3	1 de 3
registros mal atribuidos	~7 %	0
incidencias de datos cruzados	3	0
duración del cierre mensual	52 min	14 min
costo mensual de la capa de aplicación	155 USD	64 USD

La lección que esta clase traslada al resto de la parte 04: los tres primeros problemas tienen la misma raíz y no la tenía ninguna plataforma anterior — una instancia sirve a muchas peticiones a la vez, y el proceso se congela cuando no hay ninguna. Ese modelo divide el costo por seis y exige del código dos propiedades que antes se podían ignorar: aislamiento entre peticiones simultáneas y ningún trabajo que dependa de ejecutarse «después». No es una configuración: es un contrato con la aplicación, y el único de los tres proveedores que lo pide de forma explícita.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/055-cloud-run-cloud-functions-y-api-gateway/lab.py
```

El laboratorio selecciona el motor de práctica serverless y produce `labresult.json`. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee `exercise.steps` y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de `negativetest` y explica la señal observada.
4. Materializa `api-serverless-gcp` en `evidence/` usando la plantilla indicada.
5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es una función con límites, reintentos e idempotencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `api-serverless-gcp` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El costo por petición es varias veces mayor de lo esperado	La concurrencia se puso a 1 por costumbre del modelo de una función por petición	Sube la concurrencia midiendo el percentil 95: si no empeora, la instancia estaba desaprovechada.
Faltan registros o aparecen atribuidos a la petición siguiente	La CPU solo se asigna durante la petición y el volcado por lotes queda congelado	Vuelca antes de responder o activa la CPU siempre asignada; el trabajo en segundo plano exige lo segundo.
Bajo carga, una respuesta contiene datos de otra petición	Un objeto global mutable se comparte entre las peticiones simultáneas de la instancia	Pasa el estado por argumento y añade una prueba de carga que verifique el aislamiento entre respuestas.
Un servicio interno responde desde internet	Se desplegó con <code>--allow-unauthenticated</code> y Cloud Run no está dentro de la VPC	Despliega sin acceso anónimo, concede roles/run.invoker a las cuentas llamantes y verifica que sin testigo devuelve 403.
La base de datos agota sus conexiones cuando el servicio escala	Conexiones por instancia multiplicadas por el máximo de instancias supera el límite del motor	Acota <code>--max-instances</code> y el tamaño del grupo de conexiones a la vez, con la cuenta hecha explícitamente.
Un proceso por lotes se implementa como una cadena de llamadas HTTP con estado externo	Se arrastró la limitación de tiempo de una plataforma anterior	Usa un trabajo de Cloud Run con tareas, paralelismo y reintentos; el tiempo límite es de horas.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Cómo cambia el costo al pasar la concurrencia de 1 a 80, y qué dos requisitos impone eso al código?
2. ¿Qué deja de ejecutarse con la asignación de CPU por defecto, y qué dos síntomas produce?
3. ¿Cómo pruebas una revisión con tráfico real sin que ningún usuario llegue a ella?
4. ¿Por qué un servicio interno con `--allow-unauthenticated` no está protegido por ninguna regla de red?
5. Con 100 instancias máximas y 10 conexiones por instancia, ¿qué comprobación hay que hacer contra la base de datos?

Referencias

- Google Cloud (2025). About instance concurrency in Cloud Run — concurrencia, efecto en costo y requisitos del código. <<https://cloud.google.com/run/docs/about-concurrency>>
- Google Cloud (2025). CPU allocation in Cloud Run — CPU durante la petición frente a siempre asignada. <<https://cloud.google.com/run/docs/configuring/cpu-allocation>>
- Google Cloud (2025). Rollbacks, gradual rollouts and traffic migration — revisiones, etiquetas y reparto de tráfico. <<https://cloud.google.com/run/docs/rollouts-rollbacks-traffic-migration>>
- Google Cloud (2025). Authenticating service-to-service — testigos de identidad y roles/run.invoker. <<https://cloud.google.com/run/docs/authenticating/service-to-service>>
- Google Cloud (2025). Cloud Run jobs overview — tareas, paralelismo, reintentos y tiempos límite. <<https://cloud.google.com/run/docs/create-jobs>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 04 en PDF · Recorrido de Google Cloud en PDF · Manual integral

Anterior	Índice	Siguiente
← 054 · Cloud SQL, Spanner, Firestore y Memorystore	Parte 04 · Programa	056 · Pub/Sub, Cloud Tasks y Workflows →

Evaluación — 055 Cloud Run, Cloud Functions y API Gateway

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega api-serverless-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

056 — Pub/Sub, Cloud Tasks y Workflows

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: messaging · Estado: EXECUTABLECORE

Propósito

Construir la columna asíncrona de Google Cloud, donde la pregunta que ordenó la clase 044 —¿el mensaje se consume o se observa?— ya no elige entre productos: la responden las suscripciones de un mismo tema. A cambio aparecen tres decisiones que allí no existían: qué tipo de suscripción, quién controla el ritmo de entrega hacia un destino que no escala, y qué significa exactamente la entrega «exactamente una vez» que esta plataforma ofrece y las anteriores no.

Resultados de aprendizaje

Al finalizar podrás:

1. Modelar consumo y observación con temas y suscripciones en vez de con dos productos distintos.
2. Configurar una cola de mensajes fallidos que de verdad reciba mensajes, y demostrarlo.
3. Elegir entre Pub/Sub y Cloud Tasks a partir de quién debe controlar el ritmo de entrega.
4. Explicar qué garantiza la entrega exactamente una vez y qué sigue exigiendo idempotencia.
5. Reproducir un intervalo de mensajes ya procesados para reparar un daño posterior.

Conceptos centrales

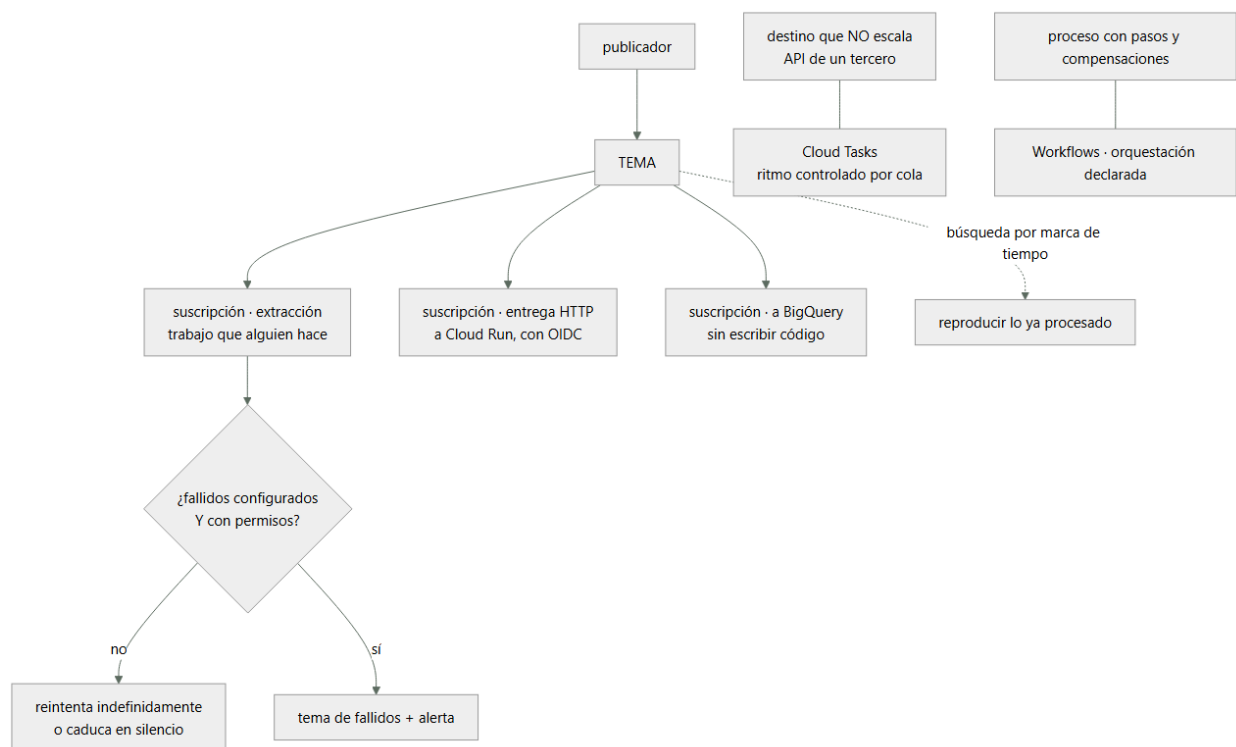
Concepto	Comprensión verificable
tema y suscripción	El tema recibe; cada suscripción tiene su propia copia y su propio estado de confirmación. Añadir un consumidor es añadir una suscripción, no otro producto.
suscripción de extracción o de entrega	La primera la consume el cliente; la segunda hace que Pub/Sub llame por HTTP a un extremo, autenticándose. La misma pieza cubre lo que en la clase 044 eran una cola y un enrutador.
plazo de confirmación	Tiempo reservado para procesar antes de reentregar. Las bibliotecas lo amplían solas si el hilo puede ejecutarse: es la tercera versión del mismo problema del programa.
entrega exactamente una vez	Garantía de que Pub/Sub no reentrega un mensaje ya confirmado dentro de una región. No garantiza que el efecto se aplique una sola vez: si el proceso muere tras aplicar y antes de confirmar, se repite.
límite de ritmo de la cola	Capacidad de Cloud Tasks para entregar como mucho N por segundo. Pub/Sub no tiene equivalente: entrega tan rápido como puede.
búsqueda por marca de tiempo	Reposicionar una suscripción en el pasado para volver a entregar lo ya procesado. Es lo que permite reparar un daño detectado tarde.

Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Un producto, y la decisión se traslada a la suscripción

La clase 044 separaba tres servicios con una pregunta: si el mensaje se consume, Service Bus; si se observa, Event Hubs; si se enruta, Event Grid. Aquí hay un tema y la respuesta está en cuántas suscripciones tiene y de qué tipo:

tema "pedidos"

- suscripción "facturacion" extracción · su propio estado de confirmación
- suscripción "almacen" extracción · su propio estado
- suscripción "notificar" entrega HTTP a Cloud Run
- suscripción "analitica" escritura directa a BigQuery, sin código

Cada suscripción recibe una copia y avanza por su cuenta. Eso significa que «observar» —varios interesados leyendo lo mismo a su ritmo— y «consumir» —trabajo que alguien hace una vez— son la misma pieza usada de dos maneras, y que añadir un consumidor nuevo no obliga a elegir otro producto.

Los tipos de suscripción y cuándo usa cada uno:

Tipo	Quién inicia	Encaja con
Extracción	El consumidor	Trabajo por lotes, control de ritmo propio, consumidores largos
Entrega HTTP	Pub/Sub	Cloud Run y funciones: el consumidor escala con la carga
A BigQuery	Pub/Sub	Analítica sin escribir ni operar un consumidor
A Cloud Storage	Pub/Sub	Archivo en bruto, como la captura de la clase 044

Las dos últimas merecen conocerse porque eliminan código: una suscripción a BigQuery sustituye a un consumidor que solo insertaba filas, con su despliegue, su vigilancia y sus fallos.

Y hay una diferencia que hay que anticipar respecto de la clase 044: una suscripción creada hoy no recibe lo publicado ayer. Es el mismo comportamiento de las suscripciones de un tema de Service Bus, y aquí tiene un remedio que allí no existía —la reproducción, que veremos más abajo— siempre que el tema tenga retención configurada.

```

$ gcloud pubsub topics create pedidos --message-retention-duration 7d
$ gcloud pubsub subscriptions create facturacion --topic pedidos \
  --ack-deadline 60 --retain-acked-messages --message-retention-duration 7d

```

Esa retención tiene costo de almacenamiento y es lo que hace posible reparar un error detectado tres días después. Sin ella, un mensaje confirmado desaparece y la única fuente es el sistema que lo originó.

2. La cola de fallidos que no recibe nada

Este es el fallo de configuración más frecuente de Pub/Sub y es completamente silencioso.

A diferencia de Service Bus, donde la subcola de fallidos existe siempre (clase 044), aquí hay que configurarla:

```
$ gcloud pubsub topics create pedidos-fallidos
$ gcloud pubsub subscriptions update facturacion \
  --dead-letter-topic pedidos-fallidos --max-delivery-attempts 5
```

Y ahí termina la mayoría de las configuraciones, que es donde empieza el problema: Pub/Sub necesita permisos propios para publicar en el tema de fallidos y para confirmar en la suscripción de origen. Sin ellos, no hay error visible; el mensaje simplemente se reintenta indefinidamente hasta caducar.

```
$ AGENTE="service-${NUM_PROYECTO}@gcp-sa-pubsub.iam.gserviceaccount.com"
$ gcloud pubsub topics add-iam-policy-binding pedidos-fallidos \
  --member "serviceAccount:$AGENTE" --role roles/pubsub.publisher
$ gcloud pubsub subscriptions add-iam-policy-binding facturacion \
  --member "serviceAccount:$AGENTE" --role roles/pubsub.subscriber
```

Y la prueba negativa, que es lo único que demuestra que funciona:

```
$ gcloud pubsub topics publish pedidos --message '{"roto": true}'
# el consumidor rechaza el mensaje cinco veces
$ gcloud pubsub subscriptions pull fallidos-sub --limit 1 --auto-ack \
  --format="value(message.data)" | base64 -d
{"roto": true}
```

✓

Sin esa comprobación, un equipo puede pasar meses creyendo que tiene una red de seguridad que no existe. Y el resto de la disciplina de la clase 044 se aplica igual, empezando por la alerta:

```
mensajes en el tema de fallidos > 0 durante N minutos → aviso
edad del mensaje más antiguo sin confirmar > umbral → aviso
```

La segunda es la que detecta el sistema que dejó de trabajar sin caerse, que es el fallo silencioso que este programa ha encontrado ya en tres plataformas.

El plazo de confirmación repite el patrón del bloqueo de Service Bus, con los mismos números que importan:

```
por defecto          10 s
máximo por mensaje   600 s
las bibliotecas lo amplían solas mientras el mensaje se procesa
→ SI el hilo puede ejecutarse
```

Y con suscripción de entrega HTTP hay una precisión propia: el plazo de confirmación es el tiempo que Pub/Sub espera la respuesta HTTP. Un manejador en Cloud Run que tarda 90 segundos con un plazo de 60 recibe una reentrega mientras sigue trabajando — el mismo escenario que en la clase 044 cobró dos pedidos duplicados. La corrección es la misma que allí y no ha cambiado en tres plataformas: el manejador registra la intención y devuelve; el trabajo largo ocurre fuera.

Y el control de flujo del cliente, que en las plataformas anteriores no era tan visible:

```
from google.cloud import pubsub_v1

flujo = pubsub_v1.types.FlowControl(max_messages=100, max_bytes=50 * 1024 * 1024)
suscriptor.subscribe(ruta, callback=manejar, flow_control=flujo)
```

Sin acotarlo, un consumidor que se reincorpora tras una caída puede traerse decenas de miles de mensajes a memoria y morir por falta de ella — repitiendo el ciclo indefinidamente. Es un fallo que solo aparece después de un incidente, es decir, en el peor momento.

3. «Exactamente una vez» dicho con precisión

Pub/Sub ofrece entrega exactamente una vez en suscripciones de extracción dentro de una región. Es una capacidad real y conviene decir con exactitud qué cubre, porque la frase invita a concluir de más.

```
$ gcloud pubsub subscriptions create facturacion --topic pedidos \
  --enable-exactly-once-delivery --ack-deadline 60
```

```
lo que garantiza
un mensaje ya confirmado NO se vuelve a entregar
```

no hay reentregas por vencimiento de plazo mal calculado
la confirmación es transaccional: o vale, o falla y lo sabes

lo que NO garantiza
que el EFECTO se aplique una sola vez

La secuencia que lo demuestra, y que ninguna garantía de transporte puede evitar:

1. el consumidor recibe el mensaje
2. aplica el efecto: cobra el pedido
3. el proceso muere antes de confirmar
4. el mensaje NO estaba confirmado → se entrega otra vez
5. otro consumidor vuelve a cobrar

Es el mismo razonamiento de las clases 033 y 044, y el hueco está en el paso 3, que está fuera del alcance del sistema de mensajería. La conclusión del programa se mantiene entera y ahora con tres plataformas detrás:

La entrega exactamente una vez se compra; el efecto exactamente una vez se construye en el manejador.

```
def manejar(mensaje):
    clave = mensaje.attributes["idempotency-key"]
    if repositorio.ya_procesado(clave):
        mensaje.ack(); return
    with repositorio.transaccion():
        aplicar_efecto(mensaje)
        repositorio.marcar_procesado(clave) # en la MISMA transacción
    mensaje.ack()
```

Lo que sí cambia la garantía es el volumen de trabajo del manejador: sin ella, la idempotencia se ejercita constantemente; con ella, solo en el caso raro del fallo entre aplicar y confirmar. Eso reduce la contención sobre la tabla de claves procesadas, que en volumen alto es una diferencia medible.

Y tiene un coste que hay que conocer: la entrega exactamente una vez reduce el rendimiento máximo de la suscripción y solo funciona en extracción. Para un flujo de telemetría de alto volumen donde un duplicado es irrelevante, activarla es pagar por una garantía que nadie necesita.

Sobre el orden, la tercera aparición del mismo compromiso del programa:

```
$ gcloud pubsub topics publish pedidos --message "$CUERPO" --ordering-key "$PEDIDO_ID"
```

el orden se garantiza por CLAVE de ordenación, dentro de una región
y la clave de ordenación es también la unidad de SERIALIZACIÓN

Exactamente igual que la sesión de Service Bus (clase 044) y que la clave de partición de Cosmos DB y Spanner (clases 042 y 054). Una clave demasiado gruesa —el país, la tienda— convierte el consumo en un solo hilo. La regla es la de siempre: se pide orden donde el negocio lo exige, con la clave más fina que lo satisfaga.

4. Cloud Tasks: cuando el destino no puede con el ritmo

Pub/Sub entrega tan rápido como puede. Es lo correcto cuando el consumidor escala, y es un problema cuando el destino no escala: una API de un tercero con un límite de peticiones, un sistema heredado, un servicio con cuota.

Cloud Tasks existe para eso, y su diferencia es la cola con ritmo:

```
$ gcloud tasks queues create envio-facturas \
  --max-dispatches-per-second 10 --max-concurrent-dispatches 5 \
  --max-attempts 8 --min-backoff 10s --max-backoff 600s
```

Pub/Sub abanico de eventos a N interesados, a la velocidad del consumidor
Cloud Tasks envío controlado a UN destino, al ritmo que ese destino aguanta

Y dos capacidades más que Pub/Sub no tiene y que resuelven casos concretos:

Programar una tarea para un instante. No «dentro de un rato» sino a las 09:00 del martes:

```
tarea = {"http_request": {"http_method": "POST", "url": destino, "body": cuerpo},
        "schedule_time": momento,           # instante exacto
        "name": f"{cola}/tasks/recordatorio-{pedido_id}"}
```

Deduplicar por nombre. Dar nombre a la tarea impide crear otra igual durante aproximadamente una hora. Es útil para el caso «si el usuario pulsa cinco veces, se envía un correo», y tiene el mismo límite que la detección de duplicados de la clase 044: es una ventana, no una garantía, y la idempotencia sigue haciendo falta.

La regla de decisión, que evita la discusión:

¿varios interesados en el mismo hecho?	Pub/Sub
¿un destino concreto que hay que llamar?	Cloud Tasks
¿hay que controlar el ritmo o programar el momento?	Cloud Tasks
¿hace falta reproducir el histórico?	Pub/Sub

Y Workflows cubre el tercer caso, el que la clase 032 planteó como orquestación frente a coreografía: un proceso con pasos, condiciones, compensaciones y espera.

```
main:
  params: [pedido]
  steps:
    - reservar_stock:
        call: http.post
        args: {url: "${ALMACEN}/reservar", body: "${pedido}"}
        result: reserva
        retry: {predicate: "${http.default_retry_predicate}", max_retries: 5,
              backoff: {initial_delay: 1, max_delay: 60, multiplier: 2}}
    - cobrar:
        try:
          call: http.post
          args: {url: "${PAGOS}/cobrar", body: "${pedido}"}
        except:
          as: e
          steps:
            - compensar:
                call: http.post
                args: {url: "${ALMACEN}/liberar", body: "${reserva}"}
            - propagar:
                raise: "${e}"
    - confirmar:
        return: "${pedido.id}"
```

Lo que aporta frente a encadenar mensajes es que la compensación está escrita en un sitio y se puede leer. En una coreografía por eventos, la respuesta a «qué pasa si el cobro falla después de reservar» está repartida entre tres servicios y nadie la tiene entera.

Su precio es por paso ejecutado —del orden de céntimos por millar—, así que es barato para procesos de negocio y caro para bucles con miles de iteraciones. Y su límite es el mismo de toda orquestación: introduce un componente que conoce el proceso completo, lo que hay que asumir a cambio de poder leerlo.

5. Reproducir: la capacidad que repara un daño detectado tarde

Esta es la propiedad que en la clase 044 no existía y que conviene tener presente antes de necesitarla.

Una suscripción puede reposicionarse en el pasado:

```
# volver a entregar todo lo publicado desde un instante
$ gcloud pubsub subscriptions seek facturacion \
  --time 2026-07-28T02:00:00Z

# o saltar hacia adelante para descartar lo pendiente
$ gcloud pubsub subscriptions seek facturacion --time $(date -u +%Y-%m-%dT%H:%M:%SZ)
```

Y con instantáneas, se puede guardar el estado de confirmación de una suscripción antes de un despliegue arriesgado y volver a él:

```
$ gcloud pubsub snapshots create antes-de-v8 --subscription facturacion
# ... despliegue, y si sale mal:
$ gcloud pubsub subscriptions seek facturacion --snapshot antes-de-v8
```

Eso convierte un fallo lógico en un incidente reparable. El caso típico: un cambio introduce un error de cálculo que corrompe datos durante dos días, y se detecta al tercero. Sin reproducción, la reparación es un guion a medida que lee del sistema de origen. Con reproducción, es corregir el consumidor y reposicionar la suscripción.

Con dos condiciones que hay que haber cumplido antes:

1. el tema y la suscripción deben tener retención suficiente
la ventana de reparación es exactamente la retención configurada
2. el consumidor debe ser IDEMPOTENTE
reproducir entrega mensajes que YA se procesaron

La segunda es la que convierte la capacidad en utilizable. Un consumidor no idempotente enfrentado a una reproducción duplica todos los efectos del intervalo, así que la herramienta de reparación se convierte en la causa de

un daño mayor. Es el mismo argumento de idempotencia por cuarta vez en el programa, ahora con una razón nueva: no solo por los duplicados que ocurren, sino por los que tú vas a provocar a propósito el día que necesites reparar algo.

Y el resto del ecosistema encaja alrededor. Eventarc unifica el enrutamiento de eventos de la plataforma —un blob nuevo en un bucket de la clase 053, un cambio de configuración auditado de la clase 049— hacia Cloud Run o hacia Workflows:

```
$ gcloud eventarc triggers create indexar-factura \
  --destination-run-service svc-indexador --destination-run-region europe-west1 \
  --event-filters="type=google.cloud.storage.object.v1.finalized" \
  --event-filters="bucket=cls-facturas" \
  --service-account sa-eventarc@cls-tienda-prod-euw1-01.iam.gserviceaccount.com
```

Por debajo eso crea un tema y una suscripción de entrega, así que todo lo de esta clase se aplica igual: plazo de confirmación, fallidos con permisos, idempotencia y alertas. Saberlo evita tratarlo como una caja distinta cuando algo va mal.

Ejemplo trabajado

CloudShop monta su columna asíncrona en Google Cloud con el contrato de la clase 044 en la mano. Dos problemas conocidos se evitan de entrada, uno reaparece con otra cara, y aparecen dos propios de la plataforma — uno de ellos completamente silencioso.

Lo que se evitó por venir escrito.

manejadores idempotentes desde el primer día	(044)
alerta sobre mensajes fallidos con umbral cero	(044)
el manejador registra la intención y devuelve;	
el trabajo largo ocurre fuera	(044)
clave de ordenación fina: pedidoId, no pais	(042, 044, 054)

Cuatro decisiones tomadas antes de desplegar, cada una por un incidente pagado en otra plataforma.

Problema 1 — la cola de fallidos llevaba tres semanas sin recibir nada.

La alerta de mensajes fallidos nunca se disparó. El equipo lo interpretó como buena señal, hasta que una revisión encontró mensajes con más de 400 intentos de entrega:

```
$ gcloud pubsub subscriptions pull facturacion --limit 1 --format="value(deliveryAttempt)"
412
```

Cuatrocientos doce intentos con --max-delivery-attempts 5 configurado. La causa: el agente de servicio de Pub/Sub no tenía permiso para publicar en el tema de fallidos, así que el traslado fallaba y el mensaje volvía a la cola.

permisos del agente de servicio	ninguno	publisher + subscriber
mensajes con más de 5 intentos	1.841	0
mensajes en el tema de fallidos	0	1.841 trasladados
prueba negativa de la cola de fallidos	no	sí, ejecutada

El equipo añade la comprobación a su lista de plataforma nueva, en la forma que ya tienen para otras: una cola de fallidos sin una prueba que demuestre que recibe es una cola de fallidos que no existe.

Problema 2 — el proveedor de envíos bloquea a CloudShop por exceso de peticiones.

El servicio de notificación de envíos usaba una suscripción de entrega HTTP hacia un servicio que llamaba a la API del transportista. En una campaña, Pub/Sub entregó todo lo acumulado de golpe:

peticiones al transportista en 60 s	4.312
límite contratado	600 por minuto
resultado	bloqueo de la cuenta durante 2 h

Pub/Sub no tiene control de ritmo: entrega tan rápido como el consumidor acepta, y el consumidor escalaba. Se cambia la pieza:

mecanismo	Pub/Sub → Cloud Run → API del transportista	Pub/Sub → Cloud Run → Cloud Tasks → API del transportista
ritmo hacia el transportista	sin control	10 por segundo, 5 simultáneas
reintentos	del consumidor	de la cola, con retroceso
bloqueos del proveedor	1	0

La lección se generaliza: el ritmo lo tiene que controlar quien conoce el límite del destino, y ese no es el sistema de mensajería.

Problema 3 — el plazo de confirmación, por tercera vez.

El consumidor de facturación tardaba entre 40 y 95 segundos, con un plazo de 60.

mensajes reentregados mientras seguían procesándose	214 en 24 h
facturas duplicadas	0

Cero facturas duplicadas, porque los manejadores eran idempotentes desde el primer día. El daño fue trabajo desperdiciado, no datos incorrectos — que es exactamente la diferencia que la idempotencia compra.

plazo de confirmación	60 s	600 s
trabajo dentro del manejador	generar la factura	registrar y devolver
generación de la factura	—	trabajo de Cloud Run (055)
reentregas mientras se procesa	214	0

Problema 4 — dos días de comisiones mal calculadas.

Un cambio en el cálculo de comisiones introdujo un error que se detectó al tercer día: 61.000 pedidos con la comisión mal.

En la plataforma anterior esto habría exigido un guion de reparación leyendo del sistema de pedidos. Aquí:

```
$ gcloud pubsub snapshots create antes-de-reparar --subscription comisiones
$ gcloud pubsub subscriptions seek comisiones --time 2026-07-26T00:00:00Z
```

mensajes reentregados	61.412	
duración del reproceso	38 min	
efectos duplicados	0	← por la idempotencia
comisiones corregidas	61.412	
reparación manual necesaria	ninguna	

La fila de los efectos duplicados es la que hace que esto funcione. La idempotencia, que hasta ahora se justificaba por los duplicados que ocurren, aquí se justifica por los duplicados que se provocan a propósito. Sin ella, la herramienta de reparación habría multiplicado el daño por dos.

Y una simplificación que quitó código.

El consumidor de analítica solo insertaba filas en el almacén de datos:

consumidor de analítica	servicio propio en Cloud Run	suscripción a BigQuery
líneas de código	310	0
despliegues que mantener	1	0
fallos de ese consumidor en 3 meses	4	—
costo mensual	12 USD	2 USD

Resumen de la columna asíncrona:

productos usados	1 (Pub/Sub)	3, por criterio
mensajes atascados sin llegar a fallidos	1.841	0
bloqueos del proveedor externo	1	0
reentregas por plazo corto	214	0
consumidores escritos a mano	4	3
reparación de un daño de 2 días	no posible	38 min, sin duplicados
costo mensual de mensajería	47 USD	29 USD

La lección que esta clase traslada al resto de la parte 04: el contrato de la clase 044 se reutilizó entero y evitó cuatro incidentes antes de ocurrir, y los dos problemas nuevos fueron de la misma familia que los de las plataformas anteriores — una red de seguridad que parecía existir y no existía, y un destino que no aguantaba el ritmo de quien le enviaba. Lo genuinamente nuevo fue la capacidad de reproducir, y su valor dependió por completo de una propiedad que ya estaba escrita en el contrato: sin manejadores idempotentes, reproducir habría sido peor que no poder hacerlo.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/056-pub-sub-cloud-tasks-y-workflows/lab.py
```

El laboratorio selecciona el motor de práctica messaging y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee `exercise.steps` y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de `negativetest` y explica la señal observada.
4. Materializa `flujo-eventos-gcp` en `evidence/` usando la plantilla indicada.
5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es un flujo que declara orden, entrega y manejo de errores. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `flujo-eventos-gcp` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
La cola de mensajes fallidos nunca recibe nada y hay mensajes con cientos de intentos	El agente de servicio de Pub/Sub no tiene permiso para publicar en el tema de fallidos ni para confirmar en el origen	Concede <code>publisher</code> y <code>subscriber</code> al agente y demuéstalo publicando un mensaje que el consumidor rechace.
Un proveedor externo bloquea la cuenta por exceso de peticiones	Pub/Sub entrega tan rápido como el consumidor acepta y el destino tiene un límite	Interpón <code>Cloud Tasks</code> con <code>max-dispatches-per-second</code> acorde al límite del destino.
Mensajes reentregados mientras el manejador sigue trabajando	El plazo de confirmación es menor que el tiempo de proceso; en entrega HTTP, ese plazo es el tiempo de espera de la respuesta	Registra la intención y devuelve; ejecuta el trabajo largo en un trabajo aparte y amplía el plazo si hace falta.
Se activa la entrega exactamente una vez y sigue habiendo efectos duplicados	La garantía es de entrega, no de aplicación: un fallo entre aplicar y confirmar reentrega	Idempotencia en el manejador, marcando la clave en la misma transacción que el efecto.
Un consumidor muere por falta de memoria al reincorporarse tras una caída	Sin control de flujo, se trae a memoria todo el acumulado	Configura <code>maxmessages</code> y <code>maxbytes</code> en el cliente antes de que ocurra el primer incidente.
No se puede reparar un daño detectado tres días después	La retención del tema y de la suscripción era menor que ese plazo	Configura la retención según la ventana de reparación que quieras tener, y verifica que los consumidores son idempotentes antes de reproducir.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Cómo se resuelve con un solo producto la distinción entre consumir y observar de la clase 044?
2. ¿Qué tres cosas hacen falta para que una cola de mensajes fallidos reciba de verdad, y cómo lo demuestras?
3. ¿Qué garantiza exactamente la entrega exactamente una vez y qué secuencia sigue produciendo efectos duplicados?
4. ¿En qué caso concreto Cloud Tasks es la pieza correcta y Pub/Sub no puede sustituirlo?
5. ¿Qué dos condiciones hay que haber cumplido antes para poder reparar un daño reproduciendo mensajes?

Referencias

- Google Cloud (2025). Pub/Sub subscription types — extracción, entrega HTTP, BigQuery y Cloud Storage.
<<https://cloud.google.com/pubsub/docs/subscriber>>
- Google Cloud (2025). Handling message failures — cola de fallidos, permisos del agente de servicio y reintentos.
<<https://cloud.google.com/pubsub/docs/handling-failures>>
- Google Cloud (2025). Exactly-once delivery — alcance de la garantía y sus límites.
<<https://cloud.google.com/pubsub/docs/exactly-once-delivery>>
- Google Cloud (2025). Replay and purge messages — búsqueda por marca de tiempo e instantáneas.
<<https://cloud.google.com/pubsub/docs/replay-overview>>
- Google Cloud (2025). Cloud Tasks overview — control de ritmo, programación y deduplicación por nombre.
<<https://cloud.google.com/tasks/docs/dual-overview>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 04 en PDF · Recorrido de Google Cloud en PDF · Manual integral

Anterior	Índice	Siguiente
← 055 · Cloud Run, Cloud Functions y API Gateway	Parte 04 · Programa	057 · Cloud Logging, Monitoring, Trace y Audit Logs →

Evaluación — 056 Pub/Sub, Cloud Tasks y Workflows

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega flujo-eventos-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.

- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

057 — Cloud Logging, Monitoring, Trace y Audit Logs

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: observability · Estado: EXECUTABLECORE

Propósito

Construir la evidencia operativa de una plataforma de Google Cloud, donde el problema es el contrario al de la clase 045: aquí casi todo se registra desde el primer minuto, así que un incidente nunca se queda sin datos y la factura llega antes que la pregunta. Y donde existe la pieza que faltaba en las dos plataformas anteriores: alertar sobre el consumo del presupuesto de error en lugar de sobre umbrales de recursos, que es la diferencia entre 340 avisos al mes y once que importan.

Resultados de aprendizaje

Al finalizar podrás:

1. Distinguir qué registros están activos y son gratuitos de los que hay que habilitar y se pagan.
2. Reducir la factura de ingesta con filtros de exclusión y destinos, sin perder capacidad de investigación.
3. Convertir un patrón de registro en una métrica para alertar en un minuto en vez de en quince.
4. Definir un SLI y un SLO, y alertar por velocidad de consumo del presupuesto de error.
5. Reconstruir una petición completa entre servicios y localizar el salto que corta la traza.

Conceptos centrales

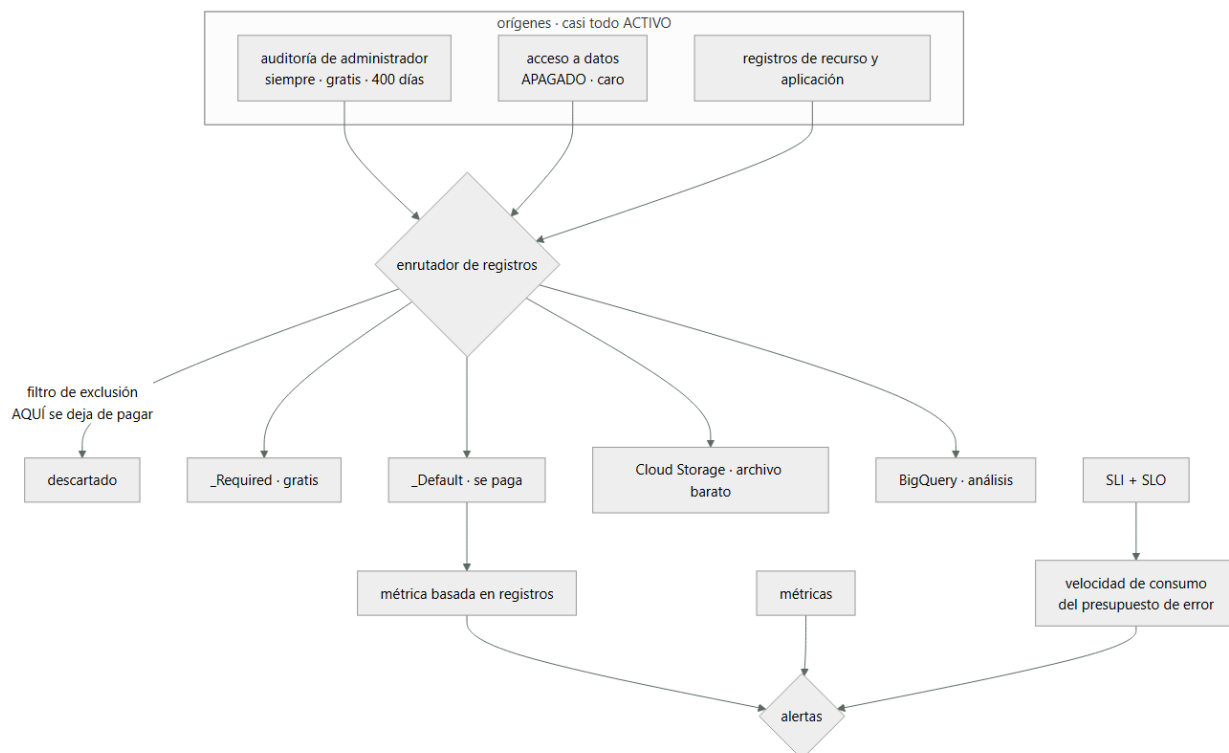
Concepto	Comprensión verificable
registro de auditoría de actividad de administrador	Quién creó, modificó o borró qué. Está siempre activo, es gratuito y se conserva 400 días sin configurar nada — justo lo que en la clase 045 exigía exportar.
registro de acceso a datos	Quién leyó qué dato. Está apagado por defecto —salvo excepciones— y es el más voluminoso: habilitarlo en todo es la forma más rápida de multiplicar la factura.
enrutador de registros	Punto por el que pasa todo antes de almacenarse. Es donde se excluye antes de pagar y donde se decide el destino: bucket de registros, almacenamiento, BigQuery o Pub/Sub.
métrica basada en registros	Contador o distribución extraído de un patrón de texto. Convierte una señal que solo existe en los registros en una métrica sobre la que se alerta en un minuto.
presupuesto de error	Fracción de peticiones que el SLO permite fallar en un periodo. Alertar sobre su velocidad de consumo sustituye a los umbrales de CPU y elimina la mayoría de los avisos inútiles.
perfilador continuo	Muestreo de CPU y memoria en producción con sobrecarga mínima. Responde a «dónde se va el tiempo» sin reproducir nada en un entorno de pruebas.

Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Aquí el problema no es la falta de datos, es la factura

La clase 045 empezaba con una frase que aquí no vale: «nada se registra hasta que alguien lo enciende». En Google Cloud el reparto es el contrario, y conviene conocerlo con precisión porque decide dónde está el riesgo.

siempre activo y GRATUITO

auditoría de actividad de administrador quién creó, modificó o borró
auditoría de eventos del sistema acciones de la propia plataforma
→ se guardan 400 días en el bucket _Required, que no se puede modificar

activo y facturado

registros de aplicación y de la mayoría de servicios
registros de peticiones de los balanceadores y de Cloud Run
auditoría de política denegada

APAGADO por defecto

auditoría de ACCESO A DATOS: quién leyó qué

La primera línea resuelve de fábrica el problema que la clase 045 tuvo que resolver exportando a almacenamiento inmutable: quién borró un recurso hace cuatro meses tiene respuesta, sin haber configurado nada, y sin que nadie pueda borrar esa respuesta.

```
$ gcloud logging read 'protoPayload.methodName:"delete" AND
timestamp>="2026-04-01T00:00:00Z"' --limit 20 \
--format="table(timestamp, protoPayload.authenticationInfo.principalEmail,
protoPayload.methodName, protoPayload.resourceName)"
```

La tercera línea es la asimetría propia, y es una decisión con precio. Los registros de acceso a datos responden a «quién leyó la tabla de clientes», que es la pregunta de cualquier investigación sobre datos personales — y son, con diferencia, los más voluminosos, porque hay muchas más lecturas que cambios de configuración.

```
$ gcloud projects get-iam-policy $PROYECTO --format=json > politica.json
# habilitar SOLO donde importa, no en toda la organización
```

El criterio que evita la factura sin renunciar a la evidencia: habilitarlos por servicio y por proyecto, en los que contienen datos personales o financieros, y dejarlos apagados en el resto. Activarlos en la organización entera «por si acaso» es

la forma más rápida de multiplicar el gasto de telemetría por varias veces.

Y el modelo de precios, que cambia lo que es razonable guardar:

ingesta	~0,50 USD por GiB
retención	30 días incluidos en _Default; más allá, por GiB y mes
_Required	gratis, 400 días, no configurable
métricas de plataforma	gratuitas

Compararlo con la clase 045 explica por qué el diseño cambia: allí la ingesta costaba unas cuatro veces más, así que la palanca era el plan por tabla. Aquí la ingesta es más barata y el volumen es mayor porque todo está encendido, de modo que la palanca principal es otra: descartar antes de ingerir.

2. El enrutador: excluir antes de pagar, archivar lo que solo se guarda

Todo registro pasa por el enrutador antes de almacenarse, y ahí hay dos operaciones distintas que se confunden:

destino (sink)	copia los registros que coinciden hacia un destino → se paga la ingesta en el destino que corresponda
exclusión	DESCARTA antes de almacenar → no se paga

La exclusión es la que baja la factura. Y se escribe con el mismo lenguaje de filtro que las consultas:

```
$ gcloud logging sinks update _Default \  
  --add-exclusion=name=peticiones-correctas,\  
  filter='resource.type="cloud_run_revision" AND httpRequest.status<400'  
  
$ gcloud logging sinks update _Default \  
  --add-exclusion=name=flujos-vpc-muestreo,\  
  filter='resource.type="gce_subnetwork" AND log_id("compute.googleapis.com/vpc_flows")  
  AND sample(insertId, 0.05)'
```

La segunda usa muestreo: conserva el 5 % de los registros de flujo de la clase 051 y descarta el resto. Para detectar patrones de tráfico, una muestra basta; para una investigación forense concreta, no — y ahí es donde entra el archivo barato:

```
$ gcloud logging sinks create archivo-crudo \  
  storage.googleapis.com/projects/$P/buckets/cls-registros-crudos \  
  --log-filter='resource.type="gce_subnetwork" OR resource.type="http_load_balancer"'
```

Ese destino escribe a Cloud Storage, donde el GB cuesta una fracción y las reglas de ciclo de vida de la clase 053 lo retiran solas. La combinación resuelve las dos necesidades sin pagar dos veces:

lo que se consulta a diario	en el bucket de registros, con retención corta
lo que se consulta una vez	en Cloud Storage, barato, con ciclo de vida
lo que no se consulta nunca	excluido: no se paga

La tercera línea exige una comprobación honesta que casi nadie hace: mirar qué se ha consultado de verdad en los últimos noventa días. Un tipo de registro que nadie ha buscado nunca es candidato a exclusión o a archivo, y esa revisión es la que convierte la reducción de costo en una decisión con datos en vez de en un recorte a ciegas.

Y para el análisis, Log Analytics convierte un bucket de registros en algo consultable con SQL, lo que evita mover los datos a otro sitio solo para poder agregarlos:

```
SELECT  
  JSON_VALUE(resource.labels.service_name) AS servicio,  
  COUNT(*) AS peticiones,  
  COUNTIF(CAST(JSON_VALUE(http_request.status) AS INT64) >= 500) AS errores  
FROM `cls-obs.global._Default._AllLogs`  
WHERE timestamp > TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 1 HOUR)  
GROUP BY servicio ORDER BY errores DESC
```

Y un límite del enrutador que produce una sorpresa: las exclusiones se aplican al bucket de destino, no al origen. Un registro excluido de Default sigue llegando a otros destinos que lo seleccionen. Es lo que se quiere —excluir del caro y archivar en el barato— y hay que tenerlo presente al calcular el ahorro.

3. Alertar por presupuesto de error, no por umbral de recurso

Esta es la pieza que las clases 034 y 045 no tenían y que cambia la operación más que ninguna otra de la parte.

El problema con las alertas de umbral es conocido: la CPU al 85 % puede ser perfectamente normal, y el 40 % puede ser catastrófico si la mitad de las peticiones fallan. Un panel lleno de avisos que nadie atiende no es vigilancia, es ruido

con guardia.

Google Cloud modela el objetivo directamente:

SLI indicador: proporción de peticiones buenas sobre el total
SLO objetivo: 99,9 % en 28 días
presupuesto de error 0,1 % de las peticiones
con 10 M de peticiones al mes → 10.000 peticiones pueden fallar

Y la alerta se define sobre la velocidad a la que se consume ese presupuesto:

velocidad de consumo = tasa de error observada / tasa que agota justo el presupuesto

velocidad 1 a este ritmo, el presupuesto se agota exactamente al final del periodo
velocidad 14,4 el presupuesto de 28 días se agota en 2 días
→ si dura una hora, ya se ha gastado el 2 % del mes

La configuración estándar usa dos ventanas, y su lógica merece entenderse porque es lo que elimina el ruido:

ventana corta + velocidad alta (5 min, ×14,4) incidente agudo → avisar YA
ventana larga + velocidad baja (6 h, ×1) degradación lenta → avisar,
pero sin urgencia

y se exige que AMBAS ventanas coincidan
→ un pico de 30 segundos no dispara nada
→ una degradación sostenida sí

```
$ gcloud alpha monitoring policies create --policy-from-file - <<'YAML'
displayName: "Consumo rápido del presupuesto de error · tienda"
conditions:
- displayName: "velocidad de consumo > 14,4 en 5 min y 1 h"
  conditionThreshold:
    filter: 'select_slo_burn_rate("projects/.../services/tienda/serviceLevelObjectives/disponibilidad-
999", "3600s")'
    comparison: COMPARISON_GT
    thresholdValue: 14.4
    duration: 300s
YAML
```

Lo que se gana no es solo menos ruido. Se gana que cada alerta corresponde a algo que el usuario nota, y que la urgencia está calibrada: una alerta de velocidad 14,4 significa que si nadie hace nada en dos días no queda presupuesto, lo que es una frase que un responsable de guardia puede entender a las tres de la madrugada. «CPU al 85 %» no lo es.

Y las alertas de umbral no desaparecen: se quedan para lo que sí es un límite duro, que es distinto de una degradación:

SLO / presupuesto de error la experiencia del usuario
umbral cuotas, disco lleno, certificado que caduca,
conexiones al máximo, mensajes fallidos > 0

La segunda columna incluye la alerta que este programa ha justificado en tres plataformas: trabajo que se acumula. Un consumidor detenido no consume presupuesto de error de un servicio HTTP, así que un sistema que solo vigile el SLO de la API no lo detecta.

Y para lo que solo existe en los registros, la métrica basada en registros cierra el hueco que la clase 045 dejó abierto —allí, alertar sobre un registro costaba entre siete y veinte minutos—:

```
$ gcloud logging metrics create errores-cobro \
--description "Excepciones del servicio de cobro" \
--log-filter='resource.type="cloud_run_revision"
AND resource.labels.service_name="svc-pagos"
AND severity>=ERROR AND jsonPayload.tipo="CobroRechazado"'
```

Una vez extraída, es una métrica normal: se alerta sobre ella en aproximadamente un minuto, con el mismo mecanismo rápido que cualquier otra.

4. Trazas, errores y dónde se va el tiempo

Tres herramientas responden tres preguntas distintas, y las tres están integradas sin montar nada.

Cloud Trace reconstruye la petición completa entre servicios. La correlación viaja en la cabecera estándar del W3C —o en la propia de Google— y tiene la misma condición que en la clase 045: cada salto debe propagarla.

X-Cloud-Trace-Context: TRACE_ID/SPAN_ID;o=1

```
traceparent: 00-<trace-id>-<span-id>-01
```

Y el mismo síntoma cuando falta: la traza se corta justo en el servicio interno que se escribió a mano, que es normalmente el interesante. La comprobación es directa —buscar una traza y contar los tramos esperados— y conviene hacerla el día que se despliega un servicio nuevo, no durante un incidente.

En Cloud Run hay un detalle que ahorra trabajo: la plataforma ya emite un tramo por petición, así que el mapa básico existe sin instrumentar nada. Lo que hay que añadir es la propagación entre servicios propios y los tramos de las llamadas a bases de datos, que es donde suele estar el tiempo.

Error Reporting agrupa las excepciones por firma y cuenta ocurrencias, primera y última vez. Sustituye a buscar cadenas en los registros y responde a la pregunta que abre casi todo incidente de aplicación: qué ha empezado a fallar ahora que antes no fallaba. Funciona sin configuración si las excepciones se escriben con la severidad y el formato adecuados, lo que es un argumento concreto para registrar en estructura y no en texto libre:

```
import json, sys
print(json.dumps({
    "severity": "ERROR",
    "message": str(exc),
    "stack_trace": traceback.format_exc(),
    "logging.googleapis.com/trace": f"projects/{proyecto}/traces/{trace_id}",
}), file=sys.stderr)
```

Ese campo de traza es el que enlaza el error con su petición completa: desde la excepción se llega a la traza, y desde la traza a la dependencia que la causó. Es la cadena que la clase 034 pedía y que aquí cuesta una línea.

Cloud Profiler responde a una pregunta que ninguna de las plataformas anteriores contestaba en producción: dónde se va el tiempo de CPU y la memoria. Muestra continuamente con una sobrecarga muy baja y presenta el resultado como gráfico de llamas.

```
import googlecloudprofiler
googlecloudprofiler.start(service="svc-tienda", service_version="v8")
```

Su valor es que encuentra cosas que nadie buscaría: un serializador que consume un tercio de la CPU, una expresión regular recompilada en cada petición, una biblioteca de fechas que domina el perfil. Son mejoras que no aparecen en ninguna traza porque están dentro de un tramo, y que suelen tener una corrección de pocas líneas.

Y el Servicio Gestionado para Prometheus merece mención porque decide el diseño de la parte 06: Google Cloud ingiere métricas en formato Prometheus de forma nativa, con consultas en PromQL. Eso significa que la instrumentación de una aplicación no es específica del proveedor: las mismas métricas y los mismos paneles valen aquí, en un clúster propio o en otra nube. Es exactamente el tipo de contrato portable que la clase 048 pedía identificar, y aparece en la capa donde más veces se pierde.

5. El método, ya idéntico en tres plataformas

Con tres proveedores recorridos, el procedimiento de investigación ya no es una receta por producto. Es el mismo, y solo cambia el nombre del comando:

- | | |
|-------------------------------|--|
| 1. ¿el usuario lo nota? | SLO y presupuesto de error, no CPU |
| 2. ¿desde cuándo? | percentil de latencia y tasa de error en serie |
| 3. ¿qué cambió en la ventana? | auditoría de administrador + despliegues |
| 4. ¿qué traza lo demuestra? | una petición fallida, de extremo a extremo |
| 5. ¿qué dependencia? | tramos de la traza, o errores agrupados |
| 6. ¿quién y con qué permiso? | auditoría, y el sistema de identidad (050) |

Y las consultas de cada paso se escriben una vez y se guardan, por la razón que la clase 045 ya dio: un incidente no es el momento de aprender a consultar.

```
# 3. qué cambió en las últimas 6 horas
$ gcloud logging read 'logName:"cloudaudit.googleapis.com%2Factivity"
  AND timestamp>="$(date -u -d "6 hours ago" +%Y-%m-%dT%H:%M:%SZ)'" \
  --format="table(timestamp, protoPayload.authenticationInfo.principalEmail,
    protoPayload.methodName, resource.labels.service_name)"
```

Y la comparación de las tres plataformas en observabilidad, que es lo que se lleva a la clase 060:

registros de recurso	parcial	APAGADOS	activos
auditoría, retención gratuita	90 días	90 días	400 días
precio de ingesta	medio	~2,30 USD/GiB	~0,50 USD/GiB
registros de acceso a datos	opcionales	opcionales	apagados, caros
alerta rápida sobre registros	métrica	~9 min	métrica de registro

de ellas, con impacto en usuarios	6
de ellas, atendidas	31
de ellas, silenciadas	46

El resto se ignoraron. Se sustituyen los umbrales de recurso por objetivos de servicio:

SLI	peticiones con estado < 500 y latencia < 800 ms / total	
SLO	99,9 % en 28 días	
presupuesto de error	0,1 % → ~10.200 peticiones al mes	
alertas	velocidad ×14,4 en ventanas de 5 min y 1 h	→ aviso inmediato
	velocidad ×1 en ventanas de 6 h y 3 días	→ aviso sin urgencia
alertas al mes	340	11
con impacto real en usuarios	6	11
falsos positivos	334	0
alertas silenciadas	46	0
tiempo medio hasta la detección	19 min	3 min 40 s

Once alertas, once incidentes reales. Y se conservan seis alertas de umbral para lo que sí es un límite duro: cuota al 80 %, disco al 70 %, certificado a 30 días, conexiones al 85 %, mensajes fallidos mayor que cero y acumulación en cola — la última, por tercera vez en el programa.

Problema 4 — la traza se cortaba en el servicio de precios.

tramos esperados en una petición de pedido	6
tramos observados	3

el corte, siempre en la llamada a svc-precios

El cliente HTTP de ese servicio se había escrito a mano y no reenviaba la cabecera de traza. Tercera vez que este fallo aparece en el programa, con la misma corrección.

propagación de la cabecera	4 de 6 saltos	6 de 6
tiempo hasta localizar una dependencia lenta	~25 min	~3 min
comprobación de continuidad de traza	ninguna	en el despliegue de cada servicio

Problema 5 — el 38 % de la CPU estaba en un sitio que nadie habría mirado.

Con el perfilador activo una semana:

serialización a JSON del catálogo completo	38 % del tiempo de CPU	
causa:	se serializaba el objeto entero para devolver 4 campos	
CPU en serialización	38 %	6 %
p95 del listado de catálogo	214 ms	131 ms
instancias en el pico	9	5
costo mensual de Cloud Run	31 USD	19 USD

Una corrección de once líneas, encontrada porque el perfilador estaba encendido y no porque nadie sospechara.

Resumen de la observabilidad:

ingesta diaria	62 GiB/día	10,1 GiB/día
costo mensual de telemetría	930 USD	206 USD
registros de acceso a datos	apagados	activos en 4 proyectos
alertas al mes	340	11
falsos positivos	334	0
tiempo medio hasta la detección	19 min	3 min 40 s
propagación de traza	4 de 6 saltos	6 de 6

La lección que esta clase traslada al proyecto de la clase 060: el problema de la observabilidad cambia de forma según la plataforma —en la parte 03 era la ausencia de datos, aquí es su exceso— y no cambia el criterio. En ambos casos la pregunta correcta es la misma: ¿qué señal necesito para responder a un incidente, y qué estoy pagando que nunca he consultado? La respuesta se obtiene mirando el historial de consultas, y casi siempre sorprende.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/057-cloud-logging-monitoring-trace-y-audit-logs/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.

2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa evidencia-operativa-gcp en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye evidencia-operativa-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
La factura de telemetría es varias veces la prevista desde el primer mes	Casi todo se registra por defecto, incluidos los flujos de red y las peticiones correctas	Excluye en el enrutador antes de ingerir, muestrea lo voluminoso y archiva el volumen íntegro en Cloud Storage con ciclo de vida.
No se puede saber quién leyó un dato personal	Los registros de auditoría de acceso a datos están apagados por defecto	Habílitelos de forma selectiva en los proyectos y servicios con datos sensibles, no en toda la organización.
Cientos de alertas al mes y casi ninguna corresponde a impacto real	Se alerta sobre umbrales de recursos en vez de sobre la experiencia del usuario	Define SLI y SLO y alerta por velocidad de consumo del presupuesto de error, con dos ventanas que deban coincidir.
Una alerta basada en registros llega demasiado tarde	Se evalúa como consulta programada, con la latencia acumulada de la clase 045	Convierte el patrón en una métrica basada en registros y alerta sobre ella como sobre cualquier métrica.
La traza se corta siempre en el mismo servicio	Ese salto no propaga la cabecera de contexto de traza	Propágala en todos los clientes HTTP y comprueba el número de tramos al desplegar cada servicio.
Se optimiza el servicio y la latencia no mejora	El tiempo se va dentro de un tramo, donde ninguna traza lo muestra	Activa el perfilador continuo en producción: su sobrecarga es mínima y localiza el consumo real.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué registros están activos y son gratuitos aquí, y qué problema concreto de la clase 045 resuelven?
2. ¿Cuál es la diferencia entre un destino y una exclusión en el enrutador, y cuál baja la factura?
3. Explica qué significa una velocidad de consumo de 14,4 y por qué se exigen dos ventanas coincidentes.
4. ¿Qué tipo de alerta sigue necesitando un umbral aunque exista el presupuesto de error?
5. ¿Qué pregunta responde el perfilador que ninguna traza puede responder, y por qué?

Referencias

- Google Cloud (2025). Cloud Audit Logs overview — tipos, cuáles están activos y cuáles se pagan. <<https://cloud.google.com/logging/docs/audit>>
- Google Cloud (2025). Routing and storage overview — enrutador, destinos, exclusiones y buckets Required y Default. <<https://cloud.google.com/logging/docs/routing/overview>>
- Google Cloud (2025). Log-based metrics — contadores y distribuciones extraídos de registros. <<https://cloud.google.com/logging/docs/logs-based-metrics>>
- Google Cloud (2025). SLO monitoring and burn rate alerts — SLI, SLO, presupuesto de error y ventanas múltiples. <<https://cloud.google.com/stackdriver/docs/solutions/slo-monitoring>>
- Google Cloud (2025). Cloud Profiler concepts — muestreo continuo en producción y su sobrecarga. <<https://cloud.google.com/profiler/docs/concepts-profiling>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 04 en PDF · Recorrido de Google Cloud en PDF · Manual integral

Anterior	Índice	Siguiente
← 056 · Pub/Sub, Cloud Tasks y Workflows	Parte 04 · Programa	058 · Cloud KMS, Secret Manager y Security Command Center →

Evaluación — 057 Cloud Logging, Monitoring, Trace y Audit Logs

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega evidencia-operativa-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.

- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

058 — Cloud KMS, Secret Manager y Security Command Center

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

Propósito

Cerrar la seguridad de la plataforma de Google Cloud con las tres piezas que la sostienen, y con dos precisiones que producen incidentes reales: un secreto referenciado como variable de entorno se resuelve al arrancar la instancia, así que rotarlo no cambia nada hasta que se despliega otra vez; y una clave rotada no deja de usarse, porque lo ya cifrado sigue necesitando la versión antigua. La clase 046 dejó la disciplina —cada control con su prueba negativa—; aquí cambian los mecanismos y aparece una herramienta que ninguna de las dos plataformas anteriores tenía: la simulación de rutas de ataque.

Resultados de aprendizaje

Al finalizar podrás:

1. Estructurar claves en anillos y versiones sabiendo qué se puede destruir y qué no se puede borrar nunca.
2. Rotar una clave entendiendo qué queda cifrado con la versión anterior y cuándo se puede destruir.
3. Referenciar secretos desde un servicio sabiendo cuándo se resuelven y qué exige una rotación.
4. Priorizar hallazgos de seguridad por ruta de ataque explotable en vez de por puntuación.
5. Verificar cada control de la parte con una prueba negativa ejecutada.

Conceptos centrales

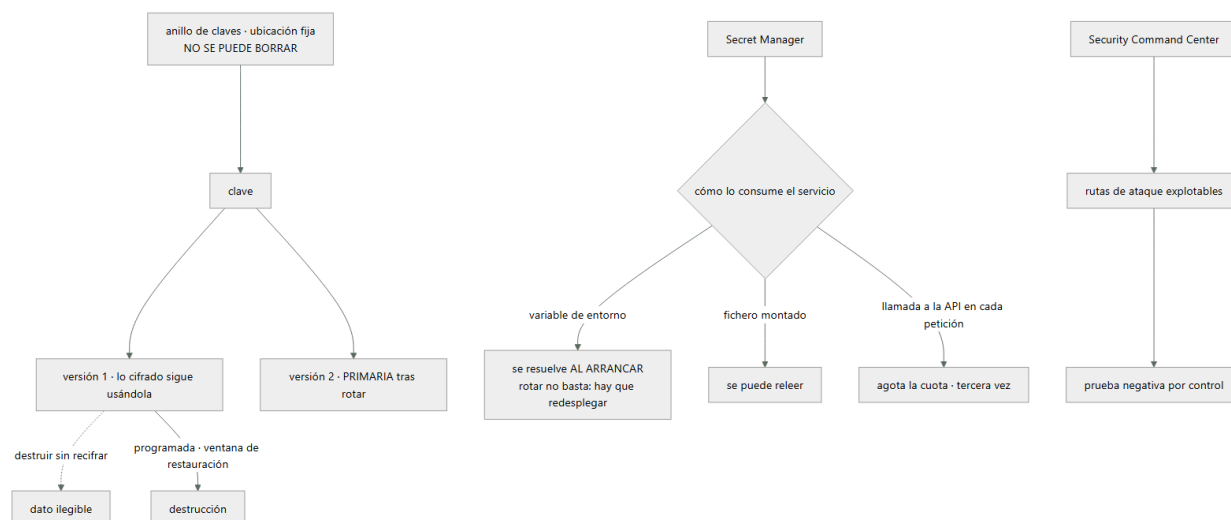
Concepto	Comprensión verificable
anillo de claves	Agrupación de claves ligada a una ubicación. No se puede borrar, ni él ni las claves que contiene: solo se destruyen versiones. Una jerarquía mal pensada se queda para siempre.
versión de clave	Material criptográfico concreto. La rotación crea una versión primaria nueva y no reescribe lo ya cifrado, que sigue dependiendo de la anterior.
destrucción programada	Una versión no se borra al instante: entra en un plazo —24 horas por defecto— durante el cual se puede restaurar. Es la red de seguridad ante el error irreversible.
nivel de protección	Software, HSM o externo. El externo mantiene la clave fuera de Google, lo que satisface requisitos de soberanía y acopla la disponibilidad del dato a la de un sistema ajeno.
referencia a secreto	Forma en que un servicio obtiene un secreto. Como variable de entorno se resuelve al arrancar la instancia; montado como fichero puede releerse en cada lectura.
simulación de rutas de ataque	Cálculo de los caminos por los que un atacante podría llegar a un recurso valioso. Prioriza por explotabilidad real, no por número de hallazgos.

Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Claves: lo que no se borra y lo que sí se destruye

La jerarquía de Cloud KMS tiene tres niveles y una propiedad que sorprende:

anillo de claves	ligado a una ubicación · NO SE PUEDE BORRAR
clave	NO SE PUEDE BORRAR
versión	sí se puede destruir, con plazo

Las dos primeras líneas son literales: un anillo creado por error se queda en el proyecto para siempre. No cuesta dinero vacío y sí ensucia el inventario, así que la estructura conviene decidirla antes de crear la primera:

```

anillo por ENTORNO y UBICACIÓN, no por equipo
kr-prod-europe-west1
clave por PROPÓSITO
  k-almacenamiento cifra buckets
  k-datos cifra Cloud SQL
  k-aplicacion cifra a nivel de aplicación
  
```

El criterio para separar claves es el mismo de la clase 046 y tiene consecuencias operativas: una clave es la unidad de revocación. Todo lo que cifre la misma clave se vuelve inaccesible a la vez cuando se revoca. Si eso incluye la evidencia de auditoría —como ocurrió en el simulacro de la clase 048—, el radio de impacto es mayor del previsto.

La destrucción de una versión tiene una protección que conviene conocer porque salva errores:

```

$ gcloud kms keys versions destroy 3 --key k-almacenamiento \
  --keyring kr-prod-europe-west1 --location europe-west1
# → estado DESTROY_SCHEDULED durante el plazo configurado

$ gcloud kms keys versions restore 3 --key k-almacenamiento \
  --keyring kr-prod-europe-west1 --location europe-west1
  
```

El plazo por defecto es de 24 horas y puede fijarse hasta 120 días al crear la clave. Es el equivalente funcional de la protección contra purga de la clase 046, con una diferencia importante: aquí es el comportamiento por defecto, no un interruptor que alguien tiene que acordarse de activar.

Y los niveles de protección, en orden de control y de acoplamiento:

software	la clave la custodia Google, en software
HSM	módulo de hardware certificado, gestionado por Google
externo	la clave vive FUERA de Google, en un gestor propio o de un tercero
	Google nunca la tiene: pide la operación criptográfica

El externo satisface requisitos de soberanía que ningún otro nivel cubre, y trae una consecuencia que hay que aceptar por escrito antes de elegirlo: la disponibilidad del dato pasa a depender de un sistema que no está en esta nube. Si el gestor externo no responde, los datos no se pueden leer. Es un intercambio legítimo cuando la obligación existe y es un riesgo gratuito cuando se elige por prudencia genérica.

Y la rotación, con la precisión que la clase 046 ya estableció y que aquí tiene una consecuencia concreta:

```
$ gcloud kms keys update k-almacenamiento --keyring kr-prod-europe-west1 \
  --location europe-west1 --rotation-period 90d --next-rotation-time 2026-09-01T00:00:00Z
```

rotar crea una versión nueva y la hace primaria
 lo NUEVO se cifra con ella
no rota lo ya cifrado, que sigue dependiendo de la versión anterior

De ahí la regla que evita el error irreversible: una versión antigua no se destruye hasta haber recifrado lo que dependía de ella, y comprobarlo no es opcional:

```
$ gcloud kms keys versions list --key k-almacenamiento \
  --keyring kr-prod-europe-west1 --location europe-west1 \
  --format="table(name.basename(), state, createTime)"
```

En los servicios que soportan clave del cliente, el recifrado se dispara reescribiendo el objeto o mediante la operación que el servicio ofrezca. Y hasta que termine, destruir la versión antigua deja datos ilegibles — que es la forma más cara de descubrir que «rotado» no significa «ya no se usa».

2. Secretos: cuándo se resuelven decide si la rotación sirve

Secret Manager guarda secretos con versiones inmutables y un alias latest. Y la decisión que produce el incidente de esta clase no es dónde se guarda, sino cómo lo consume el servicio.

Hay tres formas y no son equivalentes:

- A · variable de entorno con referencia
 la plataforma resuelve el valor AL ARRANCAR la instancia
 → rotar el secreto NO cambia nada hasta que arranque una instancia nueva
- B · fichero montado con referencia
 el fichero se puede releer
 → una rotación se recoge sin redesplegar, si la aplicación relea
- C · llamada a la API desde la aplicación
 control total, y hay que gestionar caché y cuota

La forma A es la más usada porque es la más cómoda, y su comportamiento sorprende siempre:

```
$ gcloud run deploy svc-pedidos \
  --set-secrets "DB_PASSWORD=db-password:latest"
```

latest se evalúa cuando la instancia arranca. Con instancias vivas de antes de la rotación y otras nuevas, el servicio usa dos contraseñas a la vez durante horas, y la mitad de las peticiones falla. El síntoma —errores intermitentes de autenticación que se corrigen solos con el tiempo— es difícil de atribuir.

Las dos correcciones, según el caso:

rotación planificada	desplegar una revisión nueva como último paso de la rotación: es un cambio de tráfico, segundos
rotación de emergencia	fichero montado y relectura ante fallo de autenticación

Y la segunda línea repite el patrón de dos credenciales de la clase 046, que sigue siendo la única forma de rotar sin ventana de fallo:

1. dos credenciales activas: primaria y secundaria
2. rotar la secundaria, que nadie usa
3. desplegar la revisión que apunta a la secundaria ya rotada
4. rotar la antigua primaria, ahora libre

La forma C tiene un riesgo conocido y es la tercera vez que aparece en el programa: leer el secreto en cada petición agota la cuota del proyecto y el fallo se manifiesta como error de la base de datos, no del gestor de secretos. La corrección es la misma que en la clase 046: cachear en memoria con vencimiento y releer solo ante un fallo de autenticación.

Dos decisiones más que se toman al crear el secreto y no se cambian:

```
$ gcloud secrets create db-password \
  --replication-policy user-managed --locations europe-west1,europe-west4
```

La política de replicación gestionada por el usuario permite elegir las regiones, que es el mismo requisito de residencia que la clase 053 resolvió eligiendo la birregión y que la clase 041 no podía resolver. Es coherente con el resto de la plataforma y hay que decidirlo al crear, porque después no cambia.

Y una limitación que conviene decir sin adornos, porque es una diferencia real frente a otra plataforma del programa: Secret Manager no rota por sí solo. Puede publicar una notificación en un tema de Pub/Sub según un calendario, y la rotación —conectarse al motor, generar la credencial nueva, guardarla como versión— la escribe el equipo.

```
$ gcloud secrets update db-password \
  --next-rotation-time 2026-09-01T03:00:00Z --rotation-period 2592000s \
  --topic projects/$P/topics/rotacion-secretos
```

Eso es un disparador, no una rotación. Compararlo con la rotación integrada de otros gestores es justo: aquí hay que construir la función que la ejecuta, y conviene presupuestarlo en lugar de descubrirlo.

3. Priorizar por ruta de ataque, no por número de hallazgos

Security Command Center hace tres cosas y se paga por niveles:

inventario de recursos y hallazgos de configuración	nivel básico
detección de amenazas (eventos, contenedores, máquinas)	niveles de pago
simulación de RUTAS DE ATAQUE y cumplimiento normativo	niveles de pago

La tercera es la que no tenía equivalente en las dos plataformas anteriores y la que cambia cómo se prioriza.

Un escáner de configuración produce cientos de hallazgos y los ordena por severidad genérica. La simulación de rutas hace otra cosa: parte de los recursos que declaras valiosos y calcula los caminos por los que alguien podría llegar a ellos, atravesando permisos, suplantaciones, reglas de red y servicios expuestos.

```
escáner clásico      "este bucket no tiene registro de acceso"    severidad media
                    "esta máquina tiene IP pública"    severidad media
                    ...320 hallazgos más
```

```

ruta de ataque      internet → svc-informes (sin autenticar)
                   → cuenta de servicio sa-informes
                   → suplantación de sa-datos
                   → lectura de gs://cls-facturas
exposición          del recurso valioso: ALTA

```

Esa segunda forma es exactamente el tipo de cadena que la clase 050 obligó a buscar a mano con el analizador de políticas. Aquí se calcula sola y se recalcula, que es la diferencia entre una auditoría puntual y un control continuo: una cuenta de servicio creada la semana siguiente a la auditoría aparece sin que nadie repita el ejercicio.

La priorización que se deduce es más honesta que una puntuación:

se corrige primero lo que ESTÁ en una ruta hacia un recurso valioso después, lo que aumentaría la superficie si algo más fallara al final, lo que es una buena práctica sin camino conocido

Y conviene decir lo que cuesta, porque es una partida real: los niveles de pago se facturan en proporción al gasto de la organización o por recurso protegido, así que la decisión es de alcance —qué carpetas y qué proyectos— exactamente como los planes de la clase 046. Activarlo en toda la organización sin mirar el inventario repite el error que allí costó 2.460 dólares al mes.

Y las detecciones tienen la misma exigencia que cualquier otro control:

activar una detección no demuestra nada
lo demuestra provocarla y ver el hallazgo

Una prueba segura y estándar para la detección de amenazas en la capa de identidad es crear una clave de cuenta de servicio en un proyecto donde la política de la clase 050 lo prohíbe, y comprobar que aparece el hallazgo correspondiente; para la de contenedores, ejecutar un binario desde un directorio temporal dentro de un contenedor. Lo importante no es la prueba concreta sino la regla: la fecha de la última verificación de cada detección debe existir y ser reciente.

4. La lista de pruebas negativas de la parte 04

La clase 046 estableció que un control sin prueba negativa es una afirmación. Cerrar la parte 04 exige la lista equivalente, y conviene compararla con la de allí porque las pruebas se parecen y los mensajes de error no:

control	prueba negativa que lo demuestra
bucket no público	conceder a allUsers → 412, violación de restricción (049, 053)
sin claves de cuenta de servicio	crear una clave → denegado por política (050)

federación acotada por sujeto	desde otro repositorio → permiso denegado (050)	
salida a internet cerrada	curl con tiempo límite → agota	(051)
firewall dirigido por identidad	prueba de conectividad → UNREACHABLE	(051)
Cloud SQL sin IP pública	conexión directa → tiempo agotado	(054)
servicio interno no público	curl sin testigo → 403	(055)
cola de fallidos operativa	publicar un mensaje que falla → aparece en el tema de fallidos	(056)
auditoría de acceso a datos activa	leer un dato y encontrar la entrada	(057)
destrucción de versión de clave	programar y restaurar dentro del plazo	
rotación de secreto sin corte	rotar con tráfico en curso: 0 errores	
recuperación de bucket	borrar y restaurar uno de prueba	(053)

Doce afirmaciones con doce comprobaciones. Y la comparación de los tres proveedores en un aspecto concreto —qué protege la plataforma por defecto y qué hay que activar— es lo que el proyecto de la clase 060 tiene que poder resumir:

borrado de contenedor protegido	opcional	2 interruptores	por defecto
auditoría de administrador	90 días	90 días	400 días, gratis
auditoría de acceso a datos	opcional	opcional	apagada, cara
claves: destrucción con ventana	opcional	opcional	por defecto
creación de credenciales de larga duración bloqueable por política	sí	sí	sí
rutas de ataque calculadas	—	—	integrado
rotación de secretos integrada	sí	parcial	hay que construirla

La última fila es la única en la que esta plataforma queda por detrás de forma clara, y merece figurar como riesgo declarado en lugar de omitirse.

Y la prueba de segundo orden que la clase 046 introdujo sigue siendo la más rentable: comprobar que el control sigue activo pasado un tiempo. Aquí el mecanismo son las políticas de organización de la clase 049 y los hallazgos continuos de Security Command Center, que detectan la desviación en horas en vez de en la siguiente auditoría. Los controles no se revierten por malicia: se revierten porque alguien recreó un recurso desde una plantilla vieja, y esa es una causa que ninguna norma escrita evita.

5. Lo que este programa ya puede afirmar sobre seguridad en la nube

Con tres plataformas recorridas, hay conclusiones que ya no dependen del proveedor y conviene enunciarlas, porque son las que se llevan a las partes 11 y 22.

Primera: las credenciales de larga duración son el problema, y las tres plataformas ofrecen la misma salida. Rol de instancia, identidad administrada, cuenta de servicio adjunta; federación con sujeto acotado para lo externo. La prueba negativa ha sido idéntica tres veces y ha fallado dos de tres la primera vez que se ejecutó.

Segunda: el permiso suma y lo que resta vive en otro sistema. Políticas de control de servicios, Azure Policy, políticas de organización y de denegación. Los tres producen un error que no habla de permisos, y leerlo ahorra buscar en el sistema equivocado. Tres vocabularios, un método.

Tercera: el gobierno guarda la puerta y no limpia la casa. En las tres, imponer una restricción no corrige lo existente, y la secuencia correcta ha sido siempre inventariar, corregir y después imponer. Hacerlo al revés produjo en la clase 046 un panel en verde con catorce recursos incumpliendo.

Cuarta: el control de la clave es control real y responsabilidad real. Poder revocar significa poder dejar un dato ilegible, y el radio de impacto se estima mal siempre que un mismo recurso sirve a varios propósitos — como demostró el simulacro de la clase 048.

Quinta, y la que más veces se salta: un informe que dice «se ha habilitado X» no es comparable con uno que dice «se ha habilitado X y este es el error que devuelve al intentar lo que impide». La primera forma es una intención. La segunda es evidencia, y es la única que sobrevive a una auditoría y a un cambio de proveedor.

Lo que queda por comprobar, y que la clase 060 tiene que responder con datos: si la lista de doce pruebas negativas de esta parte se puede generar a partir de la línea base en lugar de escribirse a mano cada vez. Si la respuesta es que sí, el trabajo de seguridad de la cuarta plataforma será una fracción del de esta — que es exactamente la promesa que este programa hizo en la parte 01 y que ya se puede empezar a medir.

Ejemplo trabajado

CloudShop cierra la seguridad de su plataforma en Google Cloud. Llega con la línea base de la clase 048 y la lista de pruebas negativas de la 046, así que tres controles se montan sin incidentes. Los cuatro problemas de este mes son mecanismos nuevos mal entendidos.

Lo que se montó sin incidentes, por venir escrito:

sin credenciales de larga duración: bloqueadas por política	(046 → 050)
inventariar, corregir y después imponer	(046 → 049)
separar por propósito lo que comparte clave	(048 → 058)
cada control con su prueba negativa	(046 → 058)

La tercera venía directamente del simulacro de la clase 048, donde revocar una clave detuvo tres servicios en vez de uno. Aquí las claves se separaron desde el principio:

k-facturas	cifra el bucket de facturas
k-auditoria	cifra el archivo de registros
k-datos	cifra Cloud SQL

Problema 1 — la mitad de las instancias con la contraseña antigua.

Se rota la contraseña de la base de datos a las 03:00. A las 09:20 siguen apareciendo errores intermitentes de autenticación.

```
$ gcloud run services describe svc-pedidos \
  --format="value(spec.template.spec.containers[0].env)"
DB_PASSWORD → secretKeyRef: db-password:latest
```

La referencia se resuelve al arrancar la instancia. Las instancias vivas desde antes de las 03:00 seguían con la contraseña anterior; las nuevas, con la nueva. Seis horas con dos credenciales en circulación.

referencia al secreto	variable de entorno	fichero montado
relectura ante fallo de autenticación	no	sí
rotación	cambiar la versión	dos credenciales: rotar la secundaria, desplegar, rotar la otra
errores durante la rotación siguiente	3.412	0

Problema 2 — errores 500 que parecían de la base de datos. Tercera vez.

```
RESOURCE_EXHAUSTED: Quota exceeded for quota metric
'Access secret versions' of service 'secretmanager.googleapis.com'
```

Un servicio nuevo leía el secreto en cada petición. Es el mismo incidente de la clase 046 con Key Vault y de la 051 con los puertos de traducción: el mismo defecto de código produce el mismo fallo en la tercera plataforma.

lecturas del gestor de secretos	1 por petición	1 cada 30 min
relectura ante fallo de auth	no	sí
errores 500 en el pico	4,1 %	0,0 %

Problema 3 — una versión de clave destruida por error, recuperada en la ventana.

Durante una limpieza, alguien programó la destrucción de la versión 2 de k-facturas creyendo que la rotación a la versión 3 la había dejado sin uso.

```
$ gcloud kms keys versions list --key k-facturas --keyring kr-prod-europe-west1 \
  --location europe-west1 --format="table(name.basename(), state)"
2  DESTROY_SCHEDULED
3  ENABLED
```

El 71 % de los objetos del bucket seguían cifrados con la versión 2. Destruirla los habría dejado ilegibles de forma permanente.

```
$ gcloud kms keys versions restore 2 --key k-facturas \
  --keyring kr-prod-europe-west1 --location europe-west1
```

plazo de destrucción programada	24 h (por defecto)	30 días
recifrado tras rotar	no había	proceso

comprobación previa a destruir	ninguna	automatizado obligatoria:
objetos aún cifrados con la versión 2	71 %	objetos por versión 0 %

La ventana de destrucción salvó el caso. Es el mismo papel que la protección contra purga de la clase 046 y con una diferencia importante: aquí venía puesta. El equipo la amplía a 30 días de todos modos, porque 24 horas no cubren un fin de semana.

Problema 4 — una ruta de ataque que la auditoría de la clase 050 no podía ver.

Security Command Center señala un camino con exposición alta:

```
internet → svc-informes (desplegado sin autenticación hace 9 días)
          → cuenta de servicio sa-informes-v2
          → suplantación de sa-datos
          → lectura de gs://cls-facturas
```

La auditoría de suplantaciones se había hecho tres semanas antes y estaba limpia. sa-informes-v2 y el servicio sin autenticar se crearon después.

revisión de rutas de suplantación	puntual, manual	continua
servicios sin autenticación	2 de 9	0 de 9
cadenas de suplantación hacia datos	1	0
tiempo desde la creación hasta la detección	—	4 horas

La diferencia no es la herramienta: es que una auditoría es una foto y un hallazgo continuo es una película. La clase 050 obligó a mirar los caminos; esta hace que se miren solos.

Y una decisión documentada como riesgo aceptado.

Se evaluó el gestor de claves externo para las facturas, por un requisito de soberanía que resultó ser una preferencia y no una obligación:

lo que aporta	la clave nunca está en Google
lo que cuesta	la disponibilidad del dato depende de un sistema externo y su indisponibilidad hace ilegible el bucket
decisión	no se adopta; se documenta con la condición que obligaría a revisarlo: una obligación contractual explícita

Resumen del cierre de seguridad:

claves por propósito	1 para todo	3
plazo de destrucción de versiones	24 h	30 días
objetos cifrados con una versión programada para destrucción	71 %	0 %
errores durante una rotación de secreto	3.412	0
errores 500 por cuota del gestor de secretos	4,1 %	0,0 %
servicios sin autenticación	2 de 9	0 de 9
rutas de ataque hacia datos valiosos	1	0
controles con prueba negativa ejecutada	0 de 12	12 de 12

La lección que esta clase traslada al proyecto de la clase 060: los cuatro problemas fueron de mecanismos nuevos, y dos de ellos eran el mismo defecto de código que ya había fallado en dos plataformas anteriores. Lo que evitó los otros tres controles fue una lista escrita, no la experiencia de nadie. La seguridad de la tercera plataforma costó menos porque la segunda dejó doce pruebas escritas, y ese es exactamente el rendimiento que este programa prometió medir.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/058-cloud-kms-secret-manager-y-security-command-center/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa controles-gcp en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye controles-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Tras rotar un secreto, parte de las peticiones sigue fallando durante horas	La referencia como variable de entorno se resuelve al arrancar la instancia	Móntalo como fichero y relee ante fallo de autenticación, o despliega una revisión nueva como último paso de la rotación.
Errores 500 que parecen de la base de datos y no lo son	Se lee el secreto en cada petición y se agota la cuota del proyecto	Cachea en memoria con vencimiento y relee solo ante un fallo de autenticación.
Destruir una versión de clave deja datos ilegibles	La rotación no recifra lo existente: sigue dependiendo de la versión anterior	Comprueba cuántos objetos usan cada versión antes de destruir, amplía el plazo de destrucción y automatiza el recifrado.
Un anillo o una clave creados por error no se pueden eliminar	En Cloud KMS solo se destruyen versiones; anillos y claves son permanentes	Decide la jerarquía antes de crear la primera: anillo por entorno y ubicación, clave por propósito y unidad de revocación.
Una auditoría de permisos limpia deja pasar una ruta de acceso a datos	La auditoría es una foto y los recursos nuevos se crean después	Usa la simulación de rutas de ataque como control continuo y corrige primero lo que está en un camino hacia un recurso valioso.
Se espera rotación automática de secretos y no ocurre	Secret Manager notifica según un calendario pero no ejecuta la rotación	Escribe la función de rotación disparada por la notificación y presupuesta ese trabajo; declara el riesgo mientras no exista.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué no se puede borrar nunca en Cloud KMS y qué implica para el diseño de la jerarquía?
2. Rotas una clave. ¿Qué queda cifrado con la versión anterior y cuándo puedes destruirla?

3. ¿Cuándo se resuelve un secreto referenciado como variable de entorno, y qué incidente produce durante una rotación?
4. ¿En qué se diferencia priorizar por ruta de ataque de priorizar por puntuación de seguridad?
5. Enumera cinco controles de la parte 04 con la prueba negativa concreta que demuestra cada uno.

Referencias

- Google Cloud (2025). Cloud KMS resource hierarchy — anillos, claves y versiones, y qué no se puede eliminar. <<https://cloud.google.com/kms/docs/resource-hierarchy>>
- Google Cloud (2025). Key rotation — versión primaria, datos ya cifrados y recifrado. <<https://cloud.google.com/kms/docs/key-rotation>>
- Google Cloud (2025). Use Secret Manager with Cloud Run — referencias como variable de entorno y como volumen montado. <<https://cloud.google.com/run/docs/configuring/services/secrets>>
- Google Cloud (2025). Secret rotation — notificaciones programadas y qué hay que construir. <<https://cloud.google.com/secret-manager/docs/secret-rotation>>
- Google Cloud (2025). Attack path simulations in Security Command Center — exposición de recursos valiosos y priorización. <<https://cloud.google.com/security-command-center/docs/attack-exposure-learn>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 04 en PDF · Recorrido de Google Cloud en PDF · Manual integral

Anterior	Índice	Siguiente
← 057 · Cloud Logging, Monitoring, Trace y Audit Logs	Parte 04 · Programa	059 · Terraform y despliegues reproducibles en GCP →

Evaluación — 058 Cloud KMS, Secret Manager y Security Command Center

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega controles-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

059 — Terraform y despliegues reproducibles en GCP

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: iac · Estado: EXECUTABLECORE

Propósito

Declarar la infraestructura de Google Cloud con Terraform, entendiendo la pieza que la clase 047 no tenía y que lo cambia todo: el archivo de estado. Da lo que a Bicep le faltaba —saber qué gestiona este código, destruirlo ordenadamente, detectar que alguien borró algo— y trae tres problemas propios: hay que guardarlo, hay que bloquearlo, y contiene los secretos en claro. A eso se suman dos trampas del proveedor que destruyen recursos sanos sin que nadie lo pida.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar qué capacidades da el archivo de estado y qué obligaciones impone su custodia.
2. Garantizar que lo que se aplica es exactamente lo que se revisó, y no una nueva planificación.
3. Evitar la destrucción de recursos ajenos al cambio por reindexación y por estado compartido.
4. Autenticar la canalización sin ninguna clave, con federación y suplantación.
5. Comparar un modelo con estado y otro sin él con criterios operativos, no de preferencia.

Conceptos centrales

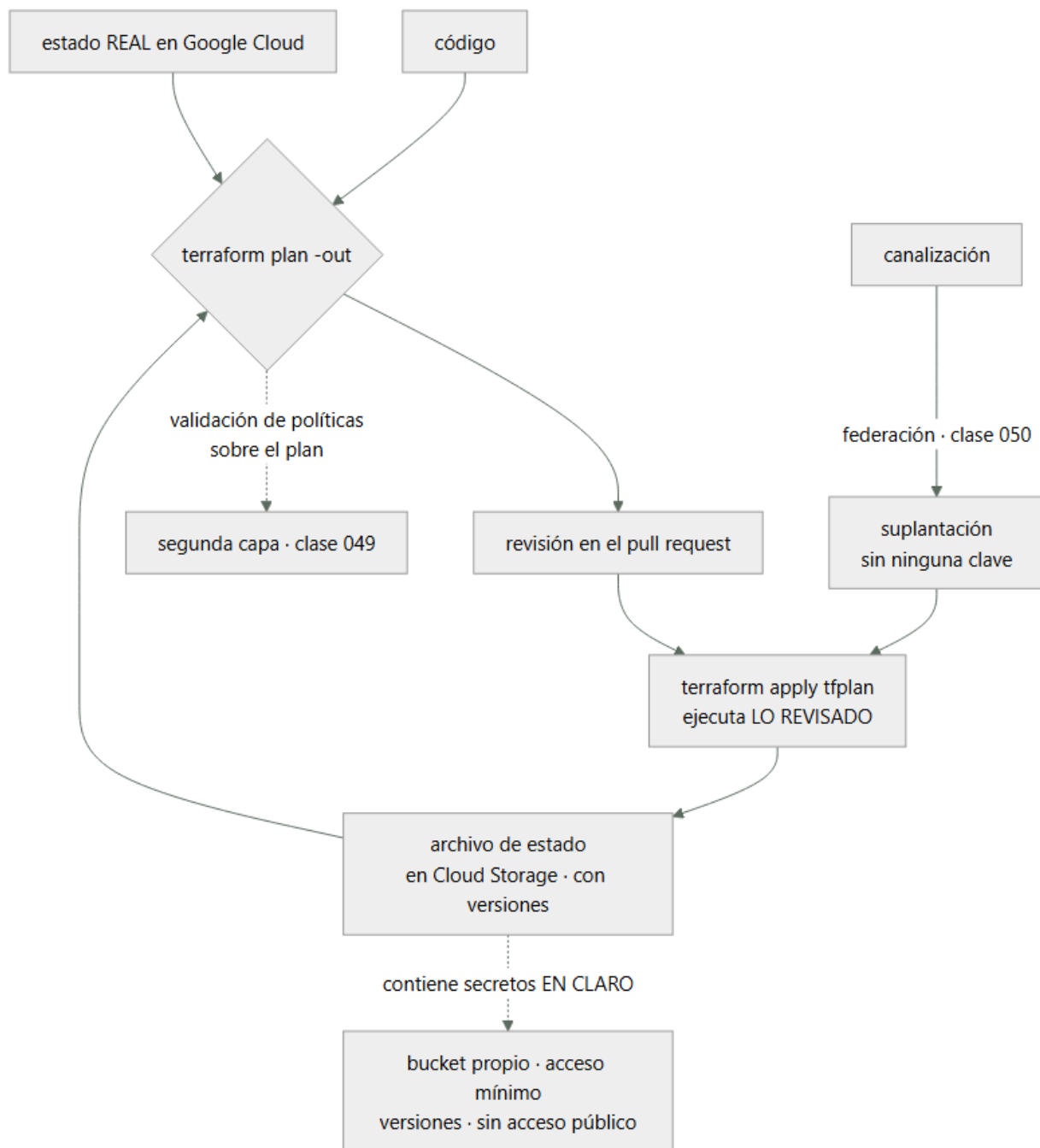
Concepto	Comprensión verificable
archivo de estado	Registro de lo que este código gestiona y de sus atributos. Permite destruir, detectar desviación e importar — y guarda valores sensibles en claro, así que es el fichero más delicado del proyecto.
bloqueo de estado	Exclusión mutua para que dos ejecuciones simultáneas no escriban a la vez. Sin él, dos canalizaciones concurrentes pueden corromper el estado.
plan guardado	Resultado de la planificación escrito a fichero. Aplicar ese fichero garantiza que se ejecuta lo revisado; sin él, apply vuelve a planificar y puede hacer otra cosa.
reindexación por count	Las instancias de un recurso con count se identifican por posición. Quitar una del medio desplaza a todas las siguientes, y Terraform destruye y recrea las que no cambiaron.
suplantación del proveedor	Autenticación de Terraform como una cuenta de servicio sin ninguna clave, partiendo de la identidad de la canalización. Es la aplicación directa de la clase 050.
desviación	Diferencia entre lo declarado y lo que existe de verdad. Con estado se detecta en la planificación; sin estado, un borrado ajeno pasa desapercibido.

Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. El estado: lo que resuelve y lo que obliga

La clase 047 terminó con una lista de lo que se pierde al no tener estado. Terraform lo tiene, así que esa lista se convierte en capacidades:

saber qué gestiona este código	el estado es la lista de pertenencia
destruir ordenadamente	terraform destroy, en orden de dependencias
detectar que alguien BORRÓ algo	la planificación lo muestra como cambio
adoptar lo creado a mano	import, sin recrear nada

La tercera es la que más se echa de menos sin estado: si alguien elimina un recurso desde la consola, un modelo sin estado simplemente lo vuelve a crear en el siguiente despliegue, sin decir que había desaparecido. Con estado, la planificación lo señala:

```
# google_compute_firewall.permitir_datos has been deleted
- resource "google_compute_firewall" "permitir_datos" { ... }
Plan: 1 to add, 0 to change, 0 to destroy.
```

Y las tres obligaciones que trae, en orden de importancia:

Contiene secretos en claro. Cualquier valor que haya pasado por un recurso —una contraseña generada, una clave, un certificado— queda escrito. No hay cifrado a nivel de campo:

```
$ terraform state pull | jq -r '.resources[].instances[].attributes | select(.password)' | head -3
```

Eso convierte al estado en el fichero más sensible del repositorio, y su custodia en una decisión de seguridad, no de almacenamiento:

```
terraform {
  backend "gcs" {
    bucket = "cls-tfstate-prod"
    prefix = "red"
  }
}
```

bucket propio, en un proyecto de plataforma
acceso uniforme y prevención de acceso público (clase 053)
versionado activo: un estado corrupto se recupera
IAM mínimo: quien puede leer el estado puede leer los secretos
clave del cliente si el requisito lo exige (clase 058)

Hay que bloquearlo. Dos ejecuciones a la vez sobre el mismo estado lo corrompen. El backend de Cloud Storage lo gestiona, y conviene comprobarlo la primera vez en lugar de suponerlo: una segunda ejecución debe esperar o fallar con un error de bloqueo, no continuar.

Hay que decidir su granularidad, y esa decisión es la misma que en la clase 047 con otra pieza. Allí el radio de impacto lo marcaba el grupo de recursos; aquí lo marca el estado:

un estado para toda la plataforma
→ terraform destroy en pruebas puede tocar lo que comparte estado
→ cada planificación consulta cientos de recursos: lenta
→ dos equipos se bloquean mutuamente

un estado por unidad de despliegue
cls-tfstate-prod/red
cls-tfstate-prod/datos
cls-tfstate-prod/tienda
→ cada uno se destruye y se planifica por separado

Y para conectarlos, salidas remotas o —mejor— fuentes de datos que consultan Google Cloud directamente, que no crean acoplamiento entre estados:

```
data "google_compute_subnetwork" "tienda" {
  name     = "snet-tienda-euw1"
  region   = "europe-west1"
  project  = var.proyecto_red
}
```

Una dependencia por dato es más robusta que una por estado: si el estado de red se recrea, el consumidor no se entera.

2. Aplicar lo revisado, y las dos formas de destruir sin querer

El plan guardado no es opcional. Sin él, apply vuelve a planificar en el momento de ejecutarse, y entre la revisión y la ejecución pueden haber cambiado cosas: una fuente de datos, una versión de imagen, un recurso tocado a mano.

```
$ terraform plan -out=tfplan -lock-timeout=5m
$ terraform show -no-color tfplan > plan.txt      # esto es lo que se revisa
$ terraform apply tfplan                          # esto ejecuta EXACTAMENTE eso
```

La diferencia con terraform apply a secas es que el segundo pide confirmación sobre un plan nuevo que nadie ha revisado. En una canalización con aprobación manual, eso rompe la garantía entera: se aprueba un texto y se ejecuta otro.

Y ahora las dos formas de destruir recursos sanos, que son las que producen los incidentes de esta clase.

La primera: count y la reindexación. Las instancias creadas con count se identifican por su posición:

```
resource "google_compute_firewall" "reglas" {
  count = length(var.reglas)
  name  = var.reglas[count.index].nombre
  # ...
}
```

```
}
```

Quitar el segundo elemento de una lista de cinco desplaza a los tres siguientes, y Terraform los ve como cambios de identidad:

```
# google_compute_firewall.reglas[1] must be replaced
# google_compute_firewall.reglas[2] must be replaced
# google_compute_firewall.reglas[3] must be replaced
Plan: 3 to add, 0 to change, 4 to destroy.
```

Se pidió quitar una regla y se van a recrear tres que no cambiaron. Con reglas de firewall el efecto es una ventana sin protección; con bases de datos o discos, es pérdida de datos.

La corrección es identificar por clave estable en lugar de por posición:

```
resource "google_compute_firewall" "reglas" {
  for_each = { for r in var.reglas : r.nombre => r }
  name     = each.value.nombre
  # ...
}
```

Con foreach, quitar un elemento afecta solo a ese elemento. La regla de equipo es corta y ahorra incidentes: count únicamente para «cero o uno»; para colecciones, siempre foreach.

La segunda: el estado compartido y destroy. Es el mismo incidente que en la clase 047 causó el modo completo, con otro mecanismo: terraform destroy en un directorio cuyo estado incluye recursos compartidos se los lleva.

Las protecciones, que se ponen antes y no después:

```
resource "google_sql_database_instance" "pedidos" {
  # ...
  deletion_protection = true           # protección del propio proveedor
  lifecycle { prevent_destroy = true } # Terraform se niega a planificar su borrado
}
```

Las dos capas hacen cosas distintas: preventdestroy hace fallar la planificación —así que el error aparece en la revisión, antes de ejecutar nada—, y deletionprotection lo impide en el servicio aunque alguien lo intente por otra vía. Y para el proyecto entero existe un gravamen que impide su eliminación, que es el equivalente al bloqueo de recurso de la clase 042:

```
$ gcloud resource-manager liens create --restrictions=resourcemanager.projects.delete \
  --parent=projects/cls-datos-prod-euw1-01 --reason="produccion"
```

Y una comprobación que conviene automatizar en la canalización: rechazar cualquier plan que destruya recursos no esperados.

```
$ terraform show -json tfplan | jq -r '
  .resource_changes[] | select(.change.actions | index("delete"))
  | .address' | tee borrados.txt
$ [ ! -s borrados.txt ] || echo "revisar: el plan destruye recursos"
```

No se trata de prohibir los borrados, sino de que ninguno pase sin que alguien lo haya leído.

3. Autenticar sin claves, que es lo que la clase 050 exigía

La forma cómoda de autenticar Terraform en una canalización es una clave de cuenta de servicio en un secreto del sistema de CI. Es exactamente lo que la clase 050 eliminó, y lo que la política de organización de la clase 049 impide crear.

La forma correcta encadena las dos piezas ya montadas:

```
- uses: google-github-actions/auth@v2
  with:
    workload_identity_provider: projects/418293047512/locations/global/workloadIdentityPools/cls-ci/
    providers/github
    service_account: tf-plan@cls-plataforma.iam.gserviceaccount.com
```

Y dentro de Terraform, la suplantación permite que la identidad de planificación y la de aplicación sean distintas:

```
provider "google" {
  project      = var.proyecto
  region       = var.region
  impersonate_service_account = var.cuenta_terraform
}
```

```
tf-plan@...    permisos de LECTURA sobre todo lo que gestiona
               se usa en los pull requests: puede planificar, no puede cambiar
tf-apply@...   permisos de escritura acotados
               solo desde la rama principal, con la condición de atributo de la clase 050
```

Esa separación es la que hace seguro publicar el plan en el pull request: la identidad que lo genera no puede aplicar nada, así que una rama maliciosa no consigue más que leer. Sin ella, ejecutar plan desde una rama arbitraria es dar permiso de escritura a quien pueda abrir un pull request.

Y dos particularidades del proveedor de Google que producen fallos intermitentes si no se anticipan:

Las API hay que habilitarlas y tardan en propagarse (clase 049):

```
resource "google_project_service" "servicios" {
  for_each = toset(["run.googleapis.com", "sqladmin.googleapis.com",
                   "secretmanager.googleapis.com"])
  project      = var.proyecto
  service      = each.value
  disable_on_destroy = false
}
```

`disable_on_destroy = false` importa: al destruir el directorio, deshabilitar una API afecta a todo el proyecto, incluido lo que gestionan otros. Es un borrado con radio de impacto mayor del que el código sugiere.

Hay dos proveedores, estable y beta. Un recurso o un atributo puede existir solo en beta, y mezclarlos exige declararlo en el recurso:

```
resource "google_some_resource" "x" {
  provider = google-beta
  # ...
}
```

Y conviene saber lo que implica: un recurso en beta puede cambiar de forma incompatible entre versiones del proveedor. Fijar la versión del proveedor deja de ser una buena práctica y pasa a ser un requisito:

```
terraform {
  required_version = "~> 1.9"
  required_providers {
    google      = { source = "hashicorp/google",      version = "~> 6.12" }
    google-beta = { source = "hashicorp/google-beta", version = "~> 6.12" }
  }
}
```

Y el fichero de bloqueo de dependencias se versiona en el repositorio, por la misma razón que cualquier otro: dos ejecuciones del mismo código en semanas distintas deben usar el mismo proveedor.

4. La canalización y la segunda capa de gobierno

El flujo completo, que es el entregable de esta clase:

```
en el pull request
1. terraform fmt -check          formato
2. terraform validate            sintaxis y referencias
3. tflint                        errores propios del proveedor
4. terraform plan -out=tfplan    con la identidad de LECTURA
5. validación de políticas sobre el plan
6. publicar el plan en el pull request

al fusionar en la rama principal
7. terraform apply tfplan        con la identidad de escritura, desde main
```

El paso 5 es el que convierte esto en gobierno y no solo en automatización. La clase 049 dejó las políticas de organización, que actúan en el momento de la llamada a la API; una validación sobre el plan actúa antes, en la revisión, y puede expresar reglas que una política de organización no expresa:

```
package terraform.cloudshop

deny[msg] {
  r := input.resource_changes[_]
  r.type == "google_storage_bucket"
  not r.change.after.uniform_bucket_level_access
  msg := sprintf("%s: falta acceso uniforme a nivel de bucket", [r.address])
}
```

```
deny[msg] {
  r := input.resource_changes[_]
  r.type == "google_sql_database_instance"
  r.change.after.settings[_].ip_configuration[_].ipv4_enabled
  msg := sprintf("%s: no debe tener IP pública (clase 054)", [r.address])
}
```

Son las dos capas de la clase 048 otra vez, con otros nombres: el código fija el estado al crear y la política vigila después, y ahora se añade una tercera que avisa antes de crear, en la revisión, cuando corregir cuesta menos.

Y el resultado de las tres capas juntas, expresado como decisiones de las clases anteriores que quedan declaradas en un solo recurso:

```
resource "google_storage_bucket" "facturas" {
  name                = "cls-facturas"
  location             = "EU"
  storage_class        = "STANDARD"
  uniform_bucket_level_access = true           # clase 053
  public_access_prevention = "enforced"        # clases 049, 053
  versioning { enabled = true }                # clase 053

  soft_delete_policy { retention_duration_seconds = 604800 }

  lifecycle_rule {                               # clase 053
    condition { days_since_noncurrent_time = 90 }
    action { type = "Delete" }
  }

  encryption { default_kms_key_name = google_kms_crypto_key.facturas.id } # 058
  labels = local.etiquetas                                                # clase 049

  lifecycle { prevent_destroy = true }
}
```

Siete decisiones de cinco clases distintas, en un recurso, imposibles de olvidar al crear el siguiente. Esa es la función real de la infraestructura como código en este programa, y es idéntica a la que la clase 047 enunció con otra herramienta.

5. Con estado o sin él: la comparación honesta

Con las clases 047 y 059 hechas, se puede comparar sin partidismo. La parte 07 profundiza; aquí interesa el criterio operativo.

	Bicep / ARM (047)	Terraform (059)
Estado	No: pregunta a la nube	Fichero que hay que custodiar
Saber qué gestiona el código	No, salvo pilas de despliegue	Sí
Detectar un borrado ajeno	No	Sí, en la planificación
Destrucción ordenada	Modo completo o pilas	destroy, por dependencias
Secretos	No quedan en ningún estado	En claro en el estado
Ejecuciones simultáneas	Sin problema	Requieren bloqueo
Multiproveedor	No	Sí
Adoptar recursos existentes	Automático	import explícito
Fidelidad de la previsión	what-if, con ruido	Plan, más fiel, con known after apply

Dos filas resumen el intercambio: el estado compra visibilidad y cobra custodia. Saber qué gestionas y detectar desviación son capacidades operativas de primer orden; a cambio, el fichero más sensible del proyecto pasa a ser responsabilidad tuya.

Y hay una fila que no está en la tabla porque no es técnica y suele decidir: Terraform vale para los tres proveedores de este programa. Un equipo que opera en más de una nube paga el coste de aprender una herramienta en vez de tres, y —más importante— puede aplicar el mismo flujo de revisión, la misma validación de políticas y la misma canalización en todas. Eso es exactamente el tipo de contrato portable que la clase 048 pedía identificar, y aparece en la capa que

más trabajo repetido genera.

Lo que no hace portable a Terraform, y conviene decirlo para no crear una expectativa falsa: los recursos son específicos del proveedor. Un módulo de red de Google Cloud no vale en Azure. Lo portable es el método —planificar, revisar, validar, aplicar, versionar—, no el código.

Y dos límites que comparten las dos herramientas y que la parte 07 desarrollará:

no hay transacción	un fallo a mitad deja lo ya aplicado → las plantillas deben ser reejecutables
no cubren el día dos	ninguna gestiona una migración de datos, un cambio de esquema ni un despliegue progresivo

La segunda es la que más veces se olvida al defender la infraestructura como código: describe el estado deseado, no el camino para llegar a él cuando el camino importa. Para eso están las clases de entrega continua de la parte 08.

Ejemplo trabajado

CloudShop pasa su plataforma de Google Cloud a Terraform. El equipo llega con la experiencia de la clase 047 y evita dos de sus incidentes; los cuatro que aparecen son propios del modelo con estado.

Lo que se evitó por venir escrito de la clase 047:

- un estado por unidad de despliegue, no uno compartido
- decidir una sola forma por tipo de recurso hijo
- plan obligatorio y revisado antes de aplicar
- nada de secretos en salidas

Incidente 1 — catorce personas podían leer todas las contraseñas.

Una revisión de accesos sobre el bucket de estado:

```
$ gcloud storage buckets get-iam-policy gs://cls-tfstate-prod \
  --format="value(bindings.members)" | tr ';' '\n' | wc -l
14
$ terraform state pull | grep -c "password"
7
```

El bucket había heredado los permisos del proyecto de plataforma, donde catorce personas tenían lectura. Siete contraseñas en claro, incluidas las de las bases de datos de producción.

proyecto del bucket de estado	plataforma	proyecto propio
principales con lectura	14	2
versionado del bucket	no	sí
prevención de acceso público	no aplicada	aplicada
cifrado con clave del cliente	no	sí (clase 058)
contraseñas rotadas tras el hallazgo	—	las 7

La rotación no era opcional: un secreto que estuvo accesible se considera comprometido, exactamente como en la clase 047 con el historial de despliegues.

Incidente 2 — se pidió quitar una regla y se recrearon tres.

Plan: 3 to add, 0 to change, 4 to destroy.

La revisión del pull request lo detectó antes de aplicar, porque el plan se publicaba y alguien lo leyó. Las reglas de firewall estaban declaradas con count sobre una lista, y quitar el segundo elemento desplazó a los tres siguientes.

identificación de las instancias	por posición	por nombre (`for_each`)
recursos afectados al quitar uno	4	1
ventana sin regla de firewall	~40 s por regla	ninguna
comprobación de borrados en el plan	ninguna	falla si hay borrados no declarados

Incidente 3 — se aprobó un plan y se ejecutó otro.

Una aplicación en producción creó una instancia con un tipo de máquina distinto al revisado. La causa:

```
# la canalización hacía esto
$ terraform plan          # se publica y se aprueba
$ terraform apply -auto-approve # ← vuelve a planificar
```

Entre la revisión y la ejecución, una fuente de datos que resolvía la última imagen devolvió una versión nueva.

plan	sin guardar	-out=tfplan
------	-------------	-------------

aplicación	replanifica	aplica el fichero
diferencias entre lo aprobado y lo aplicado	1	0
plan publicado en el pull request	sí	sí

Incidente 4 — la canalización usaba una clave, contra la política.

```
$ gcloud asset search-all-resources --scope organizations/$ORG_ID \
  --asset-types iam.googleapis.com/ServiceAccountKey \
  --query "NOT name:*/keys/system-managed*" --format="value(name)"
projects/cls-plataforma/serviceAccounts/terraform@.../keys/9f2c...
```

Una clave creada antes de aplicar la política de la clase 050, guardada en el sistema de CI. Y con permisos de escritura, usada también en los pull requests: cualquiera que abriera uno podía ejecutar una planificación con credenciales de escritura.

autenticación	clave en el CI	federación (050)
identidad en pull request	escritura	lectura, tf-plan@
identidad en la rama principal	la misma	escritura, tf-apply@
condición de atributo	ninguna	repositorio + rama main
claves de cuenta de servicio	1	0

Y la prueba negativa, la cuarta vez que este programa la ejecuta:

plan desde una rama de trabajo	funciona, solo lectura	✓
apply desde una rama de trabajo	permiso denegado	✓
apply desde main	funciona	✓

Incidente 5 — destroy en pruebas tocó algo compartido.

El directorio de pruebas incluía el recurso googleprojectservice de las API, con el valor por defecto:

```
google_project_service.servicios["run.googleapis.com"] will be destroyed
```

Deshabilitar esa API afecta al proyecto entero, no solo a lo que gestiona ese directorio. La destrucción del entorno de pruebas dejó sin servicio a dos despliegues que compartían proyecto.

disable_on_destroy	true (defecto)	false
proyectos por entorno	compartido	uno por entorno
protección de recursos críticos	ninguna	prevent_destroy + deletion_protection + gravamen
duración del incidente	26 min	—

Resumen del paso a infraestructura declarada:

principales con acceso al estado	14	2
secretos en claro accesibles	7	0
recursos destruidos sin pedirlo	4 en 1 cambio	0
planes aplicados distintos del revisado	1	0
claves de cuenta de servicio en la canalización	1	0
estados de Terraform	1	4
duración de una planificación	3 min 40 s	38 s
reglas de política validadas sobre el plan	0	11

La lección que esta clase traslada al proyecto de la clase 060 y a la parte 07: el archivo de estado da tres capacidades que Bicep no tenía y cobra por ellas con una responsabilidad concreta —es el fichero que contiene todos los secretos, y su control de acceso es un control de seguridad de primer nivel, no una configuración de almacenamiento—. Y las dos formas de destruir sin querer que aparecieron aquí, la reindexación y el estado compartido, son la misma lección que el modo completo de la clase 047 con otro mecanismo: una herramienta declarativa hace exactamente lo que dice el código, y el código dice más cosas de las que parece.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/059-terraform-y-despliegues-reproducibles-en-gcp/lab.py
```

El laboratorio selecciona el motor de práctica iac y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.

3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa infraestructura-gcp en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un plan reproducible sin secretos ni cambios inesperados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye infraestructura-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Los secretos de producción son legibles por más gente de la prevista	El archivo de estado guarda los valores en claro y su bucket heredó permisos amplios	Bucket propio en un proyecto de plataforma, IAM mínimo, versionado, sin acceso público, y rota lo que estuvo expuesto.
Quitar un elemento de una lista destruye y recrea varios recursos	Las instancias con count se identifican por posición y se reindexan	Usa foreach con una clave estable; reserva count para el caso de cero o uno.
Se aplica algo distinto de lo que se revisó	apply sin plan guardado vuelve a planificar en el momento de ejecutarse	terraform plan -out=tfplan y terraform apply tfplan; publica el plan revisado en el pull request.
Cualquiera que abre un pull request puede modificar infraestructura	La canalización usa una única identidad con permisos de escritura para planificar y aplicar	Dos cuentas: lectura para planificar en ramas, escritura solo desde la principal, con federación y condición de atributo.
Destruir un entorno de pruebas deja sin servicio a otro despliegue	El estado incluía recursos de alcance de proyecto, como la habilitación de una API	Un proyecto por entorno, disableondestroy = false, y protecciones en los recursos críticos.
Dos ejecuciones simultáneas corrompen el estado	No hay bloqueo configurado o se está usando un backend que no lo soporta	Usa el backend de Cloud Storage y comprueba que una segunda ejecución espera o falla, en vez de suponerlo.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué tres capacidades da el archivo de estado que la clase 047 no tenía, y qué obligación impone cada una?
2. ¿Por qué terraform apply sin plan guardado rompe la garantía de una aprobación manual?

3. Explica la reindexación por count con un ejemplo y di cuándo sigue siendo correcto usarlo.
4. ¿Por qué la identidad que planifica en un pull request no debe poder aplicar?
5. ¿Qué hace portable a Terraform entre proveedores y qué no lo hace?

Referencias

- HashiCorp (2025). State: purpose and sensitive data — qué guarda el estado y por qué se protege.
<<https://developer.hashicorp.com/terraform/language/state/sensitive-data>>
- HashiCorp (2025). count and foreach — identificación por posición frente a clave estable.
<<https://developer.hashicorp.com/terraform/language/meta-arguments/foreach>>
- HashiCorp (2025). Command: plan and saved plan files — planificar, guardar y aplicar exactamente lo revisado.
<<https://developer.hashicorp.com/terraform/cli/commands/plan>>
- Google Cloud (2025). Terraform GCS backend and state management — bucket de estado, bloqueo y versionado.
<<https://cloud.google.com/docs/terraform/resource-management/store-state>>
- Google Cloud (2025). Authenticate Terraform with Workload Identity Federation — suplantación sin claves en la canalización.
<<https://cloud.google.com/blog/products/identity-security/enabling-keyless-authentication-from-github-actions>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar:
 [Parte 04 en PDF](#) ·
 [Recorrido de Google Cloud en PDF](#) ·
 [Manual integral](#)

Anterior	Índice	Siguiente
← 058 · Cloud KMS, Secret Manager y Security Command Center	Parte 04 · Programa	060 · Proyecto: aplicación de tres capas en Google Cloud →

Evaluación — 059 Terraform y despliegues reproducibles en GCP

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega infraestructura-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

060 — Proyecto: aplicación de tres capas en Google Cloud

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Integrar las once clases anteriores en una plataforma de Google Cloud que funcione, se observe, falle de forma controlada y cueste algo explicable — y calificar con evidencia la hipótesis que escribió la clase 048. Esa hipótesis acertó en dos de tres afirmaciones y falló en la tercera, y el fallo es más valioso que los aciertos: la familia de incidentes más cara de esta parte no fue la que se predijo, y corregir la ley con lo observado es lo que se lleva a la parte 05.

Resultados de aprendizaje

Al finalizar podrás:

1. Trazar cada componente hasta la clase que lo decidió y la alternativa descartada.
2. Calificar la hipótesis de la clase 048 con evidencia, incluida la parte que resultó falsa.
3. Enunciar las leyes que ya se han observado en tres plataformas independientes.
4. Provocar tres fallos y medir detección, impacto y recuperación, incluida la recuperación bloqueada.
5. Comparar tres plataformas por costo por unidad y explicar por qué la mayor partida es la misma en las tres.

Conceptos centrales

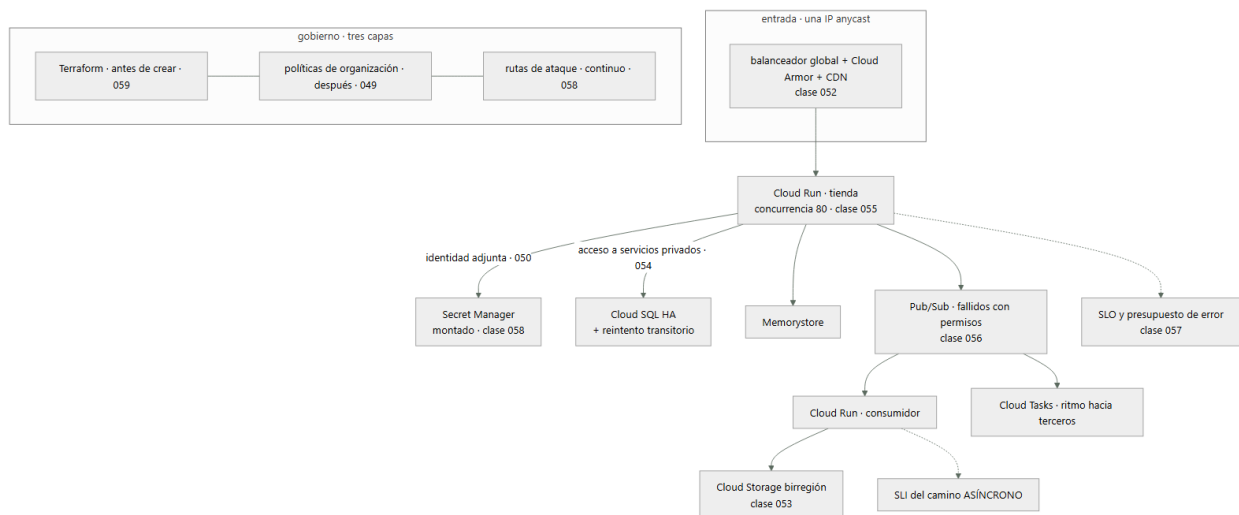
Concepto	Comprensión verificable
hipótesis calificada	Predicción escrita antes de la evidencia y evaluada después, incluida la parte errónea. Sin la parte errónea no es una hipótesis: es un resumen.
ley observada	Comportamiento que ha aparecido en tres plataformas independientes con tres mecanismos distintos. Deja de ser una peculiaridad del proveedor y pasa a ser una propiedad del problema.
recuperación bloqueada	Situación en la que la herramienta necesaria para reparar depende de lo que se ha roto. Es el fallo que ningún plan detecta sin ensayarlo.
conmutación parcial	El tráfico se traslada de región y los datos no. La infraestructura responde correctamente y el servicio incumple su objetivo.
SLI del camino asíncrono	Indicador de frescura del trabajo en segundo plano. Un SLO de la API no lo cubre: un consumidor detenido no afecta a ninguna petición HTTP.
costo por unidad de negocio	Única comparación accionable entre plataformas. Compara arquitecturas antes que proveedores, y por eso la mayor partida suele coincidir.

Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. De dónde sale cada decisión, y qué trampa evitó

La arquitectura entregada es el resultado de once decisiones con su alternativa descartada. La cuarta columna es la que solo se puede escribir después de recorrer la parte:

Componente	Requisito	Alternativa descartada	Trampa que evita
Proyectos por servicio y entorno	Cuotas y superficie (049)	Tres proyectos por entorno	La cuota de región que bloquea una campaña
Cuenta de servicio dedicada por carga	Privilegio mínimo (050)	La cuenta por defecto	Ejecutar con Editor sin saberlo
Federación con condición de atributo	Sin claves (050)	Clave en el sistema de CI	Que cualquier repositorio obtenga credenciales
Una VPC global, compartida	Simplicidad y gobierno (051)	Una red por equipo	Cinco redes en la sombra sin revisar
Firewall dirigido por cuenta de servicio	Integridad del control (051)	Etiquetas de red	Abrir un puerto poniéndose una etiqueta
Acceso privado a Google en todas las subredes	Costo y aislamiento (051)	Salida por NAT	Pagar 108 USD/mes por tráfico que no debía salir
Cloud Run con concurrencia 80	Costo y densidad (055)	Concurrencia 1	Pagar seis veces por costumbre
Cloud SQL con reintento transitorio	Continuidad (054)	Confiar en la alta disponibilidad	Perder peticiones en cada conmutación
Pub/Sub con fallidos y permisos	Trabajo visible (056)	Fallidos solo configurados	Una red de seguridad que no existe
SLO con presupuesto de error	Alertas útiles (057)	Umbrales de recurso	340 avisos y 6 incidentes
Terraform con estado por unidad	Radio de impacto (059)	Un estado compartido	Destruir lo que comparte estado

Y cuatro decisiones tomadas en contra de la traducción literal desde Azure, que son las que un revisor cuestionará:

- No se replicó el diseño de concentrador y radios.
La VPC es global: los emparejamientos y las tablas de rutas sobran (051).
- No se agruparon servicios en pocos proyectos grandes.
Las cuotas son por proyecto y los proyectos son baratos: la granularidad fina es una decisión de capacidad, no de orden (049).
- No se usó una única cuenta de almacenamiento por entorno.
Aquí la ubicación y la clase son propiedades del bucket y del objeto, no de una cuenta compartida (053).
- No se copió la política de ciclo de vida de la parte 03.

El precio por operación sube al enfriar, y la política traducida habría costado más de lo que ahorra (053).

La cuarta es la que mejor resume el riesgo de la portabilidad mal entendida: un patrón correcto en una plataforma puede destruir valor en otra sin que nada falle.

2. La hipótesis de la clase 048, calificada

La clase 048 dejó escrito, para poder equivocarse de forma comprobable:

El contrato de once filas se conservará entero en Google Cloud. Las excepciones serán otras diez, distintas de estas, y la que más incidentes producirá volverá a ser una en la que el sistema funcione por el camino equivocado en lugar de dar un error.

Tres afirmaciones. Dos se cumplieron y una es falsa.

Afirmación 1 — el contrato se conserva. CIERTA, con dos matices.

contrato	¿se conservó?
identidad de carga sin secretos	sí · cuenta de servicio adjunta
sujeto de federación acotado	sí · y volvió a fallar la 1. ^a vez
frontera de aislamiento del entorno	sí · proyecto, pero BARATO
salida declarada con costo por GB	sí · Cloud NAT
acceso privado a servicios gestionados	sí · acceso privado a Google
entrega al menos una vez	MATIZADO ↓
cola de fallidos con alerta a cero	sí · pero hay que darle permisos
idempotencia en el manejador	REFORZADO ↓
clave de partición irreversible	sí · tercera forma, silenciosa
prueba negativa por control	sí · doce ejecutadas
costo por unidad de negocio	sí

Los dos matices son mejoras del contrato, no excepciones:

entrega al menos una vez
Pub/Sub ofrece entrega EXACTAMENTE UNA VEZ dentro de una región.
El contrato se refina: lo que no existe en ninguna plataforma es el EFECTO exactamente una vez.

idempotencia en el manejador
gana una razón nueva: la reproducción de mensajes (056).
Ya no se justifica solo por los duplicados que ocurren, sino por los que se provocan a propósito al reparar un daño.

Afirmación 2 — las excepciones serán otras diez. CIERTA.

Ninguna de las diez de Azure se repitió, y aparecieron diez propias:

el proyecto es barato y cambia la granularidad	049
se pueden crear proyectos fuera de la organización	049
la facturación está fuera de la jerarquía y su exportación no es retroactiva	049
la cuenta de servicio es identidad Y recurso	050
la VPC es global	051
el descuento por uso sostenido se aplica solo	052
la reparación automática destruye infraestructura sana	052
el precio por operación sube al enfriar la clase	053
conurrencia por instancia y CPU solo durante la petición	055
el archivo de estado contiene los secretos en claro	059

Afirmación 3 — el peor incidente será «funcionar por el camino equivocado». FALSA.

Y es el resultado más útil de la parte. En Google Cloud, los equivalentes de aquel fallo fueron todos ruidosos:

sin acceso privado a Google	la llamada falla	→ 11 min hasta corregir
sin índice en Firestore	la consulta falla	→ visible al desplegar
Cloud SQL sin ruta privada	la conexión falla	→ visible al desplegar

Lo que sí produjo los incidentes más caros fue otra familia:

la cola de fallidos que no recibía nada por falta de permisos (056)
1.841 mensajes atascados durante tres semanas, sin una sola alerta, porque la ausencia de mensajes se interpretó como buena señal

la regla de ciclo de vida que costó 210 USD para ahorrar 5,22 (053)
nada falló: cobró

la estimación con precios de lista, que sobrevaloró el gasto (052)
nada falló: la conclusión era falsa

Las tres tienen una forma común que la clase 048 no supo nombrar. La ley corregida:

El fallo más caro no es el que ocurre por el camino equivocado. Es el de un mecanismo que parece estar haciendo algo y no lo está — y su caso más peligroso es aquel en el que la ausencia de señal se interpreta como buena noticia.

Eso incluye el camino privado silencioso de la clase 039, la cola de fallidos vacía de la 056, los registros apagados de la 045 y el control de seguridad revertido de la 046. Cuatro caras del mismo problema, en tres plataformas.

Y la consecuencia operativa, que es lo que se lleva a la parte 05: todo mecanismo de protección necesita una prueba que lo obligue a actuar. No basta con que no salten alarmas: hay que provocar la condición y ver la reacción. Una cola de fallidos sin un mensaje envenenado de prueba, una copia de seguridad sin una restauración y un antimalware sin un fichero de prueba son la misma afirmación sin evidencia.

3. Diez leyes ya observadas en tres plataformas

Un comportamiento que aparece tres veces con tres mecanismos distintos ha dejado de ser una peculiaridad del proveedor. Estas diez ya cumplen ese criterio, y son el activo que hace barata la cuarta plataforma:

1. El permiso suma; lo que resta vive en OTRO sistema, con otro error.
SCP · Azure Policy · políticas de organización y de denegación
2. El gobierno guarda la puerta y no limpia la casa.
Inventariar → corregir → imponer. Al revés, panel en verde con catorce recursos incumpliendo (046).
3. Los puertos de traducción de salida se agotan, y el error apunta al destino.
NAT gateway · puertos repartidos del balanceador · 64 por máquina
4. El caudal de disco es el MÍNIMO de dos límites, y casi nunca es el que compraste.
volumen/instancia · disco/máquina · GB del disco/vCPU
5. La unidad de orden es también la unidad de serialización.
FIFO · sesiones · particiones · claves de ordenación · claves primarias
6. La entrega exactamente una vez se compra; el efecto se construye en el manejador.
Y con reproducción, la idempotencia deja de ser opcional.
7. La conmutación de una base de datos gestionada es funcionamiento normal.
El cliente reintenta o pierde peticiones, con las alarmas en verde (048).
8. Con versionado, borrar no libera nada hasta que una regla retire las versiones.
Marcadores de borrado · versiones · versiones no actuales
9. Una traza se corta en el salto que alguien escribió a mano.
Y siempre es el interesante.
10. Leer un secreto en cada petición agota una cuota, y el error parece de la base de datos. Key Vault · Secret Manager · gestores equivalentes

Diez leyes, cada una observada de forma independiente en tres implementaciones que no comparten código. Y una observación sobre su naturaleza que merece decirse: ninguna de las diez es un fallo de las plataformas. Todas son consecuencias de problemas reales —traducción de direcciones, orden distribuido, límites de E/S, entrega en red no fiable— que cualquier implementación tiene que resolver de alguna manera.

Eso convierte la lista en algo más útil que un resumen: es un cuestionario para la plataforma siguiente. Ante un proveedor nuevo, las diez preguntas ya están escritas y solo hay que averiguar cómo se llaman aquí las respuestas y dónde está la excepción.

Y dos afirmaciones que no han alcanzado ese estatus y conviene no dar por buenas:

"el archivo de estado es un problema"	depende de la herramienta, no del proveedor
"lo global es mejor que lo regional"	la VPC global simplifica y la conmutación parcial es un riesgo nuevo (abajo)

4. Tres fallos provocados y lo que enseñó cada uno

Los tres se eligieron para atacar supuestos distintos, y los tres dejaron un hallazgo que ninguna revisión de configuración habría dado.

Fallo 1 — pérdida de una zona. Se retiran las instancias de europe-west1-b y se fuerza la conmutación de Cloud SQL.

detección por alerta de consumo de presupuesto	1 min 10 s
errores durante la conmutación de la base	0 ← el reintento estaba (048)
p95 durante el episodio	142 ms
recuperación completa	2 min 05 s

Cero errores en la conmutación. Es la segunda vez que un problema conocido no cuesta nada, porque estaba escrito en el contrato.

Y el hallazgo, que es nuevo. Al perder capacidad en la región europea, el balanceador global desbordó tráfico a us-east1, tal como se había diseñado. Pero la base de datos seguía en Europa:

peticiones servidas desde us-east1	18.400
p99 de esas peticiones	612 ms (SLO: 500 ms)
causa	latencia transatlántica hacia la base de datos primaria

La conmutación funcionó en la capa de red y rompió el objetivo en la capa de datos. El desbordamiento por capacidad de la clase 052 mueve el cómputo y no mueve los datos, y ninguna de las dos piezas está mal configurada.

- correcciones
1. réplica de lectura en us-east1, y lecturas dirigidas a la local
 2. las escrituras siguen yendo a Europa: se acepta y se documenta
 3. el escalador de capacidad de us-east1 se limita a 0,3:
absorbe un pico, no sustituye a la región
 4. el simulacro pasa a medir el p99 POR REGIÓN DE ORIGEN, no agregado

La cuarta es la que habría detectado el problema sin romper nada: el percentil agregado estaba dentro del objetivo porque solo el 12 % del tráfico venía de la otra región.

Fallo 2 — revocación de una clave del cliente. Se retira el permiso de la identidad sobre k-facturas.

servicios afectados esperados	1 (subida de facturas)	
servicios afectados reales	1	← la separación funcionó
recuperación tras restaurar	4 min	

La separación por propósito de la clase 058, que venía del simulacro de la clase 048, evitó el radio de impacto ampliado. Pero apareció otra cosa:

El hallazgo: la recuperación estaba bloqueada por lo que se había roto. El bucket del estado de Terraform usaba la misma clave, así que el intento de restaurar el permiso mediante la canalización falló:

```
Error: googleapi: Error 403: The caller does not have permission  
to use kms key ... for bucket cls-tfstate-prod
```

La herramienta que arregla la plataforma dependía de la clave que se acababa de revocar. La recuperación se hizo a mano, con una identidad de emergencia, y tardó cuatro minutos porque alguien conocía el camino — no porque estuviera escrito.

- correcciones
1. el bucket de estado usa una clave PROPIA, en otro anillo
 2. procedimiento de recuperación con identidad de emergencia, escrito y ensayado
 3. regla general añadida a la línea base:
ninguna herramienta de recuperación puede depender de lo que recupera

Fallo 3 — consumidor detenido en silencio. El fallo que este programa ha provocado en las tres plataformas.

detección	no hubo	4 min 20 s	3 min 40 s
mensajes acumulados	14	680	512

El hallazgo: la alerta de acumulación se disparó y la alerta de presupuesto de error no, exactamente como la clase 057 anticipó — un consumidor detenido no afecta a ninguna petición HTTP, así que el SLI de la API no se mueve. Al revisarlo, apareció el hueco real:

SLO definidos	1 · disponibilidad y latencia de la API
SLO del camino asíncrono	ninguno

La plataforma tenía un objetivo para lo que el usuario ve al pulsar y ninguno para lo que espera recibir después.

corrección

SLI de frescura: proporción de notificaciones enviadas en menos de 5 min

SLO 99 % en 28 días, con alerta por velocidad de consumo

→ el mismo mecanismo de la clase 057, aplicado a un camino que no es HTTP

Los tres hallazgos comparten forma con los de la clase 048 y merece señalarlo: la infraestructura aguantó los tres fallos, y en los tres había algo que ninguna revisión de configuración podía mostrar — una latencia entre capas, una dependencia circular de recuperación y un objetivo que nunca se definió.

5. La entrega, las tres plataformas y la pregunta que abre la parte 05

La entrega, sin conocimiento tácito.

infraestructura	Terraform, 4 estados, plan revisado y validado por políticas
línea base	24 afirmaciones, cada una con su prueba negativa
verificar.sh	ejecuta las 24 y devuelve código de salida
políticas	de organización, y validación sobre el plan
ADR	11 decisiones con su alternativa descartada
riesgos residuales	5, con responsable y condición de revisión
SLO	2: camino síncrono y camino asíncrono
consultas guardadas	4, enlazadas desde el manual de operación
procedimientos	3 fallos ensayados, con verificación posterior
línea base medida	rendimiento, costo y costo por pedido

Los cinco riesgos residuales:

1. las escrituras solo ocurren en Europa: la conmutación a us-east1 es parcial
2. Secret Manager no rota por sí solo: la función de rotación es código propio
3. el gestor de claves externo se descartó; se revisará si hay obligación contractual
4. dos cuentas de emergencia con acceso permanente, documentadas
5. los registros de acceso a datos solo están activos en 4 proyectos

La comparación de las tres plataformas, hecha de forma que signifique algo.

rps sostenidos	987,4	984,1	985,7
p50	38,4 ms	41,2 ms	39,8 ms
p95	91,2 ms	96,8 ms	94,1 ms
p99	184,7 ms	203,5 ms	197,2 ms
p99/p50	4,8×	4,9×	5,0×
errores	0	0	0

No hay diferencia significativa de rendimiento a esta escala. Es la segunda vez que se mide y la conclusión se repite, así que ya se puede afirmar: a este volumen, el rendimiento no es un criterio de elección entre proveedores. Lo que decide son capacidades, costo por unidad, requisitos de residencia y competencias del equipo.

Y el costo, desglosado hasta que cada partida tenga nombre:

entrada y borde	48	185	62
cómputo de aplicación	164	344	84
base de datos relacional	210	390	390
caché	46	55	58
mensajería	31	22	29
almacenamiento y salida	42	48	34
salida a internet (NAT)	58	69	68
telemetría	52	110	88
seguridad y gobierno	37	96	48
	■■■■■	■■■■■	■■■■■
total	688	940	861
costo por pedido	0,000702	0,000959	0,000879

Dos lecturas, y la segunda es la importante.

La primera: Google Cloud queda entre las dos, un 25 % por encima de AWS y un 8 % por debajo de Azure. Las diferencias tienen nombre: el cómputo de aplicación es menos de la mitad que en Azure por la concurrencia de la clase 055, y el borde cuesta un tercio que la pasarela con WAF.

La segunda: la mayor partida es la misma en las tres, y en dos de ellas es idéntica. La base de datos relacional aprovisionada es entre el 30 % y el 45 % del total en todos los casos, porque es la misma decisión de arquitectura tomada tres veces. Y de ahí la conclusión que cierra las cuatro partes:

Comparar proveedores compara tu arquitectura contigo mismo. La partida que domina la factura la eligió el equipo, no el proveedor.

Cambiar de nube habría movido el 25 %. Cambiar la decisión de datos —a un motor que escala a cero, a un modelo que no necesita una instancia encendida 720 horas al mes— movería más, y no depende de ningún proveedor.

Y la pregunta que abre la parte 05.

En las tres plataformas, la capa de aplicación acabó ejecutando contenedores: Cloud Run aquí, Container Apps allí, servicios gestionados de contenedor en AWS. Las tres convergieron sin que nadie lo planificara, y eso reorienta la pregunta:

Si el contenedor es la unidad de despliegue en las tres nubes, ¿qué garantiza exactamente ese contrato y dónde deja de garantizar? ¿Qué parte de lo aprendido en estas cuatro partes es del proveedor, y qué parte es de la plataforma que ejecuta el contenedor?

La hipótesis que se escribe ahora, para poder equivocarse otra vez de forma comprobable:

El contrato del contenedor —imagen, proceso, puerto, variables, señales— será portable de verdad, y las fugas estarán en los bordes: almacenamiento persistente, red, identidad y todo lo que ocurre antes del primer proceso y después de la señal de terminación. Y volverá a aparecer al menos una de las diez leyes de esta clase, con un mecanismo nuevo.

La parte 05 la califica.

Ejemplo trabajado

Entrega del capstone de la parte 04, con las cifras que se llevan a la parte 05.

Verificación completa. Las 24 afirmaciones de la línea base:

\$./verificar.sh	
✓ federación deniega desde otro repositorio	permiso denegado
✓ ninguna clave de cuenta de servicio	0 encontradas
✓ creación de claves bloqueada	violación de restricción
✓ bucket no público	412 · publicAccessPrevention
✓ recuperación de bucket	50 objetos restaurados
✓ facturado converge con lo visible	430 GiB / 412 GiB
✓ URL firmada sin claves, 15 min	caduca correctamente
✓ subred sin salida a internet	tiempo agotado a los 5 s
✓ firewall dirigido por identidad	UNREACHABLE por 22
✓ acceso privado a Google activo	resuelve interno
✓ Cloud SQL sin IP pública	tiempo agotado
✓ reintento de errores transitorios	conmutación sin errores
✓ punto caliente de Spanner	6.400 escrituras/s
✓ servicios internos no públicos	403 sin testigo
✓ concurrencia verificada bajo carga	0 respuestas cruzadas
✓ cola de fallidos recibe	mensaje envenenado llega
✓ acumulación en cola alerta	aviso en 3 min 40 s
✓ auditoría de acceso a datos activa	entrada localizada
✓ traza completa entre servicios	6 de 6 tramos
✓ destrucción de versión de clave	restaurada en la ventana
✓ rotación de secreto con tráfico	0 errores
✓ estado de Terraform con acceso mínimo	2 principales
✓ plan aplicado idéntico al revisado	tfplan verificado
✓ pedido con comilla simple y punto y coma	201
24/24 correctas	

Línea base medida:

rps 985,7 · p50 39,8 ms · p95 94,1 ms · p99 197,2 ms · errores 0
 $L = 985,7 \times 0,0398 = 39,2$ de 50 solicitados → medida válida
 $p99/p50 = 5,0\times$
costo 861 USD/mes · 0,000879 USD/pedido

Los tres fallos, con lo aprendido en cada uno:

pérdida de zona	1 min 10 s	142 ms	0 peticiones 18.400 fuera del SLO	la conmutación mueve el cómputo y no los datos
revocación de clave	inmediata	—	1 servicio (lo previsto)	la herramienta de recuperación no puede depender de lo que recupera

consumidor detenido	3 min 40 s	sin cambio	512 mensajes retrasados	faltaba un SLO para el camino asíncrono
---------------------	------------	------------	-------------------------	---

El hallazgo que justificó el capstone. Durante la pérdida de zona, todos los paneles agregados estuvieron dentro del objetivo:

p99 agregado durante el episodio	213 ms	(SLO 500 ms)	✓ verde
presupuesto de error consumido	0,7 %		✓ verde

Y al desglosar por región de origen:

```
SELECT region_origen,
       APPROX_QUANTILES(latencia_ms, 100)[OFFSET(99)] AS p99,
       COUNT(*) AS peticiones
FROM peticiones
WHERE ts BETWEEN '2026-07-30 14:02' AND '2026-07-30 14:07'
GROUP BY region_origen
```

europa-west1	p99	148 ms	134.900 peticiones	
us-east1	p99	612 ms	18.400 peticiones	← fuera del SLO

Dieciocho mil cuatrocientas peticiones fuera del objetivo, invisibles en el agregado porque eran el 12 % del tráfico. La infraestructura hizo exactamente lo que se le pidió: desbordar a la región con capacidad. Nadie había comprobado qué latencia tendría esa región contra una base de datos que seguía en Europa.

síntoma observable	ninguno: percentil agregado en verde, presupuesto de error casi intacto
consecuencia real	12 % de los usuarios con latencia 4x durante 5 minutos
causa	el desbordamiento mueve el cómputo, no los datos
qué lo destapó	desglosar por región, no el simulacro en sí

Repetido el simulacro con las cuatro correcciones:

p99 desde us-east1	612 ms	171 ms
peticiones fuera del SLO	18.400	0
escalador de capacidad de us-east1	1,0	0,3
SLO medidos por región de origen	no	sí

Se entrega a la parte 05 con:

diez leyes observadas en tres plataformas independientes
 contrato portable de once filas, dos de ellas refinadas
 la ley corregida sobre la familia de fallos más cara
 24 afirmaciones y su guion de verificación
 11 decisiones con alternativa descartada
 5 riesgos residuales con responsable
 comparación de costo por pedido en tres nubes, desglosada

Y la hipótesis escrita para la parte 05, que ya se puede calificar dentro de doce clases:

El contrato del contenedor será portable de verdad y las fugas estarán en los bordes —almacenamiento, red, identidad, arranque y terminación—. Volverá a aparecer al menos una de las diez leyes, con un mecanismo nuevo.

La lección que esta parte deja al programa: la hipótesis de la clase 048 acertó en el contrato y en las excepciones, y falló en predecir qué familia de fallos costaría más. Corregir esa predicción con evidencia vale más que haberla acertado: la familia real —un mecanismo que parece estar protegiendo y no lo está— explica el fallo más caro de las tres partes, y su antídoto es una sola regla que ahora está escrita en la línea base: todo mecanismo de protección necesita una prueba que lo obligue a actuar.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/060-proyecto-aplicacion-de-tres-capas-en-google-cloud/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.

2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa plataforma-gcp en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye plataforma-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El percentil agregado está dentro del objetivo y hay usuarios claramente afectados	Una minoría del tráfico, desde otra región u origen, queda diluida en el agregado	Mide y alerta por región de origen y por segmento, no solo agregado; el simulacro debe desglosarlo.
El tráfico conmuta de región y el servicio incumple su objetivo igualmente	El desbordamiento por capacidad mueve el cómputo y no los datos	Réplica de lectura local, escalador de capacidad limitado en la región secundaria y decisión escrita sobre dónde se escribe.
La recuperación de un incidente falla porque la herramienta depende de lo que se rompió	El estado o la canalización usan el mismo recurso o la misma clave que se ha revocado	Aísla lo que recupera de lo que se recupera, y ensaya el procedimiento con una identidad de emergencia.
Un consumidor detenido no dispara ninguna alerta de objetivo de servicio	El SLI mide peticiones HTTP y el camino asíncrono no tiene objetivo propio	Define un SLI de frescura para el trabajo en segundo plano y aplícale el mismo mecanismo de presupuesto de error.
Una política correcta en otra plataforma cuesta dinero en esta	El modelo de precios cambia el punto de equilibrio, aunque el patrón sea el mismo	Recalcula la aritmética con los precios de la plataforma antes de trasladar cualquier política de ciclo de vida o de escalado.
Se elige proveedor por precio y la factura no cambia lo esperado	La mayor partida es una decisión de arquitectura idéntica en las tres plataformas	Desglosa hasta que cada partida tenga nombre; la decisión de datos suele mover más que la de proveedor.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Cuáles de las tres afirmaciones de la hipótesis de la clase 048 se cumplieron, y qué dice la ley corregida?
2. Enumera cuatro de las diez leyes observadas en tres plataformas y el mecanismo distinto que tuvo cada una.
3. Durante la pérdida de zona todo estaba en verde y 18.400 peticiones incumplían el SLO. ¿Por qué, y qué medición lo habría mostrado?
4. ¿Qué significa que la herramienta de recuperación dependa de lo que recupera, y cómo se evita?
5. ¿Por qué la mayor partida de costo coincide en las tres plataformas y qué implica eso al elegir proveedor?

Referencias

- Google Cloud (2025). Architecture Framework — los pilares como rejilla de revisión de la entrega. <<https://cloud.google.com/architecture/framework>>
- Google Cloud (2025). Patterns for scalable and resilient apps — desbordamiento entre regiones y ubicación de los datos. <<https://cloud.google.com/architecture/scalable-and-resilient-apps>>
- Google Cloud (2025). Disaster recovery planning guide — RTO, RPO y ensayo de procedimientos. <<https://cloud.google.com/architecture/dr-scenarios-planning-guide>>
- Google (2018). The Site Reliability Workbook, cap. 2 — SLI, SLO y alertas por consumo de presupuesto de error. <<https://sre.google/workbook/implementing-slos/>>
- Google Cloud (2025). Landing zone design in Google Cloud — jerarquía, red e identidad como línea base declarada. <<https://cloud.google.com/architecture/landing-zones>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 04 en PDF · Recorrido de Google Cloud en PDF · Manual integral

Anterior	Índice	Siguiente
← 059 · Terraform y despliegues reproducibles en GCP	Parte 04 · Programa	061 · Imágenes, capas, registros y estándar OCI →

Evaluación — 060 Proyecto: aplicación de tres capas en Google Cloud

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega plataforma-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.

- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 06 — Kubernetes y plataformas administradas

083 — EKS, AKS y GKE: similitudes y diferencias

Parte: 06 — Kubernetes y plataformas administradas Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: decision · Estado: EXECUTABLECORE

Propósito

Comparar las tres plataformas gestionadas de Kubernetes por lo que de verdad diferencia la operación, y no por una tabla de funciones. Tres ejes deciden casi todo: quién decide cuándo actualizas, cuántos pods caben en tu plan de direcciones y cómo obtiene una carga la identidad de la nube. Y la conclusión que interesa al programa es cuantificable: qué parte del trabajo de las clases 073 a 082 se conserva al cambiar de proveedor y qué parte hay que rehacer.

Resultados de aprendizaje

Al finalizar podrás:

1. Delimitar qué gestiona el proveedor y qué sigue siendo trabajo propio.
2. Anticipar una actualización forzada por fin de soporte y lo que puede bloquearla.
3. Calcular cuántos pods caben según el modelo de red de cada plataforma.
4. Federar la identidad de una carga con la nube, acotando la confianza correctamente.
5. Estimar qué se conserva y qué se rehace al cambiar de plataforma gestionada.

Conceptos centrales

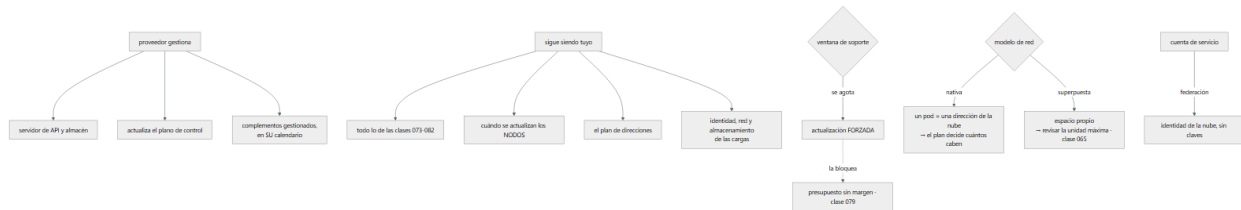
Concepto	Comprensión verificable
plano de control gestionado	El proveedor opera el servidor de API y el almacén de estado. No opera tus cargas ni tus complementos, que es donde está casi todo el trabajo de esta parte.
ventana de soporte	Periodo durante el cual una versión recibe correcciones. Al agotarse, el proveedor actualiza el clúster aunque nadie lo pida.
dirección de pod nativa de la red	Cada pod recibe una dirección del espacio de la nube. Simplifica el enrutamiento y consume el plan de direcciones de las clases 039 y 051.
red superpuesta	Los pods usan un espacio propio y su tráfico se encapsula. Ahorra direcciones y añade una capa —con la trampa de la unidad máxima de transmisión de la clase 065.
identidad federada de carga	La cuenta de servicio del clúster obtiene credenciales de la nube sin ninguna clave. Séptima aparición del mismo contrato del programa.
complemento gestionado	Pieza que el proveedor instala y actualiza en su calendario. Quita trabajo y quita control: su versión puede cambiar sin que nadie lo decida.
desviación de versiones	Diferencia admitida entre el plano de control y los nodos. Permite actualizar por fases y tiene un límite que, si se cruza, deja nodos sin soporte.

Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Qué gestiona el proveedor, y qué no

«Gestionado» se entiende como más de lo que es, y conviene delimitarlo antes de comparar nada:

el proveedor gestiona
 el servidor de API y el almacén de estado: disponibilidad, copias, parches
 la actualización del plano de control
 algunos complementos, en su calendario
 la integración con su red, su almacenamiento y su identidad

sigue siendo trabajo propio
 ABSOLUTAMENTE todo lo de las clases 073 a 082
 cuándo y cómo se actualizan los NODOS
 el plan de direcciones y sus consecuencias
 las políticas de red, de admisión y de permisos
 la observabilidad, las copias de las cargas con estado y su restauración

Es decir: el proveedor quita el trabajo que menos se nota y deja el que produce los incidentes de esta parte. Y merece decirse con claridad porque la expectativa contraria lleva a no dedicar equipo:

Un clúster gestionado no reduce el trabajo de operar aplicaciones en Kubernetes. Reduce el de operar Kubernetes.

Sobre el coste del plano de control, las tres cobran de forma distinta y la cifra rara vez decide:

una tarifa por hora y por clúster, con independencia del tamaño
 o una tarifa de gestión con algún nivel gratuito acotado
 o un nivel gratuito sin acuerdo de servicio y uno de pago con él

Lo que sí decide es la consecuencia estructural: un clúster tiene un coste fijo, así que muchos clústeres pequeños cuestan más que uno grande. Y esa aritmética empuja a compartir clúster entre equipos, lo que devuelve al aislamiento de la clase 080 y a su límite: los espacios de nombres no bastan para inquilinos no confiables.

Y la decisión que aparece pronto, con la misma forma en las tres plataformas:

un clúster por entorno frontera clara, coste fijo × entornos
 un clúster con espacios por equipo más barato, aislamiento más débil
 un clúster por equipo y entorno lo más aislado y lo más caro de operar

La respuesta razonable en la mayoría de las organizaciones es un clúster por entorno con espacios de nombres por equipo, y producción separada de todo lo demás — que es la misma conclusión a la que llegaron las clases 025, 037 y 049 con cuentas, suscripciones y proyectos. Cuarta aparición del mismo criterio.

2. Quién decide cuándo actualizas

Este es el eje que más cambia la operación y el que menos aparece en las comparativas.

Cada versión tiene una ventana de soporte de aproximadamente un año, y al agotarse el proveedor actualiza el clúster:

se publica una versión	cada ~4 meses
soporte estándar	~12-14 meses desde su publicación
soporte extendido, si existe	más meses, con sobrecoste
al agotarse	ACTUALIZACIÓN AUTOMÁTICA por el proveedor

De ahí salen tres consecuencias operativas:

Actualizar es una tarea recurrente, no un proyecto. Con una ventana de un año y una versión cada cuatro meses, un clúster se actualiza dos o tres veces al año, siempre. Un equipo que trate cada actualización como un proyecto excepcional vive permanentemente en uno.

Lo que bloquea la actualización lo has puesto tú. El proveedor respeta los presupuestos de interrupción de la clase 079, así que un presupuesto sin margen bloquea también su actualización automática. Y ahí ocurre lo peor:

- el proveedor intenta actualizar
- un presupuesto sin margen bloquea el vaciado del nodo
- la actualización se detiene o se agota su plazo
- el clúster se queda con nodos en dos versiones, o sin actualizar
- y el aviso llega por correo a una dirección que nadie lee

Es el incidente de la clase 079 con un origen distinto y una consecuencia mayor, porque termina con nodos ejecutando una versión sin soporte.

La desviación de versiones limita el orden. El plano de control se actualiza primero y los nodos después, y hay un margen máximo entre ambos:

- los nodos pueden ir por detrás del plano de control, hasta un límite nunca por delante
- y saltarse versiones intermedias no está soportado:
- hay que pasar por cada una

La última línea es la que convierte un retraso en un problema: un clúster tres versiones por detrás necesita tres actualizaciones encadenadas, cada una con su ventana y su riesgo.

Y las funciones que se retiran son la otra mitad del trabajo. Cada versión elimina versiones de API antiguas, y un manifiesto que las use deja de aplicarse:

```
$ kubectl get --raw /metrics | grep apiserver_requested_deprecated_apis
```

Esa métrica dice qué APIs obsoletas se están usando ahora mismo y quién. Debería revisarse antes de cada actualización, y es la comprobación que evita descubrir el problema con el clúster ya actualizado.

Y una recomendación que se paga sola: un clúster de pruebas una versión por delante del de producción. Cuesta el plano de control y unos nodos pequeños, y convierte cada actualización en algo ya ensayado.

3. Cuántos pods caben: el modelo de red decide

Aquí está la divergencia real entre plataformas, y tiene consecuencias que se descubren tarde.

Hay dos modelos:

DIRECCIONES NATIVAS

- cada pod recibe una dirección del espacio de la nube
- ventaja: el enrutamiento es directo; los cortafuegos y el registro de flujo de las clases 039 y 051 ven al pod, no al nodo
- coste: consume el plan de direcciones, y mucho

RED SUPERPUESTA

- los pods usan un espacio propio y el tráfico se encapsula
- ventaja: no consume direcciones de la nube
- coste: una capa más, y la unidad máxima de transmisión de la clase 065

Y el cálculo que hay que hacer antes de crear el clúster, no después:

- con direcciones nativas y una subred /22 (1.024 direcciones)
- reservadas por la nube ~5
- una por nodo × nodos
- una por pod × pods
- y direcciones en tránsito durante los despliegues, que no se liberan al instante

40 nodos × 30 pods = 1.200 direcciones necesarias
→ una /22 NO llega; hace falta al menos una /21, con margen

La última línea del cálculo es la que se olvida: durante un despliegue coexisten pods viejos y nuevos, así que el pico de direcciones es mayor que el estado estable. Y el síntoma del agotamiento es característico y despiستا:

- pods en ContainerCreating de forma intermitente, sobre todo al desplegar
- failed to allocate for range 0: no IP addresses available in range

Parece un problema del complemento de red y es un problema de aritmética de la clase 051.

Y hay un segundo límite propio de algunos modelos: el número de pods por nodo puede depender del tipo de máquina, porque las direcciones se asignan al nodo en bloques ligados a sus interfaces. Un nodo pequeño puede admitir muy pocos pods aunque le sobren CPU y memoria:

síntoma nodos con recursos libres y pods en Pending
 el suceso menciona pods insuficientes, no CPU ni memoria

Ese caso se corrige con máquinas mayores o con modos que reparten direcciones por prefijo en vez de una a una, y hay que conocerlo porque contradice la intuición de la clase 078: no siempre falta capacidad; a veces faltan direcciones.

Y los rangos secundarios que la clase 051 pedía reservar aparecen aquí con su factura: en las plataformas que los usan, pods y servicios consumen bloques propios, grandes y fijados al crear el clúster. Ampliarlos después no siempre es posible, lo que convierte el plan de direcciones en otra decisión de un solo sentido.

Y la conclusión que enlaza las dos cosas:

el tamaño máximo del clúster lo decide, en la práctica,
el plan de direcciones que alguien hizo antes de crearlo

4. La identidad, por séptima vez

Las tres plataformas resuelven lo mismo con el mismo mecanismo y distinta sintaxis: una carga del clúster obtiene credenciales de la nube sin ninguna clave.

la cuenta de servicio del pod recibe un testigo firmado por el clúster
la nube confía en el emisor del clúster
y cambia ese testigo por credenciales suyas

Es el mismo contrato de las clases 026, 038, 050, 069 y 080. Séptima aparición, con la misma pieza crítica:

la confianza del lado de la NUBE debe acotarse al espacio de nombres
y a la cuenta de servicio concretos

Sin acotar, cualquier pod del clúster puede pedir esa identidad. Y es exactamente el mismo fallo que el sujeto sin acotar de las federaciones anteriores, con un cuarto vocabulario:

condición de confianza correcta
emisor = el del clúster
sujeto = system:serviceaccount:<espacio>:<cuenta>

La comprobación es la misma prueba negativa que este programa ha ejecutado cuatro veces:

```
# desde un pod con la cuenta correcta
$ kubectl exec -n tienda api-x -- /probar-acceso-nube
ok
# desde un pod de otro espacio con otra cuenta
$ kubectl exec -n desarrollo prueba-y -- /probar-acceso-nube
acceso denegado
```

✓

Y la parte de la sintaxis, que es lo único que cambia entre plataformas:

```
# el mecanismo es el mismo; la anotación y el nombre del recurso, no
apiVersion: v1
kind: ServiceAccount
metadata:
  name: api
  namespace: tienda
  annotations:
    # <clave específica del proveedor>: <identidad de la nube>
```

Y los complementos gestionados merecen su nota porque cambian el reparto de responsabilidades:

complemento gestionado
el proveedor lo instala y lo actualiza en su calendario
menos trabajo, y su versión puede cambiar sin que nadie lo decida

complemento propio
se elige la versión y el momento
más trabajo, y es la única forma de fijar el comportamiento

La decisión importa sobre todo en el complemento de red, por lo que la clase 080 mostró: si no implementa políticas, los objetos no filtran nada. Y algunas plataformas ofrecen varios, con soporte distinto de políticas, así que la elección

del complemento al crear el clúster decide si la segmentación de red es posible. Es una decisión de instalación con consecuencias de seguridad, y hay que tomarla con la clase 080 delante.

5. Qué se conserva al cambiar de plataforma

La pregunta que este programa lleva haciendo desde la clase 048 tiene aquí una respuesta cuantificable, y es la mejor noticia de la parte.

se conserva SIN CAMBIOS

- todos los manifiestos de carga: despliegues, servicios, configuración, presupuestos, políticas de red, perfiles de seguridad, escalado
- las imágenes y su cadena de suministro (clases 061-067)
- el contrato de la aplicación: señales, comprobaciones, apagado (068)
- la instrumentación: el formato de métricas es el mismo (082)
- las herramientas de gestión de manifiestos (081)

hay que REHACER

- la creación del clúster y su plan de direcciones
- la sintaxis de la federación de identidad
- los grupos de nodos y su escalado
- la elección e instalación de complementos
- la integración con el almacenamiento: nombres de clases y parámetros
- la integración con el balanceador de entrada: anotaciones
- la recogida de registros y métricas hacia el sistema del proveedor

El primer bloque es grande y el segundo es la capa de plataforma, no las aplicaciones. Expresado como proporción, en una migración real entre dos proveedores:

ficheros de manifiesto de aplicaciones	~95 % sin cambios
capa de plataforma	~100 % reescrita
esfuerzo total	concentrado en 6-8 componentes

Y eso confirma con datos lo que la clase 072 predijo: el contrato del contenedor es portable y las fugas están en los bordes. Aquí los bordes tienen nombre: red, almacenamiento, identidad y entrada — las mismas cuatro, por tercera vez.

Y hay una conclusión práctica que conviene sacar y que casi nadie saca: si la capa de plataforma se va a reescribir de todos modos, conviene aislarla desde el principio.

repositorio de aplicaciones	manifiestos portables, sin nada del proveedor
repositorio de plataforma	creación del clúster, complementos, clases de almacenamiento, anotaciones de entrada, federación de identidad

Con esa separación, cambiar de proveedor toca un repositorio y no doscientos ficheros. Y tiene un beneficio inmediato aunque nunca se cambie: hace visible qué parte del sistema está atada al proveedor, que es una cifra que casi ninguna organización conoce.

Y un aviso honesto sobre la portabilidad, para no exagerarla: que los manifiestos sean portables no significa que el sistema lo sea. Los servicios gestionados que las cargas usan —bases de datos, colas, almacenamiento de objetos— siguen siendo del proveedor, y suelen ser la parte más difícil de mover. Kubernetes hace portable el cómputo, que es la parte que ya era más fácil.

La lista de comprobación para elegir y montar una plataforma gestionada:

- plan de direcciones calculado para el tamaño máximo previsto, con margen de despliegue, y comprobado el límite de pods por nodo
- complemento de red que implementa políticas, verificado con prueba negativa
- federación de identidad acotada a espacio y cuenta, con prueba negativa
- ventana de soporte anotada, con fecha de la próxima actualización obligatoria
- clúster de pruebas una versión por delante
- métrica de APIs obsoletas revisada antes de cada actualización
- presupuestos con margen, para que la actualización del proveedor no se bloquee
- separación entre repositorio de aplicaciones y de plataforma
- complementos: decidido cuáles son gestionados y cuáles propios, y por qué

Ejemplo trabajado

CloudShop opera un clúster en una nube y evalúa mover una parte a otra. El ejercicio se hace en serio —con una migración real de un entorno— y produce cinco datos que ninguna comparativa daba.

Dato 1 — el 94 % de los manifiestos no cambió.

manifiestos de aplicaciones	214	13
de esos 13:		
clase de almacenamiento	4	
anotaciones del objeto de entrada	5	
anotaciones de cuenta de servicio	3	
tolerancias de nodos especiales	1	
capa de plataforma	31	31

Los trece cambios son exactamente las cuatro fugas de la clase 072, y ninguno estaba en la lógica de la aplicación.

reescribir la capa de plataforma	9 días-persona
adaptar los 13 manifiestos	1 día-persona
validar y comparar	4 días-persona

Y la conclusión que se documentó: el coste de cambiar de plataforma gestionada está en seis componentes de infraestructura, no en las aplicaciones.

Dato 2 — el plan de direcciones limitaba el clúster a 27 nodos.

Al dimensionar el clúster nuevo:

subred asignada	/22 · 1.024 direcciones
modelo	direcciones nativas
Pods por nodo previstos	30
nodos que caben	$1.024 / 31 \approx 33$
menos margen de despliegue (~20 %) ≈ 27	

El clúster original tenía el mismo problema y no lo sabían: llevaban meses sin poder pasar de 24 nodos, y lo atribuían a cuotas del proveedor.

\$ kubectl describe node nodo-14 grep -i 'pods:'		
pods: 29	← límite por tipo de máquina, no por CPU ni memoria	
subred del clúster	/22	/20
nodos posibles	27	110
límite de pods por nodo	29	110 (modo por prefijo)
sucesos "no IP addresses available"	41 en un mes	0
pods en Pending por límite de pods	sí, en los picos	no

Dos correcciones distintas: la subred resolvía el techo del clúster y el modo de asignación por prefijo resolvía el techo por nodo. Ninguna se puede aplicar sobre un clúster existente sin recrearlo, lo que confirma que es una decisión de un solo sentido.

Dato 3 — una actualización forzada que llevaba tres meses bloqueada.

versión del clúster	fin de soporte hace 6 semanas
intentos del proveedor	4, todos detenidos
nodos actualizados	11 de 24
motivo	presupuesto sin margen en 2 servicios (clase 079)
aviso	correo a una lista que nadie leía

El clúster llevaba seis semanas con la mitad de los nodos en una versión sin soporte, y con el plano de control ya actualizado por el proveedor.

presupuestos que bloquean	2	0
notificación del proveedor	correo a lista	alerta en el canal de guardia
clúster de pruebas por delante	no había	sí, una versión
métrica de APIs obsoletas revisada	nunca	antes de cada actualización
duración de la actualización completa	no terminaba	4 h 10 min

Y la revisión de APIs obsoletas, hecha por primera vez, encontró dos manifiestos que la versión siguiente habría rechazado.

Dato 4 — el complemento de red decidía si la segmentación era posible.

La plataforma de destino ofrecía dos complementos, y solo uno implementaba políticas. La elección por defecto del asistente de creación era el otro.

políticas de red efectivas	0	14
direcciones consumidas por pod	1	1
unidad máxima de transmisión	sin cambios	revisada (clase 065)
complemento gestionado por el proveedor	sí	sí

Si la migración se hubiera hecho aceptando el valor por defecto, las catorce políticas de la clase 080 habrían vuelto a no filtrar nada — el mismo incidente, en una plataforma nueva, por una casilla del asistente.

Dato 5 — la federación de identidad, sin acotar por defecto.

La primera configuración funcionaba desde el primer intento, lo que hizo sospechar:

```
$ kubectl run prueba -n desarrollo --rm -it --image=... -- /probar-acceso-nube
ok ← desde OTRO espacio, con OTRA cuenta
```

La condición de confianza del lado de la nube aceptaba cualquier testigo del clúster.

condición de confianza	emisor del clúster	emisor + espacio + cuenta
podés que podían obtener la identidad	todos	solo el previsto
prueba negativa	ninguna	en el guion de verificación

Cuarta vez en el programa que esta misma prueba negativa se ejecuta y tercera vez que falla la primera vez.

Resumen de la evaluación:

manifiestos de aplicación reescritos	—	13 de 214
capa de plataforma reescrita	—	31 de 31
nodos que caben en el plan de direcciones	27	110
políticas de red efectivas	14	14
federación acotada	sí	sí (tras corregir)
semanas con versión sin soporte	6	0
esfuerzo total de la migración	—	14 días-persona

La lección que esta clase traslada al proyecto de la clase 084: la comparación entre plataformas gestionadas no se decide por funciones sino por tres cosas que se fijan al crear el clúster y no se pueden cambiar después — el plan de direcciones, el complemento de red y el modelo de identidad. Las tres son decisiones de un solo sentido, las tres se toman en un asistente de creación en cinco minutos, y las tres deciden el techo de crecimiento, la posibilidad de segmentar y el alcance de una credencial. El resto —el 94 % de los manifiestos— es portable, y esa es la confirmación cuantificada de la hipótesis que abrió la parte 05.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-06-kubernetes-managed-platforms/083-eks-aks-y-gke-similitudes-y-diferencias/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa matriz-kubernetes-administrado en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye matriz-kubernetes-administrado para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.

- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El clúster no puede crecer más allá de cierto número de nodos	El plan de direcciones no da para más pods, y el pico de despliegue consume más que el estado estable	Calcula el tamaño máximo previsto con margen antes de crear el clúster; ampliarlo después suele exigir recrearlo.
Hay nodos con CPU y memoria libres y pods en Pending	El límite de pods por nodo depende del tipo de máquina y del modo de asignación de direcciones	Usa máquinas mayores o el modo de asignación por prefijo; el suceso menciona pods insuficientes, no recursos.
El proveedor intenta actualizar y el clúster queda a medias durante semanas	Un presupuesto de interrupción sin margen bloquea también la actualización automática	Mantén margen en los presupuestos y dirige los avisos del proveedor al canal de guardia, no a una lista.
Tras una actualización, algunos manifiestos dejan de aplicarse	La versión nueva retiró versiones de API que esos manifiestos usaban	Revisa la métrica de APIs obsoletas antes de cada actualización y mantén un clúster de pruebas por delante.
Las políticas de red no filtran en la plataforma nueva	El complemento elegido en el asistente de creación no las implementa	Elige el complemento con soporte de políticas y verifica con una prueba negativa antes de dar por buena la migración.
Cualquier pod del clúster obtiene la identidad de la nube	La condición de confianza acepta cualquier testigo del clúster, sin acotar espacio ni cuenta	Acota la confianza al espacio de nombres y a la cuenta concretos, y comprueba desde otro espacio que se deniega.
Se espera que la plataforma gestionada reduzca el trabajo de operación	Gestiona el plano de control, no las aplicaciones ni los complementos	Dimensiona el equipo para el trabajo de las clases 073 a 082, que es el que produce los incidentes.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué gestiona exactamente el proveedor y qué sigue siendo trabajo propio?
2. ¿Qué ocurre cuando se agota la ventana de soporte y qué puede impedir esa actualización?
3. Calcula cuántos nodos caben en una subred /22 con 30 pods por nodo y margen de despliegue.
4. ¿Por qué la elección del complemento de red al crear el clúster es una decisión de seguridad?
5. ¿Qué proporción de manifiestos se conserva al cambiar de plataforma, y dónde se concentra el esfuerzo?

Referencias

- Kubernetes (2025). Version skew policy — desviación admitida entre plano de control y nodos.
<<https://kubernetes.io/releases/version-skew-policy/>>
- Kubernetes (2025). Deprecated API migration guide — versiones retiradas y cómo detectarlas.
<<https://kubernetes.io/docs/reference/using-api/deprecation-guide/>>

- AWS (2025). Amazon EKS networking and IP address planning — direcciones por nodo y modos de asignación.
<<https://docs.aws.amazon.com/eks/latest/userguide/eks-networking.html>>
- Microsoft (2025). AKS networking concepts — modelos de red y sus consecuencias de direccionamiento.
<<https://learn.microsoft.com/en-us/azure/aks/concepts-network>>
- Google Cloud (2025). GKE cluster networking and IP allocation — rangos secundarios y límites por nodo.
<<https://cloud.google.com/kubernetes-engine/docs/concepts/alias-ips>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 06 en PDF · Manual integral

Anterior	Índice	Siguiente
← 082 · Logs, métricas, eventos y depuración	Parte 06 · Programa	084 · Proyecto: plataforma Kubernetes portable →

Evaluación — 083 EKS, AKS y GKE: similitudes y diferencias

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega matriz-kubernetes-administrado con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 09 — Datos, mensajería, serverless e integración

114 — Pub/sub, streams, particiones y orden

Parte: 09 — Datos, mensajería, serverless e integración Nivel: avanzado · Horas estimadas: 4 Laboratorio: messaging · Estado: EXECUTABLECORE

Propósito

Pasar de la cola —que entrega y borra— al registro ordenado que conserva, donde cada consumidor lleva su propia posición y puede volver atrás. Ese cambio resuelve tres cosas que la clase 113 no podía: varios consumidores independientes del mismo hecho, releer el pasado y reconstruir estado desde cero. Y trae dos decisiones que se toman al crear y no se deshacen: cuántas particiones y qué clave las elige, que juntas deciden el paralelismo máximo, el orden y dónde aparecerá la partición caliente.

Resultados de aprendizaje

Al finalizar podrás:

1. Distinguir cuándo hace falta un registro conservado y cuándo basta una cola.
2. Elegir número de particiones y clave, entendiendo lo que fijan para siempre.
3. Razonar sobre grupos de consumidores, reparto y lo que cuesta un rebalanceo.
4. Gestionar la posición de lectura sin perder ni repetir rangos enteros.
5. Usar retención y compactación para reconstruir estado.

Conceptos centrales

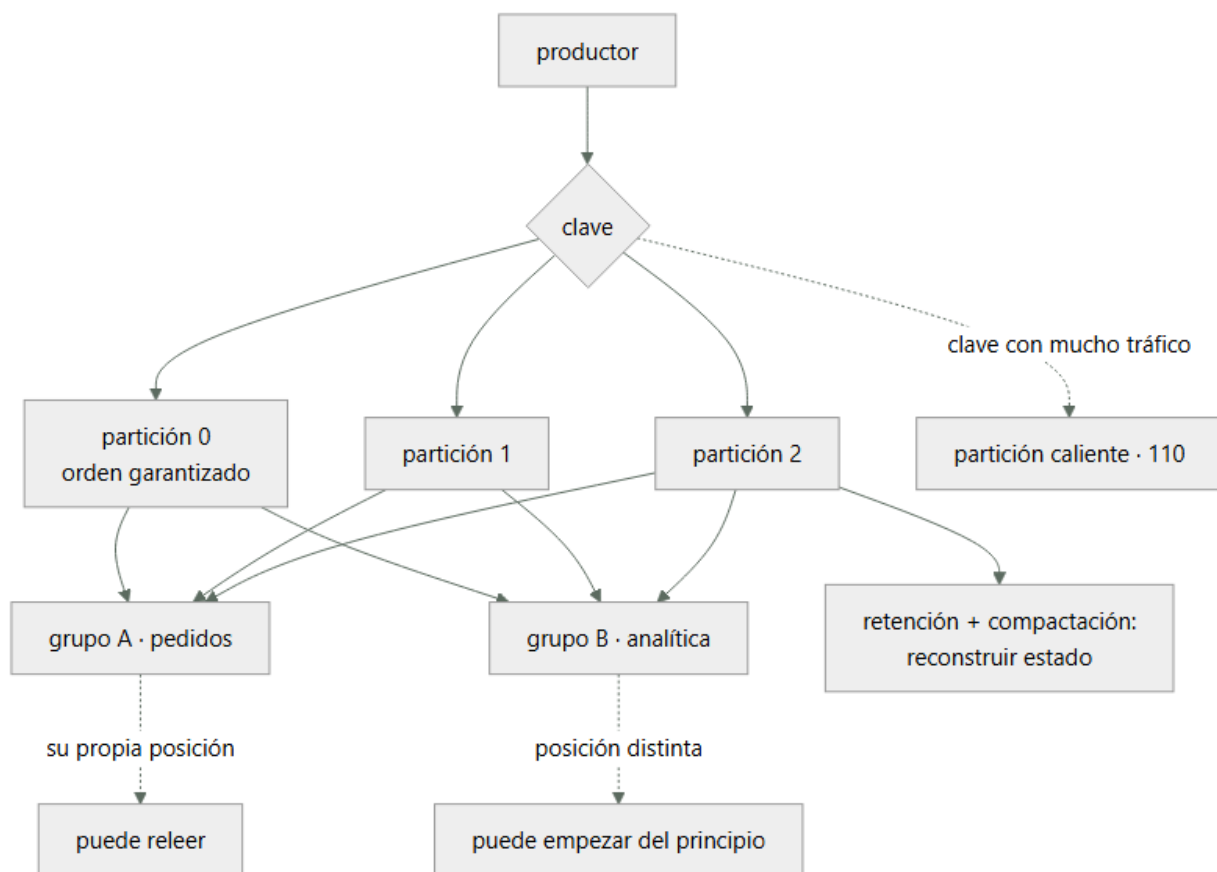
Concepto	Comprensión verificable
registro conservado	Secuencia ordenada e inmutable de mensajes que no se borran al leerse. Cada consumidor avanza por su cuenta.
partición	Subsecuencia ordenada del tema. Es a la vez la unidad de paralelismo y la única dentro de la cual hay orden.
posición de lectura	Índice del último mensaje procesado por un grupo. Confirmarla mal salta o repite rangos enteros, no un mensaje.
grupo de consumidores	Conjunto que se reparte las particiones. Ninguna partición la leen dos miembros a la vez, así que el paralelismo máximo es el número de particiones.
rebalanceo	Reparto de particiones cuando entra o sale un miembro. Durante él el consumo se detiene y suele haber reproceso.
compactación por clave	Conservar solo el último valor de cada clave. Convierte el registro en una fotografía reconstruible del estado actual.

Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Conservar cambia lo que se puede hacer

La diferencia con la clase 113 es una sola y tiene muchas consecuencias:

COLA	se entrega y se borra un mensaje, un consumidor lo procesado ya no existe
REGISTRO	se escribe y se conserva cada consumidor lleva su posición lo procesado sigue ahí

Y lo que eso permite:

varios consumidores independientes del mismo hecho
facturación, analítica, notificación, búsqueda
→ y añadir uno nuevo no afecta a los demás

releer

un consumidor con un defecto reprocesa desde una posición anterior
→ sin pedir nada al productor

empezar del principio

un servicio nuevo reconstruye su estado leyendo el histórico
→ esto es lo que no se puede hacer con una cola

Y el criterio para elegir uno u otro:

cola	trabajo que alguien tiene que hacer una vez «envía este correo», «cobra este pedido»
registro	hecho que ya ocurrió y que interesa a varios «el pedido 1421 pasó a pagado»

La distinción se nota en el nombre del mensaje: un imperativo va a una cola; un hecho en pasado va a un registro.

Y lo que cuesta conservar:

- el almacenamiento se paga por el periodo de retención
- los mensajes viejos siguen ahí cuando el código ya cambió
 - un consumidor debe poder leer formatos anteriores
 - es la convivencia N/N+1 de la clase 102, aplicada a los datos
- borrar un dato concreto es difícil: el registro es inmutable
 - y hay requisitos legales que lo exigen (clase 112)

La última línea tiene dos respuestas conocidas: compactar con una marca de borrado por clave, o guardar los datos personales fuera del registro y publicar solo referencias. La segunda es la que menos disgustos da.

2. Particiones: paralelismo y orden a la vez

La partición es la pieza central y hace dos trabajos que conviene ver separados:

es la unidad de ORDEN	dentro de una partición hay orden total
	entre particiones no hay ninguno
es la unidad de PARALELISMO	cada partición la lee un solo miembro del grupo
	→ paralelismo máximo = número de particiones

Y de ahí sale la primera decisión irreversible en la práctica:

¿cuántas particiones?

pocas	techo de paralelismo bajo
muchas	más metadatos, más ficheros, rebalanceos más lentos
	y más coste por partición en los servicios administrados

Y por qué aumentar el número después es un problema:

$\text{partición} = f(\text{clave}) \bmod N$

si N cambia, la misma clave pasa a otra partición

- los mensajes viejos de esa clave están en la partición vieja
- y el orden por clave, que era la garantía, se rompe en la frontera

Así que ampliar es posible y no conserva la garantía de orden para las claves ya existentes. Es la ley 14 en su forma más limpia: una decisión de creación con consecuencias que no se pueden deshacer sin migrar.

La regla práctica al dimensionar:

particiones $\geq$ consumidores máximos previstos
y con margen: 2 a 3 veces el paralelismo actual
pero no cientos «por si acaso»

La clave, que es la segunda decisión y hereda todo lo de la clase 110:

misma clave → misma partición → orden garantizado entre esos mensajes

clave = identificador de pedido	orden por pedido, reparto bueno
clave = país	5 valores → 5 particiones útiles
clave = nula (reparto rotatorio)	máximo reparto, ningún orden

Y la partición caliente vuelve exactamente igual que en la clase 110: una clave con mucho tráfico satura su partición mientras las demás están ociosas, y no se puede repartir porque repartirla rompería el orden que justificaba la clave.

si una clave concentra demasiado tráfico:

¿de verdad necesita orden esa clave?

- no → quítale la clave y reparte
- sí → hay un límite físico, y hay que reducir el trabajo por mensaje

3. Grupos, rebalanceo y posición

El grupo de consumidores reparte las particiones entre sus miembros:

12 particiones, 4 consumidores	→ 3 particiones cada uno
12 particiones, 12 consumidores	→ 1 cada uno
12 particiones, 20 consumidores	→ 8 CONSUMIDORES OCIOSOS

La tercera línea es la que sorprende a quien viene de las colas: añadir consumidores por encima del número de particiones no aumenta nada. El autoescalado por retraso tiene ahí su techo, y hay que declararlo.

El rebalanceo ocurre cuando entra o sale un miembro, y su coste es real:

se detiene el consumo de todo el grupo

se reparte de nuevo
cada consumidor empieza desde la última posición confirmada
→ lo procesado y no confirmado se REPITE

Y el error más frecuente es provocarlo sin querer:

el consumidor tarda más de lo permitido entre lecturas
→ el coordinador lo da por muerto
→ rebalanceo
→ el consumidor vuelve y provoca otro rebalanceo
→ y en ese bucle el grupo no avanza

Lo que lo corrige:

leer lotes más pequeños, para volver antes a pedir
separar el latido del proceso, si el cliente lo permite
subir el tiempo máximo entre lecturas al p99 real del proceso
y usar reparto incremental, que no detiene a todo el grupo

La posición de lectura. Aquí la diferencia con una cola es importante y peligrosa: no se confirma un mensaje, se confirma una posición, y todo lo anterior queda dado por hecho.

confirmar ANTES de procesar	si falla, se saltan mensajes: se PIERDEN
confirmar DESPUÉS de procesar	si falla, se repite el lote: al menos una vez
confirmar cada mensaje	seguro y lento
confirmar por lotes	rápido, y repite el lote entero al fallar

Y el caso que rompe a mucha gente: procesar en paralelo dentro de un lote y confirmar la posición mayor. Si el mensaje 7 falla y el 9 va bien, confirmar 9 da por hecho el 7.

si se procesa en paralelo, la posición confirmable es
la del último mensaje SIN huecos anteriores

Y la señal de estado, que aquí tiene dos formas y hay que mirar las dos:

retraso en mensajes	cuántos faltan por leer
retraso en tiempo	cuánto hace que se produjo el que se está leyendo

La segunda es la que se compara con lo que el negocio tolera; la primera dice si se está recuperando.

4. Retención, compactación y reconstrucción

La retención es lo que convierte el registro en algo más que un transporte.

por tiempo	conserva N días
por tamaño	conserva N GB por partición
permanente	con almacenamiento por capas, el histórico completo

Y lo que la retención permite, que es lo que justifica todo el modelo:

un consumidor con un defecto retrocede su posición y reprocesa
un servicio nuevo empieza en el principio y construye su estado
un incidente se investiga leyendo lo que pasó de verdad

Y la advertencia correspondiente: la retención es también el plazo de recuperación. Si se conservan tres días y un defecto se descubre a los cinco, no hay nada que reprocesar.

retención $\geq$ tiempo típico hasta descubrir un defecto
→ en la práctica, 7 días es poco y 30 suele ser suficiente

La compactación por clave es distinta de la retención por tiempo y resuelve otro problema:

conserva el ÚLTIMO mensaje de cada clave, para siempre
descarta los anteriores

→ el tema deja de ser un histórico y pasa a ser una fotografía
→ leerlo entero reconstruye el estado actual de todas las claves

Y para qué sirve de verdad:

reconstruir un caché tras perderlo (clase 111)
dar estado inicial a un servicio nuevo sin consultar a la base
mantener una copia local de una tabla de referencia

Y el detalle imprescindible: para borrar una clave se publica un mensaje con valor vacío. Sin eso, una clave borrada en el origen vive para siempre en el tema compactado.

Y dos cosas que conviene no mezclar:

tema de HECHOS	histórico, retención por tiempo, no compactar «el pedido 1421 pasó a pagado»
tema de ESTADO	compactado, última versión por clave «el pedido 1421 está así»

Mezclarlas es un error frecuente: compactar un tema de hechos borra historia que alguien necesitaba.

Y un último aviso sobre los reintentos, que enlaza con la clase 113: reintentar un mensaje fuera del registro rompe el orden. Si el mensaje 5 falla y se manda a una cola de reintentos mientras el 6 y el 7 se procesan, se ha perdido la garantía que justificaba la partición. Las dos salidas honestas:

- detenerse en el fallo y no avanzar esa partición
 - conserva el orden y bloquea a los de detrás
- enviar a un tema de reintentos y ACEPTAR que se pierde el orden
 - hay que declararlo, no descubrirlo

Y la lista de comprobación de la clase:

- está justificado por qué esto es un registro y no una cola
- el número de particiones cubre el paralelismo máximo previsto
- está entendido que ampliar particiones rompe el orden por clave
- la clave reparte y no concentra tráfico en una partición
- el máximo de consumidores útiles está declarado y no se escala más allá
- el tiempo máximo entre lecturas es mayor que el p99 de proceso
- la posición se confirma después de procesar y sin huecos
- se vigilan retraso en mensajes y retraso en tiempo
- la retención cubre el tiempo típico hasta descubrir un defecto
- los temas de hechos no están compactados
- el borrado por clave publica un mensaje vacío
- está declarado si los reintentos rompen el orden

Y el cierre que enlaza con la clase siguiente: si varios sistemas leen los mismos hechos, el formato de esos hechos es un contrato entre equipos que no se ven. Cómo se define, cómo se cambia sin romper a quien lee el histórico y quién es su dueño es la materia de la clase 115.

Ejemplo trabajado

CloudShop publica los hechos de pedido en un registro conservado para que tres equipos los consuman por su cuenta. El diseño inicial es razonable y contiene dos decisiones de creación que costarán una migración.

Por qué un registro y no una cola.

consumidores del mismo hecho con una cola	facturación, analítica, búsqueda 3 colas, y el productor tiene que publicar en las tres
añadir un cuarto consumidor	tocar al productor
reprocesar tras un defecto	pedir al productor que reenvíe

Y con registro, los cuatro problemas desaparecen: el productor publica una vez y cada consumidor avanza a su ritmo.

Decisión 1: seis particiones. Se queda corta a los cinco meses.

al diseñar	1.200 mensajes/s, 4 consumidores
particiones elegidas	6
mes 5	4.900 mensajes/s
consumidores necesarios	11
consumidores útiles	6 ← techo
consumidores ociosos	5
retraso en tiempo	de 2 s a 4 min 20 s

Y al ampliar a 24 particiones apareció lo del apartado segundo:

mensajes de un mismo pedido antes de ampliar	en la partición 3
mensajes del mismo pedido después	en la partición 17
ventana en la que el orden por pedido no está garantizado	la retención: 30 días
casos de orden invertido detectados	41

La migración honesta consistió en publicar en un tema nuevo con 24 particiones, consumir de los dos durante treinta días y retirar el viejo. Doce minutos de decisión inicial, cinco semanas de migración.

Decisión 2: la clave. La primera fue el país.

clave = país	
valores distintos	190

particiones que reciben tráfico	6
tráfico de España	61 % del total
carga de la partición de España	61 %
carga de la partición menos usada	0,4 %
retraso de la partición de España	9 min
retraso del resto	< 1 s

Es la partición caliente de la clase 110, en otro sistema y con el mismo diagnóstico. Y la pregunta del apartado segundo, respondida:

¿necesita orden el conjunto de un país?	no	
¿qué necesita orden?	los mensajes de un mismo pedido	
→ clave = identificador de pedido		
particiones con tráfico	6	24
desviación entre la más y la menos cargada	×150	×1,3
retraso máximo	9 min	2 s
casos de orden invertido	0	0

El bucle de rebalanceo, que paralizó el consumo 40 minutos.

02:10	un consumidor tarda 6 min en un lote (una dependencia lenta)	
02:15	el coordinador lo da por muerto: tiempo máximo entre lecturas, 5 min	
02:15	rebalanceo; el grupo se detiene	
02:16	el consumidor vuelve → otro rebalanceo	
02:16	... se repite	
02:50	se detecta y se corrige a mano	
tamaño del lote	500 mensajes	50 mensajes
tiempo de proceso por lote, p99	6 min	38 s
tiempo máximo entre lecturas	5 min	5 min
reparto	detiene el grupo	incremental
rebalanceos por semana	14	0-1
minutos de consumo detenido por semana	47	< 1

El error de posición que perdió 9.000 mensajes.

Un consumidor procesaba el lote en paralelo con veinte hilos y confirmaba la posición mayor al terminar:

lote de 500, mensajes 12.000 a 12.499	
el mensaje 12.203 falla y se registra el error	
el resto termina bien	
se confirma la posición 12.499	
→ el 12.203 nunca se procesa y nadie vuelve a él	
mensajes perdidos así en 3 semanas	9.140
detectado por	un descuadre en facturación de 21.400 €
corrección	confirmar la posición del último SIN huecos anteriores
mensajes perdidos tras la corrección	0
reproceso	se retrocedió la posición 3 semanas y se repitió
	→ posible SOLO porque la retención era de 30 días

La última línea es el argumento del apartado cuarto: con siete días de retención, esos 21.400 € no se habrían podido recuperar.

La compactación, para reconstruir el caché de la clase 111.

El caché portante de la clase 111 tardaba catorce minutos en recalentarse contra la base de datos. Con un tema de estado compactado:

tiempo de recalentamiento	14 min	1 min 50 s
carga sobre la base durante el proceso	12.000/s	0
tamaño del tema (2,1 M de productos)	—	3,4 GB

Y el fallo que se cometió al montarlo: se compactó también el tema de hechos.

hechos conservados antes de compactar	4.100 millones
después	2,1 millones (uno por pedido)
histórico perdido	todo lo intermedio
recuperado desde	la zona bruta del lago (clase 112)

Se recuperó porque la zona bruta existía. Sin ella, el histórico de pedidos se habría perdido de forma irreversible.

A los ocho meses.

consumidores independientes	3	6
particiones	6	24

clave	país	pedido
desviación de carga entre particiones	×150	×1,3
retraso en tiempo, p99	4 min 20 s	1,8 s
rebalanceos por semana	14	0-1
mensajes perdidos por confirmación con huecos	9.140	0
retención	30 días	30 días
temas de hechos compactados por error	1	0
recalentamiento del caché	14 min	1 min 50 s

La lección que esta clase traslada a la parte 09: los dos problemas caros —ampliar particiones y cambiar la clave— no fueron fallos de operación ni de código. Fueron dos decisiones tomadas en la primera media hora de diseño, cuando el sistema tenía la cuarta parte del tráfico, y ninguna de las dos se podía deshacer sin migrar. Es la tercera confirmación seguida de la predicción de la clase 108, y ya se puede afinar: en los sistemas con estado, las decisiones baratas de tomar son las caras de cambiar, y se toman antes de tener datos para tomarlas bien.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-09-data-messaging-serverless-integration/114-pub-sub-streams-particiones-y-orden/
lab.py
```

El laboratorio selecciona el motor de práctica messaging y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa stream-particionado en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un flujo que declara orden, entrega y manejo de errores. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye stream-particionado para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Añadir consumidores no mejora el retraso	Ya hay tantos consumidores como particiones; el resto están ociosos	Declara el máximo útil, no escales por encima, y dimensiona las particiones con margen desde el principio.
El orden por clave se rompe tras ampliar particiones	La asignación de clave a partición depende del número de particiones	Publica en un tema nuevo con el tamaño correcto y consume de los dos durante la retención; ampliar en sitio no conserva la garantía.
Una partición acumula retraso y las demás están ociosas	La clave concentra el tráfico, como en la clase 110	Elige una clave con reparto real; si la que concentra no necesita orden, quítale la clave.
El grupo entra en un bucle de rebalanceos y no avanza	El proceso de un lote dura más que el tiempo máximo entre lecturas	Reduce el tamaño del lote, ajusta el tiempo máximo al percentil 99 real y usa reparto incremental.
Se pierden mensajes sin ningún error	Se confirmó una posición por delante de un mensaje que falló	Confirma la posición del último mensaje sin huecos anteriores, nunca la mayor del lote.
Se pierde histórico que alguien necesitaba	Se compactó un tema de hechos, que conserva solo el último por clave	Separa temas de hechos (retención por tiempo) de temas de estado (compactados) y no mezcles las políticas.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué tres cosas permite un registro conservado que una cola no permite?
2. ¿Por qué el número de particiones limita el paralelismo y por qué ampliarlo rompe el orden por clave?
3. ¿Qué ocurre durante un rebalanceo y cómo se evita un bucle de rebalanceos?
4. ¿Qué posición se puede confirmar si se procesa un lote en paralelo?
5. ¿Para qué sirve la compactación por clave y qué tema no debe compactarse nunca?

Referencias

- Apache Kafka (2025). Design: topics, partitions and consumer groups — orden por partición y reparto del grupo.
<<https://kafka.apache.org/documentation/#design>>
- Apache Kafka (2025). Log compaction — retención por clave y borrado con mensaje vacío.
<<https://kafka.apache.org/documentation/#compaction>>
- Google Cloud (2025). Pub/Sub: ordering keys and message retention — orden por clave y relectura.
<<https://cloud.google.com/pubsub/docs/ordering>>
- AWS (2025). Kinesis Data Streams: shards and resharding — paralelismo por fragmento y coste de reparticionar.
<<https://docs.aws.amazon.com/streams/latest/dev/kinesis-using-sdk-java-resharding.html>>
- Kleppmann, M. (2017). Designing Data-Intensive Applications, cap. 11 — registros, posiciones y reproceso.
<<https://dataintensive.net/>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 09 en PDF · Manual integral

Anterior	Índice	Siguiente
← 113 · Colas, entrega, reintentos y dead-letter queues	Parte 09 · Programa	115 · Arquitectura dirigida por eventos y contratos →

Evaluación — 114 Pub/sub, streams, particiones y orden

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega stream-particionado con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 19 — Google Cloud: arquitectura de datos y operación en producción

229 — Resource Manager, folders, Shared VPC y guardrails

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: governance · Estado: EXECUTABLECORE

Propósito

Diseñar la jerarquía de Google Cloud —organización, carpetas y proyectos— y su red compartida, con la advertencia que abre la parte: el mayor riesgo aquí no es lo que se desconoce, sino trasladar suposiciones de las dos nubes anteriores. La clase explica por qué el proyecto es una frontera más fuerte que una suscripción, qué controlan las políticas de organización frente a los permisos, y por qué la red compartida separa quién opera la red de quién la usa.

Resultados de aprendizaje

Al finalizar podrás:

1. Repartir cargas entre carpetas y proyectos con criterio.
2. Distinguir política de organización de permiso, y usar cada una.
3. Diseñar red compartida separando operación de uso.
4. Evitar los errores de trasladar suposiciones de otras nubes.
5. Corregir una jerarquía con demasiados o muy pocos proyectos.

Conceptos centrales

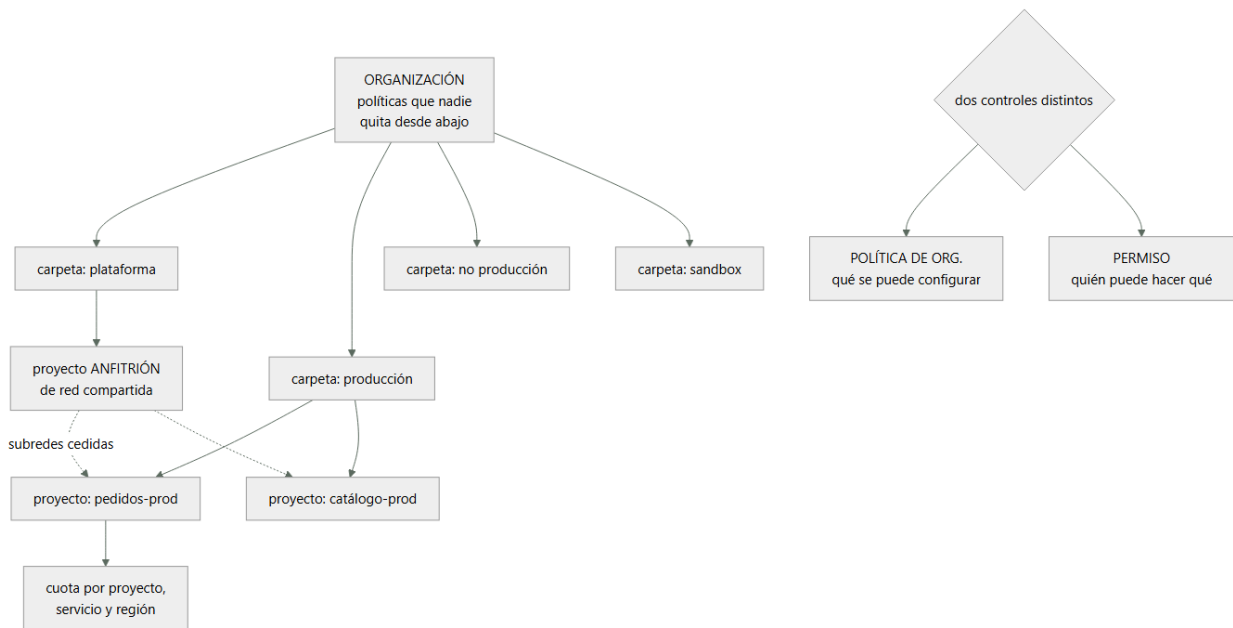
Concepto	Comprensión verificable
organización	Raíz de la jerarquía, ligada al dominio. Donde se aplican las políticas que nadie puede quitar desde abajo.
carpeta	Agrupación de proyectos y otras carpetas. Nivel intermedio donde se heredan políticas y permisos.
proyecto	Frontera de recursos, facturación, cuota, API habilitadas y red. Más fuerte que una suscripción.
política de organización	Restricción sobre qué se puede configurar. Distinta de un permiso: limita el qué, no el quién.
red compartida	Red de un proyecto anfitrión, usada por proyectos de servicio que no la administran.
cuota	Límite por proyecto, servicio y región. Se topa antes que la capacidad y se pide con antelación.

Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. La jerarquía, y por qué el proyecto es fuerte

La estructura tiene tres niveles y el del medio es opcional, pero el de abajo tiene más peso que sus equivalentes en otras nubes.

ORGANIZACIÓN

raíz, ligada al dominio
aquí van las políticas de organización y los permisos que se heredan

CARPETA

agrupación de proyectos, anidable
nivel natural para políticas por entorno o por unidad

PROYECTO

frontera de
recursos
facturación
CUOTAS
API habilitadas ← esto no existe en otras nubes
red (salvo que use una compartida)
y ámbito natural de muchos permisos

Y la diferencia que hay que interiorizar:

UN PROYECTO ES MÁS BARATO Y MÁS FUERTE que una suscripción
crear uno es inmediato y gratis
las API se habilitan por proyecto: lo que no está
habilitado, no existe
y borrar un proyecto se lleva todo lo de dentro

→ y por eso el error frecuente aquí es el CONTRARIO al de Azure: demasiados proyectos, no demasiado pocos

Y el criterio de reparto, con la tensión que hay que resolver:

UN PROYECTO POR CARGA Y ENTORNO

+ aislamiento fuerte, cuota propia, facturación clara
- más proyectos que gestionar: políticas, permisos, presupuestos, conexiones de red

UN PROYECTO POR EQUIPO Y ENTORNO

+ menos objetos que gestionar
- las cargas comparten cuota y se afectan

CRITERIO

por carga y entorno para producción
agrupado para desarrollo, si el volumen no lo justifica
→ y con la entrega AUTOMATIZADA, para que crear uno cueste
minutos y no semanas ley 16

Y las carpetas, que casi siempre se organizan de una de dos formas:

POR ENTORNO producción · no producción · sandbox
+ las políticas por entorno son las que más varían
- las unidades de negocio quedan mezcladas

POR UNIDAD, y dentro por entorno
+ refleja la organización
- la empresa se reorganiza antes que la nube clase 183

→ por entorno arriba y por unidad debajo suele envejecer mejor

Y una advertencia de traslado, la primera de muchas:

X «esto se hereda como allí»
las políticas de organización se heredan, sí
pero su combinación tiene reglas propias: algunas se fusionan y otras se sustituyen
→ hay que leerlas, no suponerlas

2. Políticas de organización y permisos: dos cosas

Aquí hay una separación que en otras nubes está mezclada, y confundirla produce controles que no controlan.

POLÍTICA DE ORGANIZACIÓN
restringe QUÉ se puede configurar
ejemplos
prohibir direcciones IP externas en máquinas
restringir las regiones donde se puede crear
prohibir claves de cuenta de servicio
exigir que los recursos no sean públicos
restringir qué dominios pueden recibir permisos
→ se aplica en organización, carpeta o proyecto
→ y NO depende de quién lo intente: ni el dueño puede

PERMISO
decide QUIÉN puede hacer QUÉ sobre QUÉ recurso
→ y se hereda hacia abajo

→ la política de organización es la barrera; el permiso es la puerta
→ y una barrera bien puesta hace que el permiso amplio sea menos peligroso clase 169

Y las políticas que conviene tener desde el primer día:

prohibir claves de cuenta de servicio
→ obliga a usar federación e identidad de carga clase 230

prohibir IP externas salvo donde se declare
restringir regiones
prohibir que los recursos se hagan públicos
restringir a qué dominios se pueden dar permisos
→ impide dar acceso a una cuenta personal por error
prohibir el reenvío de puertos y el acceso directo
exigir el uso de la red compartida

Y la disciplina de implantación, que es la de siempre:

- 1 aplicar en modo de solo registro donde exista
- 2 medir el incumplimiento actual
- 3 corregir o exceptuar, con dueño y fecha
- 4 aplicar

→ y recordando que no actúa hacia atrás ley 27

Las API habilitadas, que son un control que sorprende:

un servicio no se puede usar si su API no está habilitada en el proyecto
→ es un control de superficie muy efectivo y gratuito
→ habilitar solo lo que se usa reduce el alcance de un

compromiso

clase 189

y la trampa

habilitar una API es fácil y nadie la deshabilita

→ inventario periódico de API habilitadas y sin uso

ley 25

Y las cuotas, con la lección de la clase 222:

son por proyecto, servicio y región

se topan antes que la capacidad

se piden con antelación y tardan

→ margen del 30 % sobre el máximo del escalado

→ alerta al 80 %

clase 262

→ y no actualizar ni escalar si el margen no da

3. Red compartida: separar operar de usar

La red compartida es la pieza que más distingue a esta nube en gobierno, y resuelve un problema real.

EL PROBLEMA

si cada proyecto tiene su red, hay que conectarlas todas

entre sí

→ y cada equipo administra su red, con lo que eso implica

LA RED COMPARTIDA

un PROYECTO ANFITRIÓN contiene la red y sus subredes

los PROYECTOS DE SERVICIO usan esas subredes

→ los recursos viven en el proyecto de servicio

→ la red la administra el equipo de plataforma

QUÉ SEPARA

quién OPERA la red: plataforma

quién la USA: los equipos

→ y un equipo no puede crear rutas, emparejamientos ni

reglas por su cuenta

Y las decisiones que hay que tomar:

¿UNA RED COMPARTIDA O VARIAS?

una por entorno es lo habitual

→ producción y no producción NUNCA en la misma

¿QUÉ SUBREDES SE CEDEN A QUIÉN?

se ceden subredes concretas a proyectos concretos

→ un proyecto de servicio solo ve las subredes cedidas

→ y eso es un control de alcance real

clase 199

¿QUIÉN PUEDE CREAR REGLAS DE CORTAFUEGOS?

plataforma, con etiquetas o cuentas de servicio como

selector

→ y los equipos piden, no crean

→ si pedir tarda días, crearán recursos con IP pública

ley 16

Y las particularidades de red que hay que conocer para no trasladar suposiciones:

LA RED ES GLOBAL, las subredes son regionales

→ una sola red puede abarcar todas las regiones

→ y por tanto no hace falta emparejar entre regiones

→ es distinto de las otras dos nubes

clase 231

LAS REGLAS DE CORTAFUEGOS SON DE LA RED, no de la subred

con prioridades y con selectores por etiqueta o por

cuenta de servicio

→ seleccionar por cuenta de servicio es mucho mejor que

por etiqueta: la etiqueta la puede poner cualquiera con

permiso de instancia

LA SALIDA A INTERNET ESTÁ PERMITIDA por defecto

→ y hay que cerrarla explícitamente

ley 26

Y el error de traslado más caro en esta parte:

✗ «cada proyecto tendrá su red, como cada cuenta la suya»

→ produce decenas de redes que hay que conectar

- y pierde la ventaja principal de esta nube
- la red compartida es la elección por defecto salvo razón concreta

4. Facturación, entrega y corrección

La facturación, que se organiza distinto:

- una CUENTA DE FACTURACIÓN se asocia a muchos proyectos
 - y el desglose por proyecto es directo y limpio
 - por eso la separación por proyecto es también una decisión de atribución clase 239

y las etiquetas

- aquí se llaman de otra forma y hay dos mecanismos
 - unas sirven para facturación y organización
 - otras para condiciones de permiso y políticas
 - y no son intercambiables: hay que saber cuál se usa para qué

La entrega de proyectos, que decide si el gobierno se cumple:

- UN PROYECTO ENTREGADO trae
 - colocado en la carpeta correcta
 - cuenta de facturación asociada
 - API habilitadas mínimas
 - subredes cedidas de la red compartida
 - permisos base y cuentas de servicio
 - destinos de registro configurados clase 238
 - presupuesto y alertas
 - y las políticas heredadas aplicando

Y EL PLAZO

- si tarda semanas, los equipos usarán proyectos personales o pedirán permisos amplios en los que ya tienen ley 16
- automatizar la entrega es lo que hace cumplir el resto

Corregir una jerarquía, con el problema propio de esta nube:

DEMASIADOS PROYECTOS

- el síntoma: cientos de proyectos, muchos sin uso
 - y cada uno con su presupuesto, permisos y API
- corrección
 - inventario con último uso y dueño
 - los sin actividad en 90 días, a retirar ley 25
 - y consolidar los de desarrollo por equipo

PROYECTOS EN LA CARPETA EQUIVOCADA

- mover es sencillo y las políticas empiezan a aplicar
 - pero lo ya creado sigue incumpliendo ley 27
 - hace falta remediación, no solo mover

PROYECTOS SIN CARPETA, colgando de la organización

- no reciben las políticas por entorno
- es el hallazgo típico del primer inventario

Y lo que hay que vigilar:

- proyectos sin carpeta, sin dueño o sin presupuesto
- API habilitadas y sin uso
- cuotas frente al máximo del escalado
- políticas de organización con excepciones, y su antigüedad
- subredes cedidas y no usadas
- y recursos creados fuera de la red compartida

Y la lista de comprobación de la clase:

- hay carpetas por entorno, y proyectos por carga
- ningún proyecto cuelga directamente de la organización
- las políticas de organización están en organización o carpeta
- están las políticas mínimas: sin claves de cuenta de servicio, sin IP externas, regiones, no público, dominios
- se midió el incumplimiento antes de aplicar
- hay plan de remediación de lo existente

- solo están habilitadas las API que se usan
- hay red compartida por entorno, con subredes cedidas
- las reglas de cortafuegos seleccionan por cuenta de servicio, no por etiqueta
- la salida a internet está cerrada explícitamente
- las cuotas tienen margen y alerta al 80 %
- la entrega de un proyecto está automatizada y tarda minutos
- hay inventario de proyectos con último uso y dueño

Y el cierre que enlaza con la clase siguiente: con la jerarquía y la red repartidas, el control que decide el alcance real vuelve a ser la identidad, y aquí hay un elemento que no existe en las otras dos nubes: las condiciones en las asignaciones. Es la materia de la clase 230.

Ejemplo trabajado

CloudShop monta su organización en Google Cloud, con la experiencia de las dos nubes anteriores. Lo que sigue son los tres errores de traslado que cometió el equipo, el inventario que encontró 214 proyectos, y la red compartida que redujo 41 redes a dos.

El punto de partida, tras dos años de uso sin gobierno:

proyectos	214
colgando de la organización, sin carpeta	189
con dueño identificable	94
sin actividad en 90 días	71
sin presupuesto asociado	203
redes	41
una por proyecto que la necesitaba	
emparejamientos entre ellas	67
políticas de organización aplicadas	2
claves de cuenta de servicio	341
con más de 1 año	218

Los tres errores de traslado, cometidos en las primeras semanas:

ERROR 1 · «cada proyecto con su red»
 el equipo venía de un modelo de cuenta por carga con red propia
 creó 12 redes nuevas antes de que alguien preguntara
 → y la conexión entre ellas exigía emparejamientos, que no son transitivos

al descubrir la red compartida
 12 redes nuevas → 0
 41 redes existentes → 2 (producción y no producción)
 67 emparejamientos → 4
 y una diferencia que no habían previsto
 la red es GLOBAL: una sola red cubre las 3 regiones
 → no hacen falta emparejamientos entre regiones

ERROR 2 · «las reglas de cortafuegos son por subred»
 se crearon reglas asumiendo que aplicaban a la subred
 → aquí son de la RED, con selectores
 → y las primeras se escribieron con selectores por ETIQUETA
 → cualquiera con permiso sobre una instancia podía ponerle la etiqueta que abría el acceso a la base

corrección
 selectores por CUENTA DE SERVICIO
 → poner una cuenta de servicio a una instancia exige permiso sobre esa cuenta
 → y eso sí es un control clase 230

ERROR 3 · «los permisos amplios ya los limita la barrera»
 se asumió el modelo de la nube anterior
 → aquí la política de organización limita QUÉ se configura, no QUIÉN accede a los datos
 → una asignación amplia sobre almacenamiento seguía

dando acceso a los datos, con la barrera puesta

corrección

los dos controles, y no uno en lugar del otro
clase 230

Y la conclusión que el equipo escribió:

los tres errores fueron de SUPOSICIÓN, no de desconocimiento
cada uno venía de dar por válido algo de otra nube
→ y los tres se detectaron por casualidad, no por una comprobación
→ desde entonces, la primera tarea de cada servicio nuevo es leer qué se hereda, qué viene activado y qué se propaga
clase 228

La jerarquía nueva:

```
organización
■■■■ plataforma
■ ■■■■ red-compartida-prod      (proyecto anfitrión)
■ ■■■■ red-compartida-nopro    (proyecto anfitrión)
■ ■■■■ identidad
■ ■■■■ observabilidad
■ ■■■■ seguridad
■■■■ produccion
■ ■■■■ pedidos-prod
■ ■■■■ catalogo-prod
■ ■■■■ precios-prod
■ ■■■■ ... (14 proyectos, uno por carga)
■■■■ no-produccion
■ ■■■■ pedidos-pre, pedidos-dev
■ ■■■■ ... (28 proyectos)
■■■■ sandbox                    con caducidad de 60 días
```

Las políticas de organización, implantadas por orden:

política	incumplimiento inicial
sin claves de cuenta de servicio	341
sin IP externas en máquinas	118
regiones restringidas a la UE	41
sin recursos públicos	29
dominios permitidos para permisos	14
sin reenvío de puertos	62
uso obligatorio de red compartida	n/a (nuevo)

el orden que se siguió

- 1 dominios permitidos ← 14 casos, 2 días
y encontró 3 permisos concedidos a cuentas personales de ex empleados ley 25
- 2 sin recursos públicos ← 29 casos, 1 semana
de ellos, 4 almacenes con datos de clientes
- 3 regiones ← 41 casos, 3 semanas
- 4 sin IP externas ← 118 casos, 6 semanas
sustituidas por acceso mediante servicio de administración clase 256
- 5 sin claves de cuenta de servicio ← 341, 4 meses
el más largo, y el de más valor clase 230

y la remediación, que no vino sola

aplicar las políticas corrigió 0 recursos existentes
→ tareas de remediación planificadas aparte ley 27

El inventario de proyectos:

214 proyectos, con último uso y dueño

con actividad y dueño	94
con actividad y SIN dueño	49
→ 12 resultaron ser de equipos que ya no existen	
→ 3 tenían cargas en producción que nadie sabía que existían ley 20	
sin actividad en 90 días	71
→ apagados, y borrados 30 días después	

→ 2 reclamaciones, ambas restauradas en minutos

proyectos tras la limpieza	143
→ y consolidando los de desarrollo por equipo:	86

coste liberado	4.900 €/mes
de los cuales, en los 71 sin actividad	3.100 €

Y el hallazgo más incómodo:

uno de los 71 sin actividad tenía
una base de datos con una copia de la tabla de clientes
de 2023
sin cifrado con clave propia
sin registro de acceso
y con permisos de lectura para «todos los usuarios
autenticados» ← es decir, cualquier cuenta de Google

llevaba así	19 meses
lo encontró	el inventario, no una alerta ley 15

La red compartida, montada:

dos proyectos anfitriones: producción y no producción
red global por entorno; subredes por región
subredes cedidas a proyectos concretos
→ pedidos-prod ve su subred y nada más

reglas de cortafuegos
denegación por defecto en entrada y en SALIDA
selectores por cuenta de servicio
creadas por plataforma, pedidas por los equipos
→ y el plazo de una petición: 40 minutos, automatizado
con revisión

resultado	
redes	41 → 2
emparejamientos	67 → 4
reglas de cortafuegos	940 → 118
equipos que pueden crear rutas	41 → 1

La entrega de proyectos, automatizada:

antes petición por correo → 2 semanas
y 189 proyectos creados fuera del proceso
después plantilla que crea el proyecto con
carpeta, facturación, API mínimas, subred cedida,
permisos base, destinos de registro, presupuesto
plazo 14 min
proyectos creados fuera del proceso en 6 meses: 0

El resultado:

proyectos	214	86
sin carpeta	189	0
sin dueño	120	0
sin presupuesto	203	0
redes	41	2
claves de cuenta de servicio (las 12, con excepción registrada)	341	12
recursos públicos	29	0
permisos a cuentas fuera del dominio	14	0
coste mensual	28.700 €	21.400 €
plazo de entrega de un proyecto	2 semanas	14 min

La lección que esta clase deja: los tres errores del arranque no fueron por desconocer esta nube, sino por dar por válidas suposiciones de las otras dos, y ninguno lo detectó una comprobación: se descubrieron por casualidad. Y el inventario de proyectos —que no requería tecnología ninguna— encontró setenta y uno sin actividad, tres con cargas de producción que nadie conocía y una copia de la tabla de clientes legible por cualquier cuenta de Google desde hacía diecinueve meses.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/229-resource-manager-folders-shared-vpc-y-guardrails/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa gcp-foundation en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye gcp-foundation para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
Aparecen decenas de redes que hay que conectar entre sí	Se trasladó el modelo de una red por cuenta o suscripción	Usa red compartida por entorno; la red es global y no hacen falta emparejamientos entre regiones.
Cualquiera puede abrir un acceso poniendo una etiqueta a su instancia	Las reglas de cortafuegos seleccionan por etiqueta	Selecciona por cuenta de servicio; asignarla exige permiso sobre esa cuenta.
Las barreras están puestas y siguen existiendo accesos amplios a los datos	Se confundió política de organización con permiso: la primera limita qué se configura, no quién accede	Usa los dos controles, no uno en lugar del otro.
Hay cientos de proyectos y nadie sabe cuáles sirven	Crear un proyecto es inmediato y nadie los retira	Inventario con último uso y dueño; apaga los inactivos, bórralos después y consolida los de desarrollo.
Mover proyectos a la carpeta correcta no corrige nada	Las políticas no actúan sobre lo ya creado	Planifica una tarea de remediación además del movimiento.
Los equipos crean recursos fuera del proceso	Conseguir un proyecto o una regla de cortafuegos tarda semanas	Automatiza la entrega de proyectos y las peticiones de reglas para que tarden minutos.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué hace que un proyecto sea una frontera más fuerte que una suscripción?
2. ¿Qué diferencia hay entre una política de organización y un permiso?
3. ¿Qué separa la red compartida y por qué importa?
4. ¿Por qué los selectores por cuenta de servicio son mejores que por etiqueta?
5. ¿Cuál es el error de reparto de proyectos característico de esta nube?

Referencias

- Google Cloud (2025). Resource hierarchy. <<https://cloud.google.com/resource-manager/docs/cloud-platform-resource-hierarchy>>
- Google Cloud (2025). Organization policy constraints. <<https://cloud.google.com/resource-manager/docs/organization-policy/org-policy-constraints>>
- Google Cloud (2025). Shared VPC overview. <<https://cloud.google.com/vpc/docs/shared-vpc>>
- Google Cloud (2025). VPC firewall rules and service account targets. <<https://cloud.google.com/firewall/docs/firewalls>>
- Google Cloud (2025). Landing zone design in Google Cloud. <<https://cloud.google.com/architecture/landing-zones>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 19 en PDF · Recorrido de Google Cloud en PDF · Manual integral

Anterior	Índice	Siguiente
← 228 · Proyecto: CloudShop productivo en Azure	Parte 19 · Programa	230 · Workload Identity Federation, IAM Conditions y PAM →

Evaluación — 229 Resource Manager, folders, Shared VPC y guardrails

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega gcp-foundation con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

230 — Workload Identity Federation, IAM Conditions y PAM

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: iam · Estado: EXECUTABLECORE

Propósito

Resolver la identidad en Google Cloud, donde el mecanismo tiene una pieza que no existe en las otras dos nubes: las condiciones en las asignaciones, que permiten conceder un permiso solo sobre ciertos recursos y durante cierto tiempo. La clase cubre la federación y las cuentas de servicio sin claves, la suplantación como sustituto de las credenciales guardadas, el ámbito de asignación —que vuelve a ser el error frecuente— y la elevación temporal con aprobación.

Resultados de aprendizaje

Al finalizar podrás:

1. Eliminar claves de cuenta de servicio con federación y suplantación.
2. Asignar permisos en el ámbito mínimo, con condiciones.
3. Distinguir quién actúa de qué identidad se usa, y auditarlo.
4. Configurar elevación temporal con aprobación y caducidad.
5. Detectar y reducir el alcance de las identidades más amplias.

Conceptos centrales

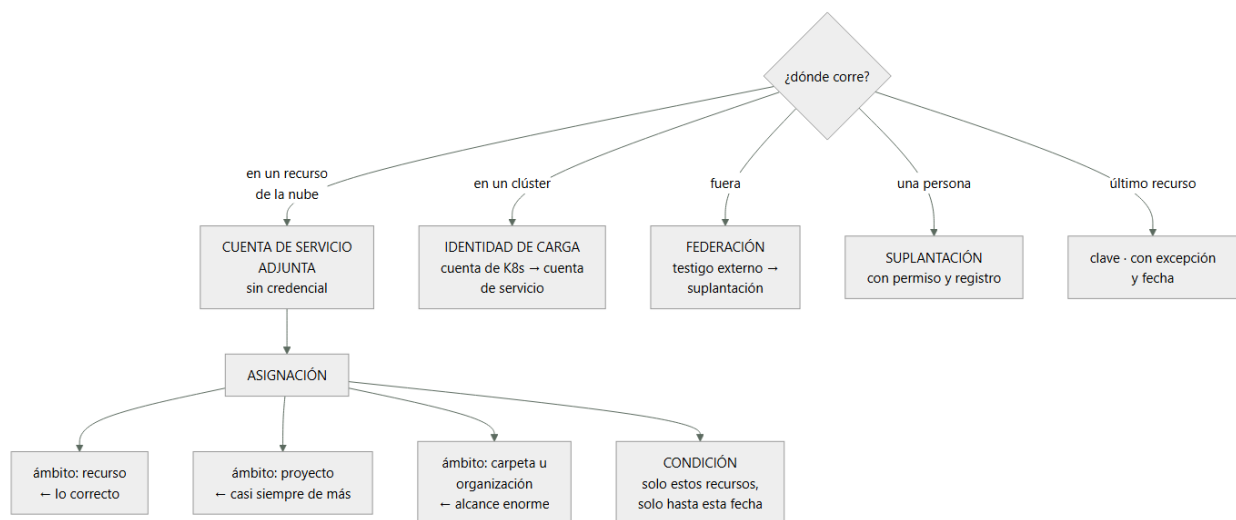
Concepto	Comprensión verificable
cuenta de servicio	Identidad de una carga. Puede usarse sin credenciales si se adjunta al recurso o se suplanta.
clave de cuenta de servicio	Credencial de larga duración en un fichero. Lo que hay que eliminar.
suplantación	Obtener un testigo de otra identidad con permiso para ello, sin credencial guardada.
federación de identidad de carga	Confianza en un emisor externo para que sus testigos suplanten una cuenta de servicio.
condición en la asignación	Expresión que limita cuándo y sobre qué recursos aplica un permiso.
identidad de carga del clúster	Vinculación entre una cuenta de servicio de Kubernetes y una de la nube, sin claves.

Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Eliminar las claves

Una clave de cuenta de servicio es un fichero con una credencial que no caduca. Es lo que hay que eliminar, y aquí hay cuatro formas de no necesitarla.

- 1 **CUENTA DE SERVICIO ADJUNTA** – para lo que corre en la nube la máquina, la función o el servicio lleva una cuenta de servicio asociada
el código obtiene testigos del servicio de metadatos
→ sin credencial en ningún sitio

y la regla que se olvida
NO usar la cuenta de servicio POR DEFECTO del proyecto
→ tiene permisos amplios y la comparten todos los recursos del proyecto
→ una cuenta por carga, con sus permisos ley 26
- 2 **IDENTIDAD DE CARGA** – para lo que corre en un clúster una cuenta de servicio de Kubernetes se vincula con una de la nube
→ el pod obtiene testigos sin claves clase 234
- 3 **FEDERACIÓN** – para lo que corre fuera se declara confianza en un emisor externo (canalización, otra nube, otro clúster)
sus testigos suplantan una cuenta de servicio
→ mismo mecanismo y mismos errores que en la clase 206
- 4 **SUPLANTACIÓN** – para personas y para encadenar una identidad con permiso obtiene un testigo de otra
→ y el registro guarda QUIÉN suplantó a QUIÉN

Y la política que lo hace efectivo:

prohibir la creación de claves de cuenta de servicio, como política de organización clase 229
→ y las existentes, inventariadas, migradas y borradas
→ con excepción registrada, dueño y fecha para lo que no se pueda ley 25

La suplantación, que es la pieza que más cambia el día a día:

UNA PERSONA no necesita permisos de producción
tiene permiso para SUPLANTAR una cuenta de servicio que sí los tiene
→ obtiene un testigo de corta duración
→ y queda registrado quién lo hizo clase 238

Y LAS CANALIZACIONES ENCADENAN

la identidad federada suplanta una cuenta de despliegue

- y esa cuenta tiene los permisos, no el testigo externo
- así se cambian los permisos sin tocar la federación

Y el error que hay que evitar:

- dar a muchos el permiso de suplantar una cuenta muy permisiva
- es equivalente a darles esos permisos, y más difícil de ver en un inventario
- el permiso de suplantar se audita como si fuera el permiso suplantado

2. Ámbito y condiciones

El error frecuente vuelve a ser el mismo de las clases 206 y 218: el ámbito demasiado amplio. Aquí hay además una herramienta que no existe en las otras dos.

- LOS ÁMBITOS, de mayor a menor
 - organización → todos los proyectos
 - carpeta → todos los de esa rama
 - proyecto → todos sus recursos
 - RECURSO → solo ese

- y la costumbre
 - asignar «Editor» en el proyecto, porque funciona
 - esa identidad puede crear, modificar y borrar casi todo
 - incluida la base de datos clase 218

Las condiciones, que permiten afinar sin crear papeles a medida:

UNA CONDICIÓN es una expresión que limita cuándo aplica el permiso

- POR RECURSO
 - «solo sobre los almacenes cuyo nombre empiece por pedidos-»
 - un permiso de proyecto que en la práctica solo toca lo suyo

- POR TIEMPO
 - «solo hasta el 31 de marzo»
 - el acceso caduca solo ley 25
 - y esto resuelve el problema del acceso temporal sin infraestructura adicional

- POR ETIQUETA
 - «solo sobre recursos con entorno = desarrollo»

- POR TIPO DE RECURSO
 - «solo sobre conjuntos de datos, no sobre trabajos»

Y el uso que más aporta:

- el acceso concedido durante un incidente, con condición de tiempo
- nadie tiene que acordarse de quitarlo
- y el registro de decisión queda en la propia asignación

Y las limitaciones que hay que conocer:

- no todos los servicios soportan condiciones por recurso
 - hay que comprobarlo por servicio
- las condiciones complican la auditoría: un inventario que no las lee muestra permisos que en realidad no aplican
 - y al revés: un permiso que parece limitado puede no estarlo si el servicio ignora la condición
- comprobar con una prueba negativa, no suponer ley 22

Los papeles, con la disciplina de siempre:

- usar el papel PREDEFINIDO más específico que exista
 - no «Editor» sino «Escritor de objetos de almacenamiento»
- y papel PERSONALIZADO cuando ninguno encaje
 - con las acciones justas, y mantenido
- y los papeles que se conceden con elevación temporal

Propietario
Administrador de organización
Administrador de facturación
Administrador de seguridad
→ nunca permanentes

3. Elevación temporal y auditoría

La elevación temporal aquí se puede montar de dos formas, y conviene conocer las dos.

CON EL SERVICIO DE GESTIÓN DE ACCESO PRIVILEGIADO
la persona es ELEGIBLE; solicita, se aprueba, caduca
→ con justificación y registro

CON CONDICIONES DE TIEMPO
una asignación con condición de caducidad
→ más simple, sin aprobación integrada
→ útil para accesos acordados fuera de línea

Y EN AMBOS CASOS
activación rápida cuando no requiere aprobación
aprobadores con turnos cuando sí ley 16
y un camino de emergencia sin aprobación, con alerta
inmediata clase 218

Y la prueba obligatoria:

usar el acceso de emergencia y cronometrar
→ en la clase 179 estaba creado, documentado y no
funcionaba ley 22

La auditoría, con lo que hay que saber leer:

LOS REGISTROS DE ACTIVIDAD DE ADMINISTRACIÓN
siempre activos y sin coste
quién creó, modificó o borró qué

LOS REGISTROS DE ACCESO A DATOS
DESACTIVADOS por defecto, salvo unos pocos ley 26
→ quién leyó qué
→ y son los que hacen falta para investigar una fuga
→ y los que más volumen generan: hay que elegir
servicios y coste clase 238

Y EL CAMPO QUE HAY QUE MIRAR
cuando alguien suplanta, el registro guarda la identidad
ORIGINAL y la suplantada
→ sin leer las dos, la auditoría atribuye la acción a la
cuenta de servicio y no a la persona

Medir el alcance, que es lo que ordena el trabajo:

el analizador de políticas responde preguntas concretas
¿quién puede leer este almacén?
¿a qué recursos puede acceder esta identidad?
¿quién puede suplantar esta cuenta de servicio?

→ y esas tres preguntas, ejecutadas periódicamente, son la
medida de alcance de la clase 133

y las recomendaciones de permisos sobrantes
la plataforma sugiere reducir papeles poco usados
→ basadas en uso real de 90 días
→ se evalúan, no se aplican a ciegas clase 227

Y una fuente de alcance que se olvida:

LOS GRUPOS
los permisos se conceden a grupos, y la pertenencia se
gestiona en el directorio
→ alguien añadido a un grupo hereda permisos que nadie
revisó
→ hay que auditar la pertenencia efectiva clase 218

4. Lo que hay que comprobar

Las pruebas negativas de esta clase, que son las mismas de siempre con otras piezas:

- crear una clave de cuenta de servicio → debe fallar
- suplantar una cuenta desde una identidad sin permiso → debe fallar
- usar una identidad federada desde otro repositorio → debe fallar
- acceder a un recurso fuera de la condición → debe fallar
- usar un acceso cuya condición de tiempo ha caducado → debe fallar
- activar un papel privilegiado sin aprobación → debe fallar
- entrar con el acceso de emergencia → debe funcionar
- comprobar que el registro guarda la identidad original en una suplantación
- y que las tres preguntas de alcance devuelven lo esperado

Y las señales que hay que vigilar:

- claves de cuenta de servicio existentes → cero
- asignaciones en organización y carpeta → mínimas
- identidades con papeles de administración
- permisos concedidos y no usados en 90 días
- condiciones de tiempo que vencen este mes
- usos del acceso de emergencia → alerta
- cambios en asignaciones de papeles → auditados
- suplantaciones de cuentas privilegiadas → alerta

Y la disciplina de revisión:

- trimestral
 - revisión de accesos por dueño
 - pertenencia efectiva de grupos
 - quién puede suplantar cada cuenta privilegiada
 - y prueba del acceso de emergencia
- y la regla que la hace útil
 - quien no responde a la revisión → se retira el acceso
 - clase 218

Los errores de traslado, que esta parte vigila especialmente:

- ✗ «la barrera limita el acceso a los datos»
 - la política de organización limita qué se configura
 - el acceso a los datos lo decide el permiso
 - clase 229
- ✗ «con quitar la clave ya está resuelto»
 - si la cuenta de servicio conserva permisos amplios, el problema sigue: solo cambia cómo se obtiene el testigo
- ✗ «la condición de recurso funciona en todos los servicios»
 - hay que comprobarlo por servicio, con una prueba

Y la lista de comprobación de la clase:

- la política prohíbe crear claves de cuenta de servicio
- no queda ninguna clave sin excepción registrada
- ningún recurso usa la cuenta de servicio por defecto
- lo que corre en la nube usa cuenta adjunta
- lo que corre fuera usa federación con sujeto exacto
- las personas acceden por suplantación, no con permisos propios de producción
- las asignaciones están en el ámbito del recurso
- se usan condiciones por recurso y por tiempo donde aportan
- se comprobó por servicio que la condición se respeta
- los papeles privilegiados requieren elevación temporal
- el acceso de emergencia se ha probado este trimestre
- los registros de acceso a datos están activados donde hacen falta
- la auditoría lee la identidad original en las suplantaciones

- se ejecutan las tres preguntas de alcance periódicamente

Y el cierre que enlaza con la clase siguiente: con jerarquía e identidad resueltas, queda la red, que aquí es global y tiene mecanismos de conectividad privada y de balanceo distintos de los de las otras dos nubes. Es la materia de la clase 231.

Ejemplo trabajado

CloudShop elimina las 341 claves de cuenta de servicio de su organización. Lo que sigue es el inventario, la migración por tipos, y el hallazgo de que quitar las claves no reducía el alcance.

El inventario de claves:

claves de cuenta de servicio	341
con más de 1 año	218
con más de 3 años	94
usadas en los últimos 90 días	127
NUNCA usadas	61
usadas desde fuera de la organización	14 ←

y las 14 usadas desde fuera

- 9 canalizaciones de un repositorio externo
- 3 herramientas de proveedores
- 1 un script en el portátil de un consultor que terminó su contrato en 2023 ley 25, 20
- 1 uso desde una dirección que nadie pudo explicar
 - se revocó inmediatamente y se investigó
 - resultó ser un entorno de pruebas de un socio, no declarado

La migración, por tipos:

tipo de uso	claves	migrado a
cargas en máquinas	88	cuenta adjunta
cargas en el clúster	64	identidad de carga
canalizaciones internas	41	federación
canalizaciones externas	9	federación
funciones y servicios	52	cuenta adjunta
scripts de personas	43	suplantación
herramientas de terceros	12	9 federadas, 3 con excepción
sin uso	32	borradas
total migrado	341 → 3 con excepción	registrada, dueño y fecha

duración 4 meses

Y los dos pasos que costaron más:

LAS 43 DE PERSONAS

cada una tenía su clave para «hacer cosas rápido»
y muchas con permisos de proyecto
→ sustituidas por suplantación de cuentas de servicio concretas
→ y aquí hubo resistencia: suplantar exige un comando más
→ se resolvió con un envoltorio que lo hace transparente ley 16

LAS 88 DE MÁQUINAS

al mirar cuál usaban, 71 usaban la CUENTA DE SERVICIO POR DEFECTO del proyecto
→ que tiene permisos amplios y la comparten todos los recursos ley 26
→ no bastaba quitar la clave: había que crear una cuenta por carga

El hallazgo: quitar las claves no redujo el alcance.

tras migrar las 341, se midió el alcance con las tres preguntas del analizador

¿a qué recursos llega cada identidad?

identidad	antes	después
cuenta por defecto de pedidos-prod	todo el proyecto	todo el proyecto
despliegue	organización	organización
cuenta de análisis	41 conjuntos	41 conjuntos

→ el alcance NO había cambiado
→ solo había cambiado cómo se obtiene el testigo

y la conclusión
eliminar credenciales resuelve el ROBO de credenciales
no resuelve el ALCANCE
→ son dos trabajos distintos, y el segundo es el que reduce el daño de un compromiso clases 189, 218

La reducción de alcance, con condiciones:

asignaciones inventariadas	2.410
en organización	18
en carpeta	140
en proyecto	1.870
en recurso	382

las tres peores

- 1 la cuenta de despliegue con «Editor» en la organización
alcance: 86 proyectos, todo
corrección: 5 cuentas, una por propósito, con asignación en el proyecto y CONDICIÓN por prefijo de nombre de recurso
alcance: entre 4 y 40 recursos cada una
- 2 el equipo de análisis con «Visor de datos» en la carpeta de producción
alcance: todos los conjuntos y tablas, incluidos los que contienen datos personales completos
corrección: asignación por conjunto, y para los sensibles, seudonimizados clases 236, 239
- 3 17 personas con «Editor» en proyectos de producción
corrección: suplantación de cuentas concretas, con elevación temporal para lo privilegiado

Y el uso de las condiciones, con la comprobación que hizo falta:

se usaron condiciones en 61 asignaciones	
por prefijo de recurso	38
por tiempo (accesos temporales)	19
por etiqueta de entorno	4

y la prueba negativa, por servicio
se intentó acceder a un recurso FUERA de la condición en cada uno de los 9 servicios implicados

respetaron la condición	7
NO la respetaron	2 ←
→ dos servicios ignoraban la condición por recurso	
→ esas 2 asignaciones se sustituyeron por papeles personalizados y asignación en el recurso	

→ y sin esa prueba, dos permisos habrían parecido limitados y no lo estaban ley 22

La elevación temporal:

papeles privilegiados retirados como permanentes	
Propietario	14 → 0
Administrador de organización	6 → 0
Administrador de facturación	4 → 0
Administrador de seguridad	3 → 0

configurados como elegibles
con aprobación de 2 personas y caducidad de 4 h
emergencia: sin aprobación, con alerta inmediata

primeros tres meses

activaciones	186
de emergencia	2
· 1 incidente real	
· 1 porque el aprobador estaba de baja y nadie cubría	
→ turno de aprobadores corregido	

y la prueba de emergencia

primera ejecución: funcionó, en 90 segundos

segunda (trimestral): la cuenta había perdido un permiso al reorganizar la carpeta ley 27

→ corregido, y añadida comprobación automática mensual

La auditoría, con el campo que faltaba leer:

al investigar un borrado accidental de una tabla

el registro decía: «cuenta de servicio de análisis borró la tabla»

→ y el equipo buscó qué carga lo había hecho

al leer el campo de identidad ORIGINAL

una persona había suplantado esa cuenta y ejecutado el borrado a mano

→ 3 horas de investigación por no leer un campo

corrección

las consultas guardadas incluyen siempre la identidad original

y una alerta: «suplantación de una cuenta privilegiada»

El resultado:

claves de cuenta de servicio	341	3
usadas desde fuera sin explicar	1	0
recursos con cuenta de servicio por defecto	71	0
asignaciones en organización	18	4
asignaciones en carpeta	140	22
alcance de la cuenta de despliegue	86 proyectos	4-40 recursos
papeles privilegiados permanentes	27	0
asignaciones con condición	0	59
condiciones que un servicio ignoraba	—	0
tiempo de investigación de una acción	3 horas	4 min

La lección que esta clase deja: eliminar las trescientas cuarenta y una claves costó cuatro meses y no redujo el alcance ni un punto: solo cambió cómo se obtiene el testigo. Reducir el alcance fue un trabajo distinto y posterior, y ahí las condiciones por recurso hicieron el trabajo que en otras nubes exige papeles a medida. Y dos de los nueve servicios ignoraban la condición, lo que se descubrió con una prueba negativa y habría dejado dos permisos aparentemente limitados sin estarlo.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/230-workload-identity-federation-iam-conditions-y-pam/lab.py
```

El laboratorio selecciona el motor de práctica iam y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa gcp-identity en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de acceso mínimo con prueba de denegación. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye gcp-identity para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Se eliminan las claves y el riesgo apenas baja	Quitar la credencial resuelve el robo, no el alcance de la identidad	Trata reducir alcance como un trabajo aparte: mide qué recursos alcanza cada identidad y baja el ámbito de las asignaciones.
Todos los recursos de un proyecto comparten una identidad muy permisiva	Usan la cuenta de servicio por defecto del proyecto	Una cuenta de servicio por carga, con sus permisos, y prohibición de usar la predeterminada.
Un permiso parece limitado por una condición y no lo está	El servicio no respeta las condiciones por recurso	Comprueba con una prueba negativa servicio por servicio; donde no se respeten, usa papeles personalizados y asignación en el recurso.
La auditoría atribuye una acción a una cuenta de servicio y no a la persona	No se lee el campo de identidad original en las suplantaciones	Incluye siempre la identidad original en las consultas y alerta sobre suplantaciones de cuentas privilegiadas.
Dar el permiso de suplantar parece inofensivo y no lo es	Equivale a conceder los permisos de la cuenta suplantada	Audita quién puede suplantar cada cuenta privilegiada como si fuera una asignación directa.
El acceso de emergencia funcionó una vez y luego no	Perdió un permiso al reorganizar la jerarquía y nadie lo comprobó	Prueba trimestral del acceso de emergencia y comprobación automática de sus permisos.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Cuáles son las cuatro formas de no necesitar una clave de cuenta de servicio?
2. ¿Qué permiten las condiciones que no permite el ámbito por sí solo?
3. ¿Por qué el permiso de suplantar debe auditarse como el permiso suplantado?
4. ¿Qué registros están desactivados por defecto y por qué hacen falta?

5. ¿Qué campo hay que leer para saber quién ejecutó realmente una acción?

Referencias

- Google Cloud (2025). Best practices for using service accounts. <<https://cloud.google.com/iam/docs/best-practices-service-accounts>>
- Google Cloud (2025). Workload identity federation. <<https://cloud.google.com/iam/docs/workload-identity-federation>>
- Google Cloud (2025). IAM conditions overview. <<https://cloud.google.com/iam/docs/conditions-overview>>
- Google Cloud (2025). Privileged Access Manager. <<https://cloud.google.com/iam/docs/pam-overview>>
- Google Cloud (2025). Policy Analyzer and IAM recommender. <<https://cloud.google.com/policy-intelligence/docs/policy-analyzer-overview>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 19 en PDF · Recorrido de Google Cloud en PDF · Manual integral

Anterior	Índice	Siguiente
← 229 · Resource Manager, folders, Shared VPC y guardrails	Parte 19 · Programa	231 · Red global, load balancing, PSC y Cloud DNS →

Evaluación — 230 Workload Identity Federation, IAM Conditions y PAM

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega gcp-identity con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

231 — Red global, load balancing, PSC y Cloud DNS

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: network · Estado: EXECUTABLECORE

Propósito

Montar la red de Google Cloud, que es la que más se diferencia de las otras dos: la red es global y una sola dirección puede servir desde todas las regiones. La clase explica qué implica eso para el balanceo, cómo funciona la conectividad privada hacia servicios propios y de terceros, y la resolución de nombres con su equivalente del problema de la clase 219: si la zona no está asociada a la red, el nombre resuelve a la dirección pública.

Resultados de aprendizaje

Al finalizar podrás:

1. Aprovechar la red global sin trasladar suposiciones regionales.
2. Elegir el balanceador adecuado entre global y regional.
3. Conectar a servicios propios y de terceros de forma privada.
4. Resolver nombres privados en todas las redes que los necesitan.
5. Controlar la salida a internet, que está abierta por defecto.

Conceptos centrales

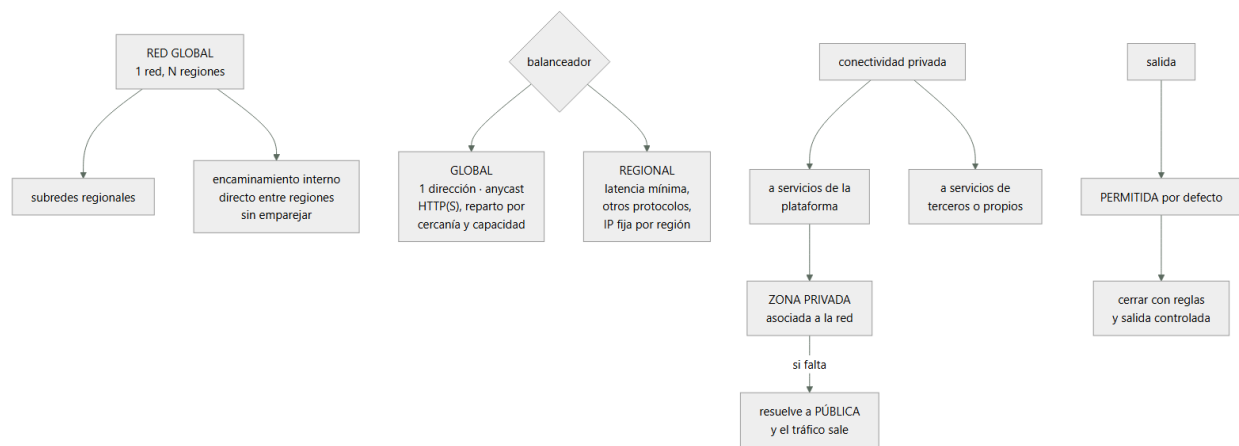
Concepto	Comprensión verificable
red global	Red virtual que abarca todas las regiones. Las subredes son regionales; el encaminamiento interno es directo.
balanceo global	Una dirección única que reparte a los grupos de la región más cercana con capacidad.
conexión de servicio privada	Mecanismo para alcanzar un servicio de la plataforma o de un tercero por una dirección de tu red.
acceso privado a servicios	Alternativa que permite alcanzar servicios de la plataforma sin dirección externa, por rutas privilegiadas.
zona privada de nombres	Zona asociada a redes concretas. Sin asociación, el nombre resuelve a la dirección pública.
salida controlada	Restricción del tráfico saliente, que está permitido por defecto y hay que cerrar.

Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. La red global, y qué cambia

Esta es la diferencia estructural que más suposiciones invalida.

LA RED ES GLOBAL

- una sola red abarca todas las regiones
- las SUBREDES son regionales
- y los recursos de subredes distintas se alcanzan directamente, sin emparejar nada

QUÉ DESAPARECE

- emparejamientos entre regiones
- concentradores de tránsito para unir regiones propias
- y buena parte de la complejidad de encaminamiento de las clases 194 y 199

QUÉ NO DESAPARECE

- el coste del tráfico entre regiones, que sigue
- el coste del tráfico entre zonas, que también
- y la latencia física

Y las suposiciones que hay que desmontar:

- ✗ «hay que emparejar las redes de cada región»
no; es una sola red
- ✗ «una subred agotada solo afecta a su región»
cierto, pero el rango se decide en el plan global
clase 193
→ y las subredes se pueden AMPLIAR sin recrear, que es una ventaja real frente a otras nubes
- ✗ «las reglas de cortafuegos son por subred»
son de la RED, con selectores y prioridades clase 229
→ una regla mal puesta afecta a todas las regiones
- ✗ «el balanceador es regional»
hay de los dos, y el global cambia el diseño

Los rangos secundarios, que hacen falta para contenedores:

- una subred puede tener rangos SECUNDARIOS
→ los pods y los servicios del clúster usan esos rangos
→ y se dimensionan aparte del rango principal
clases 193, 234

- el plan de direcciones tiene que contarlos, y son los que más consumen

Y una decisión que conviene tomar pronto:

¿SUBREDES AUTOMÁTICAS O PERSONALIZADAS?

- automáticas la red crea una subred por región, con rangos fijos

- cómodo y garantiza solapamiento con cualquier otra red creada igual
- personalizadas se declaran los rangos del plan
- la elección correcta para cualquier organización

clase 193

→ y las redes automáticas se convierten en personalizadas, pero no al revés

2. Balanceo: global y regional

El balanceador global es la pieza que más cambia el diseño frente a las otras nubes.

BALANCEADOR GLOBAL DE APLICACIÓN

UNA dirección, anunciada desde muchos puntos
el usuario llega al punto más cercano
y desde ahí se reparte a la región con capacidad

qué da
una sola entrada para todas las regiones
conmutación entre regiones sin tocar nombres
→ y por tanto SIN el retraso del TTL ni de las cachés de cliente

clase 215

terminación TLS y certificados gestionados
caché integrada en el borde
y filtrado de aplicación delante

qué implica
la salud de los grupos por región decide el reparto
→ y hay que configurarla bien, con las lecciones de la clase 196

BALANCEADOR REGIONAL

para protocolos que no son HTTP
para latencia mínima dentro de una región
y cuando hace falta una IP fija por región para listas de terceros

clase 193

BALANCEADOR INTERNO

para el tráfico entre servicios propios
regional o global según el caso

Y la consecuencia sobre continuidad, que es notable:

con balanceo global, perder una región NO exige cambiar un registro de nombres
→ el reparto deja de enviar tráfico a esa región
→ y el tramo de «redirección» de la clase 215 casi desaparece

→ lo que NO desaparece
promover los datos

clase 235

escalar la capacidad en la región que queda

clase 233

y decidir

← sigue siendo el tramo que domina

Y una advertencia:

la capacidad de los grupos por región hay que dimensionarla para absorber el tráfico de la región perdida
→ si cada región está al 80 %, perder una no funciona
→ es la misma aritmética de la clase 186, aplicada a regiones en vez de a zonas

Y el caché del borde:

el balanceador global integra caché
→ las lecciones de la clase 197 aplican igual: clave de caché, contenido rancio, invalidación por etiqueta
→ y el ahorro de salida y de origen también

3. Conectividad privada y nombres

Aquí hay dos mecanismos que se confunden y una trampa idéntica a la de la clase 219.

ACCESO PRIVADO A SERVICIOS

- las máquinas sin dirección externa alcanzan las API de la plataforma por rutas privilegiadas
- + se activa por subred, sin crear recursos
- + gratuito
- el tráfico sale por las direcciones públicas del servicio, no por una dirección de tu red
- y por tanto el control por origen es más grueso

CONEXIÓN DE SERVICIO PRIVADA

- crea una dirección DE TU RED que representa el servicio
- + alcanzable desde el centro de datos y desde otras redes
- + permite política por origen fina
- + y sirve también para servicios de TERCEROS y para publicar los propios a otros clientes
- cuesta, y consume direcciones clase 193

CRITERIO

- ¿hace falta alcanzarlo desde fuera de la red o controlar por origen? → conexión privada
- ¿solo desde dentro y sin dirección externa? → acceso privado a servicios

La trampa de los nombres, que es la misma de siempre:

- la conexión privada crea una dirección
- y hace falta que el NOMBRE del servicio resuelva a ella
- eso lo hace una ZONA PRIVADA, asociada a las redes

- X si la zona no está asociada a una red
- el nombre resuelve a la dirección PÚBLICA
- el tráfico sale a internet
- y si el acceso público está cerrado, falla; y si no, funciona pagando salida y sin control clase 219, ley 26

- y con red compartida, la asociación se hace en el proyecto anfitrión, no en el de servicio
- es el error de configuración más frecuente aquí

Y la comprobación obligatoria:

- para cada servicio con conexión privada, resolver su nombre desde CADA red y comprobar que devuelve dirección privada
- función de aptitud, diaria clase 190

La resolución con el centro de datos, con lo suyo:

- políticas de reenvío entrante y saliente
- entrante el centro de datos resuelve nombres de la nube
- saliente la nube resuelve nombres del centro de datos
- y hay que montar las DOS
- el fallo clásico es montar una y descubrir la otra el día que un sistema corporativo tiene que llamar clase 195

Y los controles de perímetro, que aquí tienen forma propia:

- un PERÍMETRO DE SERVICIO agrupa proyectos y restringe qué API pueden usarse desde fuera y hacia fuera
- impide que una identidad válida copie datos a un proyecto de otra organización
- es el equivalente del perímetro de datos de la clase 200, y es lo que cierra el hueco que deja la conexión privada

4. Salida, cortafuegos y diagnóstico

La salida está permitida por defecto, como en las otras dos nubes, y hay que cerrarla.

LAS REGLAS DE CORTAFUEGOS

- son de la red, con prioridad, y con reglas implícitas al final
- entrada: denegar todo
- SALIDA: permitir todo ley 26

selectores

por etiqueta de red ← cualquiera con permiso sobre la instancia puede ponerla
por CUENTA DE SERVICIO ← exige permiso sobre la cuenta
→ usar cuenta de servicio siempre que se pueda
clase 229

CERRAR LA SALIDA

regla de denegación de salida con prioridad alta
y permisos explícitos hacia lo declarado
con las API de la plataforma alcanzadas por acceso
privado o conexión privada

Y EL PERÍMETRO DE SERVICIO

impide sacar datos a proyectos ajenos aunque la red lo permitiera
clase 200

Y los mecanismos de salida gestionada:

LA PASARELA DE TRADUCCIÓN DE DIRECCIONES

da salida a máquinas sin dirección externa
con direcciones fijas para listas de terceros
y hay que dimensionar los puertos por instancia
→ el agotamiento de puertos aparece igual que en la
clase 221

EL CORTAFUEGOS GESTIONADO

inspección de capa 7 y control por nombre de destino
→ lo que hace falta para la lista de destinos permitidos
clase 200

El diagnóstico, con las herramientas propias:

COMPROBADOR DE CONECTIVIDAD

dice, para un origen y un destino, si el tráfico llegaría
y qué regla o ruta lo permite o lo bloquea
→ responde en segundos lo que una captura tarda horas
clase 202

REGISTROS DE FLUJO

quién habla con quién, y qué se deniega
→ con muestreo configurable: al 100 % en lo crítico

ESPEJO DE PAQUETES

copia del tráfico hacia un destino de análisis
→ caro y con datos sensibles; con procedimiento

Y LOS REGISTROS DE REGLAS DE CORTAFUEGOS

desactivados por defecto, por regla ley 26
→ activarlos en las reglas de denegación es lo que
revela lo que se está bloqueando

Y la lista de comprobación de la clase:

- la red es personalizada, con rangos del plan
- los rangos secundarios están dimensionados
- no se han creado redes por región innecesariamente
- las reglas seleccionan por cuenta de servicio
- la salida está cerrada explícitamente
- hay perímetro de servicio para los proyectos con datos
- el balanceador global se usa donde aporta, y el regional donde hace falta IP fija u otro protocolo
- la capacidad por región absorbe la pérdida de una
- las zonas privadas están asociadas a todas las redes, en el proyecto anfitrión
- hay comprobación diaria de resolución a dirección privada
- el reenvío con el centro de datos está en los dos sentidos
- los registros de flujo están al 100 % en lo crítico
- los registros de las reglas de denegación están activados

Y el cierre que enlaza con la clase siguiente: con jerarquía, identidad y red resueltas, hace falta declararlo como código con validación previa de las políticas, para que el rechazo no llegue en el momento de desplegar. Es la materia de la clase 232.

Ejemplo trabajado

CloudShop monta la red de su organización en Google Cloud. Lo que sigue es lo que ahorró la red global, el problema de zonas privadas que se repitió pese a conocerlo, y la conmutación de región que resultó ser mucho más rápida que en las otras dos nubes.

Lo que ahorró la red global:

el diseño trasladado de las otras nubes preveía	
3 redes (una por región)	
emparejamientos entre ellas	3
o un concentrador de tránsito	1
reglas duplicadas por región	
el diseño real	
2 redes compartidas (producción y no producción)	
subredes regionales dentro de cada una	
emparejamientos	0
concentradores	0
reglas de cortafuegos, únicas para las 3 regiones	
ahorro estimado frente al diseño trasladado	
coste de concentrador y tránsito	1.900 €/mes
reglas que mantener	3× menos
y la complejidad de encaminamiento entre regiones, eliminada	

Y el hallazgo que no se esperaba:

- las subredes se pueden AMPLIAR sin recrear
 - en las otras dos nubes, ampliar exigía crear otra subred o reenumerar clase 193
 - aquí, dos subredes que se estaban agotando se ampliaron de /22 a /20 en minutos, sin cortar nada
 - y eso cambió la estrategia de dimensionado: se puede empezar ajustado y ampliar

El problema de las zonas privadas, otra vez.

se montaron conexiones privadas a 19 servicios
con zonas privadas para que los nombres resolvieran a dirección privada

y al comprobar, dos semanas después
proyectos de servicio desde los que resolvía correcto 4
proyectos desde los que resolvía a dirección PÚBLICA 38

causa
con red compartida, la zona privada debe asociarse a la red del PROYECTO ANFITRIÓN
el equipo la había asociado en cada proyecto de servicio, donde no tiene efecto
→ y funcionaba en los 4 proyectos donde alguien la había asociado bien por casualidad

lo incómodo
el equipo CONOCÍA el problema: lo habían sufrido en la clase 219, con Azure
→ y aun así lo repitieron, porque el sitio donde hay que asociarla es distinto
→ es exactamente el error de traslado de la clase 229

corrección
zonas asociadas a las redes de los proyectos anfitriones
comprobación diaria: resolver los 19 nombres desde una máquina de cada proyecto de servicio
→ 4 fallos en los 6 meses siguientes, todos en proyectos creados antes de que la automatización existiera
ley 27

El balanceo global, y lo que cambió en continuidad.

montaje
1 balanceador global de aplicación

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Se crean redes por región y emparejamientos entre ellas	Se trasladó el modelo regional de otras nubes	La red es global: una red por entorno con subredes regionales, sin emparejamientos internos.
El nombre de un servicio con conexión privada resuelve a dirección pública	Con red compartida, la zona privada debe asociarse a la red del proyecto anfitrión	Asocia en el anfitrión y comprueba a diario la resolución desde una máquina de cada proyecto de servicio.
Al perder una región el servicio se degrada mientras escala	Cada región estaba dimensionada solo para su propio tráfico	Dimensiona cada región para absorber el total con utilización por debajo del codo.
Una regla de cortafuegos se puede burlar poniendo una etiqueta	El selector es por etiqueta de red	Selecciona por cuenta de servicio, que exige permiso sobre esa cuenta.
Con la salida cerrada aún se pueden copiar datos fuera	La API de la plataforma se alcanza por rutas privilegiadas y llega a proyectos ajenos	Añade un perímetro de servicio sobre los proyectos con datos.
Diagnosticar por qué no llega el tráfico lleva horas	Se revisan reglas a mano en vez de usar la comprobación de conectividad	Usa el comprobador de conectividad primero y activa los registros de las reglas de denegación.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué desaparece y qué no al ser la red global?
2. ¿Qué tramo de la conmutación de región elimina el balanceo global?
3. ¿Dónde hay que asociar una zona privada cuando se usa red compartida?
4. ¿Qué diferencia hay entre el acceso privado a servicios y la conexión de servicio privada?
5. ¿Qué cierra el perímetro de servicio que no cierra el cortafuegos?

Referencias

- Google Cloud (2025). VPC network overview. <<https://cloud.google.com/vpc/docs/vpc>>
- Google Cloud (2025). Cloud Load Balancing: global external Application Load Balancer. <<https://cloud.google.com/load-balancing/docs/https>>
- Google Cloud (2025). Private Service Connect. <<https://cloud.google.com/vpc/docs/private-service-connect>>
- Google Cloud (2025). Cloud DNS private zones and forwarding. <<https://cloud.google.com/dns/docs/zones>>

- Google Cloud (2025). Connectivity Tests.
<<https://cloud.google.com/network-intelligence-center/docs/connectivity-tests/concepts/overview>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 19 en PDF · Recorrido de Google Cloud en PDF · Manual integral

Anterior	Índice	Siguiente
← 230 · Workload Identity Federation, IAM Conditions y PAM	Parte 19 · Programa	232 · Terraform, Infrastructure Manager y policy validation →

Evaluación — 231 Red global, load balancing, PSC y Cloud DNS

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega gcp-network con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

232 — Terraform, Infrastructure Manager y policy validation

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: iac · Estado: EXECUTABLECORE

Propósito

Declarar la infraestructura de Google Cloud como código con la pieza que faltaba en las partes anteriores: validar las políticas antes de desplegar, para que el rechazo no llegue en el momento de aplicar. La clase cubre la gestión del estado y sus riesgos, la estructura por ciclo de vida, la validación previa de restricciones, y la operación que resuelve el problema de la ley 27: qué hacer con lo que ya existe y no cumple.

Resultados de aprendizaje

Al finalizar podrás:

1. Gestionar el estado de forma segura y con bloqueo.
2. Estructurar el código por ciclo de vida y por ámbito.
3. Validar políticas y coste antes de aplicar, no después.
4. Importar lo existente al código sin recrearlo.
5. Retirar recursos de forma controlada y comprobada.

Conceptos centrales

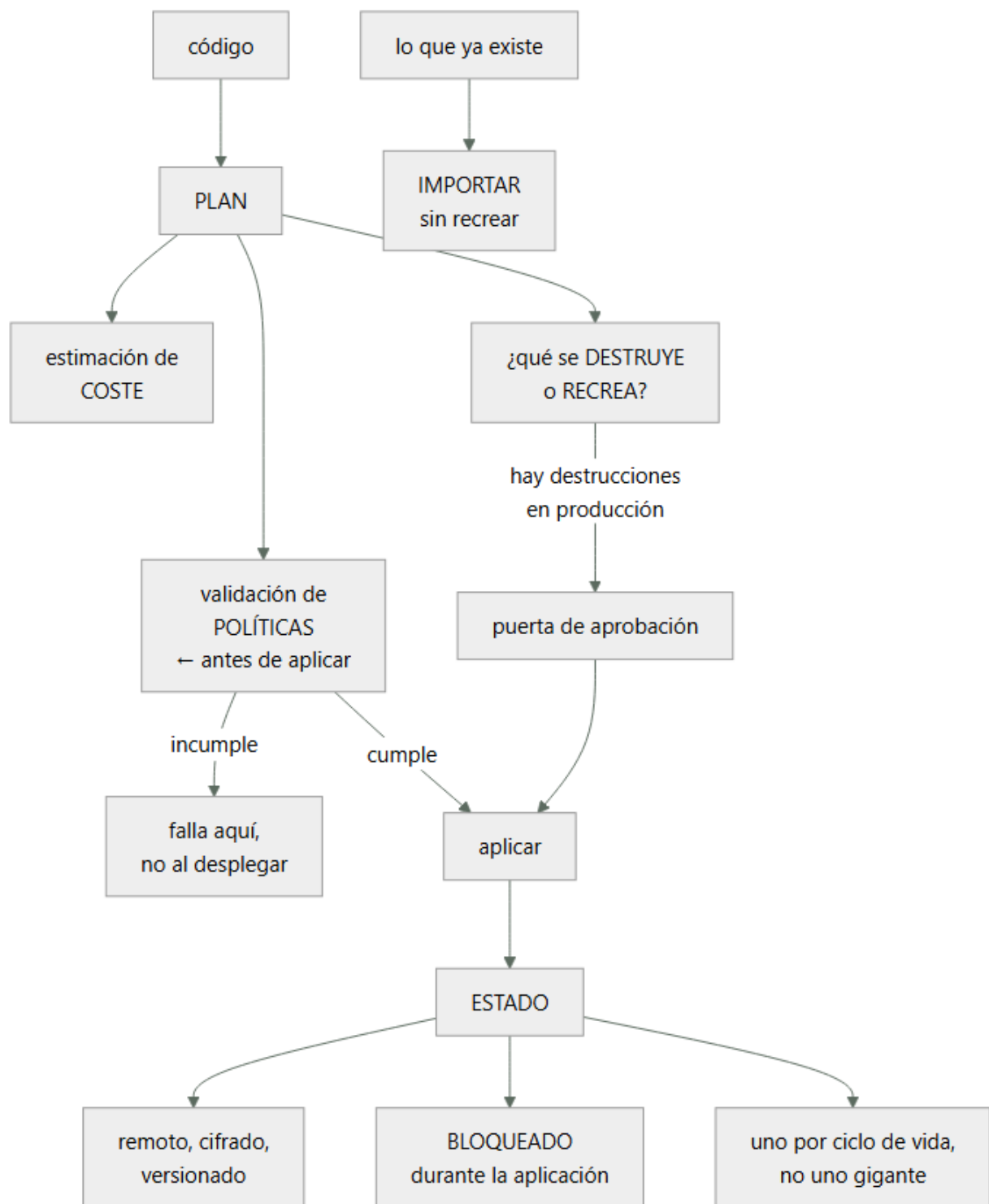
Concepto	Comprensión verificable
estado	Registro de qué recursos gestiona el código. Contiene información sensible y debe protegerse y bloquearse.
bloqueo del estado	Mecanismo que impide dos aplicaciones simultáneas. Sin él, el estado se corrompe.
plan de cambios	Previsualización de lo que se creará, modificará, recreará y destruirá. Se lee antes de aplicar.
validación de políticas	Comprobación del plan contra las restricciones de la organización, antes de aplicar.
importación	Incorporación de un recurso existente al estado, sin recrearlo.
servicio gestionado de despliegue	Alternativa que ejecuta el despliegue en la nube, con el estado gestionado por la plataforma.

Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. El estado, y cómo no perderlo

La diferencia principal frente a Bicep es que aquí hay un estado, y eso da capacidades y riesgos.

QUÉ DA EL ESTADO

saber exactamente qué gestiona el código
 detectar deriva: lo que cambió por fuera
 y destruir lo que deja de estar declarado
 → resuelve el problema de los huérfanos de la clase 220

QUÉ RIESGOS TRAE

contiene valores sensibles en claro

- contraseñas generadas, claves, cadenas de conexión
- un estado en un repositorio es una fuga
- se corrompe si dos personas aplican a la vez
- y si se pierde, el código deja de saber qué gestiona

Y la configuración mínima:

ESTADO REMOTO

- en un almacén de objetos, no en el portátil
- con VERSIONADO activado
- con cifrado
- con acceso restringido: quien lee el estado ve secretos
- y con registro de acceso clase 238

BLOQUEO

- automático al aplicar
- sin él, dos canalizaciones simultáneas corrompen el estado
- y un procedimiento para liberar un bloqueo atascado
- que hay que tener escrito ANTES de necesitarlo

UN ESTADO POR CICLO DE VIDA

- no uno gigante
- un estado enorme hace lentas todas las operaciones
- y un error afecta a todo

Y el error que más daño hace:

BORRAR O PERDER EL ESTADO

- el código deja de saber qué existe
- al aplicar, intenta CREAR lo que ya está
- y falla con errores de recurso duplicado, o peor: crea duplicados donde el nombre no es único

la recuperación

- restaurar la versión anterior del estado
- y por eso el versionado del almacén es obligatorio
- o importar todo de nuevo, que es lento y manual

Y la alternativa gestionada, que conviene conocer:

- el servicio gestionado de despliegue ejecuta el código en la nube y guarda el estado por ti
- + sin almacén que proteger, sin bloqueo que gestionar
- + con identidad de la plataforma, sin credenciales
- menos control sobre el proceso y sobre las versiones

- para equipos pequeños o que empiezan, suele ser mejor
- y para canalizaciones complejas, el modelo propio

2. Estructura y ámbitos

La estructura es la de siempre, con los ámbitos de esta nube.

SEPARADO POR CICLO DE VIDA Y POR ÁMBITO

- | | |
|---------------|---|
| organizacion/ | políticas, carpetas, permisos de organización |
| red/ | red compartida, subredes, cortafuegos, zonas privadas |
| proyectos/ | creación y configuración base |
| datos/ | bases, conjuntos, almacenes |
| | → con protección contra destrucción |
| cargas/ | servicios, trabajos, colas |

- cada uno con su estado
- y las dependencias entre ellos por salidas y fuentes de datos, no por variables copiadas

Y las reglas que evitan los problemas frecuentes:

PROTECCIÓN CONTRA DESTRUCCIÓN

- los recursos con datos llevan la marca que impide destruirlos
- y entonces un plan que los destruiría FALLA en el plan, no en el aplicar
- es la protección más barata que existe

SIN SECRETOS EN VARIABLES

- referencias al gestor de secretos
- y aun así, el valor acaba en el estado: por eso el estado se protege como un secreto

VERSIONES FIJADAS

- la del proveedor y las de los módulos
- sin fijar, el mismo código produce cosas distintas en momentos distintos ley 25

MÓDULOS DEL CATÁLOGO OFICIAL

- con los valores por defecto revisados
- misma idea que en la clase 220, contra la ley 26

Y los ámbitos de despliegue, que aquí importan:

lo que se crea en la ORGANIZACIÓN

- carpetas, políticas de organización, permisos de organización
- exige permisos muy altos
- y por eso ese estado y esa canalización van aparte, con aprobación clase 229

lo que se crea en el PROYECTO

- casi todo lo demás
- con la identidad federada del proyecto

Y una advertencia de traslado:

- ✗ «una carpeta de módulos por entorno»
produce código duplicado que diverge
- ✓ un módulo, y ficheros de variables por entorno
- y la diferencia entre entornos, visible en un solo sitio

3. Validar antes de aplicar

Esta es la pieza que las partes 17 y 18 echaban de menos: comprobar que el cambio cumplirá las políticas antes de intentarlo.

EL PROBLEMA SIN VALIDACIÓN PREVIA

- el despliegue llega a producción y la política lo rechaza
- el cambio queda a medias
- y el equipo descubre la restricción en el peor momento

LA VALIDACIÓN PREVIA

- se genera el PLAN
- se convierten los recursos previstos a su forma final
- se evalúan contra las restricciones de la organización
- y la canalización FALLA ahí, con un mensaje que dice qué restricción y qué falta

Y lo que se puede validar:

- las restricciones de organización clase 229
- las políticas propias de la empresa
- «ningún almacén sin cifrado con clave propia»
- «ninguna base sin copia de seguridad»
- «ningún recurso sin etiquetas obligatorias»
- y las convenciones
- nombres, regiones, tamaños permitidos

Y las otras dos puertas que deben estar en la misma canalización:

ESTIMACIÓN DE COSTE

- a partir del plan, cuánto costará al mes
- publicada en la revisión clase 220
- y con umbral: por encima de X, aprobación

DESTRUCCIONES Y RECREACIONES

- el plan las lista
- y una recreación destruye y crea: para una base, es pérdida de datos
- puerta automática: si hay destrucciones en producción, aprobación explícita clase 220

Y el orden completo de la canalización:

- 1 formato y análisis estático
- 2 validación de sintaxis
- 3 PLAN, con identidad federada y solo lectura
- 4 validación de políticas sobre el plan
- 5 estimación de coste
- 6 publicación del resumen: qué se crea, cambia, RECREA y destruye
- 7 puerta si hay destrucciones o si el coste supera el umbral
- 8 aplicar, con identidad de despliegue
- 9 comprobar el resultado

Y una práctica que ahorra incidentes:

el PLAN se ejecuta con una identidad de SOLO LECTURA
→ y la de escritura solo se usa en el paso de aplicar
→ así una propuesta de cambio de cualquiera puede generar plan sin poder cambiar nada clase 230

4. Importar, detectar deriva y retirar

La importación es lo que permite poner bajo código lo que ya existe, que es el problema de la ley 27.

EL ESCENARIO HABITUAL

hay cientos de recursos creados a mano
el código nuevo describe lo que debería haber
→ aplicarlo intentaría crear lo que ya existe

LA IMPORTACIÓN

incorpora el recurso existente al estado
→ y a partir de ahí, el código lo gestiona
→ sin recrearlo ni cortar nada

y el trabajo real

hay que escribir el código que COINCIDA con lo que existe
→ y el plan posterior dice qué diferencias hay
→ esas diferencias son el inventario de lo que estaba mal

Y el orden que funciona:

- 1 generar el código a partir de lo existente, donde la herramienta lo permita
- 2 importar
- 3 ejecutar el plan: si sale vacío, el código describe la realidad
- 4 y si no, cada diferencia es una decisión: ¿se ajusta el código o se corrige el recurso?
- 5 corregir hacia el estado deseado, por lotes

La deriva, que es lo que ocurre después:

alguien cambia algo por la consola durante un incidente
→ el código deja de describir la realidad
→ y el siguiente despliegue lo revierte sin avisar

LA DETECCIÓN

ejecutar el plan periódicamente y alertar si no está vacío
→ un plan que no está vacío sin que nadie haya cambiado el código significa deriva
→ y esa alerta es la que detecta los cambios manuales ley 25, clase 219

La retirada, que es lo que el estado permite y casi nadie usa:

quitar el recurso del código y aplicar → se destruye
→ y por eso la protección contra destrucción en los datos

y para lo que hay que sacar del código sin borrarlo
se quita del estado, y el recurso queda huérfano
→ útil cuando pasa a gestionarlo otro equipo
→ y peligroso si se olvida: queda sin gobierno ley 20

Y las comprobaciones de esta clase:

- borrar el estado y comprobar que se puede restaurar
 - aplicar dos veces en paralelo y ver que el bloqueo lo impide
 - intentar destruir un recurso protegido
 - desplegar algo que incumple una política y ver que falla en la validación, no al aplicar
 - ejecutar el plan sin cambios y comprobar que está vacío
 - desplegar el entorno de cero en un proyecto vacío
- clase 220

Y la lista de comprobación de la clase:

- el estado es remoto, cifrado, versionado y con acceso restringido
- el bloqueo está activo y hay procedimiento para liberarlo
- hay un estado por ciclo de vida, no uno gigante
- los recursos con datos tienen protección contra destrucción
- las versiones del proveedor y de los módulos están fijadas
- se usan módulos del catálogo oficial donde existen
- la canalización valida políticas sobre el plan
- estima el coste y lo publica
- lista destrucciones y recreaciones, con puerta de aprobación
- el plan usa identidad de solo lectura
- lo existente está importado, y el plan sale vacío
- hay detección periódica de deriva, con alerta
- el entorno se despliega de cero periódicamente

Y el cierre que enlaza con la clase siguiente: con la base declarada y validada, quedan las cargas. Las opciones de cómputo gestionado de esta nube —y la que ha cambiado más el modelo de contenedores— son la materia de la clase 233.

Ejemplo trabajado

CloudShop pone su infraestructura de Google Cloud bajo código. Lo que sigue es la importación de 4.100 recursos creados a mano, el estado que se perdió una vez, y la validación previa que evitó 61 despliegues fallidos en producción.

El punto de partida:

recursos existentes	9.400	
declarados en algún código	410	4 %
creados a mano	8.990	
estados existentes	11	
en un almacén sin versionado	7	
EN EL REPOSITORIO	2	←
en portátiles	2	

y los 2 del repositorio contenían

- 3 contraseñas de bases de datos en claro
- 1 clave de cuenta de servicio
- visibles para los 214 miembros de la organización
- desde hacía 14 meses

ley 15

Y la corrección inmediata:

estados retirados del repositorio y del historial
 contraseñas y clave rotadas
 almacén de estados: versionado, cifrado con clave propia,
 acceso restringido a 4 identidades, registro de acceso
 activado

clase 238

→ y una función de aptitud: ningún fichero de estado en
 ningún repositorio

clase 190

La importación, por lotes.

orden elegido	
1 red y cortafuegos	(más estable, menos riesgo)
2 proyectos y permisos	
3 datos	(con protección primero)
4 cargas	(las que más cambian)

el proceso por lote
generar el código desde lo existente
importar
ejecutar el plan
→ y aquí aparecía el trabajo real

lote de red: 210 recursos
plan tras importar: 118 diferencias
· 41 reglas de cortafuegos con descripción vacía
→ se ajustó el código
· 38 subredes sin registros de flujo activados
→ se corrigió el RECURSO
· 9 reglas con selector por etiqueta clase 229
→ se corrigieron
· 30 diferencias de campos irrelevantes
→ se ajustó el código

→ cada diferencia era una decisión, y 47 de las 118
revelaron configuraciones que estaban mal

Y el resultado global:

recursos importados	4.100
→ los otros 4.890 eran efímeros o de servicios gestionados que no se declaran	
diferencias encontradas al importar	1.240
ajustes al código	810
CORRECCIONES DE RECURSOS	430 ←

→ importar fue, además, una auditoría de configuración
→ y las 430 correcciones son la remediación que la ley 27
exige clase 228

El estado que se perdió.

qué pasó
una canalización mal configurada aplicó con la variable
de ruta del estado vacía
→ creó un estado nuevo, vacío, en la ruta por defecto
→ y el siguiente despliegue, con el estado bueno, entró
en conflicto

y el estado bueno se sobrescribió

qué salvó el caso
el almacén tenía versionado activado
→ se restauró la versión anterior en 4 minutos

qué habría pasado sin versionado
el código no habría sabido qué existía
→ habría intentado crear 4.100 recursos que ya estaban
→ y en los que el nombre no es único, habría creado
duplicados

correcciones
la ruta del estado, obligatoria y comprobada en la
canalización
copia diaria del estado a otro almacén
y una prueba: borrar el estado en un entorno inferior y
restaurarlo ley 22

La validación previa de políticas, que era lo que faltaba.

antes de montarla, en 3 meses	
despliegues rechazados por política EN EL MOMENTO DE	
APLICAR	61
de ellos, en producción	19
de ellos, que dejaron el cambio a medias	11
tiempo medio de resolución	2 h 20

tras montarla
la canalización convierte el plan a la forma final y lo
evalúa contra las restricciones
→ falla en la validación, con el mensaje

despliegues rechazados en el momento de aplicar
61 → 2
los 2, por restricciones que la validación no cubre
rechazados en la validación 74
→ antes de tocar nada, en 40 segundos
tiempo medio de resolución 2 h 20 → 9 min

Y las políticas propias que se añadieron a la validación:

ningún almacén sin cifrado con clave propia
ninguna base sin copia de seguridad configurada
ningún recurso sin las 5 etiquetas obligatorias
ninguna regla de cortafuegos con selector por etiqueta
ninguna cuenta de servicio con papel de proyecto
ningún recurso fuera de las regiones de la UE

→ y cada una con un mensaje que dice qué falta y enlaza el
módulo que ya lo cumple clase 217

La estimación de coste, que evitó dos sorpresas:

caso 1 una propuesta de cambio añadía un clúster con
nodos de 32 núcleos
estimación +8.400 €/mes
→ la revisión preguntó por qué; resultó ser un
valor copiado de otro entorno
→ corregido a nodos de 8: +1.100 €/mes

caso 2 una propuesta activaba registros de acceso a
datos en todos los servicios
estimación +3.900 €/mes
clase 238
→ se acotó a los servicios con datos sensibles:
+410 €/mes

La deriva, detectada:

plan ejecutado a diario, sin cambios de código
→ si no sale vacío, hay deriva

primeros 6 meses
detecciones 23
cambios hechos por consola durante incidentes 14
→ 9 de ellos deberían haberse revertido y no se
revirtieron ley 25
cambios de servicios gestionados que ajustan campos 6
→ se marcaron para ignorar
3 cambios que nadie pudo explicar
→ investigados; 2 eran automatismos de la
plataforma, 1 fue un script de un proveedor

→ y las 9 del primer grupo son exactamente lo que la deriva
sirve para encontrar

Las pruebas negativas, ejecutadas:

✓ borrar el estado y restaurarlo 4 min
✓ aplicar dos veces en paralelo bloqueado
✓ destruir un recurso protegido falla en el
plan
✓ desplegar algo que incumple una política falla en
validación

✗ plan sin cambios, vacío
→ 6 recursos con deriva permanente por campos que la
plataforma ajusta sola
→ marcados para ignorar; corregido

✓ desplegar el entorno de cero en proyecto vacío 61 min

El resultado:

recursos bajo código	4 %	97 %
estados en repositorios	2	0
secretos expuestos en estados	4	0
despliegues rechazados al aplicar	61/trim	2/trim
tiempo de resolución de un rechazo	2 h 20	9 min
configuraciones corregidas al importar	—	430

deriva detectada y corregida	no	23/6mes
sorpresas de coste en despliegues	2/trim	0
tiempo de despliegue de cero	imposible	61 min

La lección que esta clase deja: importar cuatro mil cien recursos fue además una auditoría, y produjo cuatrocientas treinta correcciones de configuración que ninguna herramienta de postura había señalado. La validación previa de políticas convirtió sesenta y un rechazos en producción en setenta y cuatro fallos de canalización de cuarenta segundos. Y el estado estuvo catorce meses en un repositorio con tres contraseñas de producción en claro, visible para doscientos catorce personas.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/232-terraform-infrastructure-manager-y-policy-validation/lab.py
```

El laboratorio selecciona el motor de práctica iac y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa gcp-iac-stack en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un plan reproducible sin secretos ni cambios inesperados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye gcp-iac-stack para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El estado contiene secretos visibles para mucha gente	Está en un repositorio o en un almacén con acceso amplio	Estado remoto, cifrado, versionado y restringido a unas pocas identidades, con registro de acceso, y una función de aptitud que prohíba ficheros de estado en repositorios.
Se pierde el estado y el código intenta crear lo que ya existe	El almacén no tiene versionado o la ruta del estado no está fijada	Versionado obligatorio, copia diaria, ruta comprobada en la canalización y prueba de restauración.
Un despliegue se rechaza por política a mitad de aplicar	No se valida el plan contra las restricciones antes de aplicar	Convierte el plan a su forma final y evalúalo contra las políticas en la canalización, con mensajes que digan qué falta.
Una recreación destruye datos	El plan no se leyó y el recurso no estaba protegido	Marca los recursos con datos contra destrucción y pon puerta de aprobación cuando el plan liste destrucciones.
Un cambio hecho en la consola desaparece en el siguiente despliegue	Deriva no detectada entre el código y la realidad	Ejecuta el plan periódicamente sin cambios de código y alerta si no sale vacío.
Aparecen sorpresas de coste tras un despliegue	No se estima el coste del plan en la revisión	Añade estimación de coste a la canalización, con umbral que exija aprobación.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué capacidades da el estado y qué riesgos trae?
2. ¿Qué ocurre si se pierde el estado y cómo se recupera?
3. ¿Qué se gana validando las políticas sobre el plan en vez de al aplicar?
4. ¿Qué revela la importación de recursos existentes además de ponerlos bajo código?
5. ¿Cómo se detecta que alguien cambió algo por la consola?

Referencias

- HashiCorp (2025). Terraform state: remote backends and locking. <<https://developer.hashicorp.com/terraform/language/state>>
- Google Cloud (2025). Infrastructure Manager. <<https://cloud.google.com/infrastructure-manager/docs/overview>>
- Google Cloud (2025). Policy validation with Policy Library. <<https://cloud.google.com/docs/terraform/policy-validation>>
- Google Cloud (2025). Cloud Foundation Toolkit modules. <<https://cloud.google.com/foundation-toolkit>>
- HashiCorp (2025). Importing existing infrastructure. <<https://developer.hashicorp.com/terraform/cli/import>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 19 en PDF · Recorrido de Google Cloud en PDF · Manual integral

Anterior	Índice	Siguiente
← 231 · Red global, load balancing, PSC y Cloud DNS	Parte 19 · Programa	233 · Cloud Run, Functions, API Gateway y Workflows →

Evaluación — 232 Terraform, Infrastructure Manager y policy validation

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega gcp-iac-stack con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

233 — Cloud Run, Functions, API Gateway y Workflows

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: serverless · Estado: EXECUTABLECORE

Propósito

Desplegar cargas sin gestionar servidores en Google Cloud, donde el modelo de contenedores gestionados es el que más ha cambiado la forma de trabajar: se despliega una imagen, escala a cero y se reparte tráfico por revisión, sin orquestador. La clase cubre la configuración que separa un servicio de demostración de uno de producción —conurrencia, instancias mínimas, tiempos de espera y conexiones—, las funciones y la orquestación de pasos largos.

Resultados de aprendizaje

Al finalizar podrás:

1. Elegir entre servicio de contenedores, funciones y flujos de trabajo.
2. Configurar concurrencia e instancias con criterio, no por defecto.
3. Conectar las cargas a la red privada, de entrada y de salida.
4. Desplegar con reparto por revisión y vuelta atrás inmediata.
5. Orquestar procesos de varios pasos sin escribir la coordinación.

Conceptos centrales

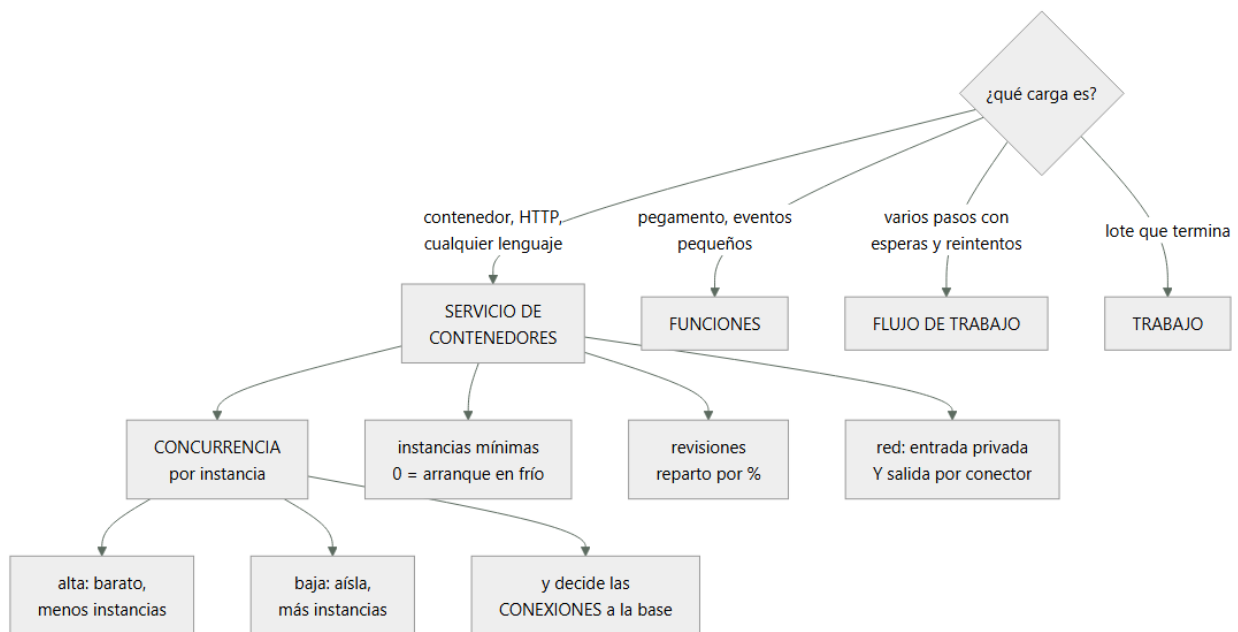
Concepto	Comprensión verificable
servicio de contenedores gestionado	Ejecuta una imagen con escalado automático, escala a cero y reparto de tráfico por revisión.
conurrencia por instancia	Cuántas peticiones atiende una instancia a la vez. Es el ajuste que más cambia coste y latencia.
instancia mínima	Número de instancias siempre encendidas. Elimina el arranque en frío y se paga.
revisión	Versión inmutable del servicio. El tráfico se reparte entre revisiones por porcentaje.
conector de red	Mecanismo que hace que el tráfico saliente del servicio pase por la red privada.
flujo de trabajo	Orquestación declarativa de pasos con reintentos, esperas y compensaciones, sin código de coordinación.

Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Elegir la forma de ejecutar

Hay cuatro opciones y se diferencian por lo que ejecutan y cuánto duran.

SERVICIO DE CONTENEDORES

- una imagen que atiende peticiones
- + cualquier lenguaje y cualquier dependencia
- + escala a cero y escala por peticiones
- + reparto de tráfico por revisión, nativo
- + sin orquestador que operar
- arranque en frío si escala a cero

FUNCIONES

- código sin empaquetar, disparado por eventos o HTTP
- + más simple para pegamento
- menos control del entorno
- y en la práctica se ejecutan sobre la misma base que los servicios: la diferencia es de empaquetado

TRABAJO

- un contenedor que se ejecuta, termina y no atiende peticiones
- + para lotes, migraciones y procesos programados
- + con paralelismo por tareas

FLUJO DE TRABAJO

- orquestación declarativa de pasos
- + reintentos, esperas largas, ramas y compensaciones sin escribir código de coordinación
- no es para lógica compleja: para coordinar

Y el criterio:

¿atiende peticiones HTTP?	servicio
¿reacciona a un evento y es pequeño?	función
¿empieza, hace y termina?	trabajo
¿coordina varios pasos con esperas y reintentos?	flujo de trabajo
¿necesita operadores, GPU o planificación fina?	clúster
	clase 234

Y la diferencia principal frente a lo visto en las otras nubes:

aquí el servicio de contenedores cubre lo que en otras nubes se reparte entre varios productos

- aplicaciones web, APIs, trabajadores y funciones
 - con un solo modelo de despliegue y de escalado
 - y eso reduce mucho lo que hay que aprender y operar
- ley 23

Y una advertencia de traslado:

- X «escala a cero, así que es lo más barato»
- con tráfico continuo, un servicio con instancias mínimas puede salir más barato que uno que arranca y para
- y con concurrencia alta, mucho más ← ver abajo

2. Concurrencia: el ajuste que más cambia

Este es el parámetro que distingue este modelo y el que más se configura mal.

CONCURRENCIA POR INSTANCIA

cuántas peticiones atiende UNA instancia a la vez
valor por defecto: alto (80 en muchos casos)

ALTA

- + menos instancias para el mismo tráfico
- + mucho más barato
- + menos arranques en frío
- una petición pesada afecta a las otras 79
- y la memoria se comparte entre todas

BAJA (1)

- + aislamiento total: como una función
- + útil si el código no es seguro para concurrencia
- una instancia por petición: caro y con muchos arranques

Y el cálculo, que es el de la clase 186:

$\text{instancias} = \text{caudal} \times \text{latencia} / \text{concurrencia}$

$1.000 \text{ pet/s} \times 0,08 \text{ s} / 80 = 1 \text{ instancia}$
 $1.000 \text{ pet/s} \times 0,08 \text{ s} / 1 = 80 \text{ instancias}$

- el mismo tráfico, con 80 veces más instancias
- y por tanto, con 80 veces más conexiones a la base

Y de ahí la consecuencia que rompe sistemas:

LAS CONEXIONES A LA BASE

cada instancia abre su agrupación
instancias máximas $\times$ tamaño de agrupación = conexiones

con concurrencia 1 y máximo 100 instancias

$100 \times 5 = 500 \text{ conexiones}$

y la base admite 200

→ agotamiento, igual que en la clase 207

con concurrencia 80 y el mismo tráfico

$2 \text{ instancias} \times 5 = 10 \text{ conexiones}$

- subir la concurrencia es también la forma de reducir conexiones
- y donde no se pueda, hace falta un intermediario

Y cómo se elige el valor:

¿el código es seguro para atender varias peticiones a la vez?

no → concurrencia 1, y arreglarlo

sí → medir

y se mide

con la carga real, subiendo la concurrencia hasta que la latencia empeora

→ el punto donde el p99 empieza a subir es el codo

clase 186

→ y se elige algo por debajo

Las instancias mínimas, con la decisión de coste:

MÍNIMO 0

- + no se paga cuando no hay tráfico
- arranque en frío en la primera petición
- aceptable para lo asíncrono y lo interno

MÍNIMO ≥ 1

- + sin arranque en frío
- se paga siempre
- obligatorio en el camino crítico

Y EL TAMAÑO DE LA IMAGEN decide cuánto dura el arranque

- imagen mínima, dependencias justas clase 212
- y el trabajo de inicialización, fuera de la petición

3. Red, despliegue y tiempos

La red, con la misma separación de la clase 221:

ENTRADA

- pública, interna o solo desde el balanceador
- «interna» significa alcanzable desde la red, no desde internet
- y la autorización sigue siendo necesaria: la red no autoriza clase 189

SALIDA

- por defecto, el tráfico saliente NO pasa por tu red
- un conector o la salida directa a la red hacen que sí
- y solo entonces se alcanza lo privado y se pasa por el cortafuegos clases 221, 231

y hay que decidir

- ¿todo el tráfico saliente por la red, o solo el privado?
- para control de salida, todo clase 200

Y el error de traslado que se repite:

- X «tiene entrada interna, ya está en la red»
- la entrada y la salida se configuran por separado
- y sin conector, el servicio no alcanza la base privada

El despliegue por revisiones, que aquí es nativo y muy bueno:

cada despliegue crea una REVISIÓN inmutable
el tráfico se reparte por porcentaje entre revisiones

- desplegar sin tráfico 0 %
- comprobar con una etiqueta que da una URL propia
- desviar 5 %, 25 %, 100 %
- y volver atrás cambiando el porcentaje segundos

→ es el despliegue escalonado de la clase 102, sin montar nada

→ y la vuelta atrás no redespliega: cambia un número

Y lo que hay que cuidar:

LAS MIGRACIONES DE ESQUEMA

- dos revisiones sirviendo a la vez significa dos versiones del código contra la misma base
- expandir y contraer, siempre clase 188

LAS SESIONES

- las instancias son efímeras y no hay afinidad garantizada
- el estado va fuera clase 149

Los tiempos de espera y el ciclo de vida:

TIEMPO MÁXIMO DE PETICIÓN

- configurable, con un máximo alto
- y el mismo error de la clase 207: poner el máximo «por si acaso» significa pagar peticiones colgadas
- múltiplo pequeño del p99 ley 26

SEÑAL DE TERMINACIÓN

- la instancia recibe aviso antes de pararse

- hay que atenderla: terminar lo que se está haciendo
- si no, se cortan peticiones clase 212

TRABAJO EN SEGUNDO PLANO

- fuera de una petición, la CPU puede no estar asignada
- un hilo que sigue trabajando tras responder puede no avanzar
- o se activa la asignación continua de CPU, o el trabajo va a una cola clase 237

Y una advertencia sobre eso último, que sorprende:

- el patrón «responder rápido y seguir trabajando en segundo plano» NO funciona por defecto aquí
- y falla de forma intermitente, que es lo peor
- el trabajo diferido va a una cola, siempre

4. Flujos de trabajo y operación

Los flujos de trabajo resuelven la coordinación sin escribirla, y evitan uno de los antipatrones de la parte 09.

QUÉ RESUELVEN

- «llama a A; si falla, reintenta 3 veces con retroceso; luego espera a que llegue B; si tarda más de 2 horas, compensa A y avisa»
- escrito en un fichero declarativo
- con el estado de cada ejecución persistido

QUÉ EVITAN

- la coordinación escrita a mano en un servicio, que acumula estado y se convierte en un punto único
- y las esperas largas ocupando concurrencia

Y LOS LÍMITES

- no son para lógica de negocio compleja
- ni para volumen muy alto
- cada paso tiene coste, y con millones de ejecuciones suma

Y la disciplina que este programa exige y aquí también aplica:

- cada paso con efecto, **IDEMPOTENTE** clase 210
- el flujo reintenta, y un reintento no debe duplicar las compensaciones, explícitas y probadas
- y recordando que la compensación hace invisible el fallo ley 19
- y el estado del flujo, consultable
- «¿por qué este pedido lleva 3 horas?» debe tener respuesta sin abrir registros

Lo que hay que vigilar en estas cargas:

- peticiones, errores y latencia por percentil clase 238
- instancias activas frente al máximo
- tocar el máximo significa rechazo
- utilización de la concurrencia
- si está muy por debajo, se puede subir y ahorrar tiempo de arranque y peticiones que lo pagan
- conexiones a la base por instancia
- peticiones que agotan el tiempo de espera
- y en flujos: ejecuciones en curso, fallidas y su antigüedad ley 13

Y el coste, con las palancas propias:

- se factura por tiempo de instancia y por peticiones

las palancas, por efecto

- 1 SUBIR LA CONCURRENCIA ← la mayor, con diferencia
- 2 reducir la latencia (menos tiempo de instancia)
- 3 instancias mínimas solo donde hacen falta
- 4 memoria y CPU ajustadas, medidas
- 5 y la asignación continua de CPU solo si se usa

Y la lista de comprobación de la clase:

- la forma de ejecutar corresponde a la carga

- la concurrencia se eligió midiendo, no por defecto
- el código es seguro para concurrencia, comprobado
- las conexiones máximas a la base están calculadas
- las instancias mínimas cubren el camino crítico
- la imagen es mínima y el arranque está medido
- la entrada está restringida y la salida pasa por la red
- el despliegue usa revisiones con reparto por porcentaje
- los cambios de esquema usan expandir y contraer
- el tiempo de espera es múltiplo pequeño del p99
- el servicio atiende la señal de terminación
- no hay trabajo en segundo plano fuera de la petición
- los pasos de los flujos son idempotentes
- hay alerta de ejecuciones de flujo antiguas o fallidas

Y el cierre que enlaza con la clase siguiente: cuando la carga exige operadores, control fino o cargas heterogéneas, aparece el clúster gestionado, que aquí tiene un modo que quita casi toda la operación. Es la materia de la clase 234.

Ejemplo trabajado

CloudShop despliega su plataforma en Google Cloud. Lo que sigue es el ajuste de concurrencia que redujo el coste un 71 % y las conexiones a la base de 500 a 12, el trabajo en segundo plano que fallaba de forma intermitente, y el flujo de trabajo que sustituyó a un coordinador escrito a mano.

El montaje inicial, trasladado de la nube anterior:

```
11 servicios, todos con
  concurrencia                1
  motivo del equipo    «así se comporta como una función,
                        que es lo que teníamos»
  instancias mínimas      0
  máximo                 100
  tiempo de espera       300 s
  memoria                512 Mi
  entrada                interna
  salida                 sin conector
```

Y los cuatro problemas que aparecieron el primer mes:

- 1 la base agotó conexiones en el primer pico
- 2 el coste era 3,4 veces el estimado
- 3 los servicios no alcanzaban la base privada
- 4 el envío de correos fallaba de forma intermitente

Problema 1 y 2 · La concurrencia.

```
el cálculo, con la carga real del servicio de pedidos
caudal en el pico          900 pet/s
latencia p50               78 ms
```

```
con concurrencia 1
  instancias = 900 × 0,078 / 1 = 71 instancias
  conexiones = 71 × 7 = 497          > 200 de la base
```

```
con concurrencia 40
  instancias = 900 × 0,078 / 40 = 2 instancias
  conexiones = 2 × 7 = 14
```

Y la medición para elegir el valor:

se subió la concurrencia por escalones, midiendo el p99

concurrencia	p99	instancias en el pico
1	142 ms	71
10	148 ms	8
25	151 ms	3
40	156 ms	2
60	189 ms	2
80	410 ms	2 ← el codo

→ el p99 empieza a subir claramente entre 60 y 80
→ se eligió 40, con margen

y antes hubo que comprobar
¿el código es seguro para atender varias a la vez?

- 2 de los 11 servicios NO lo eran: usaban una variable global para el contexto de la petición
- corregidos; y hasta corregirlos, quedaron en concurrencia 1

Y el efecto:

concurrencia	1	40
instancias en el pico	71	2
conexiones a la base	497	14
coste de cómputo	4.100 €	1.180 €
p99	142 ms	156 ms
agotamientos de conexiones	3/mes	0

Problema 3 · La salida sin conector.

síntoma los servicios no alcanzaban la base privada
ni el almacén con conexión privada clase 231

causa entrada configurada como interna
→ el equipo asumió que eso los ponía «en la red»
→ la SALIDA es un ajuste distinto

corrección
conector de red, con subred dedicada dimensionada por el máximo de instancias
y encaminamiento de CUANTO tráfico salga, por la red
→ así pasa por el cortafuegos y se registra clase 200

y la comprobación que se añadió
desde el servicio, resolver el nombre de la base
→ debe devolver dirección privada
y comprobar la dirección de salida observada
→ debe ser la de la pasarela de traducción

Problema 4 · El trabajo en segundo plano.

síntoma entre el 2 % y el 11 % de los correos de confirmación no se enviaban
sin errores en los registros
sin patrón horario

causa
el servicio respondía al cliente y luego, en un hilo aparte, enviaba el correo
→ fuera de la petición, la CPU puede no estar asignada
→ el hilo se quedaba a medias y la instancia se apagaba
→ y no se registraba nada porque no llegaba a fallar

corrección
el envío pasa a una cola; un servicio consumidor lo procesa clase 237
→ y con idempotencia, porque hay reintentos

correos no enviados 2-11 % → 0

Y la observación del equipo:

este fallo es el más difícil de los cuatro
no daba error
no tenía patrón
y funcionaba en pruebas, donde el tráfico bajo mantenía las instancias vivas
→ y es una diferencia de modelo, no un error de configuración clase 229

El flujo de trabajo que sustituyó al coordinador.

el proceso de devolución tenía 7 pasos
validar → autorizar reembolso → esperar recepción del paquete (hasta 14 días) → inspeccionar → reembolsar → actualizar inventario → notificar

la implementación anterior
un servicio que guardaba el estado en una tabla
un trabajo programado cada 10 min que buscaba

devoluciones pendientes y las avanzaba
reintentos escritos a mano
compensaciones escritas a mano
→ 1.400 líneas de coordinación
→ y 3 incidentes en el año por estados inconsistentes

la implementación con flujo de trabajo
fichero declarativo de 120 líneas
reintentos con retroceso, declarados
espera de hasta 14 días, nativa
compensación declarada
estado de cada ejecución, consultable

resultado
líneas de coordinación 1.400 → 120
incidentes por estado inconsistente 3/año → 0
«¿por qué esta devolución lleva 3 días?»
antes abrir registros: 20-40 min
después consultar la ejecución: 30 s
coste +18 €/mes

Y la disciplina que hizo falta:

los 7 pasos, idempotentes
→ el flujo reintentaba, y reembolsar dos veces cuesta dinero
→ clave de idempotencia por devolución e intento
clase 210

y la compensación, probada
→ se ejecutó a propósito 20 veces en preproducción
→ 2 de 20 dejaron el inventario mal: corregido
ley 19, 22

El despliegue por revisiones, y lo que evitó:

214 despliegues en 6 meses
desplegados sin tráfico y comprobados con URL propia
desviados al 5 %, luego al 100 %

revisiones que no llegaron al 100 % 9
· 6 detectadas en el 5 % por tasa de error
· 3 detectadas al comprobar con la URL propia
tiempo de vuelta atrás 12 s
peticiones afectadas por despliegue defectuoso
~340 (en el 5 %)

y lo que habría pasado sin reparto por porcentaje
las 9 habrían llegado al 100 % del tráfico

El resultado:

coste de cómputo	4.100 €	1.180 €
conexiones máximas a la base	497	14
agotamientos de conexiones	3/mes	0
correos no enviados	2-11 %	0
servicios que alcanzan la base privada	0/11	11/11
líneas de coordinación de devoluciones	1.400	120
incidentes por estado inconsistente	3/año	0
despliegues defectuosos que llegaron a todo el tráfico	n/d	0

La lección que esta clase deja: un solo parámetro —la concurrencia, puesta en 1 por trasladar el modelo de funciones— costaba dos mil novecientos euros al mes y agotaba las conexiones de la base. Subirlo a 40 redujo las instancias de setenta y una a dos. Y el fallo más difícil de los cuatro no dio ningún error: el trabajo en segundo plano fuera de la petición no avanza, y eso no es un error de configuración sino una diferencia de modelo que hay que conocer antes de trasladar código.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/233-cloud-run-functions-api-gateway-y-workflows/lab.py
```

El laboratorio selecciona el motor de práctica serverless y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa gcp-serverless en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una función con límites, reintentos e idempotencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye gcp-serverless para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
La base agota conexiones con poco tráfico	Concurrencia por instancia baja, que multiplica el número de instancias y de agrupaciones	Mide el codo subiendo la concurrencia y elige un valor por debajo; calcula las conexiones máximas resultantes.
El coste es varias veces el estimado	Se factura tiempo de instancia y hay muchas instancias por concurrencia baja	Subir la concurrencia es la palanca de coste con más efecto; después, reducir latencia y ajustar recursos.
El servicio no alcanza recursos privados pese a tener entrada interna	Entrada y salida se configuran por separado	Añade el conector o la salida directa a la red y comprueba la dirección de salida observada.
Un trabajo diferido falla de forma intermitente y sin errores	Fuera de la petición, la CPU puede no estar asignada y el hilo no avanza	Envía el trabajo diferido a una cola con un consumidor propio, o activa la asignación continua de CPU.
Se paga tiempo de peticiones que ya no interesan a nadie	El tiempo de espera está en el máximo por si acaso	Fíjalo en un múltiplo pequeño del p99 esperado.
Dos revisiones sirviendo a la vez rompen la base	Un cambio de esquema incompatible entre versiones	Aplica expandir y contraer: el código tolera las dos formas durante la transición.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Cómo se relacionan concurrencia, número de instancias y conexiones a la base?
2. ¿Cómo se elige el valor de concurrencia y qué hay que comprobar antes?
3. ¿Por qué tener entrada interna no basta para alcanzar recursos privados?
4. ¿Qué le ocurre al trabajo que sigue tras responder a la petición?
5. ¿Qué resuelve un flujo de trabajo y qué sigue siendo responsabilidad del código?

Referencias

- Google Cloud (2025). Cloud Run: concurrency and instance settings. <<https://cloud.google.com/run/docs/about-concurrency>>
- Google Cloud (2025). Cloud Run: traffic management and revisions. <<https://cloud.google.com/run/docs/managing/revisions>>
- Google Cloud (2025). Cloud Run: CPU allocation and background activity. <<https://cloud.google.com/run/docs/configuring/cpu-allocation>>
- Google Cloud (2025). Connecting to a VPC network from Cloud Run. <<https://cloud.google.com/run/docs/configuring/connecting-vpc>>
- Google Cloud (2025). Workflows. <<https://cloud.google.com/workflows/docs/overview>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 19 en PDF · Recorrido de Google Cloud en PDF · Manual integral

Anterior	Índice	Siguiente
← 232 · Terraform, Infrastructure Manager y policy validation	Parte 19 · Programa	234 · GKE Autopilot, Workload Identity y Config Sync →

Evaluación — 233 Cloud Run, Functions, API Gateway y Workflows

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega gcp-serverless con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

234 — GKE Autopilot, Workload Identity y Config Sync

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: kubernetes · Estado: EXECUTABLECORE

Propósito

Operar Kubernetes gestionado en Google Cloud, donde existe un modo que quita casi toda la operación de nodos y cambia la decisión de la clase 213: si el motivo para no usar clúster era el coste de mantenerlo, ese motivo se reduce mucho. La clase compara los dos modos con lo que cada uno permite y prohíbe, cubre la identidad de carga y el consumo de direcciones, y la reconciliación desde el repositorio con su alerta imprescindible.

Resultados de aprendizaje

Al finalizar podrás:

1. Elegir entre el modo gestionado y el estándar con criterios.
2. Decidir si el clúster compensa frente al servicio de contenedores.
3. Dar identidad a las cargas sin claves, atada al espacio de nombres.
4. Dimensionar los rangos de direcciones que consume el clúster.
5. Operar con reconciliación, políticas y actualizaciones planificadas.

Conceptos centrales

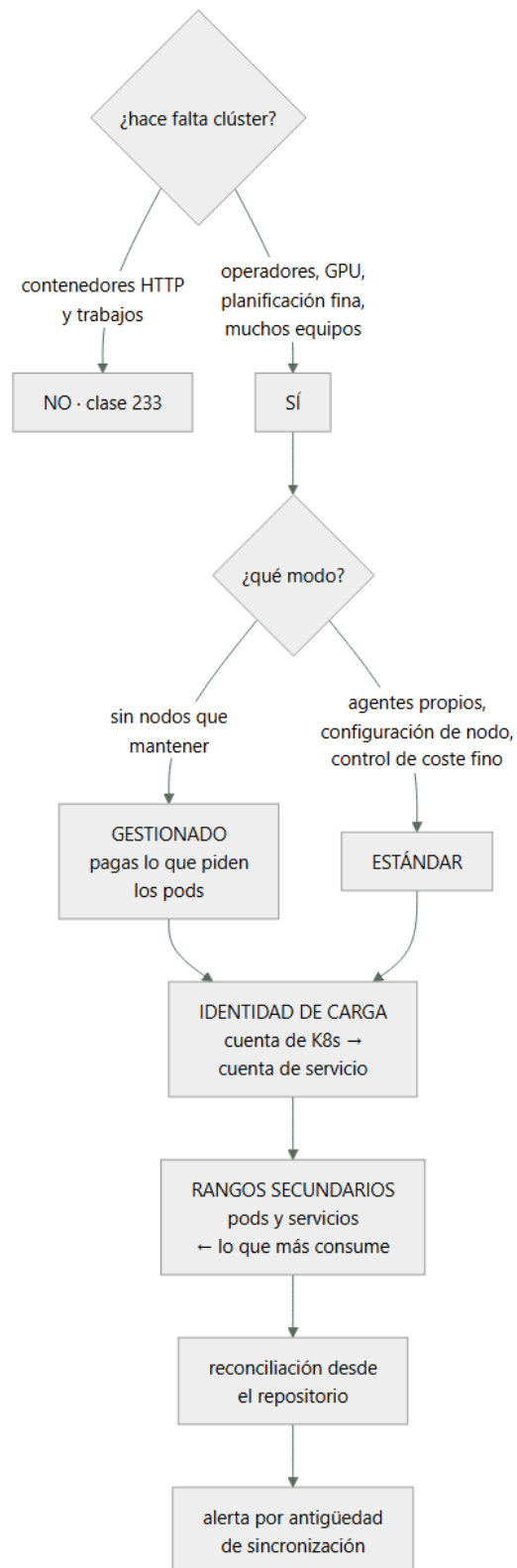
Concepto	Comprensión verificable
modo gestionado	El proveedor administra los nodos: se paga por lo que piden los pods y no hay conjunto que mantener.
modo estándar	Grupos de nodos administrados por el equipo. Más control y más trabajo.
identidad de carga	Vinculación entre una cuenta de servicio de Kubernetes y una de la nube, sin claves.
rango secundario	Rango de la subred del que salen las direcciones de pods y servicios. Es el que más consume.
canal de versión	Política de actualización automática del plano de control y de los nodos.
sincronización de configuración	Bucle gestionado que aplica el estado declarado en el repositorio y corrige la deriva.

Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Dos modos, y qué cambia cada uno

La decisión más importante al crear el clúster es el modo, y aquí hay uno que no existe en las otras nubes.

MODO GESTIONADO

- no hay grupos de nodos que administrar
- se paga por CPU, memoria y disco que PIDEN los pods

- el proveedor decide y mantiene los nodos
- + sin parchear, sin escalar el conjunto, sin drenar
- + la petición de recursos es el contrato: si pides bien, pagas bien
- + configuración segura por defecto: política de red, nodos endurecidos, identidad de carga activada
- ley 26
- no se puede tocar el nodo: nada de agentes con acceso privilegiado ni configuraciones del sistema
- menos control sobre el empaquetado
- y ciertas capacidades quedan restringidas

MODO ESTÁNDAR

- grupos de nodos propios
- + control total: agentes, configuración, tipos de máquina
- + coste por unidad menor si el empaquetado es alto
- hay que mantener el conjunto
- ley 23
- y hay que activar a mano lo que el otro trae puesto

Y el efecto sobre la decisión de la clase 213:

- el motivo principal para NO usar clúster era el coste de operarlo
- actualizaciones, complementos, nodos, parches
- el modo gestionado reduce mucho ese coste
- y por tanto la frontera se mueve: hay casos que antes no compensaban y ahora sí
- lo que NO cambia
- sigue habiendo actualizaciones de versión
- siguen los complementos que se instalan a mano
- y sigue haciendo falta saber Kubernetes

Y el criterio actualizado:

- ¿es un servicio HTTP o un trabajo?
- servicio de contenedores; más simple
- clase 233
- ¿hacen falta operadores, recursos propios o planificación fina?
- clúster
- ¿hacen falta agentes con acceso al nodo o configuración del sistema?
- clúster en modo estándar
- ¿ninguna de las dos?
- clúster en modo gestionado

Y una advertencia de coste que sorprende:

- en el modo gestionado se paga lo que PIDEN los pods
- una petición de recursos generosa «por si acaso» se paga entera, use lo que use
- y por tanto las peticiones bien puestas dejan de ser buena práctica y pasan a ser una línea de la factura
- clase 213

2. Identidad y direcciones

La identidad de carga es el mecanismo estándar aquí y conviene que esté activada desde el principio.

CÓMO FUNCIONA

- una cuenta de servicio de Kubernetes se anota con una cuenta de servicio de la nube
- el pod obtiene testigos de esa identidad
- sin claves, sin secretos montados

Y EL ENLACE

- se concede a la cuenta de Kubernetes el permiso de suplantar la de la nube
- atado a espacio de nombres Y nombre de la cuenta
- el mismo error de las clases 218 y 222: si se ata solo al proyecto, cualquier cuenta de servicio del clúster la obtiene

Y LO QUE HAY QUE DESACTIVAR

- el acceso de los pods al servicio de metadatos del nodo

- si no, obtienen la identidad del NODO, que suele tener más permisos
- en el modo gestionado viene desactivado; en el estándar, hay que hacerlo ley 26

Y la prueba obligatoria:

- desde un pod de otro espacio de nombres, intentar obtener la identidad
- debe fallar
- y desde un pod, leer las credenciales del nodo
- debe estar bloqueado ley 22

Las direcciones, que es la decisión que más consume:

LOS TRES RANGOS

- nodos del rango principal de la subred
- PODS de un rango secundario
- SERVICIOS de otro rango secundario

EL CÁLCULO

- el rango de pods se dimensiona por
- nodos máximos × pods por nodo × factor de reserva
- y el factor no es 1: se reservan bloques por nodo
- un clúster de 100 nodos con 110 pods por nodo puede necesitar un /17

Y LA TRAMPA

- el número de pods por nodo se fija al crear
- y el rango de pods NO se puede cambiar sin recrear
- es la misma decisión irreversible de la clase 222 ley 14

Y las opciones que reducen el consumo:

- reducir los pods por nodo, si no hacen falta 110
- usar rangos no enrutables para pods, donde el modelo lo permita clase 193
- y reutilizar rangos secundarios entre clústeres, donde esté soportado
- y en cualquier caso, calcularlo ANTES, con el plan de direcciones delante

Y el uso de red compartida:

- el clúster vive en un proyecto de servicio y usa subredes de la red compartida clase 229
- los rangos secundarios los cede el proyecto anfitrión
- y por tanto los pide plataforma, no el equipo

3. Operar desde el repositorio

La reconciliación es la misma idea de las clases 213 y 222, con una variante gestionada.

SINCRONIZACIÓN DE CONFIGURACIÓN GESTIONADA

- un componente del proveedor aplica lo declarado en el repositorio y corrige la deriva
- + menos que operar
- + con políticas de admisión integradas
- menos control sobre versiones y extensiones

O UN CONTROLADOR PROPIO

- + control total y portable
- hay que operarlo ley 23

Y la alerta que no puede faltar, por tercera vez:

- «la sincronización lleva N minutos sin completarse»
- o «el estado real difiere del declarado desde hace N»
- un bucle parado no da error: deja de aplicar ley 13
- y en la clase 179 esa prueba negativa falló porque la alerta iba a un canal sin nadie

Las políticas de admisión, que aquí se integran con el gobierno:

- lo que hay que impedir

imágenes que no vengan del registro propio
imágenes sin firmar clase 106
contenedores con privilegios o con red del anfitrión
recursos sin peticiones ni límites clase 213
y recursos sin etiquetas obligatorias clase 239

y la disciplina
modo de aviso primero, con medición
mensajes que digan qué falta y cómo arreglarlo
y el carril fácil que cumpla solo ley 16

Y la política de red, con la diferencia entre modos:

en el modo gestionado viene activada
en el estándar hay que activarla, y por defecto todos los
pods hablan con todos ley 26, clase 222
→ denegación por defecto entre espacios de nombres, y
reglas explícitas

Las actualizaciones, con lo que cambia aquí:

CANALES DE VERSIÓN
rápido · regular · estable
→ el plano de control y los nodos se actualizan solos
dentro del canal
→ con VENTANAS Y EXCLUSIONES de mantenimiento declaradas

Y ESO CAMBIA EL TRABAJO
no hay que planificar cada actualización menor
→ pero SÍ hay que comprobar antes las interfaces
retiradas y los complementos clase 213
→ y las exclusiones evitan que ocurra en campaña

LO QUE SIGUE SIENDO MANUAL
las versiones mayores
los complementos instalados a mano
y los operadores propios

Y una comprobación que hay que tener:

el aviso de interfaces retiradas: la plataforma detecta
manifiestos que usan API que van a desaparecer
→ conviene revisarlo antes de cada canal, no después
→ y añadirlo a la canalización clase 232

4. Coste, entrada y comprobaciones

El coste, con las palancas de cada modo:

MODO GESTIONADO
se paga lo pedido por los pods
palancas
peticiones ajustadas al p95 real clase 213
clases de cómputo adecuadas a la carga
y pods de baja prioridad para lotes tolerantes
→ y no hay empaquetado que optimizar: eso lo hace el
proveedor

MODO ESTÁNDAR
se pagan los nodos
palancas
empaquetado alto
autoescalado de nodos bien configurado
tipos de máquina adecuados
capacidad interrumpible para lotes
y compromisos de uso clase 239

Y EN AMBOS
el clúster tiene un coste fijo de plano de control
→ varios clústeres pequeños cuestan más que uno con
espacios de nombres separados
→ salvo que el aislamiento lo justifique clase 213

La entrada, con las opciones:

BALANCEADOR GLOBAL como entrada del clúster

- + una dirección para varias regiones clase 231
- + certificados gestionados y caché en el borde
- + filtrado de aplicación delante

CONTROLADOR DE ENTRADA PROPIO

- + control y portabilidad clase 158
- hay que operarlo

Y LA PUERTA DE ENLACE declarativa

- el modelo más nuevo, con separación entre quien opera la entrada y quien declara las rutas
- encaja con la separación de la clase 229

Y las comprobaciones de esta clase:

- obtener la identidad desde otro espacio de nombres
→ debe fallar
- leer credenciales del nodo desde un pod
→ debe estar bloqueado
- desplegar una imagen de un registro ajeno
→ debe rechazarse
- desplegar un pod sin peticiones de recursos
→ debe rechazarse
- desplegar un pod con privilegios
→ debe rechazarse
- hablar entre dos espacios de nombres no permitidos
→ debe fallar
- parar la sincronización y esperar la alerta
- perder una zona y comprobar el reparto de réplicas
clase 227

Y la lista de comprobación de la clase:

- la decisión de usar clúster está justificada frente al servicio de contenedores
- el modo elegido corresponde a lo que hace falta
- la identidad de carga está activada y atada a espacio de nombres y cuenta
- el acceso al servicio de metadatos está bloqueado
- los rangos secundarios se calcularon antes de crear
- los pods por nodo están ajustados a lo necesario
- el clúster usa la red compartida
- la política de red está activada, con denegación por defecto
- hay políticas de admisión con mensajes útiles
- la reconciliación está montada, con alerta por antigüedad
- el canal de versión y las ventanas están decididos
- se revisan las interfaces retiradas antes de actualizar
- las peticiones de recursos están medidas, no copiadas
- las réplicas se reparten entre zonas

Y el cierre que enlaza con la clase siguiente: con el cómputo resuelto, quedan los datos, que en esta nube ofrecen una familia amplia y una base distribuida con una propiedad que no tiene equivalente. Es la materia de la clase 235.

Ejemplo trabajado

CloudShop revisa su uso de Kubernetes en Google Cloud. Lo que sigue es la decisión de modo que cambió respecto a la de la clase 222, el cálculo de direcciones que evitó recrear el clúster, y el ahorro de ajustar las peticiones de recursos.

La decisión de modo:

- lo que se necesitaba
 - 2 cargas con GPU del equipo de datos
 - un operador de base vectorial
 - trabajos por lotes con prioridades
 - el buscador, con imagen propia

- lo que NO se necesitaba
 - agentes con acceso privilegiado al nodo
 - configuración del sistema operativo
 - tipos de máquina exóticos

→ modo GESTIONADO para todo salvo las cargas con GPU

y la comparación de esfuerzo, frente al clúster de Azure	
actualizaciones al año	3 planificadas → 1
mantenimiento de complementos	18 días/año → 4
soporte a equipos	44 días/año → 38
incidentes de plataforma	11 días/año → 3

Y la observación que el equipo registró:

El cálculo de direcciones, hecho antes de crear,

La identidad, y la prueba que falló.

pruebas negativas

Las peticiones de recursos, y el ahorro.

Multi-Cloud Engineering Program · Recorrido de Google Cloud

carga	pedido	uso p95 real
buscador	2 CPU / 4 Gi	0,7 CPU / 2,1 Gi
indexador	4 CPU / 8 Gi	2,9 CPU / 5,8 Gi
operador vect.	1 CPU / 2 Gi	0,1 CPU / 0,6 Gi
api interna	1 CPU / 2 Gi	0,2 CPU / 0,9 Gi
trabajos lote	2 CPU / 4 Gi	1,8 CPU / 3,6 Gi

ajuste a p95 con margen del 25 %	
buscador	0,9 CPU / 2,7 Gi
indexador	3,6 CPU / 7,3 Gi
operador vect.	0,2 CPU / 0,8 Gi
api interna	0,3 CPU / 1,2 Gi
trabajos lote	2,3 CPU / 4,5 Gi

coste del clúster gestionado 3.400 € → 1.640 €/mes

→ y aquí el ajuste de peticiones no es higiene: es
directamente la factura clase 213

Y un efecto que no se buscaba:

al bajar las peticiones, más pods caben por nodo
→ y el proveedor usa menos nodos
→ pero como se paga por lo pedido, el ahorro es directo y
no depende del empaquetado

La reconciliación y las políticas:

sincronización gestionada desde el repositorio
con alerta de antigüedad, a canal con guardia
→ y probada parándola: alerta en 4 minutos ley 22

políticas de admisión, en modo aviso 3 semanas	
incumplimientos medidos	
imágenes de registros ajenos	41
→ 38 eran imágenes públicas de herramientas	
→ se copiaron al registro propio y se firmaron	
pods sin peticiones de recursos	62
→ todos, de un equipo que desplegaba a mano	
contenedores con privilegios	4
→ 3 legítimos (agentes de red), 1 innecesario	
sin etiquetas obligatorias	88

tras corregir, modo bloqueo	
rechazos en el primer mes	6
todos, incumplimientos reales	
tiquetes de soporte	0
→ porque el mensaje decía qué faltaba clase 217	

Las actualizaciones, con el canal:

canal	regular
ventanas	martes y jueves, 02:00-06:00
exclusiones	del 15 de noviembre al 5 de enero (campana) y las 2 semanas de cierre trimestral

en 12 meses	
actualizaciones automáticas aplicadas	9
que causaron incidencia	0
actualizaciones mayores planificadas	1
con revisión previa de interfaces retiradas	
→ 11 manifiestos corregidos antes	
duración de la mayor	2 h

El resultado:

coste del clúster	3.400 €	1.640 €
direcciones consumidas por pods	8.192	2.048
(con el valor por defecto)		
capacidad de equipo dedicada	0,4 pers.	0,22 pers.
actualizaciones planificadas al año	3	1
enlaces de identidad mal atados	1	0
pods sin peticiones de recursos	62	0
imágenes de registros ajenos	41	0

La lección que esta clase deja: el modo gestionado redujo a la mitad el coste de operar el clúster, lo que mueve la frontera de cuándo compensa usarlo. El valor por defecto de ciento diez pods por nodo no correspondía a nada —el uso real era treinta y uno— y aceptarlo habría consumido el proyecto entero de direcciones de forma irreversible. Y el único fallo de identidad estaba, otra vez, en el componente instalado a mano: el que no pasa por la plantilla.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/234-gke-autopilot-workload-identity-y-config-sync/lab.py
```

El laboratorio selecciona el motor de práctica kubernetes y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa gcp-gke-platform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es manifiestos declarativos con estado observado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye gcp-gke-platform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El clúster consume el rango de direcciones del proyecto entero	Se aceptó el valor por defecto de pods por nodo sin medir el uso real	Calcula nodos por pods reales antes de crear; el valor se fija al crear y cambiarlo exige recrear el clúster.
La factura del clúster gestionado es alta pese a poco uso	Se paga lo que piden los pods y las peticiones estaban copiadas	Ajusta las peticiones al p95 real con margen; aquí es directamente la factura, no higiene.
Cualquier cuenta de servicio del clúster obtiene una identidad privilegiada	El enlace de identidad se ató solo al proyecto	Ata a espacio de nombres y nombre de cuenta, y compruébalo desde un pod de otro espacio.
Los pods obtienen permisos que no les corresponden	Pueden alcanzar el servicio de metadatos del nodo	Bloquéalo; en modo gestionado viene bloqueado, en estándar hay que hacerlo.
Los cambios del repositorio dejan de aplicarse sin aviso	El bucle de sincronización se detuvo en silencio	Alerta por antigüedad de la sincronización, con destino a guardia, y pruébala parándola.
Una actualización automática ocurre en plena campaña	No hay exclusiones de mantenimiento declaradas	Declara ventanas y exclusiones para los periodos críticos, y revisa las interfaces retiradas antes de cambiar de canal.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué permite y qué prohíbe el modo gestionado, y cómo cambia la decisión de usar clúster?
2. ¿Por qué las peticiones de recursos son directamente la factura en modo gestionado?
3. ¿Qué rangos consume un clúster y cuál es la decisión irreversible?
4. ¿A qué debe atarse el enlace de identidad de carga?
5. ¿Qué sigue siendo manual pese a tener canal de versión?

Referencias

- Google Cloud (2025). GKE Autopilot overview.
<<https://cloud.google.com/kubernetes-engine/docs/concepts/autopilot-overview>>
- Google Cloud (2025). Workload Identity Federation for GKE.
<<https://cloud.google.com/kubernetes-engine/docs/concepts/workload-identity>>
- Google Cloud (2025). IP address planning for GKE clusters.
<<https://cloud.google.com/kubernetes-engine/docs/concepts/alias-ips>>
- Google Cloud (2025). Config Sync and Policy Controller.
<<https://cloud.google.com/kubernetes-engine/enterprise/config-sync/docs/overview>>
- Google Cloud (2025). GKE release channels and maintenance windows.
<<https://cloud.google.com/kubernetes-engine/docs/concepts/release-channels>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 19 en PDF · Recorrido de Google Cloud en PDF · Manual integral

Anterior	Índice	Siguiente
← 233 · Cloud Run, Functions, API Gateway y Workflows	Parte 19 · Programa	235 · Cloud SQL, Spanner, Firestore y Bigtable →

Evaluación — 234 GKE Autopilot, Workload Identity y Config Sync

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega gcp-gke-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

235 — Cloud SQL, Spanner, Firestore y Bigtable

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: data · Estado: EXECUTABLECORE

Propósito

Elegir el almacén de datos en Google Cloud, donde hay cuatro familias con propósitos claramente distintos y una que no tiene equivalente en las otras nubes: una base relacional que escala horizontalmente y da consistencia fuerte global, a cambio de un coste y unas reglas de modelado propias. La clase da el criterio de elección, las decisiones que no se pueden cambiar en cada una, y la advertencia de siempre sobre la clave.

Resultados de aprendizaje

Al finalizar podrás:

1. Elegir entre las cuatro familias con criterios comprobables.
2. Modelar para la base distribuida evitando los puntos calientes.
3. Decidir si la consistencia fuerte global compensa su coste.
4. Dimensionar capacidad y detectar el estrangulamiento.
5. Combinar el camino operativo con el analítico sin forzar ninguno.

Conceptos centrales

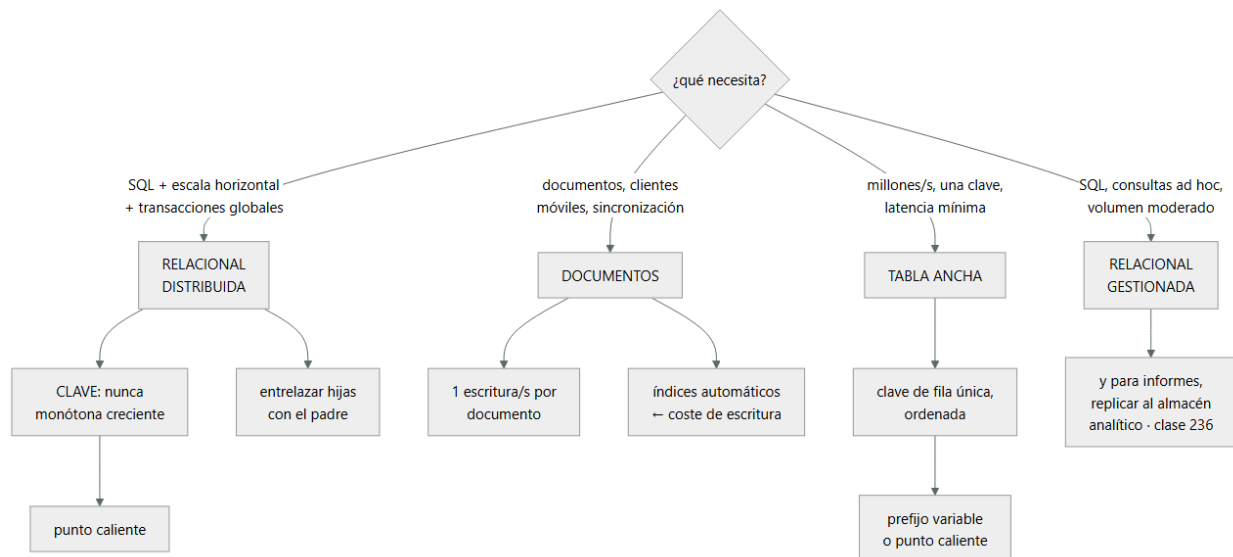
Concepto	Comprensión verificable
base relacional gestionada	Motor tradicional gestionado. Escala vertical, transacciones y consultas ad hoc.
base relacional distribuida	Escala horizontal con transacciones y consistencia fuerte global. Con reglas de modelado propias.
base de documentos	Almacén sin esquema con sincronización a clientes. Cómoda y con límites de escritura por documento.
base de tabla ancha	Almacén de altísimo volumen y baja latencia, con una sola clave ordenada.
punto caliente	Clave que concentra escrituras contiguas. En estas familias, la causa principal de mala escala.
entrelazado	Colocación física de filas hijas junto a su padre, para que la transacción no cruce servidores.

Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Cuatro familias, cuatro propósitos

Aquí las familias están más diferenciadas que en las otras nubes, y elegir mal se paga pronto.

RELACIONAL GESTIONADA

motor tradicional, gestionado
 + SQL completo, transacciones, consultas ad hoc
 + migración desde lo heredado casi directa
 – escala vertical con techo
 – y las conexiones son un recurso limitado clase 233
 para la mayoría de los sistemas de tamaño normal

RELACIONAL DISTRIBUIDA

escala horizontal CON transacciones y consistencia fuerte global
 + no hay que elegir entre SQL y escala
 + disponibilidad muy alta y multirregión real
 – coste base alto: hay un mínimo de capacidad
 – reglas de modelado propias, y la clave manda
 para volúmenes que una relacional no aguanta, o cuando se necesita consistencia fuerte entre regiones

DOCUMENTOS

almacén sin esquema, con sincronización a clientes
 + cómodo, con consultas por campo y tiempo real
 + integración directa con aplicaciones móviles
 – límite de escrituras por documento
 – índices automáticos que se pagan en cada escritura
 para perfiles, catálogos, estado de aplicaciones cliente

TABLA ANCHA

altísimo volumen, latencia de milisegundos, una sola clave ordenada
 + escala a petabytes con latencia estable
 – sin consultas por otros campos: solo por la clave
 – coste base alto (nodos)
 para series temporales, telemetría, perfiles de usuario a gran escala

Y el criterio, en preguntas:

¿el volumen cabe en una máquina grande y crecerá poco?
 → relacional gestionada; y no compliques

¿hace falta SQL y no cabe, o hace falta consistencia fuerte entre regiones?
 → relacional distribuida, midiendo el coste base

¿es estado de aplicación cliente con sincronización?

→ documentos

¿son millones de escrituras por segundo con una clave conocida?

→ tabla ancha

¿son informes y exploración?

→ ninguna de las cuatro: almacén analítico clase 236

Y la advertencia de coste que decide muchas elecciones:

la relacional distribuida y la de tabla ancha tienen COSTE
BASE ALTO

hay un mínimo de capacidad que se paga aunque no se use

→ para volúmenes pequeños salen mucho más caras

→ y por eso «empezar por la que escala» suele ser un error
de coste clase 216

2. La clave, otra vez

En las tres familias distribuidas, el error de diseño es el mismo y tiene el mismo nombre: la clave que concentra.

EL PROBLEMA

estas bases reparten los datos por RANGOS de clave

→ claves contiguas van al mismo servidor

→ y una clave que crece de forma monótona hace que TODAS
las escrituras nuevas caigan en el mismo sitio

LAS CLAVES QUE PRODUCEN PUNTOS CALIENTES

identificador autoincremental

marca de tiempo como primer componente

y cualquier secuencia ordenada

→ y es distinto del problema de la clase 208: allí la clave

se distribuía por resumen y el problema era la

concentración de TRÁFICO; aquí el problema es el ORDEN

Y las soluciones:

IDENTIFICADOR ALEATORIO

el más simple y el que mejor reparte

- se pierde el orden natural de inserción

INVERTIR LOS BITS de un valor creciente

reparte y conserva unicidad

PREFIJO CALCULADO

un valor derivado del resto de la clave, como primer
componente

→ reparte, y sigue permitiendo consultas por el resto si
se recorre cada prefijo

CLAVE COMPUESTA con algo que reparta primero

cliente + fecha, en vez de fecha + cliente

→ y sirve para las consultas por cliente clase 208

El entrelazado, que es propio de la relacional distribuida:

las filas hijas se colocan FÍSICAMENTE junto a su padre

pedido y sus líneas, juntos

→ leer un pedido con sus líneas es una sola lectura local

→ y la transacción no cruza servidores

y sin entrelazar

la transacción coordina entre servidores

→ latencia mucho mayor y más conflictos

→ es la decisión de modelado que más cambia el rendimiento

→ y hay que tomarla al crear la tabla

Y las transacciones, con lo que hay que saber:

la relacional distribuida da transacciones globales con
consistencia fuerte

→ y eso cuesta coordinación

las de SOLO LECTURA no bloquean y son mucho más baratas
→ declararlas como tales, siempre

y las de lectura con obsolescencia acotada
«lee de hace 10 segundos»
→ sirven la mayoría de las consultas y evitan la
coordinación clase 223

→ y la mayoría de los sistemas que usan transacciones
fuertes para todo están pagando coordinación que no
necesitan ley 26

3. Capacidad, límites y coste

La relacional distribuida se dimensiona por unidades de proceso.

LA CAPACIDAD
se mide en unidades; hay un mínimo por instancia
y cada unidad da un tope de almacenamiento y de caudal

→ y la regla operativa
mantener la utilización de CPU por debajo del 65 % en
una región y del 45 % en multirregión
→ por encima, la latencia se dispara clase 186

EL ESTRANGULAMIENTO
se manifiesta como latencia alta y transacciones
abortadas
→ y las abortadas hay que reintentar, lo que empeora
→ la señal directa es la CPU y la de operaciones por
servidor

Y LAS ESTADÍSTICAS DE ACCESO
la plataforma indica qué rangos de clave concentran
→ es la herramienta que encuentra el punto caliente
→ y hay que mirarla, no suponer ley 15

La base de documentos, con sus límites característicos:

UNA ESCRITURA POR SEGUNDO Y DOCUMENTO
→ un contador global en un documento no funciona
→ hay que repartirlo en varios y sumar al leer

ÍNDICES AUTOMÁTICOS
por defecto se indexa cada campo
→ cada escritura paga por todos clase 223
→ y hay que EXCLUIR lo que no se consulta

CONSULTAS LIMITADAS
no hay uniones ni agregaciones complejas
→ y las consultas compuestas exigen índices declarados

Y LAS REGLAS DE SEGURIDAD
si los clientes acceden directamente, las reglas son el
control de acceso
→ y una regla mal escrita expone todo
→ se prueban con el simulador, siempre ley 22

La tabla ancha, con lo suyo:

UNA SOLA CLAVE, ORDENADA
las consultas son por clave o por rango de claves
→ no hay índices secundarios
→ el diseño de la clave ES el diseño

GRUPOS DE COLUMNAS
separar lo que se lee junto
→ leer un grupo no trae los demás

CADUCIDAD POR CELDA
las versiones antiguas se retiran solas
→ imprescindible en series temporales

Y EL COSTE
por nodo y por almacenamiento

→ con un mínimo por instancia

Y la decisión de continuidad, que aquí es sencilla y cara:

la relacional distribuida en configuración multirregión da
consistencia fuerte y disponibilidad muy alta
→ y cuesta del orden del triple

y la relacional gestionada
réplicas de lectura, y conmutación con su plazo medido
clase 215

4. Combinar operativo y analítico

El patrón que este programa repite desde la clase 150 aquí es especialmente directo.

LO OPERATIVO en la familia que corresponda
LO ANALÍTICO en el almacén de análisis clase 236

y la conexión
flujo de cambios hacia el almacén analítico
o consulta federada desde el almacén hacia la base
o réplica gestionada, donde exista

→ y ninguna de las cuatro familias se fuerza para hacer
informes

Y las tres cosas que hay que decidir al montarlo:

¿CUÁNTO RETRASO SE ACEPTA?
segundos, minutos u horas
→ y hay que declararlo y vigilarlo ley 13

¿QUÉ DATOS NO CRUZAN?
los personales completos, seudonimizados o excluidos
clases 230, 239

¿QUIÉN ES EL ESCRITOR?
el almacén analítico es de solo lectura desde el punto de
vista del negocio
→ si alguien escribe ahí y luego vuelve al operativo, hay
dos escritores ley 21

El caché, que en muchos sistemas es lo que evita cambiar de familia:

un caché en memoria delante de la relacional gestionada
resuelve la mayoría de los problemas de lectura
→ y es mucho más barato que migrar a una distribuida
→ con las cautelas de la clase 111: el caché portante y
la avalancha

→ antes de cambiar de familia por rendimiento de lectura,
medir qué resuelve un caché

Y las comprobaciones de esta clase:

- escribir 10.000 filas con clave secuencial y observar el reparto
- consultar las estadísticas de acceso y comprobar que no hay rango caliente
- ejecutar una transacción que cruce servidores y medir
- escribir el mismo documento 10 veces por segundo
- probar las reglas de acceso con el simulador
- conmutar la base y cronometrar clase 215
- y comprobar el retraso del flujo hacia lo analítico

Y la lista de comprobación de la clase:

- la familia elegida corresponde a los patrones y al volumen
- se comparó el coste base antes de elegir una que escala
- la clave no es monótona creciente
- las tablas hijas están entrelazadas donde procede
- las lecturas usan transacciones de solo lectura
- se usa obsolescencia acotada donde basta
- la utilización se mantiene por debajo del umbral
- se revisan las estadísticas de acceso por rango

- Y el cierre que enlaza con la clase siguiente: con los datos operativos resueltos, queda la plataforma analítica, que en esta nube es la pieza más madura y la que más problemas de gobierno y de coste produce. Es la materia de la clase 236.

CloudShop elige sus almacenes en Google Cloud. Lo que sigue es la decisión que evitó pagar una base distribuida sin necesidad, el punto caliente que apareció igualmente en otra tabla, y el contador de un documento que no escalaba.

carga pedidos	volumen 41 M filas, 900 escrituras/s	elección relacional gestionada
catálogo sesiones	4,1 M productos 2 M activas, sincronización con la app	relacional gestionada documentos
telemetría de la web	41.000 eventos/s	tabla ancha
inventario global	120 M filas, consistencia fuerte entre 2 regiones	RELACIONAL DISTRIBUIDA
informes	—	almacén analítico clase 236

la propuesta inicial ponía CUANTO existe en la relacional
distribuida
motivo «escala, y así no hay que migrar nunca»

[illegible]

pedidos y catálogo en relacional gestionada	920 €/mes
inventario en distribuida	2.400 €/mes
total	3.320 €/mes

«se usa la relacional distribuida solo donde se necesita
consistencia fuerte entre regiones o el volumen no cabe»
qué la reabrirla si pedidos supera las 4.000
escrituras/s sostenidas o los 500 M de
filas clase 190

```

el modelo inicial
  tabla inventario, clave primaria: identificador
  autoincremental
  motivo «como en la base relacional de siempre»

```

Página 214

a partir de 1.200 escrituras/s
latencia p99 de 12 ms a 890 ms
transacciones abortadas 14 %
CPU media de la instancia 31 % ← engañosa

las estadísticas de acceso por rango decían
el 96 % de las escrituras caían en el ÚLTIMO rango
→ todas las claves nuevas eran contiguas
→ un solo servidor las atendía

corrección
clave primaria: identificador aleatorio
y para las consultas por producto y almacén, clave
compuesta (producto, almacén) que reparte por producto

resultado
latencia p99 a 1.200 escrituras/s 890 ms → 14 ms
transacciones abortadas 14 % → 0,2 %
caudal máximo probado 1.200 → 9.400/s

Y el entrelazado, que faltaba:

las tablas de movimientos de inventario no estaban
entrelazadas con la de inventario
→ cada transacción de ajuste coordinaba entre servidores
→ latencia de la transacción 48 ms

con movimientos entrelazados bajo inventario
→ la transacción es local
→ latencia 9 ms

→ y esta decisión se toma AL CREAR la tabla; cambiarla
exige recrear y migrar ley 14

El contador del documento.

la aplicación móvil mantenía un contador de artículos en
el carrito, en un documento por usuario
→ funcionaba

y para el panel de operaciones se añadió un contador
GLOBAL de carritos activos, en un solo documento

síntoma en campaña, el contador se quedaba atrás y
aparecían errores de contención
causa un documento admite del orden de una escritura
por segundo sostenida
y había hasta 340 modificaciones por segundo

corrección
el contador se reparte en 100 documentos
cada escritura elige uno al azar
el panel suma los 100 al leer

→ y para el panel, un valor agregado cada 30 s desde el
almacén analítico habría bastado y habría sido más
barato clase 236
→ se hizo así al final

Y los índices, recortados:

los documentos de sesión tenían 34 campos
indexados automáticamente 34
consultados 4

tras excluir los 30 no consultados
coste de escritura por documento -71 %
coste mensual de documentos 1.240 € → 380 €

Las reglas de acceso, probadas:

la aplicación móvil accede directamente a los documentos
de sesión
→ las reglas son el control de acceso, no hay servidor en
medio

el simulador de reglas, con 12 casos

- ✓ usuario lee su propio documento
- ✓ usuario escribe su propio documento
- ✗ usuario lee el documento de OTRO usuario
 - la regla comparaba el identificador del documento, no el del usuario autenticado
 - cualquiera con un identificador podía leer
- ✓ usuario sin autenticar denegado
- ✗ usuario escribe un campo que no debería
 - la regla no validaba qué campos se modifican
- ✓ el resto correcto

→ 2 de 12, y las dos exponían datos de otros usuarios
 → y sin el simulador no se habrían visto hasta que alguien las probara ley 22

La telemetría en tabla ancha:

clave de fila invertida(marca de tiempo) + sesión
 → el componente invertido reparte; sin él, todas las escrituras nuevas caerían juntas

grupos de columnas
 «evento» y «contexto» separados
 → el proceso en tiempo real lee solo «evento»

caducidad por celda 30 días
 → y el histórico va al almacén analítico clase 236

coste 610 €/mes
 frente a 3.900 € estimados con el bus de mensajes clase 224

El resultado:

coste mensual de datos (según la propuesta inicial)	6.200 €	3.320 €
latencia p99 del inventario	890 ms	14 ms
caudal máximo del inventario	1.200/s	9.400/s
transacciones abortadas	14 %	0,2 %
coste de documentos	1.240 €	380 €
reglas de acceso con fallos	2/12	0/12
contadores con contención	1	0

La lección que esta clase deja: la propuesta de poner todo en la base que escala habría costado casi el doble sin resolver ningún problema, porque el coste base de esas familias es alto y la mayoría de las cargas cabían en una relacional normal. Y el punto caliente apareció donde siempre: una clave autoincremental trasladada de la base relacional, que en un almacén repartido por rangos concentra el noventa y seis por ciento de las escrituras en un solo servidor.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/235-cloud-sql-spanner-firestore-y-bigtable/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa gcp-operational-data en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye gcp-operational-data para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Latencia altísima y transacciones abortadas con la CPU baja	Clave monótona creciente: todas las escrituras nuevas caen en el mismo rango	Usa identificador aleatorio, invierte los bits o pon delante un componente que reparta; revisa las estadísticas de acceso por rango.
Las transacciones son lentas aunque toquen pocas filas	Cruzan servidores porque las tablas hijas no están entrelazadas con el padre	Entrelaza las hijas al crear la tabla; cambiarlo después exige recrear y migrar.
Se paga mucho por una carga pequeña	Se eligió una familia con coste base alto por si escala en el futuro	Compara el coste base antes de elegir; empieza en la relacional gestionada y registra qué señal justificaría migrar.
Un contador global da errores de contención	Un documento admite del orden de una escritura por segundo	Reparte el contador en varios documentos y suma al leer, o agrega el valor en el almacén analítico.
El coste de escritura de los documentos es desproporcionado	Se indexan automáticamente todos los campos	Excluye del índice los campos que no se consultan.
Un usuario puede leer los datos de otro	Las reglas de acceso comparan el identificador equivocado o no validan los campos modificados	Prueba las reglas con el simulador para todos los casos, incluidos los de acceso ajeno.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué pregunta decide entre las cuatro familias?
2. ¿Por qué una clave autoincremental produce puntos calientes en un almacén repartido por rangos?
3. ¿Qué aporta entrelazar las tablas hijas y cuándo se decide?
4. ¿Qué límite tiene la escritura sobre un mismo documento y cómo se resuelve?
5. ¿Por qué no conviene empezar por la familia que más escala?

Referencias

- Google Cloud (2025). Spanner schema design and hotspot avoidance. <<https://cloud.google.com/spanner/docs/schema-design>>
- Google Cloud (2025). Spanner: interleaved tables and transactions. <<https://cloud.google.com/spanner/docs/schema-and-data-model>>
- Google Cloud (2025). Firestore: best practices and limits. <<https://cloud.google.com/firestore/docs/best-practices>>
- Google Cloud (2025). Bigtable schema design. <<https://cloud.google.com/bigtable/docs/schema-design>>
- Google Cloud (2025). Cloud SQL: read replicas and high availability. <<https://cloud.google.com/sql/docs/mysql/high-availability>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 19 en PDF · Recorrido de Google Cloud en PDF · Manual integral

Anterior	Índice	Siguiente
← 234 · GKE Autopilot, Workload Identity y Config Sync	Parte 19 · Programa	236 · BigQuery, Dataflow, Dataproc y gobernanza de datos →

Evaluación — 235 Cloud SQL, Spanner, Firestore y Bigtable

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega gcp-operational-data con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

236 — BigQuery, Dataflow, Dataproc y gobernanza de datos

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: data · Estado: EXECUTABLECORE

Propósito

Operar la plataforma analítica de Google Cloud, que es la pieza más madura de esta nube y la que más problemas produce de dos tipos concretos: coste, porque una consulta mal escrita puede costar cientos de euros en segundos, y gobierno, porque el acceso a datos personales se concede en la primera semana y nadie lo revisa después. La clase cubre el modelo de facturación, el diseño de tablas, el procesamiento por lotes y continuo, y el control de acceso a nivel de columna y de fila.

Resultados de aprendizaje

Al finalizar podrás:

1. Elegir el modelo de facturación adecuado y evitar las sorpresas.
2. Diseñar tablas particionadas y agrupadas para reducir lo que se lee.
3. Elegir entre procesamiento continuo, por lotes y gestionado.
4. Controlar el acceso a columnas y filas con datos sensibles.
5. Gobernar el linaje, la calidad y la retención de los datos.

Conceptos centrales

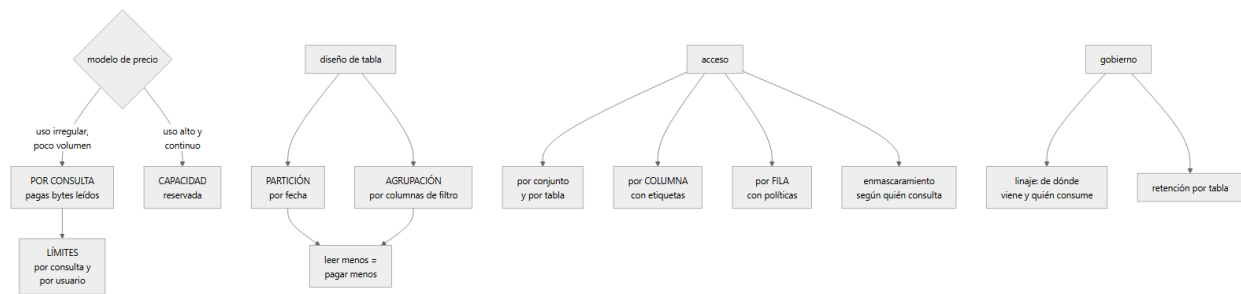
Concepto	Comprensión verificable
facturación por consulta	Se paga por bytes leídos. Una consulta sin filtro sobre una tabla grande cuesta mucho.
capacidad reservada	Modelo de precio por capacidad de proceso contratada, con coste previsible.
partición	División física de una tabla, normalmente por fecha. Permite leer solo lo necesario.
agrupación	Ordenación física por columnas dentro de la partición. Reduce lo leído en filtros por esas columnas.
enmascaramiento de columna	Transformación del valor según quién consulta, sin duplicar la tabla.
linaje	Registro de de dónde viene cada tabla y qué la consume. Necesario para retirar y para auditar.

Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. El coste, que es lo primero

Aquí el coste no es un asunto de fin de mes: una consulta puede costar cientos de euros en segundos, y por eso se aborda antes que el diseño.

POR CONSULTA

- se paga por BYTES LEÍDOS, no por filas devueltas
- «select * from tabla limit 10» sobre 40 TB lee 40 TB
- el límite no reduce lo leído
- + sin coste cuando no se consulta
- impredecible; una consulta mal escrita cuesta mucho

CAPACIDAD RESERVADA

- se paga por capacidad de proceso, con compromiso
- + coste previsible y consultas «gratis» dentro de la capacidad
- hay que dimensionarla, y las consultas compiten entre sí

CRITERIO

- uso irregular o bajo → por consulta, CON LÍMITES
- uso alto y continuo → capacidad reservada
- y lo habitual: capacidad para lo productivo y por consulta para exploración, con límites

Y los límites que hay que poner el primer día:

LÍMITE DE BYTES POR CONSULTA

- «ninguna consulta puede leer más de X»
- rechaza antes de ejecutar
- y es lo único que impide el susto de una consulta

CUOTA DIARIA POR USUARIO Y POR PROYECTO

Y LA COSTUMBRE DE ESTIMAR ANTES

- la herramienta dice cuántos bytes leerá una consulta
- ANTES de ejecutarla
- y esa estimación debe estar en la formación de cualquiera que consulte

ley 26

Y las tres causas de coste que se repiten:

- 1 SELECCIONAR TODAS LAS COLUMNAS**
el almacenamiento es por columnas: leer 3 de 80 cuesta una fracción
→ «select *» es el error más caro y el más común
- 2 NO FILTRAR POR LA COLUMNA DE PARTICIÓN**
una consulta sin filtro de fecha lee la tabla entera
→ se puede EXIGIR el filtro en la definición de la tabla
- 3 CONSULTAS PROGRAMADAS QUE NADIE REVISA**
un panel que refresca cada 5 minutos sobre una tabla grande
→ y nadie mira su coste

ley 15

2. Diseñar para leer menos

Todo el rendimiento y el coste dependen de cuántos bytes se leen. Hay dos mecanismos.

PARTICIÓN

divide la tabla, normalmente por fecha
→ una consulta con filtro de fecha lee solo esas particiones

y la opción que evita sustos
exigir el filtro de partición en la tabla
→ una consulta sin él FALLA en vez de leer todo

AGRUPACIÓN

ordena físicamente por hasta cuatro columnas dentro de la partición
→ filtrar por la primera columna de agrupación reduce mucho lo leído
→ el orden importa: se agrupa por lo que más se filtra

→ y las dos juntas son lo que convierte una tabla de 40 TB en consultas de 2 GB

Y el resto de decisiones de modelado:

DESNORMALIZAR

las uniones cuestan; repetir datos suele salir más barato
→ aquí, al contrario que en una base transaccional

CAMPOS ANIDADOS Y REPETIDOS

guardar las líneas de un pedido dentro del pedido
→ evita la unión y reduce lo leído

VISTAS MATERIALIZADAS

agregados precalculados que se refrescan solos
→ las consultas los usan automáticamente
→ y reducen mucho el coste de los paneles

TABLAS EXTERNAS

consultar ficheros del almacén sin cargarlos
→ cómodo, y más lento y más caro por consulta
→ para datos poco consultados

El almacenamiento, con su propia palanca:

las particiones que no se modifican pasan a un precio de almacenamiento menor, solas
→ y la RETENCIÓN por partición retira lo antiguo
→ sin retención, la tabla crece indefinidamente ley 25

y una decisión que hay que tomar
¿cuántos años de histórico hacen falta de verdad?
→ casi siempre menos de los que se guardan
→ y lo que haga falta por norma, en almacenamiento barato clase 239

Y la carga de datos, con lo que decide el coste:

CARGA POR LOTES	gratuita o muy barata
INSERCIÓN CONTINUA	se paga por fila, y suma
ESCRITURA POR FLUJO	el mecanismo moderno, más barato

→ si el retraso aceptable es de minutos, la carga por lotes cada pocos minutos es mucho más barata que la inserción fila a fila
→ y esa es una decisión de arquitectura, no de implementación clase 242

3. Procesar: continuo, por lotes y gestionado

Hay tres formas de transformar datos y se eligen por lo que hay que hacer, no por preferencia.

CONSULTAS PROGRAMADAS Y VISTAS MATERIALIZADAS

la transformación se escribe en SQL, dentro del almacén
+ lo más simple; sin infraestructura
+ y el linaje se registra solo
- limitado a lo que SQL puede hacer
→ y es lo que resuelve la mayoría de los casos

PROCESAMIENTO UNIFICADO (Dataflow)

- el mismo código para lotes y para flujo continuo
- + gestiona ventanas, datos que llegan tarde y estado
- + escala solo
- más que operar, y hay que entender el modelo
- para transformaciones complejas o continuas de verdad

MOTOR DE CÓDIGO ABIERTO GESTIONADO (Dataproc)

- para trabajos existentes que ya están escritos
- + migración directa de lo que ya hay
- hay que gestionar clústeres, aunque sean efímeros
- y para trabajos nuevos, rara vez es la elección

Y el criterio:

- ¿se puede escribir en SQL y el retraso de minutos vale?
 - consulta programada o vista materializada
- ¿hace falta procesamiento continuo con ventanas y estado?
 - procesamiento unificado
- ¿hay trabajos que ya existen y funcionan?
 - motor gestionado, con fecha de revisión ley 25

Y lo que hay que resolver en el procesamiento continuo, que es lo mismo de la clase 116:

DATOS QUE LLEGAN TARDE

- una ventana cerrada y un dato de hace 3 horas
- decidir: descartar, reprocesar o corregir
- y declararlo clase 242

EXACTAMENTE UNA VEZ

- se consigue con identificadores y deduplicación
- no viene gratis clase 237

Y EL RETRASO

- vigilado, con alerta ley 13
- «el flujo lleva N minutos de retraso» es la señal que dice que algo va mal antes de que falte un dato

Y la orquestación:

- los trabajos dependen unos de otros
- hace falta un orquestador que sepa el orden, reintente y avise clase 243
- y no un conjunto de trabajos programados por hora esperando que el anterior haya terminado

4. Gobierno: acceso, linaje y retención

El acceso a datos sensibles es donde este tipo de plataformas produce los hallazgos más incómodos.

LOS NIVELES DE CONTROL

- por conjunto de datos y por tabla
 - el habitual, y el más grueso
- POR COLUMNA
 - las columnas se etiquetan; el acceso se concede por etiqueta
 - «quien no tenga la etiqueta de datos personales no ve esas columnas»
- POR FILA
 - políticas que filtran según quién consulta
 - «cada país ve sus propias filas»
- ENMASCARAMIENTO
 - la columna se transforma según quién consulta
 - un analista ve el correo con formato pero sin valor
 - sin duplicar la tabla

Y el problema que resuelven, que es el de la clase 230:

- sin control por columna, dar acceso a una tabla es dar acceso a CUANTO contiene
- y por eso el acceso se concede al conjunto entero
- y por eso los equipos de análisis acaban viendo datos personales completos que no necesitan

- con etiquetas de columna
 - el acceso a la tabla no incluye las columnas etiquetadas
 - y la mayoría de los análisis funcionan igual

Y la disciplina:

- 1 CLASIFICAR las columnas: la plataforma puede detectar datos personales automáticamente
- 2 ETIQUETAR las sensibles
- 3 QUITAR el acceso amplio y conceder por etiqueta
- 4 y REVISAR trimestralmente quién la tiene clase 230

El linaje, que hace posible retirar y auditar:

qué tablas alimentan a cuáles
qué consultas y paneles consumen cada tabla
y de dónde vino cada dato

→ sin linaje, retirar una tabla es imposible: nadie sabe quién la usa ley 20

→ y ante una petición de borrado de datos personales, tampoco se sabe dónde se copiaron clases 139, 251

Y lo que hay que vigilar en esta plataforma:

coste por consulta, por usuario y por panel
consultas que leen más de X bytes
tablas sin partición ni agrupación
tablas sin retención
tablas sin consumidores en 90 días → candidatas a retirar

acceso a columnas sensibles, por identidad
retraso de los flujos continuos
y trabajos que fallan sin que nadie lo mire ley 13

Y la lista de comprobación de la clase:

- hay límite de bytes por consulta
- hay cuota diaria por usuario y por proyecto
- las tablas grandes están particionadas
- se exige el filtro de partición donde procede
- están agrupadas por las columnas de filtro habituales
- los paneles usan vistas materializadas
- hay retención por tabla
- la ingesta usa lotes o escritura por flujo, no inserción fila a fila
- las columnas sensibles están clasificadas y etiquetadas
- el acceso se concede por etiqueta, no al conjunto entero
- hay políticas de fila donde aplica
- el linaje está activado y se consulta
- hay alerta de retraso de los flujos y de trabajos fallidos
- se revisan las tablas sin consumidores

Y el cierre que enlaza con la clase siguiente: con datos operativos y analíticos resueltos, queda la mensajería que los conecta, que en esta nube tiene una propiedad que se anuncia mucho y conviene entender bien. Es la materia de la clase 237.

Ejemplo trabajado

CloudShop monta su plataforma analítica en Google Cloud. Lo que sigue son las tres consultas que costaban 4.100 € al mes, el acceso a datos personales que tenían 41 personas, y el rediseño de tablas que redujo el coste un 88 %.

La factura, al revisar:

coste mensual del almacén analítico	7.900 €	
consultas	6.200 €	
almacenamiento	1.400 €	
inserción continua	300 €	
las consultas, por origen		
paneles programados	4.100 €	←
exploración de analistas	1.400 €	
transformaciones programadas	700 €	

Y el desglose de los paneles:

panel	refresco	bytes/ejec.	€/mes
ventas en tiempo real	5 min	2,1 TB	2.900

ocupación por almacén	15 min	0,9 TB	810
embudo de conversión	1 h	3,4 TB	390

- tres consultas, 4.100 €/mes
- y los tres paneles los abría alguien 2 o 3 veces al día

Y el diagnóstico de la primera:

la consulta del panel de ventas
 select * from pedidos p join lineas l ... where
 fecha >= current_date() - 30

la tabla pedidos 14 TB
 la tabla lineas 26 TB
 ninguna estaba particionada
 → el filtro de fecha no reducía nada: se leía todo
 → y «select *» traía 80 columnas de las que se usaban 6

bytes leídos por ejecución 2,1 TB
 ejecuciones al mes 8.640

El rediseño de tablas:

pedidos
 partición por fecha de pedido, diaria
 agrupación por país, canal, estado
 filtro de partición EXIGIDO

lineas
 partición por fecha del pedido (heredada)
 agrupación por producto
 o mejor: las líneas ANIDADAS dentro de pedidos
 → se eligió anidarlas: elimina la unión

y las consultas reescritas
 columnas explícitas, no «select *»
 filtro de fecha, obligatorio

bytes por ejecución del panel de ventas 2,1 TB → 1,4 GB

Y las vistas materializadas:

los tres paneles consultaban agregados diarios
 → vista materializada con el agregado, refrescada sola
 → los paneles la usan automáticamente

bytes por ejecución 1,4 GB → 12 MB

coste de los tres paneles 4.100 € → 41 €/mes

Los límites, puestos después del susto:

un analista ejecutó una consulta exploratoria sin filtro
 sobre la tabla de eventos de 61 TB

bytes leídos	61 TB
coste de esa consulta	310 €
duración	94 s

→ y no había nada que lo impidiera

límites puestos

bytes máximos por consulta	200 GB
→ salvo excepción con aprobación	
cuota diaria por usuario	2 TB
cuota diaria por proyecto de exploración	20 TB

y formación
 la estimación previa de bytes, en la guía de
 incorporación
 → y un aviso en la herramienta cuando la estimación
 supera 50 GB

consultas rechazadas por límite, primer mes	61
de ellas, errores reales del analista	57
con excepción aprobada	4

El acceso a datos personales.

al inventariar
personas con acceso al conjunto de datos de clientes 41
de ellas, que necesitaban ver datos personales
completos 3

→ los otros 38 hacían análisis agregados y no
necesitaban el correo, el teléfono ni la dirección
→ pero el acceso se concede al conjunto, y el conjunto
los incluye clase 230

y la clasificación automática encontró
columnas con datos personales 61
en tablas donde nadie sabía que los había 9
· una tabla de análisis de campañas con correos
· dos tablas temporales de una migración de 2023
que nunca se borraron ley 25

Y la corrección:

clasificación y etiquetado de las 61 columnas
etiquetas jerárquicas
personal-directo (nombre, correo, teléfono,
dirección)
personal-indirecto (identificador, dispositivo)
financiero

acceso concedido por etiqueta, no por conjunto
personal-directo 3 personas
personal-indirecto 12 personas
financiero 6 personas
el resto del conjunto 41 personas

y enmascaramiento para los casos intermedios
el correo se ve como «a***@dominio.com»
→ suficiente para verificar y sin exponer el valor

y las políticas de fila
el equipo de cada país ve solo las filas de su país

y las 3 tablas con datos que nadie sabía
2 borradas (temporales de la migración)
1 seudonimizada

Y la comprobación:

prueba negativa
un analista sin la etiqueta consulta la columna de
correo
→ la consulta falla, indicando qué etiqueta falta
un analista de España consulta filas de Portugal
→ devuelve 0 filas
→ y ambas se ejecutan en cada despliegue del gobierno
ley 22

El procesamiento, ajustado:

antes
inserción continua fila a fila desde la aplicación
41.000 eventos/s → 300 €/mes de inserción
y transformaciones en un motor gestionado con un clúster
siempre encendido 410 €/mes

después
los eventos van al servicio de mensajería y de ahí en
lotes cada 60 s clase 237
coste de ingesta 300 € → 18 €
las transformaciones que caben en SQL, como consultas
programadas 9
las que no, en procesamiento unificado 2
clúster gestionado retirado 410 € → 140 €

y el retraso declarado
eventos disponibles para consulta ≤ 90 s

con alerta si supera 5 min

ley 13

La retención y el linaje:

retención por tabla	
eventos brutos	90 días
agregados diarios	5 años
tablas de trabajo	7 días
almacenamiento	1.400 € → 310 €
linaje activado	
tablas sin consumidores en 90 días	34
→ 28 borradas tras avisar	
→ 6 resultaron alimentar informes trimestrales	
y ante la primera petición de borrado de datos de un cliente	
tablas donde aparecía, según el linaje	7
tablas que el equipo creía	2
	clases 139, 251

El resultado:

coste mensual del almacén	7.900 €	610 €
paneles	4.100 €	41 €
exploración	1.400 €	120 €
almacenamiento	1.400 €	310 €
ingesta	300 €	18 €
bytes por ejecución del panel de ventas	2,1 TB	12 MB
personas con acceso a datos personales	41	3
columnas sensibles sin clasificar	61	0
tablas con datos personales desconocidos	3	0
tablas sin consumidores	34	6
consultas que pueden leer la tabla entera	cualquiera	0

La lección que esta clase deja: tres paneles que alguien abría dos veces al día costaban cuatro mil cien euros al mes, y el problema no era la plataforma sino tablas sin particionar consultadas con «select». Y cuarenta y una personas tenían acceso a datos personales completos porque el acceso se concede al conjunto y el conjunto los incluye; solo tres los necesitaban, y el resto siguió trabajando igual con las columnas enmascaradas.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/236-bigquery-dataflow-dataproc-y-gobernanza-de-datos/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa gcp-analytics-platform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye gcp-analytics-platform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Una consulta cuesta cientos de euros en segundos	Se paga por bytes leídos y no hay límite ni filtro de partición	Pon límite de bytes por consulta y cuota por usuario, particiona las tablas y exige el filtro de partición.
Los paneles dominan la factura	Refrescan consultas pesadas sobre tablas grandes muchas veces al día	Usa vistas materializadas con los agregados y revisa el coste por panel.
El filtro de fecha no reduce el coste	La tabla no está particionada por esa columna	Particiona por la columna de filtro habitual y agrupa por las siguientes más usadas.
Todo el equipo de análisis ve datos personales completos	El acceso se concede al conjunto de datos, que los incluye	Clasifica y etiqueta las columnas sensibles, concede por etiqueta y usa enmascaramiento para los casos intermedios.
La ingesta continua cuesta más que el proceso	Se inserta fila a fila cuando el retraso aceptable es de minutos	Agrupar en lotes o usa la escritura por flujo; decide el retraso aceptable como decisión de arquitectura.
No se puede retirar una tabla porque nadie sabe quién la usa	No hay linaje registrado	Activa el linaje y revisa periódicamente las tablas sin consumidores.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Por qué un límite de filas no reduce el coste de una consulta?
2. ¿Qué diferencia hay entre partición y agrupación, y para qué sirve cada una?
3. ¿Qué problema resuelve el control de acceso por columna?
4. ¿Cuándo conviene procesamiento unificado frente a consultas programadas?
5. ¿Para qué hace falta el linaje además de para auditar?

Referencias

- Google Cloud (2025). BigQuery: controlling costs. <<https://cloud.google.com/bigquery/docs/best-practices-costs>>
- Google Cloud (2025). Partitioned and clustered tables. <<https://cloud.google.com/bigquery/docs/partitioned-tables>>
- Google Cloud (2025). Column-level and row-level security. <<https://cloud.google.com/bigquery/docs/column-level-security-intro>>
- Google Cloud (2025). Dataflow: unified batch and streaming. <<https://cloud.google.com/dataflow/docs/concepts>>

- Google Cloud (2025). Dataplex and data lineage. <<https://cloud.google.com/dataplex/docs/about-data-lineage>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 19 en PDF · Recorrido de Google Cloud en PDF · Manual integral

Anterior	Índice	Siguiente
← 235 · Cloud SQL, Spanner, Firestore y Bigtable	Parte 19 · Programa	237 · Pub/Sub, Eventarc y entrega exactamente-una-vez →

Evaluación — 236 BigQuery, Dataflow, Dataproc y gobernanza de datos

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega gcp-analytics-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

237 — Pub/Sub, Eventarc y entrega exactamente-una-vez

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: messaging · Estado: EXECUTABLECORE

Propósito

Montar la mensajería de Google Cloud, cuyo servicio principal anuncia entrega exactamente una vez, y conviene entender con precisión qué significa y qué sigue siendo responsabilidad del código. La clase separa el servicio de mensajería del enrutador de eventos, explica los mecanismos de entrega y sus parámetros, y sostiene lo que este programa lleva demostrando: la garantía del transporte no elimina la necesidad de idempotencia en el efecto.

Resultados de aprendizaje

Al finalizar podrás:

1. Elegir entre suscripción de entrega y de extracción, y el enrutador de eventos.
2. Configurar plazos, reintentos y destino de fallidos coherentemente.
3. Entender qué garantiza la entrega exactamente una vez y qué no.
4. Ordenar mensajes por clave cuando el orden importa.
5. Operar los fallidos con alerta, diagnóstico y reproceso probado.

Conceptos centrales

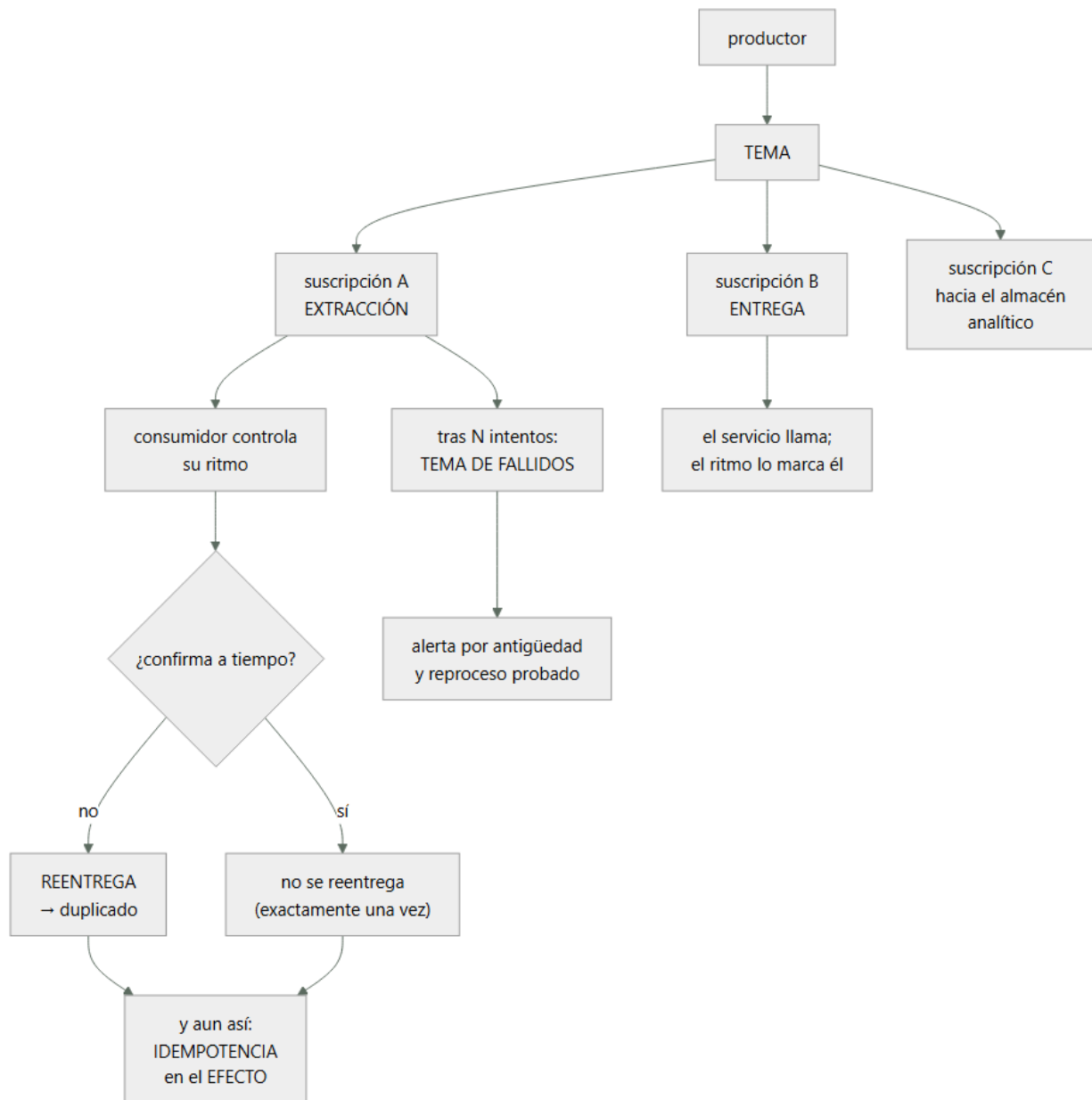
Concepto	Comprensión verificable
tema	Punto de publicación. Los consumidores se conectan mediante suscripciones independientes.
suscripción de extracción	El consumidor pide mensajes. Controla su propio ritmo.
suscripción de entrega	El servicio llama a un punto del consumidor. Más simple y con menos control de ritmo.
plazo de confirmación	Tiempo que el mensaje queda retenido tras entregarse. Si vence, se reentrega.
entrega exactamente una vez	Garantía de que un mensaje confirmado no se reentrega dentro de la ventana de confirmación.
clave de ordenación	Campo que agrupa mensajes para entregarlos en orden. Limita el caudal por clave.

Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Un tema, muchas suscripciones

El modelo aquí es más simple que el de las otras nubes: hay un servicio principal y un enrutador.

SERVICIO DE MENSAJERÍA

un TEMA donde se publica
N SUSCRIPCIONES independientes, cada una con su propio
ritmo, sus reintentos y su destino de fallidos
→ uno a muchos, sin configurar nada más
→ y añadir un consumidor es crear una suscripción: el
productor no se entera clase 148

ENRUTADOR DE EVENTOS

entrega eventos de la propia plataforma y de fuentes
propias a destinos, con filtros
→ «cuando se suba un fichero a este almacén, llama a este
servicio»
→ y por debajo usa el servicio de mensajería

Y los dos tipos de suscripción, que se eligen mal a menudo:

EXTRACCIÓN

- el consumidor pide mensajes cuando puede
- + controla su ritmo: no le llega más de lo que aguanta
- + amortigua picos de forma natural
- hay que ejecutar un proceso que extraiga

ENTREGA (a un punto HTTP)

- el servicio llama al consumidor
- + más simple: no hay proceso extractor
- + encaja con servicios que escalan por peticiones
clase 233
- el ritmo lo marca el servicio, no el consumidor
- y por tanto puede escalar el consumidor sin control

Y LA DECISIÓN

- si el consumidor tiene un recurso limitado detrás (una base), la extracción da control
- si escala bien y es idempotente, la entrega es más simple
- y en cualquier caso hay que poner un límite de concurrencia
clase 233

Y los otros destinos, que ahorran código:

SUSCRIPCIÓN HACIA EL ALMACÉN ANALÍTICO

- los mensajes se escriben directamente en una tabla
- sin consumidor que mantener
- y con esquema validado
clase 236

SUSCRIPCIÓN HACIA ALMACENAMIENTO

- vuelca por lotes a ficheros
- el histórico, sin escribir nada
clase 224

2. Exactamente una vez: qué significa

Esta es la propiedad que más se malinterpreta, y conviene ser preciso.

LO QUE GARANTIZA

- un mensaje CONFIRMADO no se vuelve a entregar dentro de la ventana de confirmación
- elimina los duplicados que produce el plazo vencido cuando el consumidor sí había terminado

LO QUE NO GARANTIZA

- que el PRODUCTOR no publique el mismo hecho dos veces
- un reintento del productor crea dos mensajes distintos
- que el efecto no se produzca dos veces si el consumidor falla DESPUÉS de actuar y ANTES de confirmar
- que el reproceso desde el tema de fallidos no repita
- ni que dos suscripciones no procesen lo mismo

→ es decir: elimina una de las cinco fuentes de duplicado de la clase 210, no las cinco

Y la conclusión, que es la de siempre:

LA IDEMPOTENCIA EN EL EFECTO SIGUE SIENDO OBLIGATORIA

- clave del suceso, generada por el productor
- registro de claves procesadas, en dos estados
- y escritura condicionada antes del efecto
clase 210

→ lo que la garantía da es que se necesitan MENOS reintentos y menos reproceso, no que se pueda prescindir de la comprobación

Y el coste, que conviene conocer:

- la entrega exactamente una vez tiene un coste
- el caudal por suscripción es menor
- la latencia es algo mayor
- y hay restricciones de configuración

→ activarla donde el duplicado cuesta dinero

→ y no en la telemetría, donde no aporta y limita el caudal

Los parámetros que producen duplicados, con la misma aritmética de siempre:

PLAZO DE CONFIRMACIÓN

tiempo que el mensaje queda retenido tras entregarse
si vence antes de confirmar, se reentrega

y la extensión automática

las bibliotecas extienden el plazo mientras se procesa

→ activarla es lo que evita el problema

→ y hay un máximo total: procesos muy largos no encajan

CONCURRENCIA DEL CONSUMIDOR

cuántos mensajes procesa a la vez

→ si es alta y el proceso es lento, se acumulan mensajes
retenidos y algunos vencen

→ es la misma relación de prebúsqueda de la clase 224

REINTENTOS Y RETROCESO

con retroceso exponencial declarado

→ sin retroceso, un consumidor caído recibe una avalancha
en cuanto vuelve

3. Orden, filtros y fallidos

El orden se consigue con clave, y tiene su coste.

CLAVE DE ORDENACIÓN

los mensajes con la misma clave se entregan en orden

→ y en la misma suscripción, al mismo consumidor

coste

el caudal por clave lo limita un solo consumidor

y si un mensaje falla, los siguientes de esa clave

ESPERAN

→ un mensaje venenoso bloquea su clave entera

y hay que decidir

¿qué se hace con el veneno? ¿se aparta y se sigue, o se
para?

→ misma decisión que en la clase 224

Y la pregunta previa, que evita el problema:

¿de verdad hace falta orden, o basta con que las
operaciones sean conmutativas?

«-1 unidad» no necesita orden

«estado = ENVIADO» tampoco, si lleva versión

«stock = 4» sí

→ y modelar para no necesitar orden es más barato que
garantizarlo clases 203, 224

Los filtros, que reducen trabajo y coste:

una suscripción puede filtrar por atributos del mensaje

→ el consumidor solo recibe lo suyo

→ y no se paga por entregar lo que se iba a descartar

y por eso conviene que los atributos lleven lo necesario
para filtrar

tipo de suceso, entidad, entorno, versión de esquema

El tema de fallidos, con la disciplina de siempre:

tras N intentos, el mensaje va a otro tema

→ y hay que crear una suscripción sobre él, o los
mensajes caducan sin que nadie los vea ley 13

LAS ALERTAS

«hay mensajes sin confirmar con más de 1 hora de
antigüedad» ← la que sirve

«el número de mensajes sin confirmar crece de forma
sostenida»

y en el tema de fallidos, las dos ley 15

EL DIAGNÓSTICO

guardar el motivo del fallo con el mensaje

→ veneno, dependencia o error de código clase 210

EL REPROCESO
procedimiento escrito, con límite de ritmo y por lotes
y probado por alguien que no lo escribió ley 22

Y una capacidad que evita reprocesos:

RETROCESO EN EL TIEMPO
una suscripción puede volver a un momento anterior o a una instantánea
→ y reprocesar desde ahí sin mover mensajes
→ útil cuando un consumidor procesó mal durante un rato
→ y exige que el consumidor sea idempotente, otra vez

4. Operar y medir

Lo que hay que vigilar, con las señales que dicen algo:

MENSAJES SIN CONFIRMAR
número y ANTIGÜEDAD del más viejo
→ la antigüedad es la que detecta el consumidor parado

PLAZOS VENCIDOS
señal directa de plazo mal puesto o proceso lento

RETRASO DE PUBLICACIÓN A CONSUMO
el tiempo entre que ocurre el hecho y se procesa
→ es la medida que le importa al negocio clase 238

MENSAJES EN EL TEMA DE FALLIDOS, y su antigüedad

ENTREGAS REPETIDAS del mismo mensaje
→ si la entrega exactamente una vez está activa y aun así hay repeticiones, algo no está confirmando

Y EL COSTE
se paga por volumen publicado, entregado y almacenado
→ una suscripción que nadie consume acumula y factura
→ suscripciones sin consumidor: inventario periódico ley 25

Y una advertencia de coste que se repite:

los mensajes se retienen hasta que se confirman o caducan
→ una suscripción abandonada retiene todo hasta la caducidad
→ y factura almacenamiento
→ y por eso las suscripciones sin consumidor son un hallazgo típico del primer inventario

Las comprobaciones de esta clase:

- enviar el mismo mensaje 50 veces y comprobar un efecto
- parar el consumidor y esperar la alerta de antigüedad
- dejar un mensaje en el tema de fallidos y esperar alerta
- reprocesar 10.000 mensajes con límite de ritmo
- publicar un mensaje con esquema inválido
- retroceder una suscripción y comprobar el reproceso
- y comprobar que un mensaje venenoso con clave de ordenación no bloquea indefinidamente

Y los errores de traslado, que esta parte vigila:

- X «como garantiza exactamente una vez, no hace falta idempotencia»
→ cubre una de las cinco fuentes de duplicado
- X «la suscripción de entrega es como una cola con consumidor»
→ el ritmo lo marca el servicio; hay que limitar la concurrencia del consumidor
- X «un tema por consumidor»
→ un tema, N suscripciones; crear un tema por consumidor obliga al productor a saber quién escucha clase 148

Y la lista de comprobación de la clase:

- hay un tema y una suscripción por consumidor
- el tipo de suscripción corresponde al consumidor
- la concurrencia del consumidor está limitada
- el plazo de confirmación supera el proceso, o se extiende
- los reintentos tienen retroceso declarado
- la entrega exactamente una vez está donde el duplicado cuesta
- toda operación con efecto es idempotente igualmente
- el orden por clave se usa solo donde hace falta
- está decidido qué hacer con un veneno en una clave ordenada
- los filtros reducen lo que llega a cada consumidor
- hay tema de fallidos CON suscripción
- hay alerta por antigüedad, no solo por número
- el motivo del fallo se guarda con el mensaje
- el reproceso está escrito y probado
- no hay suscripciones sin consumidor

Y el cierre que enlaza con la clase siguiente: con datos, cómputo y mensajería en pie, falta saber qué ocurre cuando algo va mal. La observabilidad de esta nube, con el estándar abierto, es la materia de la clase 238.

Ejemplo trabajado

CloudShop monta la mensajería de su plataforma en Google Cloud. Lo que sigue es el malentendido sobre la entrega exactamente una vez, la suscripción abandonada que acumuló 41 millones de mensajes, y la clave de ordenación que bloqueó un cliente durante 6 horas.

El malentendido, semana 2.

el equipo activó la entrega exactamente una vez en las 6 suscripciones
y retiró la comprobación de idempotencia de los consumidores
motivo «ya no hace falta: el servicio lo garantiza»

lo que pasó en la primera campaña	
correos de confirmación duplicados	214
descuentos de stock duplicados	88

el diagnóstico
la garantía cubre: un mensaje CONFIRMADO no se reentrega
no cubría lo que estaba pasando

caso 1 (correos)
el consumidor enviaba el correo y fallaba al confirmar por una desconexión
→ el mensaje se reentregaba, correctamente
→ y el correo se enviaba otra vez
→ la garantía funcionó: el problema es que el EFECTO ya se había producido

caso 2 (stock)
el productor reintentaba al no recibir confirmación de publicación
→ publicaba DOS mensajes distintos, con contenido igual
→ dos mensajes distintos, entregados una vez cada uno
→ la garantía no aplica: son mensajes distintos

corrección
idempotencia restaurada, con clave del suceso generada por el PRODUCTOR y estable entre reintentos clase 210
→ y la entrega exactamente una vez se dejó activa donde el duplicado cuesta, y se desactivó en telemetría, donde limitaba el caudal

duplicados tras la corrección	0
-------------------------------	---

Y la frase que el equipo escribió en el registro de decisión:

«la garantía del transporte reduce los duplicados de transporte; el efecto lo protege el consumidor»
→ y es la misma conclusión de las clases 117, 210 y 224, con otro producto ley 18

La suscripción abandonada.

al revisar el coste, mes 4
coste de mensajería 1.840 €/mes
esperado ~200 €

desglose
publicación 140 €
entrega 180 €
ALMACENAMIENTO DE MENSAJES SIN CONFIRMAR 1.520 € ←

causa
una suscripción creada para una prueba en el mes 1
su consumidor se apagó al terminar la prueba
la suscripción siguió recibiendo CUANTO se publicaba
mensajes retenidos 41 millones
antigüedad del más antiguo 94 días
→ y la retención estaba en 7 días... en OTRA suscripción

la abandonada tenía retención de 31 días y los mensajes
se acumulaban hasta caducar y volver a acumularse

corrección
suscripción borrada
inventario de suscripciones sin actividad de consumo
→ 4 más encontradas
alerta: «suscripción sin consumo en 24 h»
y caducidad automática de suscripciones inactivas 30 días
ley 25

coste 1.840 € → 190 €/mes

La clave de ordenación que bloqueó un cliente.

el flujo de integración con socios usaba clave de
ordenación = identificador de socio
motivo los eventos de un socio deben procesarse en orden

qué pasó
un socio envió un evento con un campo con un carácter que
el consumidor no manejaba
→ el consumidor fallaba al procesarlo
→ reintentaba con retroceso
→ y todos los eventos siguientes de ESE socio esperaban

duración 6 h 10
eventos acumulados de ese socio 4.100
otros socios sin afectación

cómo se detectó
el socio llamó
→ la alerta de antigüedad existía pero con umbral de
24 h ley 15

correcciones
1 umbral de antigüedad a 30 minutos
2 decisión escrita sobre el veneno en clave ordenada
tras 5 intentos, el mensaje se aparta al tema de
fallidos y la clave AVANZA
→ se pierde el orden estricto para esa clave, y se
registra como incidencia de datos
→ la alternativa (parar) era peor para el negocio
3 validación de esquema en la publicación
→ el mensaje con el carácter problemático se habría
rechazado al publicar clase 188

La configuración final:

consumidor	tipo	plazo	concurr.	orden
notificaciones	entrega	60 s	50	no
inventario	extracción	600 s	20	no
facturación	extracción	600 s	10	no
integración socios	extracción	300 s	5	SÍ
analítica	hacia el almacén analítico, directa			

histórico hacia almacenamiento, por lotes

entrega exactamente una vez

activa en inventario, facturación e integración
inactiva en notificaciones y telemetría

filtros

cada suscripción filtra por tipo de suceso
→ la de inventario recibía 41.000 mensajes/día y ahora
2.100
→ y no se paga por los 38.900 que descartaba

temas de fallidos

uno por suscripción, CON suscripción propia
con el motivo guardado como atributo

Y las dos suscripciones directas, que ahorraron trabajo:

ANALÍTICA

antes un consumidor que leía y escribía en el almacén
analítico: 340 líneas y un servicio que mantener
después suscripción directa hacia la tabla, con esquema
validado clase 236
→ servicio retirado ley 23

HISTÓRICO

antes un trabajo que exportaba cada noche
después suscripción hacia almacenamiento, por lotes de
5 minutos
→ trabajo retirado

Las comprobaciones, ejecutadas:

- ✓ mismo mensaje 50 veces 1 efecto
- ✓ parar el consumidor → alerta de antigüedad 3 min
- ✓ mensaje en tema de fallidos → alerta 41 s
- ✓ reprocesar 10.000 con límite de ritmo 2 min
- ✗ publicar un mensaje con esquema inválido
→ se aceptó: la validación de esquema no estaba
activada en 2 de los 6 temas
→ activada
- ✓ retroceder una suscripción y reprocesar
- ✗ veneno con clave de ordenación
→ bloqueó indefinidamente en la primera prueba
→ corregido con la decisión de apartar y avanzar

El resultado:

coste de mensajería	1.840 €	190 €
duplicados por campaña	302	0
suscripciones sin consumidor	5	0
mensajes retenidos	41 M	<10 k
bloqueo por veneno en clave ordenada	6 h 10	6 min
tiempo de detección de consumidor parado	24 h	3 min
servicios de integración retirados	—	2
mensajes entregados y descartados	38.900/día	0

La lección que esta clase deja: la garantía de entrega exactamente una vez es real y cubre una de las cinco fuentes de duplicado; quitar la idempotencia porque existe produjo trescientos dos duplicados en una campaña. Y el mayor coste del capítulo no fue de proceso ni de entrega: fue mil quinientos veinte euros al mes de almacenamiento de una suscripción que nadie consumía desde hacía tres meses.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/237-pub-sub-eventarc-y-entrega-exactamente-una-vez/lab.py
```

El laboratorio selecciona el motor de práctica messaging y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee `exercise.steps` y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de `negativetest` y explica la señal observada.
4. Materializa `gcp-event-platform` en `evidence/` usando la plantilla indicada.
5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es un flujo que declara orden, entrega y manejo de errores. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `gcp-event-platform` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
Aparecen duplicados pese a tener entrega exactamente una vez	La garantía cubre la reentrega tras confirmar, no el reintento del productor ni el fallo tras producir el efecto	Mantén la idempotencia en el consumidor con clave del suceso generada por el productor y estable entre reintentos.
La factura de mensajería es diez veces lo previsto	Una suscripción sin consumidor retiene todo lo publicado y paga almacenamiento	Inventaría suscripciones sin consumo, alerta cuando una lleve 24 h sin consumir y caduca las inactivas.
Los eventos de un cliente se detienen durante horas	Un mensaje venenoso bloquea su clave de ordenación	Decide y escribe qué ocurre con el veneno: apartarlo y avanzar, o parar; valida el esquema al publicar.
El consumidor recibe más carga de la que aguanta	Suscripción de entrega, donde el ritmo lo marca el servicio	Limita la concurrencia del consumidor o usa suscripción de extracción si hay un recurso limitado detrás.
Vencen los plazos y se reentregan mensajes que sí se estaban procesando	El plazo de confirmación es menor que el proceso y no se extiende	Activa la extensión automática del plazo y reduce la concurrencia; vigila los plazos vencidos.
Los mensajes fallidos caducan sin que nadie los vea	El tema de fallidos no tiene suscripción ni alerta	Crea suscripción sobre él, alerta por antigüedad y guarda el motivo del fallo como atributo.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué garantiza exactamente la entrega exactamente una vez y qué no?
2. ¿Cuándo conviene suscripción de extracción y cuándo de entrega?
3. ¿Qué coste tiene usar clave de ordenación y qué hay que decidir?
4. ¿Por qué una suscripción sin consumidor genera coste?
5. ¿Qué dos suscripciones directas evitan escribir un consumidor?

Referencias

- Google Cloud (2025). Pub/Sub: exactly-once delivery. <<https://cloud.google.com/pubsub/docs/exactly-once-delivery>>
- Google Cloud (2025). Pub/Sub subscription types and delivery. <<https://cloud.google.com/pubsub/docs/subscriber>>
- Google Cloud (2025). Message ordering. <<https://cloud.google.com/pubsub/docs/ordering>>
- Google Cloud (2025). Dead-letter topics and retry policies. <<https://cloud.google.com/pubsub/docs/handling-failures>>
- Google Cloud (2025). Eventarc. <<https://cloud.google.com/eventarc/docs/overview>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 19 en PDF · Recorrido de Google Cloud en PDF · Manual integral

Anterior	Índice	Siguiente
← 236 · BigQuery, Dataflow, Dataproc y gobernanza de datos	Parte 19 · Programa	238 · Cloud Operations, Trace y OpenTelemetry →

Evaluación — 237 Pub/Sub, Eventarc y entrega exactamente-una-vez

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega gcp-event-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

238 — Cloud Operations, Trace y OpenTelemetry

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: observability · Estado: EXECUTABLECORE

Propósito

Montar la observabilidad de Google Cloud, donde los registros de auditoría son la pieza distintiva y a la vez la que más coste y más sorpresas produce: el registro de acceso a datos está desactivado por defecto, y activarlo sin acotar dispara la factura. La clase cubre el enrutado y los filtros de exclusión, la instrumentación estándar, los objetivos de nivel de servicio como recurso declarado, y las alertas que este programa exige siempre.

Resultados de aprendizaje

Al finalizar podrás:

1. Enrutar registros con filtros de inclusión y exclusión, y controlar su coste.
2. Decidir qué registros de auditoría activar y sobre qué servicios.
3. Instrumentar con el estándar abierto para no quedar atado.
4. Declarar objetivos de nivel de servicio y alertar por ritmo de consumo.
5. Correlacionar desde la alerta hasta la causa sin eslabones rotos.

Conceptos centrales

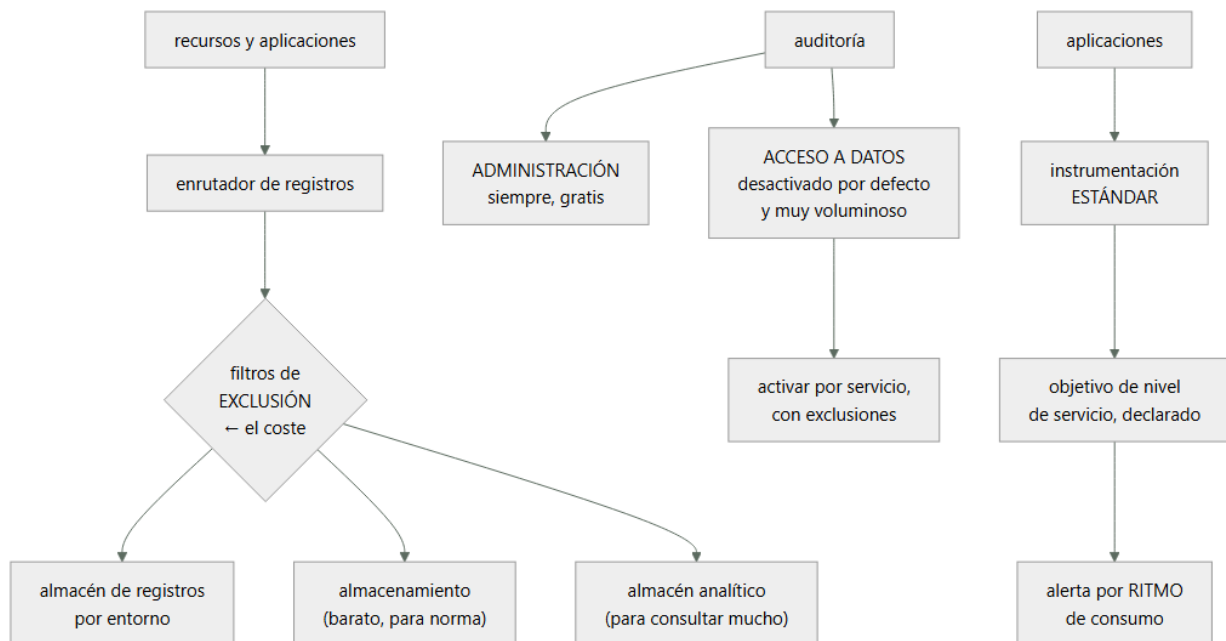
Concepto	Comprensión verificable
receptor de registros	Regla que enruta registros a un destino: almacén de registros, almacenamiento, almacén analítico o mensajería.
filtro de exclusión	Regla que descarta registros antes de almacenarlos. La palanca de coste más directa.
registro de auditoría de administración	Quién creó, modificó o borró qué. Siempre activo y sin coste.
registro de acceso a datos	Quién leyó o escribió datos. Desactivado por defecto y muy voluminoso.
objetivo de nivel de servicio	Recurso declarado con su indicador, su objetivo y su presupuesto de error.
alerta por ritmo de consumo	Se dispara cuando el presupuesto de error se gasta demasiado rápido. La que debe despertar.

Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Enrutar y excluir

El modelo aquí es un enrutador: todo pasa por él y se decide qué va a dónde.

LOS DESTINOS

almacén de registros	consultable, con retención
almacenamiento	barato, para conservar por norma
almacén analítico	para consultar mucho volumen
	clase 236
mensajería	para procesar en tiempo real

Y EL RECEPTOR

un filtro decide qué se envía a cada destino
 → y se pueden crear en la organización, para que apliquen a todos los proyectos clase 229

Los filtros de exclusión, que son la palanca de coste más directa:

se descartan ANTES de almacenar y no se pagan

lo que casi siempre se excluye
 comprobaciones de salud del balanceador
 peticiones correctas de servicios muy hablados, con muestreo parcial
 registros de nivel de información de componentes ruidosos y las lecturas de metadatos de los agentes

y con MUESTREO

«excluye el 95 % de las peticiones con código 200»
 → conserva una muestra y descarta el resto
 → y hay que tenerlo en cuenta al contar clase 225

Y el receptor a la organización, que resuelve un problema de gobierno:

un receptor en la organización envía los registros de AUDITORÍA de todos los proyectos a un destino centralizado en un proyecto de seguridad
 al que los equipos NO tienen acceso de escritura ni de borrado
 → y eso es lo que hace la auditoría creíble clase 141
 → sin ello, quien compromete un proyecto borra sus huellas

Y la retención, por destino:

almacén de registros de aplicación	30 días
almacén de auditoría	400 días

almacenamiento para conservación lo que exija la norma
→ y hay que fijarla: los valores por defecto no corresponden a nada ley 26

2. Los registros de auditoría

Esta es la pieza distintiva y la que hay que entender antes de activarla.

ADMINISTRACIÓN

quién creó, modificó o borró un recurso
SIEMPRE activo y sin coste
→ y es el que responde «¿quién cambió esto?»

ACCESO A DATOS

quién leyó o escribió datos
DESACTIVADO por defecto, salvo unos pocos ley 26
→ y es el que responde «¿quién leyó esta tabla?»
→ es decir, el que hace falta para investigar una fuga

y su volumen
cada lectura de cada objeto genera una entrada
→ en un almacén con mucho tráfico, es enorme
→ activarlo en todos los servicios dispara la factura

EVENTOS DE SISTEMA

acciones que hace la propia plataforma
siempre activo

DENEGACIÓN DE POLÍTICA

intentos rechazados por una política
→ muy útil y de poco volumen

Y la decisión que hay que tomar servicio por servicio:

¿ACTIVAR ACCESO A DATOS AQUÍ?

sí donde hay datos sensibles: almacenes con datos personales, conjuntos analíticos, gestor de secretos
sí en el gestor de claves, siempre
no en almacenes de artefactos, registros y datos públicos

y dentro de los activados

distinguir LECTURA de ESCRITURA
la escritura es poco voluminosa y muy valiosa
la lectura es lo que dispara el volumen
→ activar escritura en muchos, lectura en pocos

y excluir las identidades que generan ruido

los propios agentes, los procesos de copia
→ con una exclusión, no desactivando el servicio entero

Y la comprobación de que sirve:

«¿quién leyó la tabla de clientes en las últimas 24 h?»
→ si no se puede contestar, el registro de acceso a datos no está donde hace falta
→ y esa pregunta llega el día de un incidente, no antes clase 226

Y el acceso a los propios registros:

quien puede leer los registros de auditoría ve mucho
→ acceso restringido y auditado
→ y quien puede BORRARLOS o cambiar el enrutado, menos aún
→ con política de organización que lo impida clase 229

3. Instrumentar y declarar objetivos

La instrumentación, con la decisión de la clase 225:

ESTÁNDAR ABIERTO

trazas, métricas y registros con la API común
el destino es configuración
→ cambiar de proveedor de análisis no toca el código

→ y la misma instrumentación vale en las tres nubes
clase 158

y aquí hay un detalle a favor
los servicios gestionados ya emiten trazas y métricas
→ y el contexto se propaga si la aplicación lo respeta
→ una petición que cruza balanceador, servicio de
contenedores y base aparece como una sola traza

Y lo que hay que cuidar, que es lo de siempre:

el contexto de traza viaja también por la mensajería
→ dentro del mensaje clase 237
dimensiones de baja cardinalidad clase 211
registros estructurados con traza, servicio, versión y
entorno
y sin secretos ni datos personales completos

Los objetivos de nivel de servicio, que aquí son un recurso declarado:

se declara
el SERVICIO
el INDICADOR: disponibilidad o latencia, con su fuente
el OBJETIVO: 99,5 % en 28 días
→ y la plataforma calcula el presupuesto de error y su
consumo

y se puede declarar como código, junto al servicio
clase 232

Y LA ALERTA QUE IMPORTA
por RITMO DE CONSUMO del presupuesto
«se está gastando 14 veces más rápido de lo normal»
→ esa es la que despierta clase 211
→ y no «hubo un error»

con dos ventanas
corta, para lo que arde
larga, para la degradación lenta

Y la disciplina que este programa exige:

el objetivo se define por FLUJO DE USUARIO, no por
componente clase 123
«el 99,5 % de las confirmaciones de pedido en menos de
800 ms, medidas en el borde»
→ y medido donde el usuario lo sufre clase 126

Y las alertas que faltan siempre:

POR AUSENCIA
«este trabajo no se ha ejecutado en N minutos»
→ un trabajo programado que deja de dispararse no genera
error ley 13

POR ANTIGÜEDAD
mensajes sin confirmar, temas de fallidos, sincronización
certificados, retraso de replicación
clases 237, 234, 235

Y POR NEGOCIO
«pedidos confirmados por minuto por debajo del mínimo»
→ detecta lo que ninguna métrica técnica ve ley 15

4. Consultar, correlacionar y pagar

Las consultas preparadas, que ahorran minutos durante un incidente:

«todo lo de esta traza»
«errores de este servicio en la última hora, por tipo»
«qué cambió en los recursos en la última hora»
→ del registro de auditoría de administración
«quién accedió a estos datos»
→ del registro de acceso a datos
«peticiones más lentas y qué tramo domina»

→ guardadas y enlazadas desde el procedimiento de la alerta
clase 127

La cadena de diagnóstico, con el eslabón que aquí es directo:

alerta por ritmo de consumo del presupuesto
→ panel del servicio
→ trazas del periodo
→ registros de esas trazas
→ REGISTRO DE AUDITORÍA: ¿qué se cambió?

→ y ese último eslabón es el que suele faltar en otras plataformas y aquí está siempre activo y gratis

Y el perfilado, que resuelve una categoría de problemas:

el perfilador continuo muestra dónde se va el tiempo y la memoria en producción, con muestreo y coste bajo
→ y resuelve las preguntas que ni las trazas ni los registros contestan: «¿por qué esta función consume el 40 % de la CPU?»
→ conviene tenerlo activo antes de necesitarlo

El coste, con las palancas:

- 1 FILTROS DE EXCLUSIÓN ← la mayor
- 2 activar acceso a datos solo donde hace falta
- 3 retención por almacén, no global
- 4 destino según uso: consultable, barato o analítico
- 5 muestreo en la aplicación
- 6 y dimensiones de baja cardinalidad

y la revisión trimestral
¿cuánto cuesta la observabilidad frente al cómputo?
→ en la clase 211 costaba el doble
→ y aquí, con acceso a datos mal acotado, puede costar más
clase 239

Y la lista de comprobación de la clase:

- hay receptor en la organización para los registros de auditoría
- el destino de auditoría está en un proyecto sin acceso de escritura desde producción
- hay filtros de exclusión para lo ruidoso
- el acceso a datos está activado donde hay datos sensibles, y solo ahí
- se distingue lectura de escritura al activarlo
- la retención está fijada por almacén
- el acceso a los registros está restringido y auditado
- una política impide cambiar el enrutado de auditoría
- la instrumentación de las aplicaciones es estándar
- el contexto de traza viaja por la mensajería
- hay objetivos declarados por flujo de usuario
- hay alerta por ritmo de consumo, con dos ventanas
- hay alertas por ausencia, por antigüedad y de negocio
- las consultas de diagnóstico están guardadas
- el perfilador continuo está activo
- se puede contestar quién leyó unos datos concretos

Y el cierre que enlaza con la clase siguiente: con la observabilidad en pie, quedan la seguridad operativa y el coste, que aquí tienen herramientas propias y un mecanismo de perímetro que ya ha aparecido varias veces. Es la materia de la clase 239.

Ejemplo trabajado

CloudShop monta la observabilidad de su organización en Google Cloud. Lo que sigue es la activación del acceso a datos que multiplicó la factura por seis, la pregunta que no se podía contestar durante un incidente, y los objetivos declarados que sustituyeron a 71 alertas técnicas.

El punto de partida:

coste mensual de observabilidad	1.100 €
registros	740 €

métricas	240 €
trazas	120 €

registros de auditoría
de administración activos (gratis)
DE ACCESO A DATOS desactivados
destino el proyecto de cada equipo
→ cada equipo podía borrar sus propios registros de auditoría clase 141

El incidente que reveló el hueco.

seguridad recibió un aviso: un correo de un cliente aparecía en una lista de contactos de un tercero

la pregunta
«¿quién ha leído la tabla de clientes en los últimos 90 días?»

la respuesta
no se podía contestar
el registro de acceso a datos estaba desactivado
el de administración decía quién había concedido permisos, no quién había leído

lo que se pudo hacer
reconstruir con los registros de consulta del almacén analítico, que sí guardaban las consultas
→ 41 identidades habían consultado la tabla
→ y de ellas, 9 no debían tener acceso clase 236

tiempo de investigación 3 días
y la conclusión
la respuesta se obtuvo por casualidad, porque el almacén analítico registra sus consultas
→ sobre el almacén de objetos no habría habido nada

La activación, y la factura.

primer intento
se activó el acceso a datos en TODOS los servicios, con lectura y escritura

factura del mes siguiente	
registros	740 € → 6.400 €

desglose	
almacén de objetos, lecturas	4.100 €
→ cada descarga de una imagen del catálogo generaba una entrada; 41 M al día	
almacén analítico, lecturas	980 €
bases de datos, lecturas	710 €
resto	610 €

Y la corrección, servicio por servicio:

servicio	lectura	escritura	motivo
almacén con datos de clientes	SÍ	SÍ	datos personales
almacén de imágenes del catálogo	no	SÍ	público
almacén analítico	SÍ	SÍ	datos personales
gestor de secretos	SÍ	SÍ	siempre
gestor de claves	SÍ	SÍ	siempre
base de pedidos	no	SÍ	volumen; y el acceso va por la API
almacén de artefactos	no	SÍ	no sensible
registros	no	SÍ	

y las exclusiones dentro de los activados
identidades de los agentes de copia
identidad del proceso de replicación

→ ninguna había detectado un problema que los objetivos no detectaran antes

La prueba de las alertas:

condición provocada en las 45
no llegaron 7
4 por umbral mal puesto
2 por destino sin destinatarios
1 porque el objetivo estaba mal declarado: el
indicador medía en el servidor, no en el borde
clase 126, ley 22

El perfilador, que resolvió algo que las trazas no:

síntoma el servicio de búsqueda consumía el doble de CPU
que hacía tres meses, con el mismo tráfico
las trazas no mostraban ningún tramo lento

el perfilador continuo
el 41 % de la CPU en una función de normalización de
texto
→ un cambio de tres meses antes había añadido una
expresión regular compilada EN CADA LLAMADA

corrección compilar una vez
CPU -38 %
instancias necesarias 9 → 6

El resultado:

coste de observabilidad	1.100 €	1.640 €
(con auditoría de datos activada donde hace falta)		
frente a 6.400 € con todo activado		
volumen de registros	1,9 TB	450 GB
«¿quién leyó estos datos?»	incontestable	40 s
registros de auditoría borrables por		
los equipos	sí	no
alertas configuradas	98	45
accionables	61 (8 %)	61 (90 %)
objetivos declarados	0	6
alertas que no llegaban	n/d	0 (de 7 corregidas)

La lección que esta clase deja: activar el registro de acceso a datos en todos los servicios multiplicó la factura por seis, y acotarlo a donde hay datos sensibles la dejó en menos de la cuarta parte sin perder la capacidad que importaba. La pregunta que un incidente hizo —quién leyó una tabla— no se podía contestar, y solo se respondió por casualidad porque el almacén analítico registra sus consultas. Y setenta y una alertas técnicas se retiraron sin perder nada: ninguna había detectado un problema antes que los objetivos.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/238-cloud-operations-trace-y-opentelemetry/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa gcp-observability en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye gcp-observability para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
No se puede saber quién leyó unos datos durante un incidente	El registro de acceso a datos está desactivado por defecto	Actívalo en los servicios con datos sensibles, distinguiendo lectura de escritura, y guarda la consulta que responde esa pregunta.
La factura de registros se multiplica al activar la auditoría de datos	Se activó la lectura en todos los servicios, incluidos los de mucho volumen y datos públicos	Acota por servicio, activa escritura en muchos y lectura en pocos, y excluye las identidades de agentes y procesos de copia.
Un equipo puede borrar los registros que lo incriminan	La auditoría se guarda en el proyecto del propio equipo	Receptor en la organización hacia un proyecto de seguridad sin escritura desde producción, y política que impida modificarlo.
Hay decenas de alertas y ninguna dice si el usuario está sufriendo	Son umbrales técnicos y no objetivos por flujo de usuario	Declara objetivos por flujo, alerta por ritmo de consumo del presupuesto y retira las técnicas que no aporten.
Un objetivo declarado no detecta un problema real	El indicador se mide en el servidor y no donde el usuario lo sufre	Mide en el borde y comprueba el objetivo provocando la condición.
Un servicio consume más CPU y las trazas no muestran nada	El coste está dentro del proceso, no en las llamadas	Activa el perfilador continuo antes de necesitarlo; muestra dónde se va el tiempo en producción.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué diferencia hay entre el registro de administración y el de acceso a datos?
2. ¿Por qué el destino de la auditoría debe estar fuera del alcance de producción?
3. ¿Cuál es la palanca de coste más directa en el enrutado de registros?
4. ¿Qué alerta debe despertar a alguien y por qué no el error suelto?

5. ¿Qué resuelve el perfilador continuo que no resuelven trazas ni registros?

Referencias

- Google Cloud (2025). Cloud Logging: routing and storage. <<https://cloud.google.com/logging/docs/routing/overview>>
- Google Cloud (2025). Cloud Audit Logs. <<https://cloud.google.com/logging/docs/audit>>
- Google Cloud (2025). Service monitoring: SLOs and burn rate alerts. <<https://cloud.google.com/stackdriver/docs/solutions/slo-monitoring>>
- Google Cloud (2025). Cloud Trace and OpenTelemetry. <<https://cloud.google.com/trace/docs/setup>>
- Google Cloud (2025). Cloud Profiler. <<https://cloud.google.com/profiler/docs/about-profiler>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 19 en PDF · Recorrido de Google Cloud en PDF · Manual integral

Anterior	Índice	Siguiente
← 237 · Pub/Sub, Eventarc y entrega exactamente-una-vez	Parte 19 · Programa	239 · SCC, VPC Service Controls, KMS y FinOps →

Evaluación — 238 Cloud Operations, Trace y OpenTelemetry

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega gcp-observability con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

239 — SCC, VPC Service Controls, KMS y FinOps

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: security · Estado: EXECUTABLECORE

Propósito

Cerrar la operación de Google Cloud con la seguridad y el coste, que aquí tienen dos piezas propias: el perímetro de servicio, que es el control que impide sacar datos aunque los permisos lo permitan, y el modelo de descuentos automáticos, que cambia el cálculo de los compromisos. La clase cubre la postura y la detección con la disciplina de la clase 226, y el control de coste con la de la 214.

Resultados de aprendizaje

Al finalizar podrás:

1. Priorizar los hallazgos de seguridad por alcance y por camino de ataque.
2. Aplicar perímetros de servicio y entender qué cierran.
3. Gestionar claves de cifrado propias donde aportan.
4. Atribuir el coste y aprovechar los descuentos automáticos.
5. Comprometer capacidad solo tras retirar y redimensionar.

Conceptos centrales

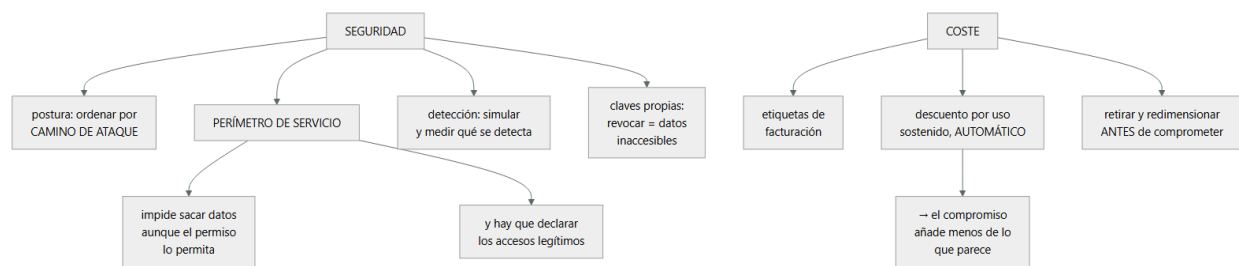
Concepto	Comprensión verificable
perímetro de servicio	Frontera que restringe qué API pueden usarse hacia dentro y hacia fuera de un grupo de proyectos.
camino de ataque	Secuencia de pasos desde un punto expuesto hasta un dato valioso. Ordena la prioridad.
clave gestionada por el cliente	Clave de cifrado bajo control propio. Permite revocar el acceso a los datos por completo.
descuento por uso sostenido	Rebaja automática por mantener recursos encendidos, sin compromiso.
compromiso de uso	Descuento a cambio de un compromiso de gasto o de capacidad durante uno o tres años.
etiqueta de facturación	Etiqueta que aparece en el desglose de costes. Distinta de las que se usan en condiciones de permiso.

Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Postura y caminos de ataque

La disciplina es la de la clase 226 y aquí hay una herramienta que la hace directa.

LO QUE PRODUCE LA EVALUACIÓN CONTINUA
 hallazgos de configuración: público, sin cifrar, sin copias, versión antigua
 hallazgos de identidad: permisos amplios, claves
 hallazgos de vulnerabilidades

Y LO QUE ORDENA EL TRABAJO
 no la gravedad declarada
 sino los CAMINOS DE ATAQUE
 «desde esta máquina expuesta, con esta cuenta de servicio, se llega a este conjunto con datos de clientes»
 → con la puntuación de exposición calculada

→ y esos caminos concentran más riesgo que cientos de hallazgos sueltos clase 226

Y el orden de trabajo, ya conocido:

- 1 cerrar los caminos que terminan en datos
- 2 reducir el alcance de las identidades más amplias clase 230
- 3 impedir por política que lo arreglado vuelva clase 229
- 4 remediar el resto por lotes ley 27
- 5 aceptar por escrito lo que no compensa clase 189

Y los hallazgos que se repiten en esta nube:

cuentas de servicio con papel de proyecto clase 230
 claves de cuenta de servicio sin caducidad
 almacenes con acceso a «todos los usuarios autenticados»
 → que significa cualquier cuenta de Google, no solo la organización
 → y es el hallazgo que más sorprende
 reglas de cortafuegos con selector por etiqueta clase 229
 conjuntos analíticos con acceso al conjunto entero clase 236
 y proyectos sin carpeta, que no reciben políticas

Y la comprobación de la detección:

simular técnicas conocidas y medir qué proporción se detecta
 → es la única medida real clase 226
 → y las que no se detecten son la lista de trabajo

2. El perímetro de servicio

Este es el control propio de esta nube y el que cierra el hueco que dejan los permisos.

EL PROBLEMA QUE RESUELVE
 una identidad con permiso legítimo sobre unos datos puede copiarlos a un proyecto de otra organización
 → el permiso lo autoriza
 → la red no lo impide, porque va por la API de la plataforma clase 200, 231

EL PERÍMETRO

agrupa proyectos y restringe qué API pueden usarse
desde fuera hacia dentro ← quién puede leer
desde dentro hacia fuera ← a dónde se puede escribir
→ y una identidad de dentro NO puede escribir en un
recurso de fuera del perímetro

Y lo que hay que hacer para que funcione sin romper nada:

- 1 DEFINIR el perímetro: qué proyectos y qué servicios
- 2 MODO DE SIMULACIÓN: registra lo que bloquearía
- 3 MEDIR semanas
- 4 DECLARAR los accesos legítimos que cruzan
reglas de entrada y de salida, por identidad, por
proyecto y por servicio
- 5 APLICAR

→ activar sin simular corta integraciones legítimas
→ y es la misma disciplina de las clases 200, 209 y 217

Y lo que suele aparecer en la simulación:

herramientas de terceros que leen datos
canalizaciones de otro proyecto
copias hacia un proyecto de análisis
el propio equipo consultando desde su portátil
→ y eso obliga a decidir: ¿se declara o se prohíbe?

Las claves gestionadas por el cliente, con lo que aportan de verdad:

QUÉ APORTAN

el control de la clave: revocarla hace los datos
inaccesibles, incluso para el proveedor
→ y eso es lo que exigen algunos requisitos
clase 177
rotación bajo control propio
y registro de cada uso de la clave
clase 238

QUÉ CUESTAN

operación: rotación, permisos, disponibilidad de la clave
y si la clave se pierde o se revoca por error, los datos
son irre recuperables
→ esa es una consecuencia real, no teórica

DÓNDE APLICARLAS

donde haya requisito o datos sensibles
no en todo
ley 26

Y una advertencia operativa:

la clave vive en una región
→ si esa región no está disponible, los datos cifrados con
ella tampoco
→ y eso entra en el cálculo del techo de disponibilidad
clase 185

3. Coste: los descuentos que ya se aplican

Aquí hay una diferencia que cambia el cálculo de los compromisos.

DESCUENTO POR USO SOSTENIDO

se aplica AUTOMÁTICAMENTE por mantener recursos
encendidos durante el mes
→ sin compromiso, sin contratar nada
→ y llega a ser apreciable

Y LA CONSECUENCIA

el compromiso de uso añade descuento SOBRE lo ya
descontado
→ el ahorro incremental es menor de lo que sugiere el
porcentaje anunciado
→ y hay que calcularlo con las cifras propias, no con el
porcentaje del catálogo
clase 227

Y los tipos de compromiso:

POR GASTO

- se compromete una cantidad por hora
- + flexible: se aplica a lo que haya
- descuento menor

POR RECURSO

- se compromete una cantidad de CPU y memoria en una región
- + descuento mayor
- rígido: si se cambia de región o de familia, se pierde

→ y el orden de la clase 227 sigue valiendo
retirar → redimensionar → apagar → y ENTONCES comprometer

La atribución, con la particularidad de las etiquetas:

HAY DOS MECANISMOS DE ETIQUETADO

- uno aparece en el desglose de FACTURACIÓN
- otro sirve para CONDICIONES de permiso y para políticas
clases 229, 230

- y no son intercambiables
- usar el equivocado hace que el gasto no se atribuya

Y LA JERARQUÍA AYUDA

- el desglose por PROYECTO es directo y limpio
- y por eso la separación por proyecto de la clase 229
es también una decisión de atribución

Y el resto del método, que es el de la clase 214:

- etiquetas obligatorias en la creación, por política
- presupuestos por proyecto, sobre previsión, con acción en
no producción
- detección de anomalías por servicio y por dueño
- retirada automática de lo ocioso: apagar antes de borrar
- y coste por unidad de negocio

Y las partidas que sorprenden en esta nube:

consultas del almacén analítico	clase 236
registros de acceso a datos	clase 238
almacenamiento de mensajes sin confirmar	clase 237
tráfico entre zonas y entre regiones	
mínimos de capacidad de las bases distribuidas	clase 235
y los proyectos sin uso que nadie retira	ley 25

4. El ritmo, y lo que se degrada solo

Coste y seguridad comparten la propiedad de degradarse sin intervención, y esta clase cierra la parte con el mismo calendario de la 227.

SEMANAL

- anomalías de coste abiertas
- hallazgos críticos nuevos
- recursos ociosos retirados

MENSUAL

- coste por unidad de negocio y su tendencia
- cobertura y utilización de compromisos
- proyectos sin uso
- y hallazgos por camino de ataque

TRIMESTRAL

- simulación de técnicas y proporción detectada clase 226
- prueba del acceso de emergencia clase 230
- revisión de accesos y de exenciones
- y un experimento de resiliencia clase 227

ANUAL

- ejercicio de pérdida de región clase 215
- revisión de las decisiones registradas clase 190

Y las señales que dicen si está sano:

SEGURIDAD

- caminos de ataque que terminan en datos → cero
- proporción de técnicas simuladas detectadas
- claves de cuenta de servicio → cero

proyectos sin carpeta → cero
y perímetros con reglas de excepción sin fecha → cero

COSTE

gasto atribuido	> 90 %
coste por unidad de negocio	tendencia
utilización de compromisos	> 95 %
proyectos sin uso	tendencia a cero

Y una comprobación honesta, la misma de la clase 226:

¿cuántos incidentes detectó el centro y cuántos se detectaron por otra vía?

→ y las vías de siempre: una factura, un tercero, un inventario, una persona

Y la lista de comprobación de la clase:

- los hallazgos se ordenan por camino de ataque
- no hay caminos que terminen en datos sensibles
- no hay almacenes con acceso a todos los usuarios autenticados
- hay perímetros de servicio sobre los proyectos con datos
- los perímetros se simulaban antes de aplicar
- las reglas de excepción del perímetro tienen dueño y fecha
- las claves propias están donde hay requisito, y su región entra en el cálculo de disponibilidad
- las etiquetas de facturación son las correctas
- el gasto atribuido supera el 90 %
- se retiró y redimensionó antes de comprometer
- el descuento incremental del compromiso se calculó sobre el ya aplicado
- hay presupuestos con acción y detección de anomalías
- se simulan técnicas y se publica la proporción detectada
- existe el calendario semanal, mensual, trimestral y anual

Y el cierre que enlaza con la clase siguiente: con todo lo de esta parte montado, queda ponerlo junto en un sistema productivo, compararlo con las otras dos nubes y cerrar la parte. Es la materia de la clase 240.

Ejemplo trabajado

CloudShop cierra la operación de Google Cloud. Lo que sigue son los almacenes accesibles por cualquier cuenta de Google, el perímetro que cortó una integración legítima, y el cálculo de compromiso que resultó dar mucho menos de lo anunciado.

La postura, ordenada por camino de ataque:

hallazgos abiertos 2.140
puntuación de exposición: los 5 caminos principales

- 1 máquina con IP externa y puerto de administración abierto
→ su cuenta de servicio tiene papel de proyecto
→ alcanza el almacén de copias y la base de pedidos
pasos: 3 · datos al final: 2,3 M de registros
- 2 almacén de exportaciones con acceso a «todos los usuarios autenticados»
→ cualquier cuenta de Google del mundo podía listarlo y descargarlo
→ contenía extractos diarios de pedidos desde 2023
pasos: 1 · datos: 41 meses de extractos
- 3 cuenta de servicio de la canalización con papel de organización clase 230
- 4 conjunto analítico con acceso al conjunto entero para 41 personas clase 236
- 5 clave de cuenta de servicio de un consultor externo, activa desde 2023 ley 25

Y el segundo merece detalle:

«todos los usuarios autenticados» NO significa «todos los de la organización»

significa cualquiera con una cuenta de Google

→ el equipo lo había configurado creyendo lo primero

→ y la política de organización que lo impide existía desde la clase 229, pero el almacén se había creado ANTES ley 27

cuánto llevaba así 19 meses

accesos externos registrados no se sabía

→ el registro de acceso a datos no estaba activo en ese almacén clase 238

→ se activó, y en 30 días: 0 accesos externos

→ lo cual no dice nada de los 19 meses anteriores

El perímetro de servicio, con su simulación.

definición

proyectos: los 14 de producción con datos

servicios restringidos: almacenamiento, almacén

analítico, bases, gestor de secretos y claves

modo de simulación, 4 semanas

accesos que se bloquearían 41.200

legítimos 610

del propio perímetro 40.400

NO IDENTIFICADOS 190 ←

los 610 legítimos

- canalización de despliegue desde el proyecto de plataforma

- herramienta de visualización de un tercero

- exportación nocturna al proyecto de análisis

- 4 personas consultando desde su portátil

los 190 no identificados

- 140 de una cuenta de servicio de un proyecto que se creía retirado en 2024 ley 25

- 38 de una herramienta de un proveedor cuyo contrato había terminado

- 12 de una dirección que resultó ser un entorno de pruebas de un socio, no declarado

→ los 190 se cortaron a propósito

→ y los 610 se declararon como reglas de entrada y salida, con dueño y fecha de revisión

Y el corte que ocurrió igualmente:

al aplicar, una integración se rompió

el equipo de finanzas exportaba un informe mensual desde una hoja de cálculo conectada

→ esa conexión no había aparecido en las 4 semanas de simulación porque es MENSUAL y cayó fuera de la ventana

→ duración del corte 2 días

→ corrección: regla declarada

→ y lección: la ventana de simulación debe cubrir el ciclo completo de negocio clase 167

Las claves propias, decididas por caso:

dónde se aplicaron

almacén con datos de clientes

conjunto analítico con datos personales

copias de seguridad

→ los tres con requisito contractual del mayor cliente

dónde NO

almacenes de imágenes y artefactos

registros de aplicación

→ sin requisito y con coste de operación

y el efecto sobre disponibilidad, calculado
la clave vive en una región
→ se configuró con réplica en la segunda región
→ sin eso, el techo del flujo habría bajado clase 185

y la prueba negativa
revocar el acceso a la clave en preproducción
→ los datos pasaron a ser inaccesibles, correctamente
→ y la restauración tardó 11 minutos
→ procedimiento escrito y probado ley 22

El cálculo de compromiso, que dio menos de lo anunciado.

gasto de cómputo 8.400 €/mes

el descuento por uso sostenido YA aplicado
precio de lista 11.900 €
descuento automático -3.500 €
gasto real 8.400 €

la oferta de compromiso a 1 año
descuento anunciado sobre lista 37 %
→ parecía un ahorro de 4.400 €/mes

el cálculo real
con compromiso, sobre lista 7.500 €
ahorro frente a los 8.400 actuales 900 €/mes
→ no 4.400 €

→ porque el descuento automático ya estaba aplicado
→ y comparar con el precio de lista es el error
clase 227

Y el orden que se siguió antes de comprometer:

- 1 RETIRAR 71 proyectos sin actividad clase 229
-3.100 €/mes
- 2 REDIMENSIONAR peticiones de recursos del clúster
clase 234
-1.760 €/mes
- 3 APAGAR entornos no productivos por horario
-1.900 €/mes
- 4 y ENTONCES comprometer, sobre la base estable
-610 €/mes

→ y el compromiso se hizo POR GASTO, no por recurso
→ motivo: el clúster iba a cambiar de familia

El coste, atribuido:

al empezar
gasto atribuido 64 %
y el 36 % restante
recursos con el tipo de etiqueta EQUIVOCADO 2.100 €
→ se habían puesto las de condiciones de permiso,
que no aparecen en el desglose
servicios compartidos 1.900 €
tráfico entre zonas 1.400 €

tras corregir el tipo de etiqueta y repartir lo compartido
gasto atribuido 95 %

La detección, medida:

simulación de 14 técnicas
detectadas la primera vez 9 de 14
las 5 que no
• exfiltración al almacén de otra organización
→ la detectó el PERÍMETRO, no la regla
→ y se aceptó así
• suplantación de una cuenta privilegiada
→ regla escrita
• creación de una clave de cuenta de servicio

- la impide la política; regla añadida igualmente
- acceso a datos desde una identidad inusual
 - dependía del registro de acceso a datos, ahora activo
- enumeración de proyectos
 - umbral corregido

segunda simulación

13 de 14

El resultado:

caminos de ataque que terminan en datos	5	0
almacenes accesibles por cualquier cuenta	3	0
perímetros de servicio	0	2
accesos no identificados que cruzaban	190	0
claves de cuenta de servicio	12	3
gasto atribuido	64 %	95 %
coste mensual	21.400 €	14.030 €
utilización del compromiso	—	97 %
técnicas simuladas detectadas	9/14	13/14

La lección que esta clase deja: un almacén con extractos de pedidos de cuarenta y un meses era descargable por cualquier persona con una cuenta de Google, porque «todos los usuarios autenticados» no significa lo que parece, y la política que lo impedía se creó después que el almacén. Y el compromiso que prometía un 37 % de descuento daba novecientos euros al mes y no cuatro mil cuatrocientos, porque el descuento automático ya estaba aplicado: comparar con el precio de lista es el error.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/239-scc-vpc-service-controls-kms-y-finops/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa gcp-security-finops en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye gcp-security-finops para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Un almacén resulta accesible desde fuera de la organización	Se concedió a «todos los usuarios autenticados», que significa cualquier cuenta del proveedor	Prohíbelo por política, revisa lo existente porque la política no actúa hacia atrás, y activa el registro de acceso a datos allí.
Una identidad legítima copia datos a otra organización	El permiso lo autoriza y la red no lo impide porque va por la API	Aplica perímetros de servicio sobre los proyectos con datos, simulando antes y declarando los accesos legítimos.
Aplicar el perímetro corta una integración que no apareció en la simulación	La ventana de simulación no cubrió un proceso mensual	Simula durante un ciclo completo de negocio antes de aplicar.
El ahorro del compromiso es mucho menor de lo anunciado	Se comparó con el precio de lista y el descuento automático ya estaba aplicado	Calcula el ahorro incremental sobre el gasto real actual, no sobre la tarifa de catálogo.
Parte del gasto no aparece atribuido pese a etiquetar	Se usó el tipo de etiqueta que sirve para condiciones de permiso y no el de facturación	Comprueba qué mecanismo aparece en el desglose y usa ese; los dos no son intercambiables.
Los datos cifrados con clave propia dejan de estar disponibles	La clave vive en una región que no está disponible	Replica la clave y cuenta su disponibilidad en el cálculo del techo del flujo.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué ordena la prioridad de los hallazgos de seguridad?
2. ¿Qué cierra un perímetro de servicio que no cierra ni el permiso ni el cortafuegos?
3. ¿Qué hay que comprobar antes de aplicar un perímetro y durante cuánto tiempo?
4. ¿Por qué el descuento por compromiso da menos de lo que anuncia el porcentaje?
5. ¿Qué efecto tiene sobre la disponibilidad usar claves de cifrado propias?

Referencias

- Google Cloud (2025). Security Command Center: attack path simulation. <<https://cloud.google.com/security-command-center/docs/attack-exposure-learn>>
- Google Cloud (2025). VPC Service Controls: dry run mode. <<https://cloud.google.com/vpc-service-controls/docs/dry-run-mode>>
- Google Cloud (2025). Customer-managed encryption keys. <<https://cloud.google.com/kms/docs/cmek>>
- Google Cloud (2025). Sustained use and committed use discounts. <<https://cloud.google.com/compute/docs/sustained-use-discounts>>
- Google Cloud (2025). Billing labels and cost breakdown. <<https://cloud.google.com/billing/docs/how-to/bq-examples>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 19 en PDF · Recorrido de Google Cloud en PDF · Manual integral

Anterior	Índice	Siguiente
← 238 · Cloud Operations, Trace y OpenTelemetry	Parte 19 · Programa	240 · Proyecto: CloudShop productivo en Google Cloud →

Evaluación — 239 SCC, VPC Service Controls, KMS y FinOps

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega gcp-security-finops con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

240 — Proyecto: CloudShop productivo en Google Cloud

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 8
Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Poner en producción el sistema completo de CloudShop en Google Cloud, compararlo con las otras dos nubes y cerrar la parte 19. La clase da el orden, el entregable y las pruebas negativas; corrige las cinco predicciones de la clase 228 —tres acertadas, una incompleta y una fallada—, actualiza el recuento de leyes, añade la ley 28 y escribe la hipótesis de la parte 20.

Resultados de aprendizaje

Al finalizar podrás:

1. Construir el sistema completo en el orden que evita rehacer.
2. Comprobar con las pruebas negativas de toda la parte.
3. Comparar las tres nubes con cifras y sacar conclusiones defendibles.
4. Corregir las cinco predicciones de la clase 228 con evidencia.
5. Escribir la hipótesis de la parte 20 en forma refutable.

Conceptos centrales

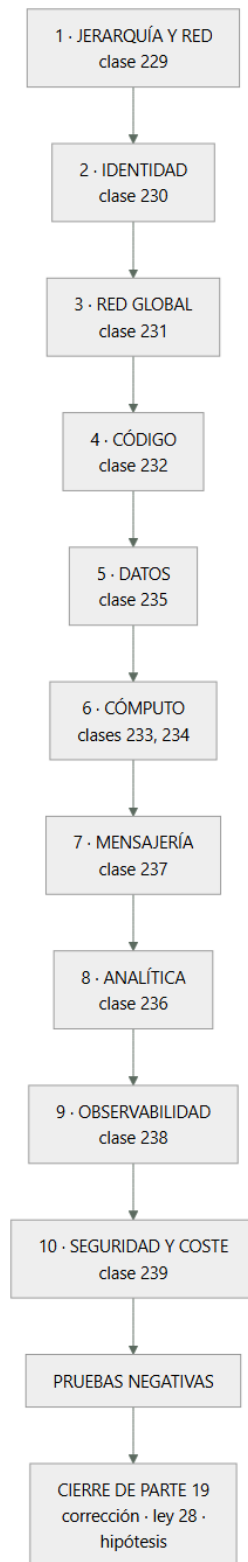
Concepto	Comprensión verificable
ley 28	Donde se paga por uso, cada valor por defecto es una factura; y nadie lo revisa porque no parece una decisión.
error de traslado	Fallo que viene de dar por válida una suposición de otra nube. La causa dominante de esta parte.
coste de operar varias nubes	Suma de modelos de gobierno, permisos, red, observabilidad y calendarios. No es proporcional al número.
equivalencia conceptual	Correspondencia de ideas entre proveedores. Alta; la operativa es la que difiere.
prueba negativa de parte	Comprobación acumulada de las once clases, ejecutada sobre el sistema entero.
hipótesis de parte	Afirmación refutable escrita antes de estudiar, que la parte siguiente corrige con evidencia.

Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. El encargo, el orden y las pruebas

El encargo. Llevar a producción la plataforma de pedidos de CloudShop en Google Cloud, con usuarios reales, guardia, presupuesto y continuidad comprobada.

El orden, por coste de cambio:

- 1 JERARQUÍA Y RED COMPARTIDA clase 229
carpetas por entorno, proyectos por carga
políticas de organización en auditoría antes de aplicar
→ y con plan de remediación de lo existente ley 27
- 2 IDENTIDAD clase 230
sin claves de cuenta de servicio
asignaciones en el ámbito del recurso, con condiciones
- 3 RED GLOBAL clase 231
una red por entorno, salida cerrada, zonas privadas
asociadas en el proyecto anfitrión
- 4 CÓDIGO clase 232
estado protegido, validación de políticas y coste antes
de aplicar
- 5 DATOS clase 235
familia por patrones y por coste base; clave que reparte
- 6 CÓMPUTO clases 233, 234
servicio de contenedores por defecto; clúster donde hace
falta
- 7 MENSAJERÍA clase 237
un tema, una suscripción por consumidor, idempotencia
- 8 ANALÍTICA clase 236
particionado, límites de consulta y acceso por columna
- 9 OBSERVABILIDAD clase 238
instrumentación estándar, objetivos declarados,
auditoría acotada
- 10 SEGURIDAD Y COSTE clase 239
perímetros, caminos de ataque, compromisos tras retirar

Las pruebas negativas de la parte:

- crear un recurso sin etiquetas obligatorias
- crear una clave de cuenta de servicio
- suplantar una cuenta sin permiso
- acceder fuera de una condición de recurso o de tiempo
- activar un papel privilegiado sin aprobación
- entrar con el acceso de emergencia
- resolver un nombre con conexión privada desde cada
proyecto
- sacar datos a un proyecto de otra organización
- desplegar algo que incumple una política
- borrar el estado y restaurarlo
- ejecutar el plan sin cambios y comprobar que sale vacío
- escribir con clave secuencial y observar el reparto
- consultar una tabla grande sin filtro de partición
- consultar una columna sensible sin la etiqueta
- enviar el mismo mensaje 50 veces
- dejar un mensaje en el tema de fallidos y esperar alerta
- obtener la identidad desde otro espacio de nombres
- perder una región y cronometrar
- provocar la condición de cada alerta
- desplegar el entorno de cero en un proyecto vacío

Y los criterios de evaluación, con el peso donde importa:

- 1 ningún proyecto sin carpeta ni sin dueño 2
- 2 cero claves de cuenta de servicio sin excepción 3
- 3 el alcance de cada identidad está medido 3
- 4 las zonas privadas resuelven desde todos los proyectos 3
- 5 hay perímetro de servicio sobre los proyectos con datos 3
- 6 la clave de las tablas distribuidas reparte 2
- 7 hay límite de bytes por consulta 3
- 8 el acceso a datos personales es por columna 3
- 9 la instrumentación es estándar 2
- 10 hay objetivos declarados con alerta por ritmo 2
- 11 la lista de valores por defecto cambiados es explícita 3

2. Cierre de la parte 19: corrección de las cinco predicciones

Las cinco predicciones de la clase 228, corregidas con la evidencia de las clases 229 a 239.

1. «la tercera nube confirmará la equivalencia conceptual y diferirá en lo operativo; y lo que más costará no será aprender lo distinto sino DESAPRENDER lo de las dos anteriores»

CORRECTA, y con cinco casos documentados. Crear una red por proyecto; asumir que las reglas de cortafuegos son por subred; confundir la barrera con el permiso; poner la concurrencia en 1 por trasladar el modelo de funciones; y usar una clave autoincremental en un almacén repartido por rangos. El más puro fue el de las zonas privadas: el equipo HABÍA SUFRIDO ese problema en la nube anterior, lo conocía, y lo repitió porque el sitio donde se asocia la zona es distinto.

2. «la jerarquía de proyectos y la red compartida volverán a condicionar todo, y el error frecuente será el CONTRARIO: demasiados proyectos»

CORRECTA y literal. 214 proyectos, 189 colgando de la organización sin carpeta, 71 sin actividad en 90 días, 120 sin dueño. Y uno de los inactivos contenía una copia de la tabla de clientes legible por cualquier cuenta, desde hacía 19 meses.

3. «la identidad volverá a ser el eje y el error más frecuente volverá a ser el ámbito amplio»

CORRECTA E INCOMPLETA, y lo que falta es lo más útil. El ámbito amplio estaba: 18 asignaciones en la organización y 140 en carpetas. Pero el hallazgo mayor fue otro: eliminar 341 claves de cuenta de servicio costó cuatro meses y NO redujo el alcance ni un punto. Quitar credenciales y reducir alcance son dos trabajos distintos, y confundirlos hace creer que el primero resuelve el segundo. Esa distinción no estaba en la predicción.

4. «los valores por defecto volverán a estar mal elegidos, y aquí fallarán sobre todo en RED»

FALLADA en la segunda mitad, y el patrón del fallo importa. Los valores por defecto de red estaban mal –la salida permitida, el selector por etiqueta–, pero los que costaron dinero de verdad fueron otros: consultar sin filtro de partición y con todas las columnas, 4.100 €/mes; activar la auditoría de acceso a datos en todo, *6 la factura de registros; el indexado automático de documentos; la concurrencia; y una suscripción abandonada reteniendo 41 millones de mensajes. Es la SEGUNDA VEZ que fallamos en dónde fallarán los valores por defecto: en la clase 228 predijimos identidad para Azure y era coste y observabilidad; aquí predijimos red y era analítica y auditoría. El patrón que no habíamos visto es que los valores por defecto caros están donde se factura por uso, no donde la nube es característica.

5. «los problemas serán otra vez de las leyes 25, 15 y 22. Lo refutable: la proporción de hallazgos que detecte una comprobación automática y periódica será MAYOR que en las partes 17 y 18»

CORRECTA, y medible. En esta parte, las comprobaciones automáticas detectaron: 4 proyectos con zonas sin asociar, 23 casos de deriva, 74 despliegues que incumplían política, 190 accesos no identificados que

cruzaban el perímetro, 5 caminos de ataque y 5 técnicas no detectadas. Frente a las partes anteriores, donde casi todo lo grave lo encontraron inventarios o facturas. PERO el matiz cualitativo se mantiene: los tres hallazgos más graves de esta parte –el almacén público de 19 meses, la suscripción de 41 millones de mensajes y las 41 personas con acceso a datos personales– los encontraron un inventario, una factura y otro inventario. Las comprobaciones detectan más cosas; las peores siguen apareciendo al mirar.

Marcador: tres correctas, una incompleta, una fallada.

3. Recuento de leyes, ley 28 e hipótesis de la parte 20

El recuento de leyes, cerrada la parte 19.

ley 13	lo que no se mira deja de funcionar en silencio	55
ley 15	la señal existe y nadie la mira	44
ley 22	un procedimiento nunca ejecutado no funciona	39
ley 14	el coste se decide al crear, no al pagar	34
ley 16	un control que estorba se rodea	32
ley 20	lo que no tiene dueño se filtra y se desperdicia	31
ley 21	el acoplamiento vive en quién escribe	25
ley 25	lo provisional sobrevive a su motivo	24
ley 26	el valor por defecto sirve a la demostración	19
ley 23	la capacidad la limita lo que ya se mantiene	19
ley 24	lo que no está en el diagrama no se analiza	13
ley 17	se optimiza la medida, no el objetivo	13
ley 19	la compensación hace invisible el fallo	10
ley 27	un control solo actúa sobre lo que cambia	9
ley 18	lo asíncrono traslada la garantía, no la elimina	9

Y la parte 19 obliga a escribir la ley que explica el fallo de la predicción 4:

LEY 28

donde se paga por uso, cada valor por defecto es una factura; y nadie lo revisa porque no parece una decisión

apariciones en esta parte	5
clase 236	consultar sin filtro de partición y con todas las columnas: 4.100 €/mes en tres paneles
clase 238	auditoría de acceso a datos activada en todo: la factura de registros x6
clase 237	una suscripción sin consumidor reteniendo 41 M de mensajes: 1.520 €/mes de almacenamiento
clase 235	indexado automático de todos los campos de un documento
clase 233	conurrencia 1 por instancia: 2.900 €/mes

y lo que la distingue de la ley 26

la 26 dice que el valor inicial está elegido para que la demostración funcione

la 28 dice DÓNDE duele: en los servicios que facturan por uso, un valor por defecto no es una configuración: es una decisión de gasto continuo

→ y por eso pasa desapercibida: nadie revisa una casilla de un asistente pensando en la factura del año que viene

ley 14

La hipótesis de la parte 20 (clases 241 a 252, plataformas de datos e IA), escrita antes de estudiarla:

1. la parte 20 volverá a demostrar lo de la clase 175: en las cargas de datos e IA el problema dominante no será el modelo ni el cómputo, será el DATO –su origen, su permiso, su forma y quién lo escribe–; y las leyes dominantes serán la 21 y la 14
2. de todo lo que se monte, lo que más problemas producirá no será entrenar ni servir modelos, sino la ORQUESTACIÓN y la calidad: trabajos que fallan sin que nadie los mire y datos que llegan mal sin que nada lo detecte

leyes 13, 15

3. los contratos de datos fracasarán por la misma razón que los controles: si publicar un dato con contrato cuesta más que publicarlo sin él, se publicará sin él ley 16
4. en la parte de IA, la evaluación será lo que más se omita: se medirá la calidad del modelo en el laboratorio y no en producción, y la deriva se descubrirá por una queja, no por una señal ley 22
5. y una refutable con cifra: **el coste de las cargas de IA estará dominado por la inferencia y no por el entrenamiento, en una proporción de al menos tres a uno**;
y dentro de la inferencia, por peticiones que no necesitaban un modelo

Y el cierre de la parte 19: de once clases, los tres hallazgos más graves los encontraron un inventario, una factura y otro inventario, y ninguno una alerta. La parte 20 sube una capa —a las plataformas de datos e inteligencia artificial, por encima de cualquier proveedor concreto— y empieza por la arquitectura de datos y sus contratos. Es la clase 241.

4. Comparar las tres nubes

Con el mismo sistema montado tres veces, se pueden defender conclusiones.

LO QUE RESULTÓ EQUIVALENTE EN LAS TRES

- el techo de disponibilidad y su aritmética clase 185
- la teoría de colas y la concurrencia clase 186
- la consistencia decidida por operación clase 187
- la idempotencia y los mensajes fallidos clase 210
- el despliegue escalonado con vuelta atrás
- la estructura del código por ciclo de vida
- el orden de implantar un control: medir, avisar, aplicar
- y las pruebas negativas, una a una

LO QUE NO

- el modelo de gobierno y su alcance
- el modelo de permisos y sus ámbitos
- la red: regional frente a global
- la observabilidad y qué viene activado
- y lo que cada una factura por uso ley 28

Y la comparación medida:

p99 del flujo de compra	412 ms	438 ms	395 ms
disponibilidad observada	99,86 %	99,84 %	99,88 %
coste mensual	16.700 €	17.400 €	15.900 €
coste por pedido	0,046 €	0,051 €	0,044 €
valores por defecto cambiados	24	19	21
pruebas negativas fallidas	7/19	8/19	6/20
tiempo hasta producción	9 semanas	7 semanas	6 semanas
líneas de infraestructura	~4.100	~3.800	~3.400
conmutación de región	12 min 40	10 min 40	8 min 15
equipo dedicado	2,1	2,4	1,9

Y las cuatro conclusiones que se pueden defender:

- 1 EL COSTE DE UN SISTEMA EQUIVALENTE ES PARECIDO
la diferencia entre la más cara y la más barata es del 9 %, dentro de lo que cambia una decisión de diseño
→ elegir nube por precio de lista es un error
clases 216, 228
- 2 EL TIEMPO HASTA PRODUCCIÓN BAJA CON CADA REPETICIÓN
9, 7 y 6 semanas
→ y no por la nube: por el método
→ lo que se traslada bien es el ORDEN, no la configuración
- 3 LO QUE MÁS DIFIERE ES LO QUE MÁS CUESTA APRENDER
el modelo operativo: gobierno, permisos, red y observabilidad
→ tres meses en cada nube nueva
→ y ese es el coste real de la multinube clase 157
- 4 EL ERROR DOMINANTE AL AÑADIR UNA NUBE NO ES LA

IGNORANCIA: ES EL TRASLADO

→ cinco casos documentados en esta parte

→ y la contramedida es un procedimiento, no un curso:
al empezar con un servicio, comprobar explícitamente
qué se hereda, qué viene activado y qué se propaga
clase 229

Y la lista de comprobación de la clase:

- el sistema se construyó en el orden por coste de cambio
- hay lista explícita de valores por defecto cambiados
- el alcance de cada identidad está medido
- los perímetros están simulados un ciclo completo
- las 20 pruebas negativas se ejecutaron y hay fallos publicados
- el entorno se despliega de cero desde el código
- el coste está atribuido por encima del 90 %
- hay comparación con las otras nubes, con cifras
- está escrito lo que no se hace

Y el cierre que enlaza con la clase siguiente: la parte 20 deja de hablar de proveedores y sube a las plataformas de datos e inteligencia artificial, que se montan sobre cualquiera de las tres. Empieza por la arquitectura de datos y sus contratos, en la clase 241.

Ejemplo trabajado

El sistema de CloudShop en producción en Google Cloud. Lo que sigue es la lista de valores por defecto cambiados, el resultado de las veinte pruebas negativas —de las que fallaron seis— y la comparación final de las tres nubes.

La lista de valores por defecto cambiados:

servicio	por defecto	cambiado a
cortafuegos	salida permitida	denegada, con permitidos
cortafuegos	selector por etiqueta	por cuenta de servicio
red proyecto	subredes automáticas	personalizadas
almacenamiento	cuenta de servicio por defecto	una por carga
almacén analítico	sin registro de acceso	activado donde hay datos
almacén analítico	sin límite de bytes	200 GB por consulta
almacén analítico	tablas sin partición	particionadas y agrupadas
almacén analítico	sin exigir filtro	exigido
documentos	indexa todos los campos	4 de 34
documentos	reglas permisivas de ejemplo	reglas probadas con simulador
servicio de contenedores	conurrencia por defecto usada como 1	40, medida
servicio de contenedores	sin conector de red	con conector y salida por la red
servicio de contenedores	tiempo de espera alto	3× el p99
mensajería	sin tema de fallidos	con tema y suscripción
mensajería	entrega exactamente una vez desactivada	activa donde el duplicado cuesta
registros	sin caducidad	30 y 400 días
registros	acceso a datos desactivado	activado por servicio, acotado
clúster	110 pods por nodo	32, medidos
estado del código	local	remoto, cifrado, versionado
Spanner	clave secuencial	aleatoria
compromisos	comparados con lista	con el gasto real
total		21
que funcionaban sin cambiarlos		21

Las veinte pruebas negativas: seis fallaron.

✓ crear recurso sin etiquetas	rechazado
✓ crear clave de cuenta de servicio	rechazado
✓ suplantar sin permiso	denegado
X acceder fuera de una condición de recurso	
→ 2 de 11 servicios ignoraban la condición	clase 230
✓ activar papel privilegiado sin aprobación	denegado
✓ acceso de emergencia	90 s
X resolver nombre con conexión privada desde cada proyecto	
→ 3 proyectos creados el mes anterior sin zonas asociadas: la automatización llegó después	ley 27
✓ sacar datos a otra organización	perímetro lo impide
✓ desplegar incumpliendo política	falla en validación
✓ borrar el estado y restaurarlo	4 min
X plan sin cambios, vacío	
→ 6 recursos con deriva permanente por campos que la plataforma ajusta; marcados para ignorar	
✓ escribir con clave secuencial	reparto correcto
✓ consultar sin filtro de partición	rechazado
X consultar columna sensible sin etiqueta	
→ 1 vista materializada creada antes del etiquetado exponía la columna sin control	ley 27
✓ mismo mensaje 50 veces	1 efecto
✓ mensaje en tema de fallidos → alerta	38 s
✓ identidad desde otro espacio de nombres	denegado
X perder una región	
→ 8 min 15 s, dentro de lo previsto, pero la clave de cifrado propia vivía en una sola región y 2 almacenes quedaron inaccesibles	clase 239
X provocar la condición de cada alerta	
→ 7 de 45 no llegaron; 4 por umbral, 2 por destino y 1 por objetivo medido en el servidor	clase 238
✓ desplegar el entorno de cero	61 min

Y el análisis de las seis:

tres por la ley 27: recursos creados antes de que existiera la comprobación o el etiquetado que los cubriría
 dos por controles que no cubrían lo que se creía (condiciones ignoradas por 2 servicios, alerta con el indicador en el sitio equivocado)
 una por una dependencia no contada en el cálculo (la región de la clave de cifrado)

→ y el diagnóstico es el mismo de las clases 216 y 228:
 el sistema creció y las comprobaciones no crecieron con él
 → con el añadido de que aquí ya existían las comprobaciones y lo que faltó fue ejecutarlas sobre lo creado después

Las cifras del sistema, tras tres meses:

p99 del flujo de compra	< 500 ms	395 ms
disponibilidad observada	99,8 %	99,88 %
coste mensual	15.000 €	15.900 €
coste por pedido	0,050 €	0,044 €
conmutación de región	15 min	8 min 15
coste atribuido	90 %	95 %
alertas por turno	< 2	0,6
técnicas simuladas detectadas	> 90 %	13/14
despliegue del entorno de cero	—	61 min

La comparación final de las tres nubes, con lo que costó de verdad:

coste mensual	16.700 €	17.400 €	15.900 €
coste por pedido	0,046 €	0,051 €	0,044 €
p99 del flujo	412 ms	438 ms	395 ms
conmutación de región	12 min 40	10 min 40	8 min 15
valores por defecto cambiados	24	19	21
pruebas fallidas	7/19	8/19	6/20
tiempo hasta producción	9 semanas	7 semanas	6 semanas

meses para aprender el modelo operativo	—	3	3
equipo dedicado	2,1	2,4	1,9

Y OPERAR LAS TRES A LA VEZ

equipo dedicado sumado	6,4
equipo real necesario	7,8

→ un 22 % más que la suma, por los cambios de contexto y por mantener tres modelos operativos al día

clase 157

Y las decisiones que se tomaron con esta comparación:

no migrar nada entre nubes: los tres sistemas funcionan
 no operar los tres a la vez para la misma carga
 → CloudShop mantiene AWS como principal, Azure por el contrato corporativo y Google para las cargas de datos y de IA

clase 157

y el criterio escrito
 «cada nube por un motivo declarado; ninguna por preferencia»

La lección que este proyecto deja: veintidós valores por defecto cambiados, todos los cuales funcionaban, y los cinco más caros estaban en servicios que facturan por uso. De las seis pruebas negativas fallidas, tres fueron por recursos creados después de que existiera la comprobación. Y comparando las tres nubes con el mismo sistema montado bien, la diferencia de coste es del nueve por ciento: la nube no decide el coste; lo deciden las decisiones de diseño y los valores por defecto que nadie revisa.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/240-proyecto-cloudshop-productivo-en-google-cloud/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa cloudshop-gcp en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye cloudshop-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.

- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Al añadir una nube nueva fallan cosas que en las anteriores funcionaban	Se trasladaron suposiciones del modelo operativo anterior	Al empezar con cada servicio, comprueba explícitamente qué se hereda, qué viene activado y qué se propaga; es un procedimiento, no un curso.
Los valores por defecto caros pasan desapercibidos	En los servicios que facturan por uso, un valor por defecto es una decisión de gasto que no parece una decisión	Revisa primero los valores por defecto de todo lo que se factura por uso, y estima su coste antes de aceptar el asistente.
Las comprobaciones pasan y aparecen huecos en lo creado después	La automatización llegó más tarde que los recursos	Toda automatización nueva se acompaña de una tarea de remediación y de una comprobación periódica sobre lo existente.
Se elige nube comparando precios de lista	El coste lo deciden el diseño y los valores por defecto, no el proveedor	Compara el coste del mismo sistema montado bien; la diferencia suele estar dentro del ruido.
Operar varias nubes consume más equipo que la suma de las partes	Se cuentan las tecnologías y no los modelos operativos ni los cambios de contexto	Justifica cada nube por un motivo declarado y cuenta el coste operativo real, que es superior a la suma.
Una clave de cifrado propia deja datos inaccesibles al perder una región	La clave vive en una región y no entró en el cálculo del techo	Replica la clave y cuenta su disponibilidad como una dependencia dura más.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Cuál de las cinco predicciones de la clase 228 falló y qué patrón revela su fallo?
2. ¿Qué dice la ley 28 y en qué se distingue de la ley 26?
3. ¿Qué distinción faltaba en la predicción sobre identidad?
4. ¿Qué resultó equivalente entre las tres nubes y qué no?
5. ¿Por qué operar tres nubes cuesta más que la suma de operar cada una?

Referencias

- Google Cloud (2025). Architecture Framework. <<https://cloud.google.com/architecture/framework>>
- Google Cloud (2025). Enterprise foundations blueprint. <<https://cloud.google.com/architecture/security-foundations>>
- Google Cloud (2025). Landing zone design. <<https://cloud.google.com/architecture/landing-zones>>
- Beyer, B. y otros (2018). The Site Reliability Workbook. <<https://sre.google/workbook/table-of-contents/>>
- Forsgren, N. y otros (2018). Accelerate — medir antes y después con las mismas definiciones. <<https://itrevolution.com/product/accelerate/>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 19 en PDF · Recorrido de Google Cloud en PDF · Manual integral

Anterior	Índice	Siguiente
← 239 · SCC, VPC Service Controls, KMS y FinOps	Parte 19 · Programa	241 · Lakehouse, warehouse, mesh y contratos de datos →

Evaluación — 240 Proyecto: CloudShop productivo en Google Cloud

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega cloudshop-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 22 — Especializaciones, certificaciones y práctica profesional

273 — Mapeo AWS, Azure, Google Cloud, Kubernetes y FinOps

Parte: 22 — Especializaciones, certificaciones y práctica profesional Nivel: intermedio-avanzado · Horas estimadas: 4
Laboratorio: assessment · Estado: EXECUTABLECORE

Propósito

Traducir entre proveedores sin engañarse: dar el mapa de equivalencias entre las tres nubes, los contenedores y la disciplina de coste, y —más importante— dónde la equivalencia se rompe. La clase enseña el método de traducción por restricción en vez de por nombre, y las diferencias reales que este programa ha medido y que ninguna tabla de equivalencias recoge.

Resultados de aprendizaje

Al finalizar podrás:

1. Traducir entre proveedores por restricción, no por nombre de servicio.
2. Reconocer dónde la equivalencia aparente esconde una diferencia real.
3. Usar el mapa de las cinco capas para orientarse en una nube nueva.
4. Evaluar una traducción con las preguntas que la validan.
5. Aplicar el mismo método a los contenedores y a la disciplina de coste.

Conceptos centrales

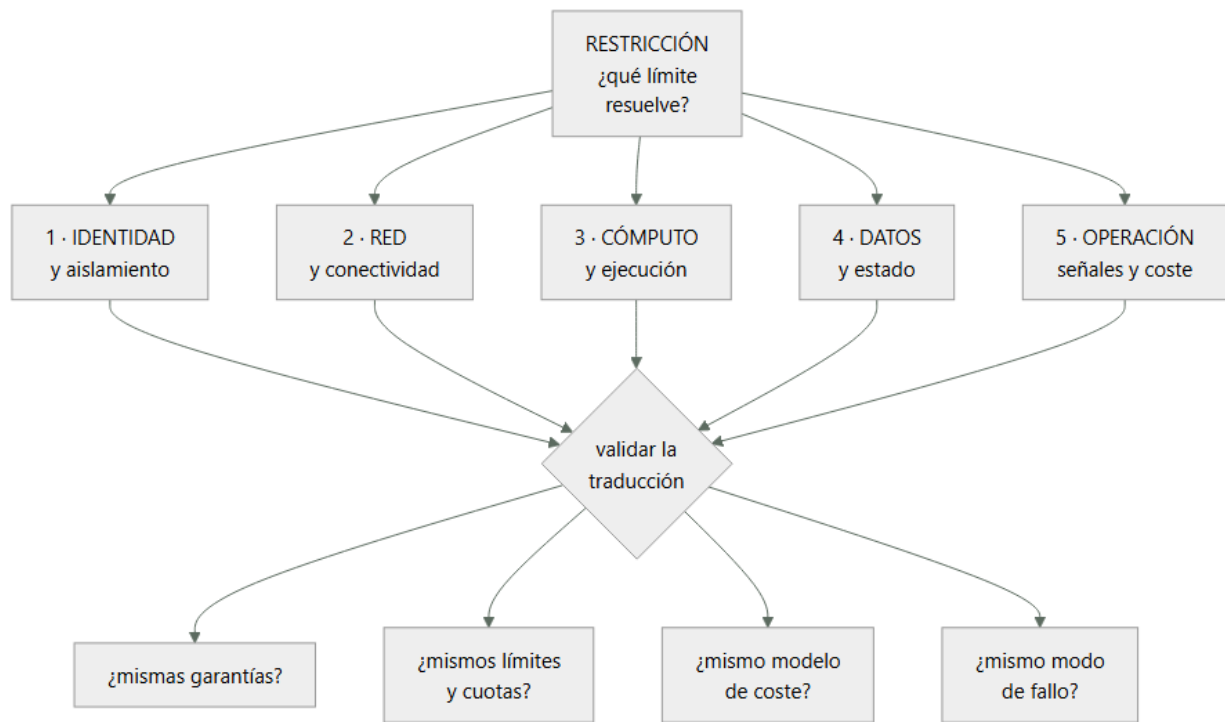
Concepto	Comprensión verificable
traducción por restricción	Buscar cómo la otra nube resuelve el mismo límite físico, en vez de buscar el servicio con nombre parecido.
equivalencia falsa	Dos servicios con el mismo propósito y garantías, límites o modelo de coste distintos.
unidad de aislamiento	El contenedor administrativo donde vive la separación: cuenta, suscripción o proyecto.
plano de control	La interfaz que crea y modifica recursos. Sus cuotas y su latencia difieren mucho entre nubes.
portabilidad aparente	Crear que usar la misma tecnología en tres nubes elimina las diferencias. No lo hace.
coste de traducción	Lo que cuesta operar en dos nubes: no es el doble de aprender, es el doble de operar.

Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento



Desarrollo

1. Traducir por restricción, no por nombre

Las tablas de equivalencias por nombre producen errores caros. El método que funciona empieza por la restricción.

EL MÉTODO

- 1 ¿qué RESTRICCIÓN resuelve esto?
→ «necesito que un servicio pruebe su identidad ante otro sin credencial permanente»
- 2 ¿cómo la resuelve la nube destino?
- 3 ¿con qué garantías, límites y coste?
- 4 ¿y con qué modo de fallo?

→ y solo entonces se escribe el nombre del servicio
→ empezar por el nombre es lo que produce las sorpresas

Y las cinco capas en las que conviene organizarse:

- 1 IDENTIDAD Y AISLAMIENTO
quién es quién, qué puede, y dónde vive la separación
→ cuenta · suscripción · proyecto
- 2 RED Y CONECTIVIDAD
direccionamiento, rutas, resolución de nombres, salida a internet, interconexión
- 3 CÓMPUTO Y EJECUCIÓN
máquinas, contenedores, funciones, escalado
- 4 DATOS Y ESTADO
objetos, bloques, relacional, no relacional, analítico, mensajería
- 5 OPERACIÓN
señales, registros, políticas, coste, cuotas

→ en toda nube existen las cinco
→ y aprender una nube nueva es rellenar esta rejilla, no leer un catálogo

Y el mapa por restricción, que es lo transferible:

RESTRICCIÓN	DÓNDE MIRAR EN CUALQUIER NUBE
«que un servicio pruebe quién es sin credencial fija»	identidad de carga de trabajo, federada
«separar entornos para que un fallo no cruce»	unidad de aislamiento administrativa

«que el tráfico solo llegue a quien debe»	segmentación de red y reglas por identidad
«que un cambio no afecte a todos a la vez»	despliegue progresivo del proveedor o propio
«que el dato sobreviva a un administrador comprometido»	copia en otra unidad de aislamiento, inmutable
«saber quién gasta qué»	etiquetas y jerarquía administrativa
«no pasar de N peticiones por segundo al plano de control»	cuotas del plano de control

→ y este mapa no cambia cuando cambian los productos

2. Dónde la equivalencia se rompe

Lo que ninguna tabla de equivalencias dice, y este programa ha medido.

- 1 LA UNIDAD DE AISLAMIENTO NO ES EQUIVALENTE
cuenta, suscripción y proyecto se parecen y difieren en cuánto cuesta crear una nueva
qué se hereda desde arriba
y qué queda compartido aunque parezca separado
→ y de aquí salen los errores de aislamiento más caros
clases 219, 231
- 2 LOS LÍMITES POR DEFECTO SON MUY DISTINTOS
dos nubes con el «mismo» servicio de funciones tienen concurrencias por defecto de órdenes distintos
→ y en la región secundaria, los valores por defecto mandan
clase 262, ley 26
- 3 EL MODELO DE COSTE CAMBIA LA ARQUITECTURA
donde el tráfico entre zonas se cobra, la arquitectura correcta es otra
donde el análisis se cobra por dato leído, el formato y la partición valen más que el motor clase 243, ley 28
→ y una traducción literal puede multiplicar la factura
- 4 LA COHERENCIA DE LOS ALMACENES DE OBJETOS
parecen iguales y sus garantías de lectura tras escritura, de listado y de sobrescritura no lo son
→ y esto rompe canales de datos silenciosamente
ley 29
- 5 LA RESOLUCIÓN DE NOMBRES Y LA SALIDA A INTERNET
quién resuelve, desde dónde, y qué pasa con las respuestas privadas difiere mucho clase 195
→ es la fuente número uno de «funciona en una nube y no en la otra»
- 6 EL PLANO DE CONTROL
su latencia y sus cuotas de peticiones difieren
→ y una herramienta de infraestructura como código que funciona en una nube se estrangula en otra
clases 128, 262
- 7 Y LA SEMÁNTICA DE LOS PERMISOS
denegar explícito, herencia, ámbito y evaluación no funcionan igual
→ una política traducida literalmente puede conceder más de lo que concedía clase 231

Y la regla que resume todo lo anterior:

LO QUE SE TRADUCE BIEN
el propósito y la forma de la arquitectura

LO QUE NO SE TRADUCE
garantías, límites, modelo de coste y modos de fallo

→ y esos cuatro son los que deciden si el sistema funciona

→ por eso la traducción se VALIDA midiendo, no leyendo

3. Contenedores y la portabilidad aparente

El caso especial que más expectativas defrauda.

LO QUE SÍ APORTA UNA CAPA COMÚN DE CONTENEDORES
el empaquetado y el modelo de despliegue se parecen
las descripciones de carga de trabajo son casi iguales
y el conocimiento del equipo se traslada

LO QUE NO ELIMINA
la identidad y su federación con la nube
la red: direccionamiento, salida, balanceo, resolución
el almacenamiento persistente y sus garantías
el registro de imágenes y su disponibilidad
las cuotas del plano de control
el coste, muy distinto por nube
y la operación: actualizaciones, versiones y su ritmo

→ la portabilidad es del ARTEFACTO, no del SISTEMA
→ y la parte que no se traslada es la que da los
incidentes clase 185

Y el coste real de operar en dos nubes:

NO ES EL DOBLE DE APRENDER: ES EL DOBLE DE OPERAR
dos inventarios clase 253
dos modelos de permisos clase 231
dos formas de red clase 194
dos conjuntos de cuotas clase 262
dos catálogos de alertas y dos guardias clase 257
dos formas de facturar clase 270
y ensayos en las dos clase 261

→ y por eso multinube por defecto es caro
→ y multinube por una RAZÓN concreta –normativa,
adquisición, un servicio que solo existe en una– se
sostiene

y la pregunta que ordena la decisión
«¿qué riesgo concreto estamos comprando con este coste?»
→ si la respuesta es «no depender de un proveedor», hay
que cuantificar cuánto vale eso y compararlo

Y el mapa de la disciplina de coste, que también se traduce:

lo que existe en las tres, con otros nombres
jerarquía administrativa para atribuir
etiquetas y su obligatoriedad
exportación detallada de facturación
compromisos con descuento
presupuestos y alertas
y capacidad interrumpible

lo que cambia de verdad
qué se cobra por transferencia entre zonas y regiones
cómo se factura el almacenamiento frío y su
recuperación
cómo se cobra la analítica: por dato leído o por
capacidad reservada
y el retraso con que llega el detalle de coste

→ y esos cuatro cambian decisiones de diseño
clase 270

4. Cómo se valida una traducción

Una traducción sobre el papel no vale nada. Estas son las comprobaciones que la convierten en fiable.

LAS CUATRO PREGUNTAS DE VALIDACIÓN

- 1 ¿MISMAS GARANTÍAS?
coherencia, durabilidad, orden, exactamente una vez
→ y aquí casi siempre hay una diferencia
- 2 ¿MISMOS LÍMITES Y CUOTAS?
por defecto y ampliables, por región
- 3 ¿MISMO MODELO DE COSTE?

qué se cobra por operación, por dato, por hora
4 ¿MISMO MODO DE FALLO?
¿falla rápido o se degrada? ¿qué se ve cuando falla?

→ y las cuatro se responden con documentación y con una prueba

Y la prueba mínima antes de comprometerse:

PROTOTIPO DE UNA SEMANA

el camino crítico de extremo a extremo
con identidad real, red real y datos de tamaño realista
midiendo latencia, coste por operación y comportamiento al fallar
y provocando un fallo a propósito clase 261

→ una semana de esto ahorra meses
→ y descubre las diferencias que ninguna tabla recoge

Y cómo se demuestra esta competencia:

LO QUE NO VALE

«conozco las tres nubes»
→ y la parte 22 predijo que esto rendiría peor que conocer una a fondo clase 264

LO QUE VALE

«migramos el canal de datos y descubrimos que las garantías de listado del almacén de objetos eran distintas; lo detectamos con una prueba de una semana antes de comprometerlos»
«traducimos la política y concedía más de lo que concedía la original; lo cogió la comprobación automática»
«el mismo diseño costaba 3,2 veces más por el cobro de tráfico entre zonas, y rediseñamos la colocación»

→ efecto, mecanismo y cifra clase 275

Y la lista de comprobación de la clase:

- traduzco por restricción, no por nombre de servicio
- uso la rejilla de cinco capas para orientarme
- compruebo garantías, límites, coste y modo de fallo
- sé que la unidad de aislamiento no es equivalente
- reviso los valores por defecto, sobre todo en la región secundaria
- compruebo la semántica de permisos, no solo su forma
- verifico resolución de nombres y salida a internet
- vigilo las cuotas del plano de control
- no confundo portabilidad del artefacto con la del sistema
- cuantifico el coste de operar en dos nubes
- hago un prototipo de una semana antes de comprometerme
- y provocho un fallo en ese prototipo

Y el cierre que enlaza con la clase siguiente: con el mapa y el método de traducción, queda la parte que examina esto por escrito y con tiempo limitado. Las preguntas de escenario y la estrategia de examen son la materia de la clase 274.

Ejemplo trabajado

CloudShop traduce tres piezas entre nubes. Lo que sigue son las cuatro diferencias que el prototipo de una semana encontró y ninguna tabla recogía, la política traducida que concedía de más, y la cuenta del coste real de operar en dos nubes.

Traducción 1 · El canal de datos.

qué se traducía
ingesta a almacén de objetos, catálogo, y consultas analíticas clase 242

la tabla de equivalencias decía
almacén de objetos → almacén de objetos

catálogo → catálogo
motor de consulta → motor de consulta

→ traducción aparentemente trivial

Y lo que el prototipo de una semana encontró:

DIFERENCIA 1 · garantías de listado
el proceso dependía de listar un prefijo justo después de escribir
→ en el origen, el listado era inmediato
→ en el destino, podía tardar
→ y el trabajo posterior procesaba ficheros de menos
SIN ERROR ley 29

cómo se detectó
prueba con 40.000 ficheros escritos y listados de inmediato
→ 61 casos de listado incompleto
cómo se arregló
manifiesto explícito de ficheros, en vez de listar

DIFERENCIA 2 · modelo de coste de la consulta
origen por dato leído
destino por capacidad reservada
→ el mismo conjunto de 41 informes
origen, tras optimizar 12 USD/informe
destino con capacidad mínima por debajo del umbral, pero con suelo mensual
→ por debajo de cierto volumen el destino era más caro
→ y por encima, mucho más barato
→ la decisión dependía del volumen, no del motor

DIFERENCIA 3 · cuota del plano de control
la herramienta de infraestructura como código aplicaba 341 recursos
→ en el destino se estrangulaba a mitad
→ aplicación fallida de forma intermitente clase 128
→ se resolvió partiendo el estado y limitando el paralelismo

DIFERENCIA 4 · zonas horarias en las particiones
la convención por defecto del catálogo difería
→ y las particiones diarias se desplazaban unas horas
→ los informes diarios cuadraban salvo en el límite del día clase 243

Y la valoración:

coste del prototipo	1 semana, 1 persona
diferencias encontradas	4
que ninguna tabla de equivalencias recogía	4
que habrían llegado a producción sin error visible	2

→ las dos silenciosas eran las peligrosas

Traducción 2 · La política que concedía de más.

política original
permitir lectura de un prefijo concreto del almacén, a un rol de servicio, con denegación explícita para todo lo demás clase 231

la traducción literal
se mantuvo la concesión y se omitió la denegación explícita
→ porque en la nube destino la herencia y la evaluación funcionaban distinto y «parecía innecesaria»

lo que produjo
el rol heredaba desde el nivel superior un permiso de lectura sobre el contenedor entero administrativo
→ y sin la denegación explícita, la herencia ganaba
→ el rol podía leer 214 conjuntos de datos en vez de 1

Y cómo se detectó:

no en la revisión humana: la revisión lo aprobó

lo cogió la comprobación automática de permisos efectivos
→ que compara «qué puede hacer realmente esta identidad»
antes y después clase 217
→ 214 recursos accesibles frente a 1 esperado

→ la lección: se compara el EFECTO, no el texto de la política
→ y esa comprobación se añadió a la cadena para toda traducción

La cuenta del coste real de operar en dos nubes.

CloudShop tenía una razón concreta para la segunda nube
un cliente del sector público exigía residencia en una región que el proveedor principal no ofrecía
clase 280

y se midió lo que costaba mantenerla

inventario y etiquetado, segunda nube	0,4 personas
modelo de permisos y su auditoría	0,3
red y conectividad	0,3
cuotas y capacidad	0,2
alertas, guardia y procedimientos	0,6
facturación y atribución	0,2
ensayos	0,2
actualizaciones y versiones	0,3
	■■■■■■■
	2,5 personas
más el coste de infraestructura	41.000 USD/mes
y el sobre coste por menor volumen (compromisos peores)	+9 %

Y la decisión que se tomó con esa cifra:

ingresos del cliente que exigía la segunda nube
1,9 M USD/año
coste de mantenerla ~2,5 personas + 492.000 USD/año

→ se mantuvo, y con la cifra escrita en el registro de decisión clase 272
→ con condición de revisión: «si estos ingresos bajan de 900.000, se revisa»

y lo que NO se hizo
no se replicó el resto de la plataforma en la segunda nube «por si acaso»
→ solo lo que ese cliente necesitaba
→ 6 servicios de 41

Y la comparación con lo que se había propuesto al principio:

propuesta inicial	«seamos multinube en todo, para no depender de un proveedor»
coste estimado	~7 personas + 1,4 M USD/año
riesgo evitado	no cuantificado

y la pregunta que la desmontó
«¿cuál es el escenario concreto del que nos protege, y cuánto nos costaría si ocurriera?»
→ el escenario realista era una subida de precios o una pérdida de servicio prolongada
→ y la protección real requería mantener el sistema EJECUTÁNDOSE en las dos, no solo poder migrar
→ lo que multiplicaba la cifra

→ se decidió: multinube donde hay una razón concreta; portabilidad razonable en lo demás
→ y «portabilidad razonable» se definió: contratos, datos

exportables y decisiones registradas, no abstracciones
propias clase 267

Y la rejilla de cinco capas, rellena para orientarse.

el equipo mantuvo una tabla de una página por nube

capa	qué mirar	dónde difiere más
identidad	unidad de aislamiento y federación	herencia y denegación
red	direccionamiento,	resolución de
	salida, balanceo	nombres privados
cómputo	escalado, arranque,	límites por
	interrumpible	defecto
datos	garantías del almacén	listado y coste
	y del relacional	por operación
operación	señales, políticas,	retraso del
	cuotas, coste	detalle de coste

→ una página por nube, actualizada cuando algo sorprende

→ y usada para entrar en una nube nueva en días

La lección que esta clase deja: la traducción del canal de datos parecía trivial y el prototipo de una semana encontró cuatro diferencias, dos de ellas silenciosas —un listado incompleto y un desplazamiento de particiones— que habrían llegado a producción sin producir ningún error. Y la política traducida pasó la revisión humana mientras concedía acceso a 214 conjuntos de datos en vez de uno: lo cogió comparar permisos efectivos, no leer el texto.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-22-specializations-certifications-career/273-mapeo-aws-azure-google-cloud-kubernetes-y-finops/lab.py
```

El laboratorio selecciona el motor de práctica assessment y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa certification-map en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una evaluación por escenarios con rúbrica y evidencia trazable. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye certification-map para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Un servicio equivalente se comporta distinto en producción	Se tradujo por nombre y no se compararon garantías, límites, coste y modo de fallo	Traduce por restricción y valida las cuatro preguntas con documentación y con un prototipo del camino crítico.
Una política traducida concede más de lo que concedía	La herencia y la evaluación de permisos difieren y la denegación explícita se omitió	Compara permisos efectivos antes y después, no el texto de la política; automatiza esa comprobación en la cadena.
El canal de datos procesa registros de menos sin dar error	Las garantías de listado y de lectura tras escritura del almacén no son iguales	Usa manifiestos explícitos en vez de listar, y prueba con volumen realista escribiendo y listando de inmediato.
La infraestructura como código se aplica a medias e intermitentemente	Se alcanzan las cuotas de peticiones del plano de control, que difieren por nube	Parte el estado, limita el paralelismo y vigila la cuota del plano de control como una más del inventario.
El mismo diseño cuesta varias veces más en la otra nube	El modelo de coste cambia lo que es una buena arquitectura	Revisa qué se cobra por transferencia, por operación y por dato leído; rediseña colocación y formato antes de migrar.
Se adopta multinube por no depender de un proveedor y el coste se dispara	No se cuantificó el riesgo evitado ni el coste de operar dos nubes	Exige un escenario concreto y su coste; multinube donde hay una razón, portabilidad razonable en lo demás.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Por qué se traduce por restricción y no por nombre de servicio?
2. ¿Cuáles son las cinco capas de la rejilla y para qué sirven?
3. ¿Qué cuatro cosas no se traducen nunca y por qué deciden el resultado?
4. ¿Qué aporta y qué no aporta una capa común de contenedores?
5. ¿Qué comprueba un prototipo de una semana que ninguna tabla recoge?

Referencias

- AWS (2024). Compare AWS and Azure services. <<https://learn.microsoft.com/azure/architecture/aws-professional/services>>
- Google Cloud (2024). Google Cloud for AWS professionals. <<https://cloud.google.com/docs/get-started/aws-comparison>>
- CNCF (2024). Kubernetes conformance and portability. <<https://www.cncf.io/certification/software-conformance/>>
- FinOps Foundation (2024). FOCUS: FinOps cost and usage specification — coste comparable entre nubes. <<https://focus.finops.org/>>
- Brewer, E. (2012). CAP twelve years later: how the rules have changed — por qué las garantías no se traducen. <<https://www.infoq.com/articles/cap-twelve-years-later-how-the-rules-have-changed/>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Descargar: Parte 22 en PDF · Manual integral

Anterior	Índice	Siguiente
← 272 · Ruta Cloud Solutions Architect	Parte 22 · Programa	274 · Preguntas de escenario y estrategia de examen →

Evaluación — 273 Mapeo AWS, Azure, Google Cloud, Kubernetes y FinOps

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega certification-map con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.